Physlib Monthly Report

Jun 2026

Joseph Tooby-Smith Nicola Bernini Robby Sneiderman Yidi Qi
Gregory Loges Zhi Kai Pong giuseppe.sorge kastch nateabr
juanjfndz Lazar Milikic Andrea Pari doxtor6 Bjørn Kjos-Hanssen
Justin Findlay Shibashish abudjum Christian Krause Florian Wiesner
LibertasSpZ Paul Lezeau Ranjan Sharma Shlok Vaibhav Singh
Tracy McSheery Vasily Ilin Wouter Deconinck Yanting Teng

Generated Thu, 30 Jul 2026 10:16:03 GMT

Abstract

Auto-generated summary of changes to `leanprover-community/physlib` on branch `upstream/master` between `d44dd71` and `05930fe`. The full diff is available at GitHub compare.

Contents

1 Introduction

During Jun 2026, 27 contributors submitted 154 commits that changed 46,837 lines (+28,676 / -18,161) across 492 files spanning 47 top-level areas of the repository. This report catalogues 1,496 newly added Lean declarations: 982 lemmas, 317 defs, 109 instances, 43 theorems, 29 abbrevs, 13 structures, 1 inductive, 1 informal definition, and 1 informal lemma.

See the full compare diff and the list of commits on GitHub for the raw source.

1.1 Contributors

Contributor	Commits
Joseph Tooby-Smith	21
Nicola Bernini	20
Robby Sneiderman	15
Yidi Qi	13
Gregory Loges	12
Zhi Kai Pong	9
giuseppe.sorge	7
kastch	7
nateabr	7
juanjfndz	6
Lazar Milikic	5
Andrea Pari	4
doxtor6	4
Bjørn Kjos-Hanssen	3
Justin Findlay	3
Shibashish	3
abudjum	2
Christian Krause	2
Florian Wiesner	2
LibertasSpZ	2
Paul Lezeau	1
Ranjan Sharma	1
Shlok Vaibhav Singh	1
Tracy McSheery	1
Vasily Ilin	1
Wouter Deconinck	1
Yanting Teng	1

1.2 Modified subfolders

Subfolder	Files	Additions	Deletions
Physlib/Relativity	85	+5,706	-7,342
Physlib/SpaceAndTime	40	+3,364	-1,393
Physlib/Mathematics	35	+2,642	-1,564
Physlib/Particles	44	+2,087	-1,693
PhyslibAlpha/ClassicalFieldTheory	18	+3,083	-0
Physlib/QuantumMechanics	33	+1,916	-791
Physlib/ClassicalMechanics	15	+1,393	-1,014

Physlib/Electromagnetism	31	+1,034	-1,107
scripts	9	+1,683	-10
PhyslibAlpha/QuantumMechanics	2	+1,308	-0
QuantumInfo/ForMathlib	38	+472	-581
Physlib/StringTheory	4	+306	-655
PhyslibAlpha/SpaceAndTime	7	+903	-0
PhyslibAlpha/Mathematics	1	+565	-0
Physlib/QFT	43	+153	-389
Physlib/StatisticalMechanics	6	+116	-365
Physlib/Thermodynamics	3	+81	-330
QuantumInfo/Entropy	4	+150	-173
Physlib/Units	9	+173	-128
Physlib/Cosmology	7	+264	-1
scripts/PhyslibAlpha	5	+242	-0
QuantumInfo/States	6	+73	-102
.github/workflows	3	+118	-54
scripts/MetaPrograms	12	+133	-38
QuantumInfo/ResourceTheory	3	+73	-81
lakefile.lean	1	+0	-125
lakefile.toml	1	+125	-0
PhyslibAlpha/ClassicalMechanics	1	+122	-0
QuantumInfo/Channels	6	+51	-57
Physlib/CondensedMatter	1	+21	-71
AGENTS.md	1	+85	-0
Physlib/FluidDynamics	1	+14	-41
lake-manifest.json	1	+27	-26
Physlib.lean	1	+39	-4
QuantumInfo/ClassicalInfo	3	+18	-15
PhyslibAlpha.lean	1	+32	-0
AI-POLICY.md	1	+31	-0
PhyslibAlpha	1	+30	-0
docs	1	+23	-0
README.md	1	+7	-1
CITATION.cff	1	+7	-0
QuantumInfo/Measurements	1	+1	-4
CONTRIBUTING.md	1	+2	-2
QuantumInfo/Operators	1	+1	-2
lean-toolchain	1	+1	-1
.codespellignore	1	+1	-0
Physlib/Meta	1	+0	-1

1.3 New declarations by kind

Kind	Count
lemma	982
def	317
instance	109
theorem	43
abbrev	29
structure	13

inductive	1
informal_definition	1
informal_lemma	1

2 Details by file

2.1 Physlib.ClassicalMechanics.FreeParticle.Basic

Physlib/ClassicalMechanics/FreeParticle/Basic.lean on GitHub.

def ClassicalMechanics.FreeParticle.linearMomentum [source]

```
def linearMomentum (s : FreeParticle) (q : Trajectory) (t : Time) : ℝ
```

lemma ClassicalMechanics.FreeParticle.linearMomentum_conserved_of_velocity_const [source]

If a free-particle trajectory has constant velocity, then its linear momentum is constant.

```
lemma linearMomentum_conserved_of_velocity_const (s : FreeParticle) (q : Trajectory)
    (h : ∃ ₀v, ∀ t, s.velocity q t = ₀v) :
    ∃ p, ∀ t, s.linearMomentum q t = p
```

theorem ClassicalMechanics.FreeParticle.linearMomentum_conserved [source]

A free particle satisfying the equation of motion conserves linear momentum.

Newton's second law implies that the acceleration vanishes, so the velocity is constant. Since the particle mass is fixed, the linear momentum is constant in time.

```
theorem linearMomentum_conserved (s : FreeParticle) (q : Trajectory)
    (h : ∀ t, s.NewtonsSecondLaw q t) (hcont : ContDiff ℝ 2 q) :
    ∃ p, ∀ t, s.linearMomentum q t = p
```

2.2 Physlib.ClassicalMechanics.HarmonicOscillator.ConfigurationSpace

Physlib/ClassicalMechanics/HarmonicOscillator/ConfigurationSpace.lean on GitHub.

lemma ClassicalMechanics.HarmonicOscillator.ConfigurationSpace.coe_apply [source]

```
lemma coe_apply (x : ConfigurationSpace) (i : Fin 1) : x i = x.val i
```

instance ClassicalMechanics.HarmonicOscillator.ConfigurationSpace.«instance@8aef359a» [source]

The topology on configuration space, induced by the chosen coordinate 'ConfigurationSpace.val' into 'EuclideanSpace ℝ (Fin 1)'.

```
instance : TopologicalSpace ConfigurationSpace
```

def ClassicalMechanics.HarmonicOscillator.ConfigurationSpace.valEquiv [source]

The coordinate equivalence between configuration space and its 'EuclideanSpace ℝ (Fin 1)' model, given by 'ConfigurationSpace.val' with its wrapper inverse.

```
def valEquiv : ConfigurationSpace ≃ EuclideanSpace ℝ (Fin 1) where
```

instance ClassicalMechanics.HarmonicOscillator.ConfigurationSpace.«instance@f9bea586» [source]

Configuration space is Hausdorff, transported from ‘EuclideanSpace ℝ (Fin 1)‘ across the coordinate homeomorphism.

```
instance : T2Space ConfigurationSpace
```

instance ClassicalMechanics.HarmonicOscillator.ConfigurationSpace.«instance@572511a8» [source]

Configuration space is second countable, transported from ‘EuclideanSpace ℝ (Fin 1)‘ across the coordinate homeomorphism.

```
instance : SecondCountableTopology ConfigurationSpace
```

instance ClassicalMechanics.HarmonicOscillator.ConfigurationSpace.«instance@9e9cd011» [source]

The structure of a charted space on ‘ConfigurationSpace‘, modeled on its ‘EuclideanSpace ℝ (Fin 1)‘ coordinate via the single global chart ‘valHomeomorphism‘.

```
instance : ChartedSpace (EuclideanSpace ℝ (Fin 1)) ConfigurationSpace where
```

instance ClassicalMechanics.HarmonicOscillator.ConfigurationSpace.«instance@66ad2370» [source]

The structure of a smooth (indeed analytic) manifold on ‘ConfigurationSpace‘. With a single global chart, the only coordinate change is the chart's self-transition, which is analytic.

```
instance : IsManifold 𝓘(ℝ, EuclideanSpace ℝ (Fin 1)) ω ConfigurationSpace where
```

2.3 Physlib.ClassicalMechanics.HarmonicOscillator.Solution

Physlib/ClassicalMechanics/HarmonicOscillator/Solution.lean on GitHub.

def ClassicalMechanics.HarmonicOscillator.InitialConditionsFromTwoPositions.toInitialConditions [source]

Convert two-position initial conditions to standard initial conditions at ‘t = 0‘.

*Obtained by solving the 2×2 linear system from the trajectory formula at ‘t_1‘ and ‘t_2‘. See ‘toInitialConditions_trajectory_at_t_1‘ and ‘toInitialConditions_trajectory_at_t_2‘ in section D.2 for the correctness proofs (valid under ‘sin (S.ω * (t_2 - t_1)) ≠ 0‘).*

```
noncomputable def toInitialConditions (S : HarmonicOscillator)
    (IC : InitialConditionsFromTwoPositions) : InitialConditions where
```

def ClassicalMechanics.HarmonicOscillator.InitialConditionsFromTwoVelocities.toInitialConditions [source]

Convert two-velocity initial conditions to standard initial conditions at ‘t = 0‘.

*Obtained by solving the 2×2 linear system from the velocity formula at ‘t_1‘ and ‘t_2‘. See ‘toInitialConditions_velocity_at_t_1‘ and ‘toInitialConditions_velocity_at_t_2‘ in section D.3 for the correctness proofs (valid under ‘sin (S.ω * (t_2 - t_1)) ≠ 0‘).*

```
noncomputable def toInitialConditions (S : HarmonicOscillator)
    (IC : InitialConditionsFromTwoVelocities) : InitialConditions where
```

lemma ClassicalMechanics.HarmonicOscillator.InitialConditionsFromTwoPositions.toInitialConditions_traj
[source]

*The trajectory from ‘toInitialConditions‘ passes through ‘x_t_1‘ at time ‘t_1‘, provided ‘sin (S.ω * (t_2 - t_1)) ≠ 0‘.*

```
lemma ₁toInitialConditions_trajectory_at_t (S : HarmonicOscillator)
    (IC : InitialConditionsFromTwoPositions)
    (Δh : sin (Sω. * (IC.₂t - IC.₁t)) ≠ 0) :
    (IC.toInitialConditions S).trajectory S IC.₁t = IC.₁x_t
```

lemma ClassicalMechanics.HarmonicOscillator.InitialConditionsFromTwoPositions.toInitialConditions_traj
[source]

*The trajectory from ‘toInitialConditions‘ passes through ‘x_t_2‘ at time ‘t_2‘, provided ‘sin (S.ω * (t_2 - t_1)) ≠ 0‘.*

```
lemma ₂toInitialConditions_trajectory_at_t (S : HarmonicOscillator)
    (IC : InitialConditionsFromTwoPositions)
    (Δh : sin (Sω. * (IC.₂t - IC.₁t)) ≠ 0) :
    (IC.toInitialConditions S).trajectory S IC.₂t = IC.₂x_t
```

lemma ClassicalMechanics.HarmonicOscillator.InitialConditionsFromTwoVelocities.toInitialConditions_vel
[source]

*The trajectory from ‘toInitialConditions‘ has velocity ‘v_t_1‘ at time ‘t_1‘, provided ‘sin (S.ω * (t_2 - t_1)) ≠ 0‘.*

```
lemma ₁toInitialConditions_velocity_at_t (S : HarmonicOscillator)
    (IC : InitialConditionsFromTwoVelocities)
    (Δh : sin (Sω. * (IC.₂t - IC.₁t)) ≠ 0) :
    ∂ₜ ((IC.toInitialConditions S).trajectory S) IC.₁t = IC.₁v_t
```

lemma ClassicalMechanics.HarmonicOscillator.InitialConditionsFromTwoVelocities.toInitialConditions_vel
[source]

*The trajectory from ‘toInitialConditions‘ has velocity ‘v_t_2‘ at time ‘t_2‘, provided ‘sin (S.ω * (t_2 - t_1)) ≠ 0‘.*

```
lemma ₂toInitialConditions_velocity_at_t (S : HarmonicOscillator)
    (IC : InitialConditionsFromTwoVelocities)
    (Δh : sin (Sω. * (IC.₂t - IC.₁t)) ≠ 0) :
    ∂ₜ ((IC.toInitialConditions S).trajectory S) IC.₂t = IC.₂v_t
```

def ClassicalMechanics.HarmonicOscillator.AmplitudePhase.toInitialConditions [source]

Convert amplitude–phase initial conditions to standard initial conditions at ‘t = 0‘, via ‘$x_0 = A \cos \varphi$‘ and ‘$v_0 = A\,\omega \sin \varphi$‘.

See ‘toInitialConditions_trajectory_eq_cos‘ and ‘toInitialConditions_velocity_eq_sin‘ in section E.3 for the correctness proofs.

```
noncomputable def toInitialConditions (S : HarmonicOscillator) (IC : AmplitudePhase) :
    InitialConditions where
```

lemma ClassicalMechanics.HarmonicOscillator.AmplitudePhase.toInitialConditions_trajectory_eq_cos [source]

The trajectory of amplitude–phase initial conditions is the cosine normal form ‘x(t) = A cos (ω t - φ)‘.

```
lemma toInitialConditions_trajectory_eq_cos (S : HarmonicOscillator) (IC : AmplitudePhase)
    (t : Time) :
    (IC.toInitialConditions S).trajectory S t
      = EuclideanSpace.single 0 (IC.A * cos (Sω. * t - ICφ.))
```

lemma ClassicalMechanics.HarmonicOscillator.AmplitudePhase.toInitialConditions_velocity_eq_sin [source]

The velocity of the amplitude–phase trajectory is ‘v(t) = -A ω sin (ω t - φ)‘.

```
lemma toInitialConditions_velocity_eq_sin (S : HarmonicOscillator) (IC : AmplitudePhase)
    (t : Time) :
    ∂ₜ ((IC.toInitialConditions S).trajectory S) t
      = EuclideanSpace.single 0 (-(IC.A * Sω. * sin (Sω. * t.val - ICφ.)))
```

def ClassicalMechanics.HarmonicOscillator.AmplitudePhase.fromInitialConditions [source]

Recover amplitude–phase data from standard initial conditions, as the polar coordinates of the phase vector ‘$(x_0,\ v_0 / \omega)$‘ embedded as ‘$z = x_0 + (v_0 / \omega)\,i$‘: the amplitude is ‘$\|z\|$‘ and the phase is ‘Complex.arg z‘.

See ‘toInitialConditions_fromInitialConditions‘ for the right-inverse identity.

```
noncomputable def fromInitialConditions (S : HarmonicOscillator) (IC : InitialConditions) :
    AmplitudePhase where
```

lemma ClassicalMechanics.HarmonicOscillator.AmplitudePhase.toInitialConditions_fromInitialConditions [source]

‘fromInitialConditions‘ is a right inverse of ‘toInitialConditions‘: converting initial conditions to amplitude–phase form and back recovers them exactly.

```
lemma toInitialConditions_fromInitialConditions (S : HarmonicOscillator)
    (IC : InitialConditions) :
    (fromInitialConditions S IC).toInitialConditions S = IC
```

lemma ClassicalMechanics.HarmonicOscillator.InitialConditions.trajectory_eq_cos [source]

Every trajectory of the harmonic oscillator is a single shifted cosine after converting its initial conditions to amplitude-phase form.

```
lemma trajectory_eq_cos (IC : InitialConditions) (t : Time) :
    IC.trajectory S t =
      EuclideanSpace.single 0 ((AmplitudePhase.fromInitialConditions S IC).A *
        cos (Sω. * t - (AmplitudePhase.fromInitialConditions S IC)φ.))
```

lemma ClassicalMechanics.HarmonicOscillator.InitialConditions.trajectory_velocity_eq_sin
[source]

The velocity of every trajectory is the corresponding shifted sine in amplitude-phase form.

```
lemma trajectory_velocity_eq_sin (IC : InitialConditions) (t : Time) :
    ∂ₜ (IC.trajectory S) t =
      EuclideanSpace.single 0 (-((AmplitudePhase.fromInitialConditions S IC).A * Sω. *
        sin (Sω. * t.val - (AmplitudePhase.fromInitialConditions S IC)φ.)))
```

lemma ClassicalMechanics.HarmonicOscillator.InitialConditions.trajectory_velocity_eq_zero_iff_sin_eq_z
[source]

For nonzero amplitude, the velocity vanishes exactly when the sine factor in amplitude-phase form vanishes.

```
lemma trajectory_velocity_eq_zero_iff_sin_eq_zero (IC : InitialConditions)
    (hA : (AmplitudePhase.fromInitialConditions S IC).A ≠ 0) (t : Time) :
    ∂ₜ (IC.trajectory S) t = 0 ↔
      sin (Sω. * t.val - (AmplitudePhase.fromInitialConditions S IC)φ.) = 0
```

lemma ClassicalMechanics.HarmonicOscillator.InitialConditions.trajectory_velocity_eq_zero_iff_exists_i
[source]

For nonzero amplitude, the velocity is zero exactly at phase times ‘$\varphi + n\pi$‘.

```
lemma trajectory_velocity_eq_zero_iff_exists_int (IC : InitialConditions)
    (hA : (AmplitudePhase.fromInitialConditions S IC).A ≠ 0) (t : Time) :
    ∂ₜ (IC.trajectory S) t = 0 ↔
      ∃ n : ℤ,
        (t : ℝ) =
          ((AmplitudePhase.fromInitialConditions S IC)φ. + n * π) / Sω.
```

lemma ClassicalMechanics.HarmonicOscillator.InitialConditions.trajectory_velocity_eq_zero_iff_norm_eq_
[source]

The velocity vanishes exactly when the trajectory has norm equal to the amplitude.

```
lemma trajectory_velocity_eq_zero_iff_norm_eq_amplitude (IC : InitialConditions)
    (t : Time) :
    ∂ₜ (IC.trajectory S) t = 0 ↔
      ‖IC.trajectory S ‖t = (AmplitudePhase.fromInitialConditions S IC).A
```

lemma ClassicalMechanics.HarmonicOscillator.InitialConditions.trajectory_eq_zero_iff_cos_eq_zero
[source]

For nonzero amplitude, the trajectory passes through zero exactly when the cosine factor in amplitude-phase form vanishes.

```
lemma trajectory_eq_zero_iff_cos_eq_zero (IC : InitialConditions)
    (hA : (AmplitudePhase.fromInitialConditions S IC).A ≠ 0) (t : Time) :
    IC.trajectory S t = 0 ↔
      cos (Sω. * t.val - (AmplitudePhase.fromInitialConditions S IC)φ.) = 0
```

lemma ClassicalMechanics.HarmonicOscillator.InitialConditions.trajectory_eq_zero_iff_exists_int [source]

For nonzero amplitude, the trajectory passes through zero exactly at phase times ‘φ + (2n + 1)π / 2‘.

```
lemma trajectory_eq_zero_iff_exists_int (IC : InitialConditions)
    (hA : (AmplitudePhase.fromInitialConditions S IC).A ≠ 0) (t : Time) :
    IC.trajectory S t = 0 ↔
      ∃ n : ℤ,
        (t : ℝ) =
          ((AmplitudePhase.fromInitialConditions S IC)φ. + (2 * n + 1) * π / 2) / Sω.
```

lemma ClassicalMechanics.HarmonicOscillator.return_time [source]

Assuming that the initial coordinate and velocity are not simultaneously zero, the time stamps when the harmonic oscillator returns to its initial coordinate and velocity is a multiple of its period

```
lemma return_time (IC : InitialConditions) (non_trivial : IC.₀x ≠ 0 ∨ IC.₀v ≠ 0)
    (t : Time) (ht : IC.trajectory S t = IC.₀x ∧ ∂ₜ (IC.trajectory S) t = IC.₀v) :
    ∃ n : ℤ,  (n : ℝ) * (T S) = t
```

2.4 Physlib.ClassicalMechanics.Lagrangian.TotalDerivativeEquivalence

Physlib/ClassicalMechanics/Lagrangian/TotalDerivativeEquivalence.lean on GitHub.

lemma ClassicalMechanics.Lagrangian.isTotalTimeDerivative_explicit [source]

Explicit reformulation (by the chain rule): $\delta L(t,\ q,\ d_t\ q) = \partial F/\partial t(t,\ q) + \langle \nabla_r F(t,\ q),\ d_t\ q\rangle$

or

$\delta L(t,\ q,\ d_t\ q) = fderiv\ \mathbb{R}\ F\ (t,\ q)\ (1,\ d_t\ q)$

```
lemma isTotalTimeDerivative_explicit δ{L : Time → X → X → ℝ} :
    IsTotalTimeDerivative δL ↔  ∃( (F : Time → X → ℝ) (_ : ContDiff ℝ ∞ ₁F),
    ∀ t q v, δL t q v = fderiv ℝ ₁F (t, q) ((1 : Time), v))
```

2.5 Physlib.Electromagnetism.Distributional.Basic

Physlib/Electromagnetism/Distributional/Basic.lean on GitHub.

def Electromagnetism.DistElectromagneticPotential.ofStaticScalarPotentialFunction [source]

The creation of an electromagnetic potential from a static scalar-potential function.

If the function is distribution-bounded, it is first promoted to a distribution using ‘Space.distOfFunction‘. Otherwise the constructor returns zero.

```
noncomputable def ofStaticScalarPotentialFunction {d} (c : SpeedOfLight)
    φ( : Space d → ℝ) : DistElectromagneticPotential d
```

lemma Electromagnetism.DistElectromagneticPotential.ofStaticScalarPotentialFunction_of_isDistBounded [source]

```
@[simp]
lemma ofStaticScalarPotentialFunction_of_isDistBounded {d} (c : SpeedOfLight)
    φ( : Space d → ℝ) (φh : Space.IsDistBounded φ) :
    ofStaticScalarPotentialFunction c φ =
      ofStaticScalarPotential c (Space.distOfFunction φ φh)
```

lemma Electromagnetism.DistElectromagneticPotential.ofStaticScalarPotentialFunction_eq_zero_of_not_isD [source]

```
@[simp]
lemma ofStaticScalarPotentialFunction_eq_zero_of_not_isDistBounded {d}
    (c : SpeedOfLight) φ( : Space d → ℝ) (φh : ¬ Space.IsDistBounded φ) :
    ofStaticScalarPotentialFunction c φ = 0
```

def Electromagnetism.DistElectromagneticPotential.ofVectorPotential [source]

The creation of an electromagnetic potential from a vector potential.

```
noncomputable def ofVectorPotential {d} (c : SpeedOfLight) :
    ((Time × Space d) →d[] EuclideanSpace ℝ (Fin d)) →ₗ[ℝ]
    DistElectromagneticPotential d where
```

def Electromagnetism.DistElectromagneticPotential.ofStaticVectorPotential [source]

The creation of an electromagnetic potential from a static vector potential.

```
noncomputable def ofStaticVectorPotential {d} (c : SpeedOfLight) :
    ((Space d) →d[] EuclideanSpace ℝ (Fin d)) →ₗ[ℝ] DistElectromagneticPotential d
```

lemma Electromagnetism.DistElectromagneticPotential.distTensorDeriv_eq_sum_sum [source]

```
lemma distTensorDeriv_eq_sum_sum {d} (A : DistElectromagneticPotential d)
    ε( : 𝓢(SpaceTime d, ℝ)) :
    distTensorDeriv A ε = ∑ μ, ∑ ν, (SpaceTime.distDeriv μ A ε ν) •
      Lorentz.CoVector.basis μ ⊗ₜ[ℝ] Lorentz.Vector.basis ν
```

lemma Electromagnetism.DistElectromagneticPotential.distTensorDeriv_basis_repr_apply [source]

```
@[simp]
lemma distTensorDeriv_basis_repr_apply {d} μν{ : (Fin 1 ⊕ Fin d) × (Fin 1 ⊕ Fin d)}
    (A : DistElectromagneticPotential d)
    ε( : 𝓢(SpaceTime d, ℝ)) :
    (Lorentz.CoVector.basis.tensorProduct Lorentz.Vector.basis).repr (distTensorDeriv A ε) μν =
    distDeriv μν.1 A ε μν.2
```

lemma Electromagnetism.DistElectromagneticPotential.toTensor_distTensorDeriv_basis_repr_apply [source]

```
lemma toTensor_distTensorDeriv_basis_repr_apply {d} (A : DistElectromagneticPotential d)
    ε( : □(SpaceTime d, ℝ)) (b : ComponentIdx (S := realLorentzTensor d)
      (Fin.append ![Color.down] ![Color.up])) :
    (Tensor.basis _).repr (Tensorial.toTensor (distTensorDeriv A ε)) b =
    distDeriv (b 0) A ε (b 1)
```

2.6 Physlib.Electromagnetism.Kinematics.EMPotential

Physlib/Electromagnetism/Kinematics/EMPotential.lean on GitHub.

instance Electromagnetism.ElectromagneticPotential.«instance@94a1323b» [source]

```
instance {d} : Zero (ElectromagneticPotential d) where
```

lemma Electromagnetism.ElectromagneticPotential.zero_val [source]

```
@[simp]
lemma zero_val {d} : (0 : ElectromagneticPotential d).val = 0
```

lemma Electromagnetism.ElectromagneticPotential.zero_apply [source]

```
@[simp]
lemma zero_apply {d} (x : SpaceTime d) : (0 : ElectromagneticPotential d) x = 0
```

instance Electromagnetism.ElectromagneticPotential.«instance@3f7e7f60» [source]

```
instance {d} : Neg (ElectromagneticPotential d) where
```

lemma Electromagnetism.ElectromagneticPotential.neg_val [source]

```
@[simp]
lemma neg_val {d} (A : ElectromagneticPotential d) :
    (- A).val = - A.val
```

lemma Electromagnetism.ElectromagneticPotential.neg_apply [source]

```
@[simp]
lemma neg_apply {d} (A : ElectromagneticPotential d) (x : SpaceTime d) :
    (- A) x = - A x
```

instance Electromagnetism.ElectromagneticPotential.«instance@6c8e2750» [source]

```
instance {d} : Sub (ElectromagneticPotential d) where
```

lemma Electromagnetism.ElectromagneticPotential.sub_val [source]

```
@[simp]
lemma sub_val {d} (A B : ElectromagneticPotential d) :
    (A - B).val = A.val - B.val
```

lemma Electromagnetism.ElectromagneticPotential.sub_apply [source]

```
@[simp]
lemma sub_apply {d} (A B : ElectromagneticPotential d) (x : SpaceTime d) :
    (A - B) x = A x - B x
```

instance Electromagnetism.ElectromagneticPotential.«instance@2040b1c1» [source]

```
instance {d} : AddCommGroup (ElectromagneticPotential d) where
```

lemma Electromagnetism.ElectromagneticPotential.action_apply [source]

```
@[simp]
lemma action_apply {d} Λ( : LorentzGroup d) (A : ElectromagneticPotential d)
    (x : SpaceTime d) :
    Λ( • A) x = Λ • A Λ⁻(¹ • x)
```

instance Electromagnetism.ElectromagneticPotential.«instance@f3d05234» [source]

```
noncomputable instance {d} :
    DistribMulAction (LorentzGroup d) (ElectromagneticPotential d) where
```

2.7 Physlib.Electromagnetism.Kinematics.FieldStrength

Physlib/Electromagnetism/Kinematics/FieldStrength.lean on GitHub.

lemma Electromagnetism.ElectromagneticPotential.toFieldStrength_eval_eq_basis_repr [source]

Evaluating both tensor indices of the field strength gives the coefficient in the standard tensor-product basis.

```
lemma toFieldStrength_eval_eq_basis_repr {d} (A : ElectromagneticPotential d)
    (x : SpaceTime d) μ( ν : Fin 1 ⊕ Fin d) :
    toField {A.toFieldStrength x | μ[] ν⊤[]} =
    (Lorentz.CoVector.basis.tensorProduct Lorentz.Vector.basis).repr
      (A.toFieldStrength x) μ(, ν)
```

lemma Electromagnetism.ElectromagneticPotential.toFieldStrength_eval_apply [source]

The evaluated components of the field strength tensor in terms of derivatives of the electromagnetic potential.

```
lemma toFieldStrength_eval_apply {d} (A : ElectromagneticPotential d)
    (x : SpaceTime d) μ( ν : Fin 1 ⊕ Fin d) :
    toField {A.toFieldStrength x | μ[] ν⊤[]} =
    ∑ κ, η( μ κ * ∂_ κ A x ν - η ν κ * ∂_ κ A x μ)
```

lemma Electromagnetism.ElectromagneticPotential.toFieldStrength_eval_apply_eq_single [source]

The evaluated components of the field strength tensor after using diagonal form of the Minkowski metric.

```
lemma toFieldStrength_eval_apply_eq_single {d} (A : ElectromagneticPotential d)
    (x : SpaceTime d) μ( ν : Fin 1 ⊕ Fin d) :
```

```
    toField {A.toFieldStrength x | μ[] ν⊤[]} =
    η μ μ * ∂_ μ A x ν - η ν ν * ∂_ ν A x μ
```

lemma Electromagnetism.ElectromagneticPotential.toFieldStrength_eval_eq_fieldStrengthMatrix [source]

Index evaluation of the field strength tensor agrees with the corresponding component of the field strength matrix.

```
lemma toFieldStrength_eval_eq_fieldStrengthMatrix {d} (A : ElectromagneticPotential d)
    (x : SpaceTime d) μ( ν : Fin 1 ⊕ Fin d) :
    toField {A.toFieldStrength x | μ[] ν⊤[]} = A.fieldStrengthMatrix x μ(, ν)
```

2.8 Physlib.Electromagnetism.Kinematics.GaugeTransformation

Physlib/Electromagnetism/Kinematics/GaugeTransformation.lean on GitHub.

def Electromagnetism.ElectromagneticPotential.ofGradient [source]

*The pure-gauge electromagnetic potential ‘A^μ = ∂^μ χ = η^{μν} ∂_ν χ‘ built from a gauge function ‘χ‘. The metric-contracted form ‘∑ κ, η μ κ * ∂_ κ χ‘ is the definition; the diagonal form (which equals ‘η μ μ * ∂_ μ χ‘ since ‘η‘ is diagonal) is derived in ‘ofGradient_apply‘.*

```
noncomputable def ofGradient {d} χ( : SpaceTime d → ℝ) : ElectromagneticPotential d where
```

lemma Electromagnetism.ElectromagneticPotential.ofGradient_apply_sum [source]

Unfolding of the summed definition of ‘ofGradient‘.

```
lemma ofGradient_apply_sum {d} χ( : SpaceTime d → ℝ) (x : SpaceTime d) μ( : Fin 1 ⊕ Fin d) :
    ofGradient χ x μ = ∑ κ, η μ κ * ∂_ κ χ x
```

lemma Electromagnetism.ElectromagneticPotential.ofGradient_apply [source]

Evaluation of ‘ofGradient‘ in the diagonal form; the off-diagonal entries of ‘η‘ vanish so only the ‘κ = μ‘ term survives.

```
lemma ofGradient_apply {d} χ( : SpaceTime d → ℝ) (x : SpaceTime d) μ( : Fin 1 ⊕ Fin d) :
    ofGradient χ x μ = η μ μ * ∂_ μ χ x
```

lemma Electromagnetism.ElectromagneticPotential.ofGradient_zero [source]

The pure-gauge potential built from the zero gauge function has all components zero.

```
lemma ofGradient_zero {d} (x : SpaceTime d) μ( : Fin 1 ⊕ Fin d) :
    ofGradient (0 : SpaceTime d → ℝ) x μ = 0
```

lemma Electromagnetism.ElectromagneticPotential.ofGradient_add [source]

‘ofGradient‘ is additive in the gauge function (when both summands are differentiable).

```
lemma ofGradient_add {d} χ₁{ χ₂ : SpaceTime d → ℝ}
    (χ₁h : Differentiable ℝ χ₁) (χ₂h : Differentiable ℝ χ₂) :
    ofGradient χ₁( + χ₂) = ofGradient χ₁ + ofGradient χ₂
```

lemma Electromagnetism.ElectromagneticPotential.differentiable_ofGradient [source]

The pure-gauge potential is differentiable when ‘χ‘ is ‘C^2‘.

```
lemma differentiable_ofGradient {d} χ{ : SpaceTime d → ℝ} (χh : ContDiff ℝ 2 χ) :
    Differentiable ℝ (ofGradient χ)
```

lemma Electromagnetism.ElectromagneticPotential.contDiff_ofGradient [source]

The pure-gauge potential is ‘C^n‘ when ‘χ‘ is ‘C^{n+1}‘.

```
lemma contDiff_ofGradient {n} {d} χ{ : SpaceTime d → ℝ} (χh : ContDiff ℝ (n + 1) χ) :
    ContDiff ℝ n (ofGradient χ)
```

lemma Electromagnetism.ElectromagneticPotential.toFieldStrength_ofGradient [source]

A pure-gauge potential has vanishing field strength.

```
lemma toFieldStrength_ofGradient {d} χ{ : SpaceTime d → ℝ} (χh : ContDiff ℝ 2 χ)
    (x : SpaceTime d) : (ofGradient χ).toFieldStrength x = 0
```

lemma Electromagnetism.ElectromagneticPotential.ofGradient_equivariant [source]

‘ofGradient‘ intertwines the Lorentz action on potentials with composition by Λ^{-1} on the gauge function: ‘$\Lambda \bullet ofGradient\ \chi = ofGradient\ (\chi \circ (\Lambda^{-1} \bullet \cdot))$‘.

```
lemma ofGradient_equivariant {d} χ( : SpaceTime d → ℝ) (χh : Differentiable ℝ χ)
    Λ( : LorentzGroup d) :
    Λ • ofGradient χ = ofGradient χ( ∘ Λ⁻(¹ • ·))
```

def Electromagnetism.ElectromagneticPotential.gaugeTransform [source]

The gauge transformation ‘$A\hat{}\mu \mapsto A\hat{}\mu + \partial\hat{}\mu\ \chi$‘ of an electromagnetic potential.

```
noncomputable def gaugeTransform {d} χ( : SpaceTime d → ℝ) (A : ElectromagneticPotential d) :
    ElectromagneticPotential d
```

lemma Electromagnetism.ElectromagneticPotential.gaugeTransform_apply [source]

Evaluation of ‘gaugeTransform‘.

```
lemma gaugeTransform_apply {d} χ( : SpaceTime d → ℝ) (A : ElectromagneticPotential d)
    (x : SpaceTime d) : gaugeTransform χ A x = A x + ofGradient χ x
```

lemma Electromagnetism.ElectromagneticPotential.toFieldStrength_gaugeTransform [source]

The field strength tensor is invariant under gauge transformations.

```
lemma toFieldStrength_gaugeTransform {d} (A : ElectromagneticPotential d)
    χ( : SpaceTime d → ℝ) (hA : Differentiable ℝ A) (χh : ContDiff ℝ 2 χ) (x : SpaceTime d) :
    (gaugeTransform χ A).toFieldStrength x = A.toFieldStrength x
```

lemma Electromagnetism.ElectromagneticPotential.fieldStrengthMatrix_gaugeTransform [source]

The field strength matrix is invariant under gauge transformations.

```
lemma fieldStrengthMatrix_gaugeTransform {d} (A : ElectromagneticPotential d)
    χ( : SpaceTime d → ℝ) (hA : Differentiable ℝ A) (χh : ContDiff ℝ 2 χ) (x : SpaceTime d) :
    (gaugeTransform χ A).fieldStrengthMatrix x = A.fieldStrengthMatrix x
```

lemma Electromagnetism.ElectromagneticPotential.gaugeTransform_zero [source]

Shifting by the zero gauge function is the identity.

```
lemma gaugeTransform_zero {d} (A : ElectromagneticPotential d) :
    gaugeTransform (0 : SpaceTime d → ℝ) A = A
```

lemma Electromagnetism.ElectromagneticPotential.gaugeTransform_gaugeTransform [source]

Two successive gauge shifts compose: shifting by χ_2 then χ_1 equals shifting by $\chi_1 + \chi_2$. This upgrades one-step F-invariance to invariance along any finite chain of gauge shifts.

```
lemma gaugeTransform_gaugeTransform {d} (A : ElectromagneticPotential d)
    χ₁( χ₂ : SpaceTime d → ℝ) (χ₁h : Differentiable ℝ χ₁) (χ₂h : Differentiable ℝ χ₂) :
    gaugeTransform χ₁ (gaugeTransform χ₂ A) = gaugeTransform χ₁( + χ₂) A
```

lemma Electromagnetism.ElectromagneticPotential.gaugeTransform_equivariant [source]

Gauge transformations commute with Lorentz transformations: applying Λ and then performing a gauge transformation by ‘χ‘ equals performing a gauge transformation by ‘$\chi \circ (\Lambda^{-1} \bullet \cdot)$‘ and then applying Λ.

```
lemma gaugeTransform_equivariant {d} (A : ElectromagneticPotential d)
    χ( : SpaceTime d → ℝ) (χh : Differentiable ℝ χ) Λ( : LorentzGroup d) :
    Λ • gaugeTransform χ A = gaugeTransform χ( ∘ Λ⁻(¹ • ·)) Λ( • A)
```

lemma Electromagnetism.ElectromagneticPotential.fieldStrengthMatrix_bareGradient_inl_inr [source]

The ‘(inl 0, inr i)‘ component of the field strength matrix of the bare-gradient potential ‘B^μ := ∂_μ χ‘ for ‘χ(x) = $x^0 \cdot x^i$‘ equals ‘2‘. This witnesses that the bare covariant gradient does not produce a gauge-invariant field strength, so the raised-index contraction ‘η^{μν} ∂_ν χ‘ in ‘ofGradient‘ is necessary (see the module overview).

```
lemma fieldStrengthMatrix_bareGradient_inl_inr {d : ℕ} (i : Fin d)
    (x : SpaceTime d) :
    let χ : SpaceTime d → ℝ
```

lemma Electromagnetism.ElectromagneticPotential.toFieldStrength_bareGradient_ne_zero [source]

The field strength of the bare-gradient potential ‘B^μ := ∂_μ χ‘ for ‘χ(x) = $x^0 \cdot x^j$‘ is nonzero (follows from ‘fieldStrengthMatrix_bareGradient_inl_inr‘).

```
lemma toFieldStrength_bareGradient_ne_zero {d : ℕ} (i : Fin d)
    (x : SpaceTime d) :
    let χ : SpaceTime d → ℝ
```

2.9 Physlib.Electromagnetism.Kinematics.MagneticField

Physlib/Electromagnetism/Kinematics/MagneticField.lean on GitHub.

lemma Electromagnetism.ElectromagneticPotential.magneticField_coord_eq_fieldStrengthMatrix [source]

```
lemma magneticField_coord_eq_fieldStrengthMatrix {i : Fin 3} {c : SpeedOfLight}
    (A : ElectromagneticPotential) (t : Time)
    (x : Space) (hA : Differentiable ℝ A) :
    A.magneticField c t x i =
    - A.fieldStrengthMatrix ((toTimeAndSpace c).symm (t, x)) (Sum.inr (i+1), Sum.inr (i+2))
```

2.10 Physlib.Mathematics.Calculus.Wirtinger.Basic

Physlib/Mathematics/Calculus/Wirtinger/Basic.lean on GitHub.

def Physlib.Wirtinger.weightedDirDeriv [source]

The base-point field ‘p ↦ (1/2)(d_{b_1} f + c·d_{b_2} f)‘, a weighted combination of the real Fréchet derivative of ‘f‘ along two directions ‘b_1‘, ‘b_2‘. The directional Wirtinger operators are its two specializations: ‘dWirtingerDir f v‘ at ‘c = -i‘, ‘(b_1, b_2) = (v, i·v)‘, and ‘dWirtingerAntiDir f v‘ at ‘c = i‘. Keeping ‘b_1‘, ‘b_2‘ free lets §G differentiate it a second time once and reuse the bridge for both operators.

```
def weightedDirDeriv (f : V → ℂ) (c : ℂ) (₁b ₂b : V) : V → ℂ
```

def Physlib.Wirtinger.dWirtingerDir [source]

The holomorphic directional Wirtinger derivative ‘∂_v f = (1/2)(d_v f − i·d_{i·v} f)‘ of ‘f : V → ℂ‘ along the direction vector ‘v : V‘, the ‘weightedDirDeriv‘ at ‘c = -i‘.

```
def dWirtingerDir (f : V → ℂ) (v u : V) : ℂ
```

def Physlib.Wirtinger.dWirtingerAntiDir [source]

The anti-holomorphic directional Wirtinger derivative ‘∂̄_v f = (1/2)(d_v f + i·d_{i·v} f)‘ of ‘f : V → ℂ‘ along the direction vector ‘v : V‘, the ‘weightedDirDeriv‘ at ‘c = i‘.

```
def dWirtingerAntiDir (f : V → ℂ) (v u : V) : ℂ
```

lemma Physlib.Wirtinger.dWirtingerDir_apply [source]

Definitional unfolding of ‘dWirtingerDir‘ to the explicit Wirtinger combination, used to expand the outer operator of a composition without touching the inner one.

```
lemma dWirtingerDir_apply (g : V → ℂ) (v u : V) :
    dWirtingerDir g v u
      = (1 / 2 : ℂ) * (fderiv ℝ g u v - Complex.I * fderiv ℝ g u (Complex.I • v))
```

lemma Physlib.Wirtinger.dWirtingerAntiDir_apply [source]

Definitional unfolding of ‘dWirtingerAntiDir‘ to the explicit Wirtinger combination.

```
lemma dWirtingerAntiDir_apply (g : V → ℂ) (v u : V) :
    dWirtingerAntiDir g v u
      = (1 / 2 : ℂ) * (fderiv ℝ g u v + Complex.I * fderiv ℝ g u (Complex.I • v))
```

lemma Physlib.Wirtinger.dWirtingerDir_add [source]

Additivity of ‘dWirtingerDir‘, ‘∂_v(g + h) = ∂_v g + ∂_v h‘.

```
lemma dWirtingerDir_add {g h : V → ℂ} (hg : DifferentiableAt ℝ g u)
    (hh : DifferentiableAt ℝ h u) (v : V) :
    dWirtingerDir (g + h) v u = dWirtingerDir g v u + dWirtingerDir h v u
```

lemma Physlib.Wirtinger.dWirtingerAntiDir_add [source]

Additivity of ‘dWirtingerAntiDir‘, ‘∂_v(g + h) = ∂_v g + ∂_v h‘.

```
lemma dWirtingerAntiDir_add {g h : V → ℂ} (hg : DifferentiableAt ℝ g u)
    (hh : DifferentiableAt ℝ h u) (v : V) :
    dWirtingerAntiDir (g + h) v u = dWirtingerAntiDir g v u + dWirtingerAntiDir h v u
```

lemma Physlib.Wirtinger.dWirtingerDir_smul [source]

Compatibility of ‘dWirtingerDir‘ with complex scalar multiplication, ‘∂_v(c·g) = c·∂_v g‘.

```
lemma dWirtingerDir_smul (c : ℂ) {g : V → ℂ} (hg : DifferentiableAt ℝ g u) (v : V) :
    dWirtingerDir (c • g) v u = c • dWirtingerDir g v u
```

lemma Physlib.Wirtinger.dWirtingerAntiDir_smul [source]

Compatibility of ‘dWirtingerAntiDir‘ with complex scalar multiplication, ‘∂_v(c·g) = c·∂_v g‘.

```
lemma dWirtingerAntiDir_smul (c : ℂ) {g : V → ℂ} (hg : DifferentiableAt ℝ g u) (v : V) :
    dWirtingerAntiDir (c • g) v u = c • dWirtingerAntiDir g v u
```

lemma Physlib.Wirtinger.fderiv_mul_apply [source]

The real Fréchet derivative of a product, evaluated at a tangent ‘v‘.

```
private lemma fderiv_mul_apply {g h : V → ℂ} (hg : DifferentiableAt ℝ g u)
    (hh : DifferentiableAt ℝ h u) (v : V) :
    fderiv ℝ (g * h) u v = g u * fderiv ℝ h u v + h u * fderiv ℝ g u v
```

lemma Physlib.Wirtinger.dWirtingerDir_mul [source]

The Wirtinger Leibniz rule for ‘dWirtingerDir‘, ‘∂_v(g·h) = ∂_v g·h + g·∂_v h‘.

```
lemma dWirtingerDir_mul {g h : V → ℂ} (hg : DifferentiableAt ℝ g u)
    (hh : DifferentiableAt ℝ h u) (v : V) :
    dWirtingerDir (g * h) v u = dWirtingerDir g v u * h u + g u * dWirtingerDir h v u
```

lemma Physlib.Wirtinger.dWirtingerAntiDir_mul [source]

The Wirtinger Leibniz rule for ‘dWirtingerAntiDir‘, ‘∂_v(g·h) = ∂_v g·h + g·∂_v h‘.

```
lemma dWirtingerAntiDir_mul {g h : V → ℂ} (hg : DifferentiableAt ℝ g u)
    (hh : DifferentiableAt ℝ h u) (v : V) :
    dWirtingerAntiDir (g * h) v u =
      dWirtingerAntiDir g v u * h u + g u * dWirtingerAntiDir h v u
```

lemma Physlib.Wirtinger.dWirtingerDir_fun_sum [source]

Finite-sum rule for ‘dWirtingerDir‘, ‘∂_v(∑_a F_a) = ∑_a ∂_v F_a‘.

```
lemma dWirtingerDir_fun_sum α{ : Type*} {s : Finset α} {F : α → V → ℂ}
    (hF : ∀ a ∈ s, DifferentiableAt ℝ (F a) u) (v : V) :
    dWirtingerDir (fun p => ∑ a ∈ s, F a p) v u = ∑ a ∈ s, dWirtingerDir (F a) v u
```

lemma Physlib.Wirtinger.dWirtingerAntiDir_fun_sum [source]

Finite-sum rule for ‘dWirtingerAntiDir‘, ‘∂_v(∑_a F_a) = ∑_a ∂_v F_a‘.

```
lemma dWirtingerAntiDir_fun_sum α{ : Type*} {s : Finset α} {F : α → V → ℂ}
    (hF : ∀ a ∈ s, DifferentiableAt ℝ (F a) u) (v : V) :
    dWirtingerAntiDir (fun p => ∑ a ∈ s, F a p) v u = ∑ a ∈ s, dWirtingerAntiDir (F a) v u
```

lemma Physlib.Wirtinger.fderiv_star_eq [source]

*Differentiation commutes with conjugation: the real Fréchet derivative of the pointwise conjugate ‘p ↦ star (f p)‘ is ‘conjCLE‘ (conjugation on ‘ℂ‘) composed with ‘fderiv ℝ f u‘; in physicists' notation, ‘d f̄ = conj(d f)‘. Conjugation is ‘ℝ‘-linear, so it slides through the real derivative unchanged, whereas it does *not* commute with the holomorphic Wirtinger derivative ‘∂_v‘. The ‘star‘ conjugates the *output* ‘f p‘, so this is not a derivative in a conjugate variable.*

This is the analytic core of the conjugation lemmas below (‘dWirtingerDir_star_comp‘ and its dual): distributed over the Wirtinger split of ‘fderiv ℝ f u‘ (‘realLinear_apply_eq_wirtinger‘, §D), the outer ‘conjCLE‘ conjugates the two coefficients and swaps the holomorphic and anti-holomorphic parts.

```
lemma fderiv_star_eq {E : Type*} [NormedAddCommGroup E] [NormedSpace ℝ E]
    {f : E → ℂ} {u : E} (hf : DifferentiableAt ℝ f u) :
    fderiv ℝ (fun p : E => star (f p)) u =
      Complex.conjCLE.toContinuousLinearMap.comp (fderiv ℝ f u)
```

lemma Physlib.Wirtinger.dWirtingerDir_star_comp [source]

Conjugating the function swaps the operators up to an outer conjugation: ‘∂_v f = conj (∂_v f)‘.

```
lemma dWirtingerDir_star_comp (hf : DifferentiableAt ℝ f u) (v : V) :
    dWirtingerDir (fun p => star (f p)) v u = star (dWirtingerAntiDir f v u)
```

lemma Physlib.Wirtinger.dWirtingerAntiDir_star_comp [source]

Conjugating the function swaps the operators up to an outer conjugation: ‘∂_v f = conj (∂_v f)‘. Dual of ‘dWirtingerDir_star_comp‘.

```
lemma dWirtingerAntiDir_star_comp (hf : DifferentiableAt ℝ f u) (v : V) :
    dWirtingerAntiDir (fun p => star (f p)) v u = star (dWirtingerDir f v u)
```

lemma Physlib.Wirtinger.realLinear_apply_eq_wirtinger [source]

*Split a real-linear map ‘ℂ → ℂ‘ into its Wirtinger components. Any real-linear ‘L : ℂ →L[ℝ] ℂ‘ splits into a holomorphic and an anti-holomorphic part with the Wirtinger coefficients ‘a = ½(L 1 - i * L i)‘, ‘b = ½(L 1 + i * L i)‘ as weights:*

*‘L w = a * w + b * star w‘.*

This is purely algebraic: ‘L‘ is an arbitrary real-linear map, no derivative involved. Its use is the Wirtinger chain rule (‘dWirtingerDir_comp‘ below), where the weights of the outer differential ‘L = fderiv ℝ g (f u)‘ are the coefficients ‘∂g/∂f‘, ‘∂g/∂f‘.

```
lemma realLinear_apply_eq_wirtinger (L : ℂ →L[ℝ] ℂ) (w : ℂ) :
    L w =
      ((1 / 2 : ℂ) * (L 1 - Complex.I * L Complex.I)) * w
        + ((1 / 2 : ℂ) * (L 1 + Complex.I * L Complex.I)) * star w
```

lemma Physlib.Wirtinger.dWirtingerDir_comp [source]

The two-term Wirtinger chain rule for ‘dWirtingerDir‘, outer ‘g : ℂ → ℂ‘ and inner ‘f : V → ℂ‘:

‘∂_v(g∘f) = (∂g/∂f)·∂_v f + (∂g/∂f̄)·∂_v f̄‘.

‘realLinear_apply_eq_wirtinger‘ splits the chain rule’s outer ‘ℝ‘-linear factor into the ‘∂g/∂f‘, ‘∂g/∂f̄‘ coefficients, each multiplying its inner directional derivative ‘∂_v f‘, ‘∂_v f̄‘.

```
lemma dWirtingerDir_comp {g : ℂ → ℂ} (hg : DifferentiableAt ℝ g (f u))
    (hf : DifferentiableAt ℝ f u) (v : V) :
    dWirtingerDir (fun p => g (f p)) v u =
      dWirtingerDir g 1 (f u) * dWirtingerDir f v u
        + dWirtingerAntiDir g 1 (f u) * dWirtingerDir (fun p => star (f p)) v u
```

lemma Physlib.Wirtinger.dWirtingerAntiDir_comp [source]

The two-term Wirtinger chain rule for ‘dWirtingerAntiDir‘, the anti-holomorphic dual of ‘dWirtingerDir_comp‘:

‘∂_v(g∘f) = (∂g/∂f)·∂_v f + (∂g/∂f̄)·∂_v f̄‘.

Same outer ‘∂g/∂f‘, ‘∂g/∂f‘ coefficients, now each multiplying its anti-holomorphic inner derivative ‘∂_v f‘, ‘∂_v f‘; same proof as ‘dWirtingerDir_comp‘.

```
lemma dWirtingerAntiDir_comp {g : ℂ → ℂ} (hg : DifferentiableAt ℝ g (f u))
    (hf : DifferentiableAt ℝ f u) (v : V) :
    dWirtingerAntiDir (fun p => g (f p)) v u =
      dWirtingerDir g 1 (f u) * dWirtingerAntiDir f v u
        + dWirtingerAntiDir g 1 (f u) * dWirtingerAntiDir (fun p => star (f p)) v u
```

lemma Physlib.Wirtinger.fderiv_comp_clm_apply [source]

Chain rule for an inner continuous linear map ‘L‘. Because the derivative of a linear map is the map itself, the real Fréchet derivative of ‘g ∘ L‘ at ‘u‘, applied to ‘x‘, equals the derivative of ‘g‘ at ‘L u‘ applied to ‘L x‘.

```
private lemma fderiv_comp_clm_apply {g : V' → ℂ} {L : V →L[ℝ] V'} {u : V}
    (hg : DifferentiableAt ℝ g (L u)) (x : V) :
    fderiv ℝ (fun p => g (L p)) u x = fderiv ℝ g (L u) (L x)
```

lemma Physlib.Wirtinger.dWirtingerDir_comp_conjLinear [source]

Domain conjugation swaps the operators: precomposing with a conjugate-‘ℂ‘-linear ‘L‘ turns the holomorphic derivative of ‘g ∘ L‘ at ‘u‘ into the anti-holomorphic derivative of ‘g‘ at the mapped point ‘L u‘, in the mapped direction ‘L v‘: ‘∂_v(g ∘ L)‘ at ‘u‘ equals ‘∂_{L v} g‘ at ‘L u‘.

```
lemma dWirtingerDir_comp_conjLinear {g : V' → ℂ} {L : V →L[ℝ] V'} {u : V}
    (hL : ∀ x : V, L (Complex.I • x) = -(Complex.I • L x))
    (hg : DifferentiableAt ℝ g (L u)) (v : V) :
    dWirtingerDir (fun p => g (L p)) v u = dWirtingerAntiDir g (L v) (L u)
```

lemma Physlib.Wirtinger.dWirtingerAntiDir_comp_conjLinear [source]

Dual of ‘dWirtingerDir_comp_conjLinear‘: the anti-holomorphic derivative of ‘g ∘ L‘ at ‘u‘ is the holomorphic derivative of ‘g‘ at the mapped point ‘L u‘, in the mapped direction ‘L v‘: ‘∂_v(g ∘ L)‘ at ‘u‘ equals ‘∂_{L v} g‘ at ‘L u‘.

```
lemma dWirtingerAntiDir_comp_conjLinear {g : V' → ℂ} {L : V →L[ℝ] V'} {u : V}
    (hL : ∀ x : V, L (Complex.I • x) = -(Complex.I • L x))
    (hg : DifferentiableAt ℝ g (L u)) (v : V) :
    dWirtingerAntiDir (fun p => g (L p)) v u = dWirtingerDir g (L v) (L u)
```

lemma Physlib.Wirtinger.dWirtingerDir_eq_of_clinear [source]

Holomorphic collapse: along a direction where the real derivative is ‘ℂ‘-linear, the holomorphic derivative is the full real derivative, ‘∂_v f = d_v f‘.

```
lemma dWirtingerDir_eq_of_clinear {v : V}
    (h : fderiv ℝ f u (Complex.I • v) = Complex.I • fderiv ℝ f u v) :
    dWirtingerDir f v u = fderiv ℝ f u v
```

lemma Physlib.Wirtinger.dWirtingerAntiDir_eq_zero_of_clinear [source]

Holomorphic collapse: the anti-holomorphic derivative vanishes along a direction of ‘ℂ‘-linearity, ‘∂_v f = 0‘.

```
lemma dWirtingerAntiDir_eq_zero_of_clinear {v : V}
    (h : fderiv ℝ f u (Complex.I • v) = Complex.I • fderiv ℝ f u v) :
    dWirtingerAntiDir f v u = 0
```

lemma Physlib.Wirtinger.dWirtingerDir_eq_zero_of_antilinear [source]

Anti-holomorphic collapse: a direction of conjugate-‘ℂ‘-linearity kills the holomorphic derivative, ‘∂_v f = 0‘.

```
lemma dWirtingerDir_eq_zero_of_antilinear {v : V}
    (h : fderiv ℝ f u (Complex.I • v) = -(Complex.I • fderiv ℝ f u v)) :
    dWirtingerDir f v u = 0
```

lemma Physlib.Wirtinger.dWirtingerAntiDir_eq_of_antilinear [source]

Anti-holomorphic collapse: the anti-holomorphic derivative is the full real derivative along a direction of conjugate-‘ℂ‘-linearity, ‘∂_v f = d_v f‘.

```
lemma dWirtingerAntiDir_eq_of_antilinear {v : V}
    (h : fderiv ℝ f u (Complex.I • v) = -(Complex.I • fderiv ℝ f u v)) :
    dWirtingerAntiDir f v u = fderiv ℝ f u v
```

lemma Physlib.Wirtinger.hasFDerivAt_fderiv_apply [source]

The field ‘p ↦ d_b f‘ is the evaluation map ‘· b‘ composed with ‘fderiv ℝ f‘, so when ‘fderiv ℝ f‘ is differentiable its derivative is ‘fderiv ℝ (fderiv ℝ f) u‘ post-composed with that evaluation.

```
private lemma hasFDerivAt_fderiv_apply (hf' : DifferentiableAt ℝ (fderiv ℝ f) u)
    (b : V) :
    HasFDerivAt (fun p => fderiv ℝ f p b)
      ((ContinuousLinearMap.apply ℝ ℂ b).comp (fderiv ℝ (fderiv ℝ f) u)) u
```

lemma Physlib.Wirtinger.hasFDerivAt_weightedDirDeriv [source]

The ‘weightedDirDeriv‘ is differentiable wherever ‘fderiv ℝ f‘ is.

```
private lemma hasFDerivAt_weightedDirDeriv (hf' : DifferentiableAt ℝ (fderiv ℝ f) u)
    (c : ℂ) (₁b ₂b : V) :
    HasFDerivAt (weightedDirDeriv f c ₁b ₂b)
      ((1 / 2 : ℂ) • ((ContinuousLinearMap.apply ℝ ℂ ₁b).comp (fderiv ℝ (fderiv ℝ f) u)
        + c • (ContinuousLinearMap.apply ℝ ℂ ₂b).comp (fderiv ℝ (fderiv ℝ f) u))) u
```

lemma Physlib.Wirtinger.fderiv_weightedDirDeriv [source]

The bridge: differentiating a ‘weightedDirDeriv‘ along a third direction ‘a‘ lands on the second real Fréchet derivative ‘fderiv ℝ (fderiv ℝ f) u a b‘ in the two slots.

```
private lemma fderiv_weightedDirDeriv (hf' : DifferentiableAt ℝ (fderiv ℝ f) u)
    (c : ℂ) (₁b ₂b a : V) :
    fderiv ℝ (weightedDirDeriv f c ₁b ₂b) u a
      = (1 / 2 : ℂ) * (fderiv ℝ (fderiv ℝ f) u a ₁b
          + c * fderiv ℝ (fderiv ℝ f) u a ₂b)
```

lemma Physlib.Wirtinger.dWirtingerAntiDir_eq_weightedDirDeriv [source]

A directional derivative is a 'weightedDirDeriv': anti-holomorphic with 'c = i'.

```
private lemma dWirtingerAntiDir_eq_weightedDirDeriv (w : V) :
    (fun p => dWirtingerAntiDir f w p) = weightedDirDeriv f Complex.I w (Complex.I • w)
```

lemma Physlib.Wirtinger.dWirtingerDir_eq_weightedDirDeriv [source]

A directional derivative is a 'weightedDirDeriv': holomorphic with 'c = -i'.

```
private lemma dWirtingerDir_eq_weightedDirDeriv (v : V) :
    (fun p => dWirtingerDir f v p) = weightedDirDeriv f (-Complex.I) v (Complex.I • v)
```

lemma Physlib.Wirtinger.fderiv_dWirtingerAntiDir [source]

Differentiating the anti-holomorphic directional derivative lands on the second real Fréchet derivative in the two slots.

```
private lemma fderiv_dWirtingerAntiDir (hf' : DifferentiableAt ℝ (fderiv ℝ f) u)
    (w a : V) :
    fderiv ℝ (fun p => dWirtingerAntiDir f w p) u a
      = (1 / 2 : ℂ) * (fderiv ℝ (fderiv ℝ f) u a w
          + Complex.I * fderiv ℝ (fderiv ℝ f) u a (Complex.I • w))
```

lemma Physlib.Wirtinger.fderiv_dWirtingerDir [source]

Differentiating the holomorphic directional derivative lands on the second real Fréchet derivative in the two slots.

```
private lemma fderiv_dWirtingerDir (hf' : DifferentiableAt ℝ (fderiv ℝ f) u)
    (v a : V) :
    fderiv ℝ (fun p => dWirtingerDir f v p) u a
      = (1 / 2 : ℂ) * (fderiv ℝ (fderiv ℝ f) u a v
          - Complex.I * fderiv ℝ (fderiv ℝ f) u a (Complex.I • v))
```

lemma Physlib.Wirtinger.differentiableAt_dWirtingerDir [source]

On a 'C^2' field the holomorphic directional derivative is itself real-differentiable.

```
lemma differentiableAt_dWirtingerDir (hf2 : ContDiffAt ℝ 2 f u) (v : V) :
    DifferentiableAt ℝ (fun p => dWirtingerDir f v p) u
```

lemma Physlib.Wirtinger.differentiableAt_dWirtingerAntiDir [source]

On a 'C^2' field the anti-holomorphic directional derivative is itself real-differentiable.

```
lemma differentiableAt_dWirtingerAntiDir (hf2 : ContDiffAt ℝ 2 f u) (w : V) :
    DifferentiableAt ℝ (fun p => dWirtingerAntiDir f w p) u
```

lemma Physlib.Wirtinger.dWirtingerDir_congr_of_eventuallyEq [source]

The holomorphic directional derivative depends only on the field near the point: fields agreeing on a neighbourhood have equal derivative.

```
lemma dWirtingerDir_congr_of_eventuallyEq {₁f ₂f : V → ℂ} {u : V}
    (h : ₁f ᶠ=[nhds u] ₂f) (v : V) :
    dWirtingerDir ₁f v u = dWirtingerDir ₂f v u
```

lemma Physlib.Wirtinger.dWirtingerAntiDir_congr_of_eventuallyEq [source]

The anti-holomorphic directional derivative depends only on the field near the point; dual of ‘dWirtingerDir_congr_of_eventuallyEq‘.

```
lemma dWirtingerAntiDir_congr_of_eventuallyEq {₁f ₂f : V → ℂ} {u : V}
    (h : ₁f ᶠ=[nhds u] ₂f) (v : V) :
    dWirtingerAntiDir ₁f v u = dWirtingerAntiDir ₂f v u
```

theorem Physlib.Wirtinger.dWirtingerDir_dWirtingerAntiDir_comm [source]

***Schwarz's theorem** for the directional Wirtinger operators: on a ‘C^2‘ field ‘f‘ the holomorphic and anti-holomorphic directional derivatives commute in any two directions, ‘∂_v ∂_w f = ∂_w ∂_v f‘.*

The commutation adds no analytic input. By the §G bridge each order expands into a real-linear combination of the second real Fréchet derivative ‘fderiv ℝ (fderiv ℝ f) u‘ on the four directions ‘v‘, ‘i·v‘, ‘w‘, ‘i·w‘. The two orders give the same combination up to transposing the two slots of that second derivative, and ‘ContDiffAt.isSymmSndFDerivAt‘, the symmetry of ordinary mixed second partials, equates them.

```
theorem dWirtingerDir_dWirtingerAntiDir_comm (hf2 : ContDiffAt ℝ 2 f u) (v w : V) :
    dWirtingerDir (fun p => dWirtingerAntiDir f w p) v u
      = dWirtingerAntiDir (fun p => dWirtingerDir f v p) w u
```

2.11 Physlib.Mathematics.Calculus.Wirtinger.Coordinate

Physlib/Mathematics/Calculus/Wirtinger/Coordinate.lean on GitHub.

def Physlib.Wirtinger.dWirtingerCoord [source]

Holomorphic Wirtinger derivative along the I-th coordinate of ‘ι → ℂ‘.

```
def dWirtingerCoord (f : ι( → ℂ) → ℂ) (I : ι) : ι( → ℂ) → ℂ
```

def Physlib.Wirtinger.dWirtingerAntiCoord [source]

Anti-holomorphic Wirtinger derivative along the I-th coordinate of ‘ι → ℂ‘.

```
def dWirtingerAntiCoord (f : ι( → ℂ) → ℂ) (I : ι) : ι( → ℂ) → ℂ
```

lemma Physlib.Wirtinger.dWirtingerCoord_apply [source]

Real-Fréchet form of ‘dWirtingerCoord‘: ‘dWirtingerCoord f I u = (1/2)(∂_x − i · ∂_y) f‘, the derivatives along the slot-I real and imaginary coordinate directions. Unconditional — the directional definition makes it definitional (‘Complex.I • Pi.single I 1 = Pi.single I Complex.I‘); no differentiability hypothesis is needed.

```
lemma dWirtingerCoord_apply {f : ι( → ℂ) → ℂ}
    {u : ι( → ℂ)} (I : ι) :
    dWirtingerCoord f I u = (1 / 2 : ℂ) * (fderiv ℝ f u (Pi.single I 1)
      - Complex.I * fderiv ℝ f u (Pi.single I Complex.I))
```

lemma Physlib.Wirtinger.dWirtingerAntiCoord_apply [source]

Real-Fréchet form of ‘dWirtingerAntiCoord‘: ‘∂_I f = (1/2)(∂_x + i · ∂_y) f‘, mirror of ‘dWirtingerCoord_apply‘ with the sign flip on the imaginary-direction term. Unconditional, as for ‘dWirtingerCoord_apply‘.

```
lemma dWirtingerAntiCoord_apply {f : ι( → ℂ) → ℂ}
    {u : ι( → ℂ)} (I : ι) :
    dWirtingerAntiCoord f I u = (1 / 2 : ℂ) * (fderiv ℝ f u (Pi.single I 1)
      + Complex.I * fderiv ℝ f u (Pi.single I Complex.I))
```

lemma Physlib.Wirtinger.clinear_of_holomorphic [source]

The real derivative of a holomorphic ‘f : E → ℂ‘ is ‘ℂ‘-linear along every direction — the hypothesis the foundation collapse ‘dWirtingerDir_eq_of_clinear‘ consumes. A holomorphic ‘f‘ has ‘fderiv ℝ f u = (fderiv ℂ f u).restrictScalars ℝ‘ (‘DifferentiableAt.fderiv_restrictScalars‘), so its real derivative agrees with the ‘ℂ‘-linear complex derivative on every direction. Used at ‘E = ι → ℂ‘ for the coordinate collapse and at ‘E = ℂ‘ for the outer-‘g‘ chain rule (§D).

```
private lemma clinear_of_holomorphic {E : Type*} [NormedAddCommGroup E] [NormedSpace ℝ E]
    [NormedSpace ℂ E] [IsScalarTower ℝ ℂ E] {f : E → ℂ} {u : E}
    (hf : DifferentiableAt ℂ f u) (d : E) :
    fderiv ℝ f u (Complex.I • d) = Complex.I • fderiv ℝ f u d
```

lemma Physlib.Wirtinger.dWirtingerCoord_congr_of_eventuallyEq_apply [source]

‘dWirtingerCoord‘ is local: functions agreeing on a neighbourhood of ‘u‘ have equal holomorphic Wirtinger derivative at ‘u‘ (‘$f_1 =^f [nhds\ u]\ f_2 \Longrightarrow \partial\_I\ f_1\ u = \partial\_I\ f_2\ u$‘).

```
lemma dWirtingerCoord_congr_of_eventuallyEq_apply {₁f ₂f : ι( → ℂ) → ℂ}
    {u : ι( → ℂ)} (h : ₁f ᶠ=[nhds u] ₂f) (I : ι) :
    dWirtingerCoord ₁f I u = dWirtingerCoord ₂f I u
```

lemma Physlib.Wirtinger.dWirtingerCoord_neg_apply [source]

Pointwise negation rule for the holomorphic coordinate derivative at ‘u‘: ‘$\partial\_I\ (-f) = -\partial\_I\ f$‘. Used with ‘dWirtingerCoord_add_apply‘ to assemble the coordinate-difference rule ‘dWirtingerCoord_coordDiff‘ (§C).

```
lemma dWirtingerCoord_neg_apply {u : ι( → ℂ)} (I : ι) :
    dWirtingerCoord (fun v => -(f v)) I u = -(dWirtingerCoord f I u)
```

lemma Physlib.Wirtinger.fderiv_coordProj [source]

The real Fréchet derivative of the J-th coordinate projection ‘v ↦ v J‘. Consumed by the Kronecker coordinate-value lemmas ‘dWirtingerCoord_coordProj‘ / ‘dWirtinger-AntiCoord_coordProj‘, which feed it the slot-I real and imaginary directions.

```
private lemma fderiv_coordProj (J : ι) (u d : ι( → ℂ)) :
    fderiv ℝ (fun v : ι( → ℂ) => v J) u d = d J
```

lemma Physlib.Wirtinger.dWirtingerCoord_add_apply [source]

Pointwise additivity of the holomorphic coordinate derivative at ‘u‘: ‘∂_I (f + g) = ∂_I f + ∂_I g‘. Used to assemble the coordinate-difference rule ‘dWirtingerCoord_coordDiff‘ (§C).

```
lemma dWirtingerCoord_add_apply {u : ι( → ℂ)}
    (hf : DifferentiableAt ℝ f u) (hg : DifferentiableAt ℝ g u) (I : ι) :
    dWirtingerCoord (f + g) I u = dWirtingerCoord f I u + dWirtingerCoord g I u
```

lemma Physlib.Wirtinger.dWirtingerCoord_smul_apply [source]

Pointwise compatibility with complex scalar multiplication at ‘u‘: ‘∂_I (c • f) = c • ∂_I f‘.

```
lemma dWirtingerCoord_smul_apply {u : ι( → ℂ)}
    (c : ℂ) (hf : DifferentiableAt ℝ f u) (I : ι) :
    dWirtingerCoord (c • f) I u = c • dWirtingerCoord f I u
```

lemma Physlib.Wirtinger.dWirtingerCoord_mul_apply [source]

Pointwise Leibniz rule for the holomorphic coordinate derivative at ‘u‘: ‘∂_I (f · g) = ∂_I f · g + f · ∂_I g‘.

```
lemma dWirtingerCoord_mul_apply {u : ι( → ℂ)}
    (hf : DifferentiableAt ℝ f u) (hg : DifferentiableAt ℝ g u) (I : ι) :
    dWirtingerCoord (f * g) I u =
      dWirtingerCoord f I u * g u + f u * dWirtingerCoord g I u
```

lemma Physlib.Wirtinger.dWirtingerCoord_fun_sum_apply [source]

Pointwise finite-sum rule for holomorphic coordinate derivatives at ‘u‘: ‘∂_I (∑ a ∈ s, F a) = ∑ a ∈ s, ∂_I (F a)‘.

```
lemma dWirtingerCoord_fun_sum_apply α{ : Type*} {s : Finset α}
    {F : α → ι( → ℂ) → ℂ} {u : ι( → ℂ)}
    (hF : ∀ a ∈ s, DifferentiableAt ℝ (F a) u) (I : ι) :
    dWirtingerCoord (fun v => ∑ a ∈ s, F a v) I u =
      ∑ a ∈ s, dWirtingerCoord (F a) I u
```

lemma Physlib.Wirtinger.dWirtingerCoord_eq_complex_fderiv_apply [source]

For a holomorphic function, ‘dWirtingerCoord‘ is the complex Fréchet derivative in the corresponding coordinate direction: ‘∂_I f = fderiv ℂ f u (Pi.single I 1)‘.

```
lemma dWirtingerCoord_eq_complex_fderiv_apply {u : ι( → ℂ)}
    (hf : DifferentiableAt ℂ f u) (I : ι) :
    dWirtingerCoord f I u = fderiv ℂ f u (Pi.single I 1)
```

def Physlib.Wirtinger.conjCLM [source]

Pointwise conjugation bundled as an $\mathbb{R}$-linear CLM (conjugate-$\mathbb{C}$-linear): the I-th component is conjugation of the I-th coordinate 'Complex.conjCLE ∘ proj I'. Its underlying function is the 'star' of 'ι → ℂ' ('conjCLM_apply'). Bundling 'star' as a '→L[ℝ]' is what lets the anti-holomorphic lemmas below feed it to the foundation 'dWirtingerDir_comp_conjLinear', which requires its domain map as a continuous linear map.

```
private def conjCLM : ι( → ℂ) →LR[] ι( → ℂ)
```

lemma Physlib.Wirtinger.conjCLM_smul_I [source]

'conjCLM' is conjugate-$\mathbb{C}$-linear: 'conj (i·d) = -(i · conj d)'. The 'hL' hypothesis the foundation 'dWirtingerDir_comp_conjLinear' / 'dWirtingerAntiDir_comp_conjLinear' consume.

```
private lemma conjCLM_smul_I (d : ι → ℂ) :
    conjCLM (Complex.I • d) = -(Complex.I • conjCLM ι( := ι) d)
```

lemma Physlib.Wirtinger.dWirtingerCoord_eq_zero_of_antiHolomorphic_apply [source]

*An *anti-holomorphic* function 'v ↦ g (star v)' (a holomorphic 'g' precomposed with conjugation) depends on the coordinates only through their conjugates 'z̄'. Its holomorphic derivative therefore vanishes, '∂_I (g ∘ star) = 0', since '∂_I' differentiates w.r.t. 'z' and treats 'z̄' as constant. Dual to '∂_I f = 0' for holomorphic 'f'.*

```
lemma dWirtingerCoord_eq_zero_of_antiHolomorphic_apply {u : ι( → ℂ)}
    (hg : DifferentiableAt ℂ g (star u)) (I : ι) :
    dWirtingerCoord (fun v : ι( → ℂ) => g (star v)) I u = 0
```

lemma Physlib.Wirtinger.dWirtingerAntiCoord_congr_of_eventuallyEq_apply [source]

'dWirtingerAntiCoord' is local ('$f_1 =^f [nhds\ u]\ f_2 \Longrightarrow ∂\_I\ f_1\ u = ∂\_I\ f_2\ u$').

```
lemma dWirtingerAntiCoord_congr_of_eventuallyEq_apply {₁f ₂f : ι( → ℂ) → ℂ}
    {u : ι( → ℂ)} (h : ₁f ᶠ=[nhds u] ₂f) (I : ι) :
    dWirtingerAntiCoord ₁f I u = dWirtingerAntiCoord ₂f I u
```

lemma Physlib.Wirtinger.dWirtingerAntiCoord_neg_apply [source]

Pointwise negation rule for the anti-holomorphic coordinate derivative at 'u': '∂_I (−f) = −∂_I f'. Used with 'dWirtingerAntiCoord_add_apply' to assemble the coordinate-difference rule 'dWirtingerAntiCoord_coordDiff' (§C).

```
lemma dWirtingerAntiCoord_neg_apply {u : ι( → ℂ)} (I : ι) :
    dWirtingerAntiCoord (fun v => -(f v)) I u = -(dWirtingerAntiCoord f I u)
```

lemma Physlib.Wirtinger.dWirtingerCoord_conjCoord [source]

'∂_I z̄^J = 0'. The conjugate of 'dWirtingerAntiCoord_coordProj' ('z̄^J = star z^J'), read off through 'dWirtingerDir_star_comp' rather than recomputed. Used to assemble the coordinate-difference rule 'dWirtingerCoord_coordDiff' (§C).

```
lemma dWirtingerCoord_conjCoord (I J : ι) :
    dWirtingerCoord (fun u : ι( → ℂ) => star (u J)) I = 0
```

lemma Physlib.Wirtinger.dWirtingerAntiCoord_conjCoord [source]

‘∂_I z̄^J = δ_IJ‘. The conjugate of ‘dWirtingerCoord_coordProj‘, read off through ‘dWirtingerAntiDir_star_comp‘ rather than recomputed. Used to assemble the coordinate-difference rule ‘dWirtingerAntiCoord_coordDiff‘ (§C).

```
lemma dWirtingerAntiCoord_conjCoord (I J : ι) :
    dWirtingerAntiCoord (fun u : ι( → ℂ) => star (u J)) I =
      fun _ => if I = J then 1 else 0
```

lemma Physlib.Wirtinger.dWirtingerAntiCoord_add_apply [source]

Pointwise additivity of the anti-holomorphic coordinate derivative at ‘u‘: ‘∂_I (f + g) = ∂_I f + ∂_I g‘. Used to assemble the coordinate-difference rule ‘dWirtingerAntiCoord_coordDiff‘ (§C).

```
lemma dWirtingerAntiCoord_add_apply {u : ι( → ℂ)}
    (hf : DifferentiableAt ℝ f u) (hg : DifferentiableAt ℝ g u) (I : ι) :
    dWirtingerAntiCoord (f + g) I u = dWirtingerAntiCoord f I u + dWirtingerAntiCoord g I u
```

lemma Physlib.Wirtinger.dWirtingerAntiCoord_smul_apply [source]

Pointwise compatibility with complex scalar multiplication at ‘u‘: ‘∂_I (c • f) = c • ∂_I f‘.

```
lemma dWirtingerAntiCoord_smul_apply {u : ι( → ℂ)}
    (c : ℂ) (hf : DifferentiableAt ℝ f u) (I : ι) :
    dWirtingerAntiCoord (c • f) I u = c • dWirtingerAntiCoord f I u
```

lemma Physlib.Wirtinger.dWirtingerAntiCoord_mul_apply [source]

Pointwise Leibniz rule for the anti-holomorphic coordinate derivative at ‘u‘: ‘∂_I (f · g) = ∂_I f · g + f · ∂_I g‘.

```
lemma dWirtingerAntiCoord_mul_apply {u : ι( → ℂ)}
    (hf : DifferentiableAt ℝ f u) (hg : DifferentiableAt ℝ g u) (I : ι) :
    dWirtingerAntiCoord (f * g) I u =
      dWirtingerAntiCoord f I u * g u + f u * dWirtingerAntiCoord g I u
```

lemma Physlib.Wirtinger.dWirtingerAntiCoord_fun_sum_apply [source]

Pointwise finite-sum rule for anti-holomorphic coordinate derivatives at ‘u‘: ‘∂_I (∑ a ∈ s, F a) = ∑ a ∈ s, ∂_I (F a)‘.

```
lemma dWirtingerAntiCoord_fun_sum_apply α{ : Type*} {s : Finset α}
    {F : α → ι( → ℂ) → ℂ} {u : ι( → ℂ)}
    (hF : ∀ a ∈ s, DifferentiableAt ℝ (F a) u) (I : ι) :
    dWirtingerAntiCoord (fun v => ∑ a ∈ s, F a v) I u =
      ∑ a ∈ s, dWirtingerAntiCoord (F a) I u
```

lemma Physlib.Wirtinger.dWirtingerAntiCoord_eq_complex_fderiv_apply [source]

For an anti-holomorphic function the anti-holomorphic Wirtinger derivative equals the complex Fréchet derivative of ‘g‘ at ‘star u‘ along the slot-I real coordinate direction, ‘∂_I (g ∘ star) = fderiv ℂ g (star u) (Pi.single I 1)‘. Dual of ‘dWirtinger-Coord_eq_complex_fderiv_apply‘.

```
lemma dWirtingerAntiCoord_eq_complex_fderiv_apply {u : ι( → ℂ)}
    (hg : DifferentiableAt ℂ g (star u)) (I : ι) :
    dWirtingerAntiCoord (fun v : ι( → ℂ) => g (star v)) I u =
      fderiv ℂ g (star u) (Pi.single I 1)
```

lemma Physlib.Wirtinger.dWirtingerAntiCoord_eq_zero_of_holomorphic_apply [source]

Holomorphic functions have zero anti-holomorphic coordinate derivative: ‘∂_I f = 0‘.

```
lemma dWirtingerAntiCoord_eq_zero_of_holomorphic_apply {u : ι( → ℂ)}
    (hf : DifferentiableAt ℂ f u) (I : ι) :
    dWirtingerAntiCoord f I u = 0
```

lemma Physlib.Wirtinger.coordDiff_eq_add_neg [source]

The coordinate difference ‘z^J − z̄^J‘ as a sum of the holomorphic coordinate and the negated conjugate coordinate — the form on which the ‘dWirtingerCoord‘ / ‘dWirtingerAntiCoord‘ additivity rules apply.

```
private lemma coordDiff_eq_add_neg (J : ι) :
    (fun v : ι( → ℂ) => v J - star (v J))
      = (fun v => v J) + (fun v => -(star (v J)))
```

lemma Physlib.Wirtinger.dWirtingerCoord_coordDiff [source]

‘∂_I (z^J − z̄^J) = δ_IJ‘, from ‘∂_I z^J = δ_IJ‘ and ‘∂_I z̄^J = 0‘.

```
lemma dWirtingerCoord_coordDiff (I J : ι) :
    dWirtingerCoord (fun v : ι( → ℂ) => v J - star (v J)) I
      = fun _ => if I = J then 1 else 0
```

lemma Physlib.Wirtinger.dWirtingerAntiCoord_coordDiff [source]

‘∂_I (z^J − z̄^J) = −δ_IJ‘, from ‘∂_I z^J = 0‘ and ‘∂_I z̄^J = δ_IJ‘.

```
lemma dWirtingerAntiCoord_coordDiff (I J : ι) :
    dWirtingerAntiCoord (fun v : ι( → ℂ) => v J - star (v J)) I
      = fun _ => if I = J then -1 else 0
```

lemma Physlib.Wirtinger.dWirtingerCoord_comp_apply [source]

The two-term coordinate chain rule for a real-differentiable outer ‘g‘, pointwise at ‘u‘: ‘∂_I (g ∘ f) = (∂g/∂f) · ∂_I f + (∂g/∂f̄) · ∂_I f̄‘, with coefficients ‘∂g/∂f = dWirtingerDir g 1 (f u)‘ and ‘∂g/∂f̄ = dWirtingerAntiDir g 1 (f u)‘. The ‘d = Pi.single I 1‘ case of the foundation ‘dWirtingerDir_comp‘. The single-term holomorphic specialization ‘dWirtingerCoord_comp_holomorphic_apply‘ is proved from this.

```
lemma dWirtingerCoord_comp_apply {u : ι( → ℂ)}
    (hg : DifferentiableAt ℝ g (f u)) (hf : DifferentiableAt ℝ f u) (I : ι) :
    dWirtingerCoord (fun v => g (f v)) I u =
      dWirtingerDir g 1 (f u) * dWirtingerCoord f I u
        + dWirtingerAntiDir g 1 (f u) *
          dWirtingerCoord (fun v : ι( → ℂ) => star (f v)) I u
```

lemma Physlib.Wirtinger.dWirtingerCoord_comp_holomorphic_apply [source]

The single-term coordinate chain rule for a holomorphic outer ‘g‘, pointwise at ‘u‘: ‘∂_I (g ∘ f) = deriv g (f u) · ∂_I f‘. From the two-term ‘dWirtingerCoord_comp_apply‘: for holomorphic ‘g‘ the anti-holomorphic coefficient ‘dWirtingerAntiDir g 1 (f u)‘ vanishes and ‘dWirtingerDir g 1 (f u)‘ collapses to ‘deriv g (f u)‘, both off the ‘ℂ‘-linearity ‘clinear_of_holomorphic‘ (§B).

```
lemma dWirtingerCoord_comp_holomorphic_apply {u : ι( → ℂ)}
    (hg : DifferentiableAt ℂ g (f u)) (hf : DifferentiableAt ℝ f u) (I : ι) :
    dWirtingerCoord (fun v => g (f v)) I u =
      deriv g (f u) * dWirtingerCoord f I u
```

lemma Physlib.Wirtinger.dWirtingerCoord_star_comp_apply [source]

Conjugating the inner field swaps the two operators: ‘∂_I f = conj (∂_I f)‘. The ‘d = Pi.single I 1‘ case of the foundation ‘dWirtingerDir_star_comp‘.

```
lemma dWirtingerCoord_star_comp_apply {u : ι( → ℂ)}
    (hf : DifferentiableAt ℝ f u) (I : ι) :
    dWirtingerCoord (fun v : ι( → ℂ) => star (f v)) I u =
      star (dWirtingerAntiCoord f I u)
```

lemma Physlib.Wirtinger.dWirtingerAntiCoord_comp_apply [source]

The two-term coordinate chain rule for a real-differentiable outer ‘g‘, anti-holomorphic version, pointwise at ‘u‘: ‘∂_I (g ∘ f) = (∂g/∂f) · ∂_I f + (∂g/∂f̄) · ∂_I f̄‘. Same outer coefficients as ‘dWirtingerCoord_comp_apply‘, now multiplying the anti-holomorphic inner derivatives. The ‘d = Pi.single I 1‘ case of the foundation ‘dWirtingerAntiDir_comp‘. The single-term holomorphic specialization ‘dWirtingerAntiCoord_comp_holomorphic_apply‘ is proved from this.

```
lemma dWirtingerAntiCoord_comp_apply {u : ι( → ℂ)}
    (hg : DifferentiableAt ℝ g (f u)) (hf : DifferentiableAt ℝ f u) (I : ι) :
    dWirtingerAntiCoord (fun v => g (f v)) I u =
      dWirtingerDir g 1 (f u) * dWirtingerAntiCoord f I u
        + dWirtingerAntiDir g 1 (f u) *
          dWirtingerAntiCoord (fun v : ι( → ℂ) => star (f v)) I u
```

lemma Physlib.Wirtinger.dWirtingerAntiCoord_comp_holomorphic_apply [source]

The single-term coordinate chain rule for a holomorphic outer ‘g‘, anti-holomorphic version, pointwise at ‘u‘: ‘∂_I (g ∘ f) = deriv g (f u) · ∂_I f‘. As in ‘dWirtingerCoord_comp_holomorphic_apply‘, the ‘∂g/∂f̄‘ channel vanishes and ‘∂g/∂f‘ collapses to ‘deriv g (f u)‘.

```
lemma dWirtingerAntiCoord_comp_holomorphic_apply {u : ι( → ℂ)}
    (hg : DifferentiableAt ℂ g (f u)) (hf : DifferentiableAt ℝ f u) (I : ι) :
    dWirtingerAntiCoord (fun v => g (f v)) I u =
      deriv g (f u) * dWirtingerAntiCoord f I u
```

lemma Physlib.Wirtinger.dWirtingerAntiCoord_star_comp_apply [source]

Conjugating the inner field swaps the two operators: ‘∂_I f̄ = conj (∂_I f)‘. Dual of ‘dWirtingerCoord_star_comp_apply‘, the ‘d = Pi.single I 1‘ case of the foundation ‘dWirtingerAntiDir_star_comp‘.

```
lemma dWirtingerAntiCoord_star_comp_apply {u : ι( → ℂ)}
    (hf : DifferentiableAt ℝ f u) (I : ι) :
    dWirtingerAntiCoord (fun v : ι( → ℂ) => star (f v)) I u =
      star (dWirtingerCoord f I u)
```

lemma Physlib.Wirtinger.differentiableAt_dWirtingerCoord [source]

On a ‘C^2‘ function the holomorphic coordinate Wirtinger derivative is itself real-differentiable (‘DifferentiableAt ℝ (∂_I f) u‘) — the ‘d = Pi.single I 1‘ case of ‘differentiableAt_dWirtingerDir‘.

```
lemma differentiableAt_dWirtingerCoord (hf2 : ContDiffAt ℝ 2 f u) (I : ι) :
    DifferentiableAt ℝ (fun v => dWirtingerCoord f I v) u
```

lemma Physlib.Wirtinger.differentiableAt_dWirtingerAntiCoord [source]

On a ‘C^2‘ function the anti-holomorphic coordinate Wirtinger derivative is itself real-differentiable (‘DifferentiableAt ℝ (∂_J f) u‘) — the ‘d = Pi.single J 1‘ case of ‘differentiableAt_dWirtingerAntiDir‘.

```
lemma differentiableAt_dWirtingerAntiCoord (hf2 : ContDiffAt ℝ 2 f u) (J : ι) :
    DifferentiableAt ℝ (fun v => dWirtingerAntiCoord f J v) u
```

theorem Physlib.Wirtinger.dWirtingerCoord_dWirtingerAntiCoord_comm [source]

***Schwarz's theorem** for the coordinate Wirtinger operators: on a ‘C^2‘ function the holomorphic and anti-holomorphic derivatives commute, ‘∂_I ∂_J f = ∂_J ∂_I f‘. The ‘d = Pi.single I 1‘, ‘e = Pi.single J 1‘ case of the general ‘dWirtingerDir_dWirtingerAntiDir_comm‘.*

```
theorem dWirtingerCoord_dWirtingerAntiCoord_comm (hf2 : ContDiffAt ℝ 2 f u) (I J : ι) :
    dWirtingerCoord (fun v => dWirtingerAntiCoord f J v) I u
      = dWirtingerAntiCoord (fun v => dWirtingerCoord f I v) J u
```

2.12 Physlib.Mathematics.ConjModule

Physlib/Mathematics/ConjModule.lean on GitHub.

def ConjModule [source]

The conjugate module of ‘M‘: the same additive group with the scalar action twisted by conjugation, ‘r • v = star r • v‘. A type synonym so the twisted action stays off ‘M‘.

```
def ConjModule (M : Type*)
```

instance ConjModule.«instance@88e8497f» [source]

```
instance : AddCommGroup (ConjModule M)
```

instance ConjModule.instModule [source]

The twisted action ‘r • v = star r • v‘, obtained by restricting scalars along the conjugation ring endomorphism ‘starRingEnd k‘.

```
instance instModule : Module k (ConjModule M)
```

def conjEquiv [source]

The canonical conjugate-linear equivalence ‘M ≃ₛₗ[starRingEnd k] ConjModule M‘, the identity on the underlying additive group.

```
def conjEquiv : M ≃ₛₗ[starRingEnd k] ConjModule M where
```

def ConjModule.involution [source]

Conjugating twice returns the original module: the ‘k‘-linear isomorphism ‘ConjModule (ConjModule M) ≃ₗ[k] M‘. It is ‘k‘-linear, not merely semilinear, because ‘starRingEnd k‘ composed with itself is the identity.

```
def involution : ConjModule (ConjModule M) ≃ₗ[k] M
```

def ConjModule.starFinsupp [source]

Coordinate-wise conjugation on ‘ι →₀ k‘, a conjugate-linear self-equivalence.

```
noncomputable def starFinsupp : ι( →₀ k) ≃ₛₗ[starRingEnd k] ι( →₀ k) where
```

def Basis.conj [source]

A basis of ‘M‘ transported to a basis of ‘ConjModule M‘: the same basis vectors, with coordinates conjugated (‘(Basis.conj b).repr v = star ∘ b.repr v‘).

```
noncomputable def _root_.Basis.conj (b : Basis ι k M) : Basis ι k (ConjModule M)
```

2.13 Physlib.Mathematics.CrossProductMatrix

Physlib/Mathematics/CrossProductMatrix.lean on GitHub.

def Matrix.crossProductMatrix [source]

*The hat map ‘[ω]ₓ‘: the skew-symmetric ‘3 × 3‘ matrix acting as the cross product with ‘ω‘, i.e. ‘[ω]ₓ *ᵥ v = ω ×₃ v‘.*

```
def crossProductMatrix ω( : Fin 3 → ℝ) : Matrix (Fin 3) (Fin 3) ℝ
```

lemma Matrix.crossProductMatrix_mulVec [source]

The hat matrix acts on a vector as the cross product.

```
@[simp]
lemma crossProductMatrix_mulVec ω( v : Fin 3 → ℝ) :
    crossProductMatrix ω ᵥ* v = ω ×₃ v
```

lemma Matrix.crossProductMatrix_transpose [source]

The hat map produces skew-symmetric matrices.

```
@[simp]
lemma crossProductMatrix_transpose ω( : Fin 3 → ℝ) :
    (crossProductMatrix ω)ᵀ = - crossProductMatrix ω
```

def Matrix.crossProductVee [source]

The vee map: reads the vector off a ‘3 × 3‘ matrix. It is a left inverse of the hat map.

```
def crossProductVee (A : Matrix (Fin 3) (Fin 3) ℝ) : Fin 3 → ℝ
```

lemma Matrix.crossProductVee_crossProductMatrix [source]

The vee map is a left inverse of the hat map.

```
@[simp]
lemma crossProductVee_crossProductMatrix ω( : Fin 3 → ℝ) :
    crossProductVee (crossProductMatrix ω) = ω
```

2.14 Physlib.Mathematics.KroneckerDelta

Physlib/Mathematics/KroneckerDelta.lean on GitHub.

def KroneckerDelta.generalizedKroneckerDelta [source]

Generalized Kronecker delta: ‘δ^{$\mu_1 \ldots \mu_n$}_{$\nu_1 \ldots \nu_n$} = det ($\delta[\mu_i, \nu_j]$)‘.

This is defined for any finite type ‘α‘ with decidable equality.

```
def generalizedKroneckerDelta α{ ι : Type} [DecidableEq α]
    [DecidableEq ι] [Fintype ι]
    μ( : ι → α) ν( : ι → α) : ℤ
```

lemma KroneckerDelta.generalizedKroneckerDelta_swap [source]

Swapping two of the upper indices of the generalized Kronecker delta negates it. This is one row transposition of the underlying determinant.

```
lemma generalizedKroneckerDelta_swap α{ ι : Type} [DecidableEq α] [DecidableEq ι] [Fintype ι]
    μ( ν : ι → α) {i j : ι} (hij : i ≠ j) :
    generalizedKroneckerDelta μ( ∘ Equiv.swap i j) ν = - generalizedKroneckerDelta μ ν
```

2.15 Physlib.Mathematics.LinearPMap

Physlib/Mathematics/LinearPMap.lean on GitHub.

lemma LinearPMap.sub_le_zero [source]

```
lemma sub_le_zero : f - f ≤ 0
```

lemma LinearPMap.neg_add_le_zero [source]

```
lemma neg_add_le_zero : -f + f ≤ 0
```

lemma LinearPMap.le_iff_neg_le_neg [source]

```
lemma le_iff_neg_le_neg : ₁g ≤ ₂g ↔ -₁g ≤ -₂g
```

lemma LinearPMap.le_neg_iff_neg_le [source]

```
lemma le_neg_iff_neg_le : ₁g ≤ -₂g ↔ -₁g ≤ ₂g
```

lemma LinearPMap.add_sub_le_cancel [source]

```
lemma add_sub_le_cancel : ₁f + (₂f - ₁f) ≤ ₂f
```

lemma LinearPMap.add_sub_le_cancel_left [source]

```
lemma add_sub_le_cancel_left : ₁f + ₂f - ₁f ≤ ₂f
```

lemma LinearPMap.add_sub_le_cancel_right [source]

```
lemma add_sub_le_cancel_right : ₁f + ₂f - ₂f ≤ ₁f
```

lemma LinearPMap.add_add_sub_le_cancel [source]

```
lemma add_add_sub_le_cancel : ₁f + ₂f + (₃f - ₂f) ≤ ₁f + ₃f
```

lemma LinearPMap.add_sub_sub_le_cancel [source]

```
lemma add_sub_sub_le_cancel : ₁f + ₂f - (₁f - ₃f) ≤ ₂f + ₃f
```

lemma LinearPMap.sub_sub_sub_le_cancel_right [source]

```
lemma sub_sub_sub_le_cancel_right : ₁f - ₂f - (₃f - ₂f) ≤ ₁f - ₃f
```

lemma LinearPMap.sub_sub_sub_le_cancel_left [source]

```
lemma sub_sub_sub_le_cancel_left : ₁f - ₂f - (₁f - ₃f) ≤ ₃f - ₂f
```

lemma LinearPMap.sub_le_of_le_add [source]

```
lemma sub_le_of_le_add (h : g ≤ ₁g + ₂g) : g - ₂g ≤ ₁g
```

lemma LinearPMap.sub_add_le_cancel [source]

```
lemma sub_add_le_cancel : ₁f - ₂f + ₂f ≤ ₁f
```

lemma LinearPMap.add_le_of_le_sub [source]

```
lemma add_le_of_le_sub (h : g ≤ ₁g - ₂g) : g + ₂g ≤ ₁g
```

lemma LinearPMap.add_left_le_of_le [source]

```
lemma add_left_le_of_le (h : ₁g ≤ ₂g) : f + ₁g ≤ f + ₂g
```

lemma LinearPMap.add_right_le_of_le [source]

```
lemma add_right_le_of_le (h : ₁g ≤ ₂g) : ₁g + f ≤ ₂g + f
```

lemma LinearPMap.sub_right_le_of_le [source]

```
lemma sub_right_le_of_le (h : ₁g ≤ ₂g) : ₁g - f ≤ ₂g - f
```

lemma LinearPMap.sub_left_le_of_le [source]

```
lemma sub_left_le_of_le (h : ₁g ≤ ₂g) : f - ₁g ≤ f - ₂g
```

lemma LinearPMap.zero_smul_le [source]

```
lemma zero_smul_le (f : E →ₗ□.[] F) : (0 : □) • f ≤ 0
```

lemma LinearPMap.zero_smul_eq [source]

```
@[simp]
lemma zero_smul_eq {f : E →ₗ□.[] F} (h : f.domain = ⊤) : (0 : □) • f = 0
```

def LinearPMap.sum [source]

A finite sum of partial linear maps.

'sum f' and '$\sum$ a, f a' are equal, but not by definition. With 'sum f' both 'domain' and 'toFun' are made explicit.

```
def sum : E →ₗ.[R] F where
```

lemma LinearPMap.sum_domain [source]

```
lemma sum_domain : (sum f).domain = ⋂ a, (f a).domain
```

lemma LinearPMap.sum_domain_le [source]

```
lemma sum_domain_le (a : α) : (sum f).domain ≤ (f a).domain
```

lemma LinearPMap.sum_apply [source]

```
@[simp]
lemma sum_apply ψ( : (sum f).domain) : sum f ψ = ∑ a, f a ⟨ψ, sum_domain_le f a ψ⟩.2
```

def LinearPMap.compRestricted [source]

'$g \circ_r f$' is the composition of 'g' with 'f' restricted to a domain consisting of exactly those '$x : f.\text{domain}$' for which '$f\, x \in g.\text{domain}$'.

```
def compRestricted : E →ₗ.[R] G
```

lemma LinearPMap.compRestricted_domain_le [source]

```
lemma compRestricted_domain_le : (g ∘ᵣ f).domain ≤ f.domain
```

lemma LinearPMap.compRestricted_domain [source]

```
lemma compRestricted_domain : (g ∘ᵣ f).domain = (g.domain.comap f.toFun).map f.domain.subtype
```

lemma LinearPMap.mem_compRestricted_domain_iff [source]

```
lemma mem_compRestricted_domain_iff {x : E} :
    x ∈ (v ∘ᵣ u).domain ↔ ∃ h : x ∈ u.domain, u ⟨x, ⟩h ∈ v.domain
```

lemma LinearPMap.mem_compRestricted_domain_iff' [source]

```
lemma mem_compRestricted_domain_iff' {x : E} :
    x ∈ (v ∘ᵣ u).domain ↔ ∃ y : u.domain, x = y ∧ ∃ y' : v.domain, u y = y'
```

lemma LinearPMap.mem_domain_of_mem_compRestricted_domain [source]

```
lemma mem_domain_of_mem_compRestricted_domain (x : (v ∘ᵣ u).domain) : u ⟨x, x⟩.2.2 ∈ v.domain
```

lemma LinearPMap.compRestricted_apply [source]

```
@[simp]
lemma compRestricted_apply (x : (v ∘ᵣ u).domain) :
    (v ∘ᵣ u) x = v ⟨u ⟨x, x⟩.2.2, mem_domain_of_mem_compRestricted_domain ⟩x
```

lemma LinearPMap.compRestricted_zero [source]

The zero map is right-absorbing.

```
@[simp]
lemma compRestricted_zero : g ∘ᵣ (0 : E →ₗ.[R] F) = 0
```

lemma LinearPMap.compRestricted_assoc [source]

```
lemma compRestricted_assoc {H : Type*} [AddCommGroup H] [Module R H]
    (₁f : G →ₗ.[R] H) (₂f : F →ₗ.[R] G) (₃f : E →ₗ.[R] F) :
    (₁f ∘ᵣ ₂f) ∘ᵣ ₃f = ₁f ∘ᵣ ₂f ∘ᵣ ₃f
```

lemma LinearPMap.compRestricted_eq_comp [source]

'compRestricted' is the same as 'comp' when the range of 'u' is contained in 'v.domain'.

```
lemma compRestricted_eq_comp (h : ∀ x : u.domain, u x ∈ v.domain) :
    v ∘ᵣ u = v.comp u h
```

lemma LinearPMap.comp_le_compRestricted [source]

'compRestricted' is maximal amongst compositions of 'v' with domain restrictions of 'u'.

```
lemma comp_le_compRestricted
    {S : Submodule R E} (h : ∀ x : (u.domRestrict S).domain, u ⟨x, x⟩.2.2 ∈ v.domain) :
    v.comp (u.domRestrict S) h ≤ v ∘ᵣ u
```

lemma LinearPMap.compRestricted_mono_left [source]

```
lemma compRestricted_mono_left {g g' : F →ₗ.[R] G} (h : g ≤ g') (f : E →ₗ.[R] F) :
    g ∘ᵣ f ≤ g' ∘ᵣ f
```

lemma LinearPMap.compRestricted_mono_right [source]

```
lemma compRestricted_mono_right (g : F →ₗ.[R] G) {f f' : E →ₗ.[R] F} (h : f ≤ f') :
    g ∘ᵣ f ≤ g ∘ᵣ f'
```

lemma LinearPMap.neg_compRestricted [source]

```
@[simp]
lemma neg_compRestricted : (-g) ∘ᵣ f = -g ∘ᵣ f
```

lemma LinearPMap.compRestricted_neg [source]

```
@[simp]
lemma compRestricted_neg : g ∘ᵣ (-f) = -g ∘ᵣ f
```

lemma LinearPMap.add_compRestricted [source]

```
lemma add_compRestricted : (₁g + ₂g) ∘ᵣ f = ₁g ∘ᵣ f + ₂g ∘ᵣ f
```

lemma LinearPMap.sub_compRestricted [source]

```
lemma sub_compRestricted : (₁g - ₂g) ∘ᵣ f = ₁g ∘ᵣ f - ₂g ∘ᵣ f
```

lemma LinearPMap.compRestricted_add_ge [source]

```
lemma compRestricted_add_ge : g ∘ᵣ ₁f + g ∘ᵣ ₂f ≤ g ∘ᵣ (₁f + ₂f)
```

lemma LinearPMap.compRestricted_sub_ge [source]

```
lemma compRestricted_sub_ge : g ∘ᵣ ₁f - g ∘ᵣ ₂f ≤ g ∘ᵣ (₁f - ₂f)
```

lemma LinearPMap.compRestricted_smul [source]

```
lemma compRestricted_smul {S : Type*} [DivisionRing S]
    [Module S E] [Module S F] [Module S G] [SMulCommClass S S F] [SMulCommClass S S G]
    {c : S} (hc : c ≠ 0) (g : F →ₗ.[S] G) (f : E →ₗ.[S] F) :
    g ∘ᵣ (c • f) = c • (g ∘ᵣ f)
```

lemma LinearPMap.smul_compRestricted [source]

```
@[simp]
lemma smul_compRestricted {M : Type*} [Monoid M] [DistribMulAction M G] [SMulCommClass R M G]
    (c : M) (g : F →ₗ.[R] G) (f : E →ₗ.[R] F) :
    (c • g) ∘ᵣ f = c • (g ∘ᵣ f)
```

instance LinearPMap.instMonoid [source]

```
instance instMonoid : Monoid (E →ₗ.[R] E) where
```

lemma LinearPMap.mul_def [source]

```
lemma mul_def (₁f ₂f : E →ₗ.[R] E) : ₁f * ₂f = ₁f ∘ᵣ ₂f
```

lemma LinearPMap.one_domain [source]

```
@[simp]
lemma one_domain : (1 : E →ₗ.[R] E).domain = ⊤
```

lemma LinearPMap.one_toFun [source]

```
@[simp]
lemma one_toFun : (1 : E →ₗ.[R] E).toFun = topEquiv.toLinearMap
```

lemma LinearPMap.one_coe [source]

```
@[simp]
lemma one_coe : (1 : E →ₗ.[R] E).toFun' = ⇑topEquiv.toLinearMap
```

lemma LinearPMap.inverse_ker [source]

```
lemma inverse_ker : f.inverse.toFun.ker = ⊥
```

lemma LinearPMap.inverse_inverse [source]

```
lemma inverse_inverse : f.inverse.inverse = f
```

lemma LinearPMap.inverse_compRestricted_eq [source]

```
lemma inverse_compRestricted_eq : f.inverse ∘ᵣ f = domRestrict 1 f.domain
```

lemma LinearPMap.compRestricted_inverse_eq [source]

```
lemma compRestricted_inverse_eq : f ∘ᵣ f.inverse = domRestrict 1 f.inverse.domain
```

2.16 Physlib.Particles.BeyondTheStandardModel.RHN.AnomalyCancellation.Basic

Physlib/Particles/BeyondTheStandardModel/RHN/AnomalyCancellation/Basic.lean on GitHub.

lemma SMνCharges.sum_one [source]

```
lemma sum_one  [AddCommMonoid M] (f : Fin (νSMSpecies 1).numberCharges → M) :
    ∑ i, f i = f ⟨0, by ⟩simp
```

2.17 Physlib.Particles.StandardModel.AnomalyCancellation.Basic

Physlib/Particles/StandardModel/AnomalyCancellation/Basic.lean on GitHub.

lemma SMCharges.sum_SMSpecies_numberCharges_one [source]

```
lemma sum_SMSpecies_numberCharges_one {M} [AddCommMonoid M]
    (f : Fin (SMSpecies 1).numberCharges → M) :
    ∑ i, f i = f ⟨0, by ⟩simp
```

lemma SMCharges.toSpecies_apply_eq [source]

```
lemma toSpecies_apply_eq (i : Fin 5) (S : (SMCharges n).Charges) :
    toSpecies i S = fun j => toSpeciesEquiv S i j
```

2.18 Physlib.Particles.StandardModel.AnomalyCancellation.NoGrav.One.LinearParameterization

Physlib/Particles/StandardModel/AnomalyCancellation/NoGrav/One/LinearParameterization.lean on GitHub.

lemma SM.SMNoGrav.One.linearParameters.toSpecies_apply_asCharges [source]

```
lemma toSpecies_apply_asCharges (S : linearParameters) (i : Fin 5) :
    toSpecies i S.asCharges = fun _ => S.asCharges i
```

2.19 Physlib.Particles.StandardModel.Basic

Physlib/Particles/StandardModel/Basic.lean on GitHub.

def StandardModel.gaugeGroup$\mathbb{Z}_6$UnitaryOfRoot [source]

The unitary complex number associated to a sixth root of unity.

```
noncomputable def ℤ₆gaugeGroupUnitaryOfRoot α( : rootsOfUnity 6 ℂ) : unitary ℂ
```

lemma StandardModel.gaugeGroup$\mathbb{Z}_6$UnitaryOfRoot_coe [source]

```
@[simp]
lemma ℤ₆gaugeGroupUnitaryOfRoot_coe α( : rootsOfUnity 6 ℂ) :
    (ℤ₆gaugeGroupUnitaryOfRoot α : ℂ) = α(( : ℂˣ) : ℂ)
```

def StandardModel.gaugeGroup$\mathbb{Z}_6$SU3OfRoot [source]

The 'SU(3)' scalar matrix associated to a sixth root of unity.

```
noncomputable def ℤ₆gaugeGroupSU3OfRoot α( : rootsOfUnity 6 ℂ) :
    specialUnitaryGroup (Fin 3) ℂ
```

def StandardModel.gaugeGroup$\mathbb{Z}_6$SU2OfRoot [source]

The 'SU(2)' scalar matrix associated to a sixth root of unity.

```
noncomputable def ℤ₆gaugeGroupSU2OfRoot α( : rootsOfUnity 6 ℂ) :
    specialUnitaryGroup (Fin 2) ℂ
```

def StandardModel.gaugeGroup$\mathbb{Z}_6$OfRoot [source]

The element of 'GaugeGroupI' associated to a sixth root of unity.

```
noncomputable def ℤ₆gaugeGroupOfRoot α( : rootsOfUnity 6 ℂ) : GaugeGroupI
```

lemma StandardModel.gaugeGroup$\mathbb{Z}_6$OfRoot_toSU3 [source]

```
@[simp]
lemma ℤ₆gaugeGroupOfRoot_toSU3 α( : rootsOfUnity 6 ℂ) :
    GaugeGroupI.toSU3 (ℤ₆gaugeGroupOfRoot α) = ℤ₆gaugeGroupSU3OfRoot α
```

lemma StandardModel.gaugeGroup$\mathbb{Z}_6$OfRoot_toSU2 [source]

```
@[simp]
lemma ℤ₆gaugeGroupOfRoot_toSU2 α( : rootsOfUnity 6 ℂ) :
    GaugeGroupI.toSU2 (ℤ₆gaugeGroupOfRoot α) = ℤ₆gaugeGroupSU2OfRoot α
```

lemma StandardModel.gaugeGroup$\mathbb{Z}_6$OfRoot_toU1 [source]

```
@[simp]
lemma ℤ₆gaugeGroupOfRoot_toU1 α( : rootsOfUnity 6 ℂ) :
    GaugeGroupI.toU1 (ℤ₆gaugeGroupOfRoot α) = ℤ₆gaugeGroupUnitaryOfRoot α
```

lemma StandardModel.gaugeGroup$\mathbb{Z}_6$OfRoot_mem_center [source]

```
lemma ℤ₆gaugeGroupOfRoot_mem_center α( : rootsOfUnity 6 ℂ) :
    ℤ₆gaugeGroupOfRoot α ∈ Subgroup.center GaugeGroupI
```

def StandardModel.gaugeGroup$\mathbb{Z}_6$Hom [source]

The homomorphism from sixth roots of unity to 'GaugeGroupI'.

```
noncomputable def ℤ₆gaugeGroupHom : rootsOfUnity 6 ℂ →* GaugeGroupI where
```

lemma StandardModel.gaugeGroup$\mathbb{Z}_6$Hom_apply [source]

```
@[simp]
lemma ℤ₆gaugeGroupHom_apply α( : rootsOfUnity 6 ℂ) :
    ℤ₆gaugeGroupHom α = ℤ₆gaugeGroupOfRoot α
```

lemma StandardModel.gaugeGroup$\mathbb{Z}_6$Hom_toSU3 [source]

```
@[simp]
lemma ℤ₆gaugeGroupHom_toSU3 α( : rootsOfUnity 6 ℂ) :
    GaugeGroupI.toSU3 (ℤ₆gaugeGroupHom α) = ℤ₆gaugeGroupSU3OfRoot α
```

lemma StandardModel.gaugeGroup$\mathbb{Z}_6$Hom_toSU2 [source]

```
@[simp]
lemma ℤ₆gaugeGroupHom_toSU2 α( : rootsOfUnity 6 ℂ) :
    GaugeGroupI.toSU2 (ℤ₆gaugeGroupHom α) = ℤ₆gaugeGroupSU2OfRoot α
```

lemma StandardModel.gaugeGroup$\mathbb{Z}_6$Hom_toU1 [source]

```
@[simp]
lemma ℤ₆gaugeGroupHom_toU1 α( : rootsOfUnity 6 ℂ) :
    GaugeGroupI.toU1 (ℤ₆gaugeGroupHom α) = ℤ₆gaugeGroupUnitaryOfRoot α
```

lemma StandardModel.gaugeGroup$\mathbb{Z}_6$OfRoot_mem [source]

```
lemma ℤ₆gaugeGroupOfRoot_mem α( : rootsOfUnity 6 ℂ) :
    ℤ₆gaugeGroupOfRoot α ∈ ℤ₆gaugeGroupSubGroup
```

lemma StandardModel.mem_gaugeGroup$\mathbb{Z}_6$SubGroup_iff [source]

```
lemma ℤ₆mem_gaugeGroupSubGroup_iff (g : GaugeGroupI) :
    g ∈ ℤ₆gaugeGroupSubGroup ↔ ∃ α : rootsOfUnity 6 ℂ, ℤ₆gaugeGroupOfRoot α = g
```

lemma StandardModel.gaugeGroup$\mathbb{Z}_6$SubGroup_le_center [source]

```
lemma ℤ₆gaugeGroupSubGroup_le_center :
    ℤ₆gaugeGroupSubGroup ≤ Subgroup.center GaugeGroupI
```

instance StandardModel.gaugeGroup$\mathbb{Z}_6$SubGroup_normal [source]

```
instance ℤ₆gaugeGroupSubGroup_normal : ℤ₆gaugeGroupSubGroup.Normal where
```

instance StandardModel.«instance@25db9a02» [source]

```
noncomputable instance : Group ℤ₆GaugeGroup
```

def StandardModel.GaugeGroup$\mathbb{Z}_6$.mk [source]

The quotient map from 'GaugeGroupI' to 'GaugeGroup$\mathbb{Z}_6$'.

```
noncomputable def mk : GaugeGroupI →* ℤ₆GaugeGroup
```

lemma StandardModel.GaugeGroup$\mathbb{Z}_6$.mk_gaugeGroup$\mathbb{Z}_6$OfRoot [source]

```
@[simp]
lemma ℤ₆mk_gaugeGroupOfRoot α( : rootsOfUnity 6 ℂ) :
    mk (ℤ₆gaugeGroupOfRoot α) = 1
```

def StandardModel.gaugeGroup$\mathbb{Z}_2$RootTo$\mathbb{Z}_6$Root [source]

The inclusion of second roots of unity into sixth roots of unity.

```
noncomputable def ℤ₂ℤ₆gaugeGroupRootToRoot : rootsOfUnity 2 ℂ →* rootsOfUnity 6 ℂ
```

def StandardModel.gaugeGroup$\mathbb{Z}_2$OfRoot [source]

The element of 'GaugeGroupI' associated to a second root of unity.

```
noncomputable def ℤ₂gaugeGroupOfRoot α( : rootsOfUnity 2 ℂ) : GaugeGroupI
```

lemma StandardModel.gaugeGroup$\mathbb{Z}_2$OfRoot_toSU3 [source]

```
@[simp]
lemma ℤ₂gaugeGroupOfRoot_toSU3 α( : rootsOfUnity 2 ℂ) :
    GaugeGroupI.toSU3 (ℤ₂gaugeGroupOfRoot α) =
      ℤ₆gaugeGroupSU3OfRoot (ℤ₂ℤ₆gaugeGroupRootToRoot α)
```

lemma StandardModel.gaugeGroup$\mathbb{Z}_2$OfRoot_toSU2 [source]

```
@[simp]
lemma ℤ₂gaugeGroupOfRoot_toSU2 α( : rootsOfUnity 2 ℂ) :
    GaugeGroupI.toSU2 (ℤ₂gaugeGroupOfRoot α) =
      ℤ₆gaugeGroupSU2OfRoot (ℤ₂ℤ₆gaugeGroupRootToRoot α)
```

lemma StandardModel.gaugeGroup$\mathbb{Z}_2$OfRoot_toU1 [source]

```
@[simp]
lemma ℤ₂gaugeGroupOfRoot_toU1 α( : rootsOfUnity 2 ℂ) :
    GaugeGroupI.toU1 (ℤ₂gaugeGroupOfRoot α) =
      ℤ₆gaugeGroupUnitaryOfRoot (ℤ₂ℤ₆gaugeGroupRootToRoot α)
```

lemma StandardModel.gaugeGroup$\mathbb{Z}_2$OfRoot_mem_center [source]

```
lemma ℤ₂gaugeGroupOfRoot_mem_center α( : rootsOfUnity 2 ℂ) :
    ℤ₂gaugeGroupOfRoot α ∈ Subgroup.center GaugeGroupI
```

def StandardModel.gaugeGroup$\mathbb{Z}_2$Hom [source]

The homomorphism from second roots of unity to 'GaugeGroupI'.

```
noncomputable def ℤ₂gaugeGroupHom : rootsOfUnity 2 ℂ →* GaugeGroupI
```

lemma StandardModel.gaugeGroup$\mathbb{Z}_2$Hom_apply [source]

```
@[simp]
lemma ℤ₂gaugeGroupHom_apply α( : rootsOfUnity 2 ℂ) :
    ℤ₂gaugeGroupHom α = ℤ₂gaugeGroupOfRoot α
```

lemma StandardModel.gaugeGroup$\mathbb{Z}_2$Hom_toSU3 [source]

```
@[simp]
lemma ℤ₂gaugeGroupHom_toSU3 α( : rootsOfUnity 2 ℂ) :
    GaugeGroupI.toSU3 (ℤ₂gaugeGroupHom α) =
      ℤ₆gaugeGroupSU3OfRoot (ℤ₂ℤ₆gaugeGroupRootToRoot α)
```

lemma StandardModel.gaugeGroup$\mathbb{Z}_2$Hom_toSU2 [source]

```
@[simp]
lemma ℤ₂gaugeGroupHom_toSU2 α( : rootsOfUnity 2 ℂ) :
    GaugeGroupI.toSU2 (ℤ₂gaugeGroupHom α) =
      ℤ₆gaugeGroupSU2OfRoot (ℤ₂ℤ₆gaugeGroupRootToRoot α)
```

lemma StandardModel.gaugeGroup$\mathbb{Z}_2$Hom_toU1 [source]

```
@[simp]
lemma ℤ₂gaugeGroupHom_toU1 α( : rootsOfUnity 2 ℂ) :
    GaugeGroupI.toU1 (ℤ₂gaugeGroupHom α) =
      ℤ₆gaugeGroupUnitaryOfRoot (ℤ₂ℤ₆gaugeGroupRootToRoot α)
```

lemma StandardModel.gaugeGroup$\mathbb{Z}_2$OfRoot_mem [source]

```
lemma ℤ₂gaugeGroupOfRoot_mem α( : rootsOfUnity 2 ℂ) :
    ℤ₂gaugeGroupOfRoot α ∈ ℤ₂gaugeGroupSubGroup
```

lemma StandardModel.mem_gaugeGroup$\mathbb{Z}_2$SubGroup_iff [source]

```
lemma ℤ₂mem_gaugeGroupSubGroup_iff (g : GaugeGroupI) :
    g ∈ ℤ₂gaugeGroupSubGroup ↔ ∃ α : rootsOfUnity 2 ℂ, ℤ₂gaugeGroupOfRoot α = g
```

lemma StandardModel.gaugeGroup$\mathbb{Z}_2$SubGroup_le_gaugeGroup$\mathbb{Z}_6$SubGroup [source]

```
lemma ℤ₂ℤ₆gaugeGroupSubGroup_le_gaugeGroupSubGroup :
    ℤ₂gaugeGroupSubGroup ≤ ℤ₆gaugeGroupSubGroup
```

lemma StandardModel.gaugeGroup$\mathbb{Z}_2$SubGroup_le_center [source]

```
lemma ℤ₂gaugeGroupSubGroup_le_center :
    ℤ₂gaugeGroupSubGroup ≤ Subgroup.center GaugeGroupI
```

instance StandardModel.gaugeGroup$\mathbb{Z}_2$SubGroup_normal [source]

```
instance ℤ₂gaugeGroupSubGroup_normal : ℤ₂gaugeGroupSubGroup.Normal where
```

instance StandardModel.«instance@6ca2807b» [source]

```
noncomputable instance : Group ℤ₂GaugeGroup
```

def StandardModel.GaugeGroup$\mathbb{Z}_2$.mk [source]

The quotient map from 'GaugeGroupI' to 'GaugeGroup$\mathbb{Z}_2$'.

```
noncomputable def mk : GaugeGroupI →* ℤ₂GaugeGroup
```

lemma StandardModel.GaugeGroup$\mathbb{Z}_2$.mk_gaugeGroup$\mathbb{Z}_2$OfRoot [source]

```
@[simp]
lemma ℤ₂mk_gaugeGroupOfRoot α( : rootsOfUnity 2 ℂ) :
    mk (ℤ₂gaugeGroupOfRoot α) = 1
```

def StandardModel.gaugeGroup$\mathbb{Z}_3$RootTo$\mathbb{Z}_6$Root [source]

The inclusion of third roots of unity into sixth roots of unity.

```
noncomputable def ℤ₃ℤ₆gaugeGroupRootToRoot : rootsOfUnity 3 ℂ →* rootsOfUnity 6 ℂ
```

def StandardModel.gaugeGroup$\mathbb{Z}_3$OfRoot [source]

The element of 'GaugeGroupI' associated to a third root of unity.

```
noncomputable def ℤ₃gaugeGroupOfRoot α( : rootsOfUnity 3 ℂ) : GaugeGroupI
```

lemma StandardModel.gaugeGroup$\mathbb{Z}_3$OfRoot_toSU3 [source]

```
@[simp]
lemma ℤ₃gaugeGroupOfRoot_toSU3 α( : rootsOfUnity 3 ℂ) :
    GaugeGroupI.toSU3 (ℤ₃gaugeGroupOfRoot α) =
      ℤ₆gaugeGroupSU3OfRoot (ℤ₃ℤ₆gaugeGroupRootToRoot α)
```

lemma StandardModel.gaugeGroup$\mathbb{Z}_3$OfRoot_toSU2 [source]

```
@[simp]
lemma ℤ₃gaugeGroupOfRoot_toSU2 α( : rootsOfUnity 3 ℂ) :
    GaugeGroupI.toSU2 (ℤ₃gaugeGroupOfRoot α) =
      ℤ₆gaugeGroupSU2OfRoot (ℤ₃ℤ₆gaugeGroupRootToRoot α)
```

lemma StandardModel.gaugeGroup$\mathbb{Z}_3$OfRoot_toU1 [source]

```
@[simp]
lemma ℤ₃gaugeGroupOfRoot_toU1 α( : rootsOfUnity 3 ℂ) :
    GaugeGroupI.toU1 (ℤ₃gaugeGroupOfRoot α) =
      ℤ₆gaugeGroupUnitaryOfRoot (ℤ₃ℤ₆gaugeGroupRootToRoot α)
```

lemma StandardModel.gaugeGroup$\mathbb{Z}_3$OfRoot_mem_center [source]

```
lemma ℤ₃gaugeGroupOfRoot_mem_center α( : rootsOfUnity 3 ℂ) :
    ℤ₃gaugeGroupOfRoot α ∈ Subgroup.center GaugeGroupI
```

def StandardModel.gaugeGroup$\mathbb{Z}_3$Hom [source]

The homomorphism from third roots of unity to 'GaugeGroupI'.

```
noncomputable def ℤ₃gaugeGroupHom : rootsOfUnity 3 ℂ →* GaugeGroupI
```

lemma StandardModel.gaugeGroup$\mathbb{Z}_3$Hom_apply [source]

```
@[simp]
lemma ℤ₃gaugeGroupHom_apply α( : rootsOfUnity 3 ℂ) :
    ℤ₃gaugeGroupHom α = ℤ₃gaugeGroupOfRoot α
```

lemma StandardModel.gaugeGroup$\mathbb{Z}_3$Hom_toSU3 [source]

```
@[simp]
lemma ℤ₃gaugeGroupHom_toSU3 α( : rootsOfUnity 3 ℂ) :
    GaugeGroupI.toSU3 (ℤ₃gaugeGroupHom α) =
      ℤ₆gaugeGroupSU3OfRoot (ℤ₃ℤ₆gaugeGroupRootToRoot α)
```

lemma StandardModel.gaugeGroup$\mathbb{Z}_3$Hom_toSU2 [source]

```
@[simp]
lemma ℤ₃gaugeGroupHom_toSU2 α( : rootsOfUnity 3 ℂ) :
    GaugeGroupI.toSU2 (ℤ₃gaugeGroupHom α) =
      ℤ₆gaugeGroupSU2OfRoot (ℤ₃ℤ₆gaugeGroupRootToRoot α)
```

lemma StandardModel.gaugeGroup$\mathbb{Z}_3$Hom_toU1 [source]

```
@[simp]
lemma ℤ₃gaugeGroupHom_toU1 α( : rootsOfUnity 3 ℂ) :
    GaugeGroupI.toU1 (ℤ₃gaugeGroupHom α) =
      ℤ₆gaugeGroupUnitaryOfRoot (ℤ₃ℤ₆gaugeGroupRootToRoot α)
```

lemma StandardModel.gaugeGroup$\mathbb{Z}_3$OfRoot_mem [source]

```
lemma ℤ₃gaugeGroupOfRoot_mem α( : rootsOfUnity 3 ℂ) :
    ℤ₃gaugeGroupOfRoot α ∈ ℤ₃gaugeGroupSubGroup
```

lemma StandardModel.mem_gaugeGroup$\mathbb{Z}_3$SubGroup_iff [source]

```
lemma ℤ₃mem_gaugeGroupSubGroup_iff (g : GaugeGroupI) :
    g ∈ ℤ₃gaugeGroupSubGroup ↔ ∃ α : rootsOfUnity 3 ℂ, ℤ₃gaugeGroupOfRoot α = g
```

lemma StandardModel.gaugeGroup$\mathbb{Z}_3$SubGroup_le_gaugeGroup$\mathbb{Z}_6$SubGroup [source]

```
lemma ℤ₃ℤ₆gaugeGroupSubGroup_le_gaugeGroupSubGroup :
    ℤ₃gaugeGroupSubGroup ≤ ℤ₆gaugeGroupSubGroup
```

lemma StandardModel.gaugeGroup$\mathbb{Z}_3$SubGroup_le_center [source]

```
lemma ℤ₃gaugeGroupSubGroup_le_center :
    ℤ₃gaugeGroupSubGroup ≤ Subgroup.center GaugeGroupI
```

instance StandardModel.gaugeGroup$\mathbb{Z}_3$SubGroup_normal [source]

```
instance ℤ₃gaugeGroupSubGroup_normal : ℤ₃gaugeGroupSubGroup.Normal where
```

instance StandardModel.«instance@2ccf6670» [source]

```
noncomputable instance : Group ℤ₃GaugeGroup
```

def StandardModel.GaugeGroup$\mathbb{Z}_3$.mk [source]

The quotient map from 'GaugeGroupI' to 'GaugeGroup$\mathbb{Z}_3$'.

```
noncomputable def mk : GaugeGroupI →* ℤ₃GaugeGroup
```

lemma StandardModel.GaugeGroup$\mathbb{Z}_3$.mk_gaugeGroup$\mathbb{Z}_3$OfRoot [source]

```
@[simp]
lemma ℤ₃mk_gaugeGroupOfRoot α( : rootsOfUnity 3 ℂ) :
    mk (ℤ₃gaugeGroupOfRoot α) = 1
```

instance StandardModel.«instance@e87cb8a7» [source]

```
noncomputable instance (q : GaugeGroupQuot) : Group (GaugeGroup q)
```

def StandardModel.GaugeGroupQuot.subgroup [source]

The central subgroup of 'GaugeGroupI' quotiented by a gauge-group quotient choice.

```
noncomputable def subgroup : GaugeGroupQuot → Subgroup GaugeGroupI
  | ℤ₆. => ℤ₆gaugeGroupSubGroup
  | ℤ₂. => ℤ₂gaugeGroupSubGroup
  | ℤ₃. => ℤ₃gaugeGroupSubGroup
  | .I => ⊥
```

lemma StandardModel.GaugeGroupQuot.subgroup_le_center [source]

The subgroup attached to a gauge-group quotient choice lies in the center of ‘GaugeGroupI‘.

```
lemma subgroup_le_center (q : GaugeGroupQuot) :
    subgroup q ≤ Subgroup.center GaugeGroupI
```

instance StandardModel.GaugeGroupQuot.subgroup_normal [source]

The subgroup attached to a gauge-group quotient choice is normal in ‘GaugeGroupI‘.

```
instance subgroup_normal (q : GaugeGroupQuot) : (subgroup q).Normal
```

def StandardModel.GaugeGroupQuot.quotientMap [source]

The quotient map from ‘GaugeGroupI‘ to the gauge group selected by a quotient choice.

```
noncomputable def quotientMap (q : GaugeGroupQuot) : GaugeGroupI →* GaugeGroup q
```

lemma StandardModel.GaugeGroupQuot.quotientMap_I_apply [source]

```
@[simp]
lemma quotientMap_I_apply (g : GaugeGroupI) :
    quotientMap .I g = g
```

lemma StandardModel.GaugeGroupQuot.quotientMap_$\mathbb{Z}_6$_gaugeGroup$\mathbb{Z}_6$OfRoot [source]

```
@[simp]
lemma ℤ₆ℤ₆quotientMap_gaugeGroupOfRoot α( : rootsOfUnity 6 ℂ) :
    quotientMap ℤ₆. (ℤ₆gaugeGroupOfRoot α) = 1
```

lemma StandardModel.GaugeGroupQuot.quotientMap_$\mathbb{Z}_2$_gaugeGroup$\mathbb{Z}_2$OfRoot [source]

```
@[simp]
lemma ℤ₂ℤ₂quotientMap_gaugeGroupOfRoot α( : rootsOfUnity 2 ℂ) :
    quotientMap ℤ₂. (ℤ₂gaugeGroupOfRoot α) = 1
```

lemma StandardModel.GaugeGroupQuot.quotientMap_$\mathbb{Z}_3$_gaugeGroup$\mathbb{Z}_3$OfRoot [source]

```
@[simp]
lemma ℤ₃ℤ₃quotientMap_gaugeGroupOfRoot α( : rootsOfUnity 3 ℂ) :
    quotientMap ℤ₃. (ℤ₃gaugeGroupOfRoot α) = 1
```

lemma StandardModel.GaugeGroupQuot.mem_subgroup_iff_quotientMap_eq_one [source]

The kernel of the quotient map is the subgroup selected by the quotient choice.

```
lemma mem_subgroup_iff_quotientMap_eq_one (q : GaugeGroupQuot) (g : GaugeGroupI) :
    g ∈ subgroup q ↔ quotientMap q g = 1
```

lemma StandardModel.GaugeGroupQuot.quotientMap_eq_iff [source]

Two representatives have the same image under the selected quotient map exactly when their quotient lies in the subgroup selected by the quotient choice.

```
lemma quotientMap_eq_iff (q : GaugeGroupQuot) (g h : GaugeGroupI) :
    quotientMap q g = quotientMap q h ↔ g / h ∈ subgroup q
```

2.20 Physlib.Particles.StandardModel.HiggsBoson.Basic

Physlib/Particles/StandardModel/HiggsBoson/Basic.lean on GitHub.

instance StandardModel.HiggsVec.«instance@d58bd5cc» [source]

```
instance : SMulCommClass ℂ GaugeGroupI HiggsVec where
```

instance StandardModel.HiggsVec.«instance@34481c4b» [source]

```
instance : SMulCommClass ℝ GaugeGroupI HiggsVec where
```

def StandardModel.HiggsVec.toRealScalars [source]

The underlying real values of the Higgs vector.

```
def toRealScalars : HiggsVec →ₗ[ℝ] (Fin 4 → ℝ) where
```

lemma StandardModel.HiggsVec.toRealScalars_smul_real [source]

```
lemma toRealScalars_smul_real (a : ℝ) φ( : HiggsVec) :
    toRealScalars (a • φ) = a • toRealScalars φ
```

lemma StandardModel.HiggsVec.ofReal_toRealScalars [source]

```
lemma ofReal_toRealScalars (a : ℝ) :
    toRealScalars (ofReal a) = ₄![Real.sqrt a, 0, 0, 0]
```

lemma StandardModel.HiggsVec.ofReal_toRealScalars_norm [source]

```
lemma ofReal_toRealScalars_norm φ( : HiggsVec) :
    toRealScalars (ofReal ‖φ‖( ^ 2)) = ₄‖φ‖![, 0, 0, 0]
```

2.21 Physlib.Particles.StandardModel.HiggsBoson.EffectivePotential

Physlib/Particles/StandardModel/HiggsBoson/EffectivePotential.lean on GitHub.

abbrev StandardModel.HiggsField.EffectivePotential [source]

A general potential of the Higgs field.

```
abbrev EffectivePotential : Type
```

def StandardModel.HiggsField.EffectivePotential.IsInvariant [source]

The proposition that the general potential is invariant under the global action of the gauge group.

```
def IsInvariant (V : EffectivePotential) : Prop
```

lemma StandardModel.HiggsField.EffectivePotential.IsInvariant.eq_on_orbits [source]

An invariant potential is equal on gauge orbits.

```
lemma eq_on_orbits φ{1 φ2 : HiggsVec} {V : EffectivePotential} (h : IsInvariant V)
    (φh : φ1 ∈ MulAction.orbit GaugeGroupI  φ2) :
    V φ1 = V φ2
```

lemma StandardModel.HiggsField.EffectivePotential.IsInvariant.eq_of_norm_eq [source]

An invariant potential is equal on Higgs vectors with identical norms.

```
lemma eq_of_norm_eq φ{1 φ2 : HiggsVec} {V : EffectivePotential} (h : IsInvariant V)
    (φh : ‖φ‖1 = ‖φ‖2) :
    V φ1 = V φ2
```

lemma StandardModel.HiggsField.EffectivePotential.IsInvariant.factors_through_norm [source]

```
lemma factors_through_norm {V : EffectivePotential} (h : IsInvariant V) :
    ∃ (f : ℝ → ℝ), V = f ∘ norm
```

def StandardModel.HiggsField.EffectivePotential.HasMaxMassDimLE [source]

The proposition that the potential ‘V‘ has a maximum mass dimension less then or equal to ‘n‘ - also implying it is a polynomial.

```
def HasMaxMassDimLE (V : EffectivePotential) (n : ℕ) : Prop
```

def StandardModel.HiggsField.EffectivePotential.polynomial [source]

The polynomial associated to a potential ‘V‘ with a maximum mass dimension less than or equal to ‘n‘.

```
def polynomial (V : EffectivePotential) {n : ℕ} (h : HasMaxMassDimLE V n) :
    MvPolynomial (Fin 4) ℝ
```

lemma StandardModel.HiggsField.EffectivePotential.polynomial_totalDegree [source]

```
lemma polynomial_totalDegree {V : EffectivePotential} {n : ℕ} (h : HasMaxMassDimLE V n) :
    (polynomial V h).totalDegree ≤ n
```

lemma StandardModel.HiggsField.EffectivePotential.apply_eq_polynomial [source]

```
lemma apply_eq_polynomial {V : EffectivePotential} {n : ℕ} (h : HasMaxMassDimLE V n)
    φ( : HiggsVec) : V φ = (polynomial V h).eval φ.toRealScalars
```

def StandardModel.HiggsField.EffectivePotential.termOfMassDim [source]

The part of a potential at a given mass-dimension.

```
def termOfMassDim (V : EffectivePotential) {n : ℕ} (h : HasMaxMassDimLE V n) (m : ℕ) :
    HiggsVec → ℝ
```

lemma StandardModel.HiggsField.EffectivePotential.termOfMassDim_eq_zero_of_max_lt [source]

```
lemma termOfMassDim_eq_zero_of_max_lt {V : EffectivePotential} {n : ℕ} (h : HasMaxMassDimLE V n
    )
    {m : ℕ} (hm : n < m) φ( : HiggsVec) :
    termOfMassDim V h m φ = 0
```

lemma StandardModel.HiggsField.EffectivePotential.termOfMassDim_homogeneity [source]

```
lemma termOfMassDim_homogeneity {V : EffectivePotential} {n : ℕ} (h : HasMaxMassDimLE V n) (m :
     ℕ)
    φ( : HiggsVec) (t : ℝ) : termOfMassDim V h m (t • φ) = t ^ m * termOfMassDim V h m φ
```

lemma StandardModel.HiggsField.EffectivePotential.apply_eq_sum_termOfMassDim [source]

```
lemma apply_eq_sum_termOfMassDim {V : EffectivePotential} {n : ℕ} (h : HasMaxMassDimLE V n)
    φ( : HiggsVec) :
    V φ = ∑ m ∈ Finset.range (n + 1), termOfMassDim V h m φ
```

lemma StandardModel.HiggsField.EffectivePotential.apply_smul_eq_sum_termOfMassDim [source]

```
lemma apply_smul_eq_sum_termOfMassDim {V : EffectivePotential} {n : ℕ} (h : HasMaxMassDimLE V n
    )
    φ( : HiggsVec) (t : ℝ) :
    V (t • φ) = ∑ m ∈ Finset.range (n + 1), t ^ m * termOfMassDim V h m φ
```

lemma StandardModel.HiggsField.EffectivePotential.termOfMassDim_isInvariant [source]

```
lemma termOfMassDim_isInvariant {V : EffectivePotential} {n : ℕ} (h : HasMaxMassDimLE V n)
    (m : ℕ) (hV : IsInvariant V) : IsInvariant (termOfMassDim V h m)
```

lemma StandardModel.HiggsField.EffectivePotential.termOfMassDim_eq_mul_norm [source]

```
lemma termOfMassDim_eq_mul_norm {V : EffectivePotential} {n : ℕ}
    (h : HasMaxMassDimLE V n) (m : ℕ) (hV : IsInvariant V) φ( : HiggsVec) :
    ∃ c, termOfMassDim V h m φ = c * ‖φ‖ ^ m
```

lemma StandardModel.HiggsField.EffectivePotential.termOfMassDim_zero_of_odd [source]

```
lemma termOfMassDim_zero_of_odd {V : EffectivePotential} {n : ℕ} (h : HasMaxMassDimLE V n) (m :
     ℕ)
    (hV : IsInvariant V) φ( : HiggsVec) (hodd : Odd m)  :
    termOfMassDim V h m φ = 0
```

lemma StandardModel.HiggsField.EffectivePotential.apply_eq_sum_even_termOfMassDim [source]

```
lemma apply_eq_sum_even_termOfMassDim {V : EffectivePotential} {n : ℕ} (h : HasMaxMassDimLE V n
    )
    (hV : IsInvariant V) φ( : HiggsVec) :
    V φ = ∑ m ∈ Finset.range (n / 2 + 1), termOfMassDim V h (2 * m) φ
```

lemma StandardModel.HiggsField.EffectivePotential.apply_eq_sum_even_termOfMassDim_fin [source]

```
lemma apply_eq_sum_even_termOfMassDim_fin {V : EffectivePotential} {n : ℕ} (h : HasMaxMassDimLE
     V n)
    (hV : IsInvariant V) φ( : HiggsVec) :
    V φ = ∑ m : Fin (n/2 + 1), termOfMassDim V h (2 * m) φ
```

lemma StandardModel.HiggsField.EffectivePotential.apply_eq_sum_norm_pow [source]

The potential is equal to the sum of norms to even powers.

```
lemma apply_eq_sum_norm_pow {V : EffectivePotential} {n : ℕ} (h : HasMaxMassDimLE V n)
    (hV : IsInvariant V) φ( : HiggsVec) :
    ∃ c : Fin (n/2 + 1) → ℝ, V φ = ∑ m, c m • ‖φ‖ ^ (2 * m.1)
```

2.22 Physlib.Particles.SuperSymmetry.MSSMNu.AnomalyCancellation.Basic

Physlib/Particles/SuperSymmetry/MSSMNu/AnomalyCancellation/Basic.lean on GitHub.

lemma MSSMCharges.sum_MSSMSpecies_numberCharges_eq_expand [source]

```
lemma sum_MSSMSpecies_numberCharges_eq_expand [AddCommMonoid M]
    (f : Fin MSSMSpecies.numberCharges → M) :
    ∑ i, f i = f ⟨0, by ⟩simp + f ⟨1, by ⟩simp + f ⟨2, by ⟩simp
```

2.23 Physlib.Particles.SuperSymmetry.N1.Basic

Physlib/Particles/SuperSymmetry/N1/Basic.lean on GitHub.

abbrev SUSY.N1.ChiralScalarConfiguration [source]

The chiral scalar configuration: a complex value for each chiral label. This is the sector's only field data. Declared as an 'abbrev' so that unification sees through it to 'ChiralIndexingType → ℂ' and applies Mathlib's function-space calculus lemmas directly.

```
abbrev ChiralScalarConfiguration (ChiralIndexingType : Type*)
```

inductive SUSY.N1.ChiralColor [source]

The four colours carried by a chiral-sector index: holomorphy ('chiral' versus 'anti', a scalar versus its complex conjugate) crossed with variance ('up' versus 'down', contravariant versus covariant). Carrying both axes here lets the single index type 'ι' label the scalars.

```
inductive ChiralColor | chiralUp | chiralDown | antiUp | antiDown
```

def SUSY.N1.ChiralColor.tau [source]

The dual colour: flips variance and preserves holomorphy. Two indices may contract exactly when their colours are ‘τ‘-related, so ‘V⊗I‘ pairs only with ‘V_I‘ (same holomorphy, opposite variance) and never with a conjugate index.

```
def tau : ChiralColor → ChiralColor
  | chiralUp => chiralDown
  | chiralDown => chiralUp
  | antiUp => antiDown
  | antiDown => antiUp
```

def SUSY.N1.ChiralColor.bar [source]

The conjugate colour: flips holomorphy (‘chiral‘↔‘anti‘) and preserves variance. Complex conjugation sends an index to its conjugate carrier, so ‘bar‘ swaps ‘chiral‘ with ‘anti*‘. Distinct from the variance dual ‘tau‘; the two commute (‘bar_tau‘).*

```
def bar : ChiralColor → ChiralColor
  | chiralUp => antiUp
  | antiUp => chiralUp
  | chiralDown => antiDown
  | antiDown => chiralDown
```

abbrev SUSY.N1.chiralModule [source]

The carrier module of each colour, distinct for all four: the holomorphic vectors ‘ι → ℂ‘ and their dual ‘Module.Dual ℂ (ι → ℂ)‘ on the chiral side, and the conjugate module ‘ConjModule …‘ of each (where ‘i‘ acts as ‘−i‘) on the anti side. Variance is the vector/dual axis, holomorphy the conjugate-module axis; both are genuine carrier data, not labels tracked separately.

```
abbrev chiralModule : ChiralColor → Type
  | .chiralUp   => ι → ℂ
  | .chiralDown => Module.Dual ℂ ι( → ℂ)
  | .antiUp     => ConjModule ι( → ℂ)
  | .antiDown   => ConjModule (Module.Dual ℂ ι( → ℂ))
```

instance SUSY.N1.instAddCommGroupChiralModule [source]

```
instance instAddCommGroupChiralModule : ∀ c, AddCommGroup (chiralModule ι( := ι) c)
  | .chiralUp | .chiralDown | .antiUp | .antiDown => inferInstance
```

instance SUSY.N1.instModuleChiralModule [source]

```
noncomputable instance instModuleChiralModule : ∀ c, Module ℂ (chiralModule ι( := ι) c)
  | .chiralUp | .chiralDown | .antiUp | .antiDown => inferInstance
```

def SUSY.N1.chiralRep [source]

The representation on each colour, taken trivial over the trivial group ‘Unit‘: the chiral scalars carry no charge in this sector.

```
def chiralRep : (c : ChiralColor) → Representation ℂ Unit (chiralModule ι( := ι) c)
```

def SUSY.N1.piBasis [source]

The standard basis of the vector carrier ‘ι → ℂ‘ (the indicator functions); the basis of ‘chiralUp‘ and the reference basis for the δ pairing.

```
def piBasis : Basis ι ℂ ι( → ℂ)
```

def SUSY.N1.chiralBasis [source]

The basis of each colour's carrier, all indexed by ‘ι‘: ‘piBasis‘ on the holomorphic vectors, its dual ‘piBasis.dualBasis‘ on the holomorphic covectors, and the ‘Basis.conj‘ of each on the anti-holomorphic side (coordinates ‘star‘-ed).

```
noncomputable def chiralBasis : (c : ChiralColor) → Basis ι ℂ (chiralModule ι( := ι) c)
  | .chiralUp   => piBasis
  | .chiralDown => piBasis.dualBasis
  | .antiUp     => Basis.conj piBasis
  | .antiDown   => Basis.conj piBasis.dualBasis
```

def SUSY.N1.deltaCap [source]

The δ cap ‘∑_I b_I ⊗ b_I‘: the rank-2 tensor in ‘M ⊗ M‘ with two upper indices, whose components in the basis ‘b‘ are ‘δ^{ij}‘. It is an element of ‘M ⊗ M‘ (the inverse-metric "cap"), not a linear map, and serves as the ‘metric‘ field of the species (whose two slots share a colour).

```
def deltaCap (b : Basis ι ℂ M) : M ⊗[ℂ] M
```

def SUSY.N1.deltaBil$_2$ [source]

The δ pairing between two based modules sharing the index ‘ι‘: the dot product of coordinate vectors ‘(x, y) ↦ ∑_I (b x)_I (b' y)_I‘. Built from Mathlib's ‘Basis.toDual b‘ (the canonical δ map ‘M → Module.Dual M‘, sending ‘b‘ to its dual basis) precomposed on the second slot with the basis transport ‘b' ≃ b‘.

```
def ₂deltaBil (b : Basis ι ℂ M) (b' : Basis ι ℂ N) : M →ₗ[ℂ] N →ₗ[ℂ] ℂ
```

lemma SUSY.N1.deltaBil$_2$_apply [source]

‘deltaBil$_2$ b b' x y = ∑_I (b x)_I (b' y)_I‘.

```
lemma ₂deltaBil_apply (b : Basis ι ℂ M) (b' : Basis ι ℂ N) (x : M) (y : N) :
    ₂deltaBil b b' x y = ∑ I, b.equivFun x I * b'.equivFun y I
```

def SUSY.N1.deltaContr$_2$ [source]

The two-module δ contraction ‘M ⊗ N → ℂ‘.

```
def ₂deltaContr (b : Basis ι ℂ M) (b' : Basis ι ℂ N) : M ⊗[ℂ] N →ₗ[ℂ] ℂ
```

lemma SUSY.N1.deltaContr$_2$_tmul [source]

‘deltaContr$_2$ b b' (x ⊗ₜ y) = ∑_I (b x)_I (b' y)_I‘.

```
lemma ₂deltaContr_tmul (b : Basis ι ℂ M) (b' : Basis ι ℂ N) (x : M) (y : N) :
    ₂deltaContr b b' (x ⊗ₜ[ℂ] y) = ∑ I, b.equivFun x I * b'.equivFun y I
```

lemma SUSY.N1.deltaContr$_2$_tmul_basis [source]

'deltaContr$_2$ b b' (x ⊗$_t$ b' J) = x_J': pairing with the second basis reads off a coordinate.

```
lemma ₂deltaContr_tmul_basis (b : Basis ι ℂ M) (b' : Basis ι ℂ N) (x : M) (J : ι) :
    ₂deltaContr b b' (x ⊗ₜ[ℂ] b' J) = b.equivFun x J
```

lemma SUSY.N1.deltaContr$_2$_basis_basis [source]

'deltaContr$_2$ b b' (b I ⊗$_t$ b' J) = δ_{IJ}': the two bases are δ-dual.

```
lemma ₂deltaContr_basis_basis (b : Basis ι ℂ M) (b' : Basis ι ℂ N) (I J : ι) :
    ₂deltaContr b b' (b I ⊗ₜ[ℂ] b' J) = if I = J then 1 else 0
```

lemma SUSY.N1.deltaContr$_2$_comm [source]

'deltaContr$_2$ b b' (x ⊗$_t$ y) = deltaContr$_2$ b' b (y ⊗$_t$ x)': swapping slots swaps the two bases.

```
lemma ₂deltaContr_comm (b : Basis ι ℂ M) (b' : Basis ι ℂ N) (x : M) (y : N) :
    ₂deltaContr b b' (x ⊗ₜ[ℂ] y) = ₂deltaContr b' b (y ⊗ₜ[ℂ] x)
```

def SUSY.N1.deltaCap$_2$ [source]

The two-module δ cap '∑_I b_I ⊗ b'_I ∈ M ⊗ N'.

```
def ₂deltaCap (b : Basis ι ℂ M) (b' : Basis ι ℂ N) : M ⊗[ℂ] N
```

lemma SUSY.N1.deltaCap$_2$_comm [source]

'comm (deltaCap$_2$ b b') = deltaCap$_2$ b' b': swapping the two factors swaps the two bases.

```
lemma ₂deltaCap_comm (b : Basis ι ℂ M) (b' : Basis ι ℂ N) :
    TensorProduct.comm ℂ M N (₂deltaCap b b') = ₂deltaCap b' b
```

lemma SUSY.N1.deltaUnit$_2$_symm [source]

The 'unit_symm' law (two-module, 'toSpanSingleton' form): 'deltaCap$_2$ b' b' is the swap of 'deltaCap$_2$ b b'.

```
lemma ₂deltaUnit_symm (b : Basis ι ℂ M) (b' : Basis ι ℂ N) :
    LinearMap.toSpanSingleton ℂ _ (₂deltaCap b' b) 1 =
      LinearMap.lTensor N (LinearEquiv.refl ℂ M).toLinearMap
        (TensorProduct.comm ℂ M N (LinearMap.toSpanSingleton ℂ _ (₂deltaCap b b') 1))
```

lemma SUSY.N1.deltaContr$_2$_unit [source]

The snake identity (two-module, ‘contr_unit‘ law): contracting ‘x ∈ M‘ into the ‘M‘-leg of ‘deltaCap$_2$ b’ b ∈ N ⊗ M‘ returns ‘x‘.

```
lemma ₂deltaContr_unit (b : Basis ι ℂ M) (b' : Basis ι ℂ N) (x : M) :
    (TensorProduct.lid ℂ M) ((₂deltaContr b b').rTensor M
      ((TensorProduct.assoc ℂ M N M).symm
        (x ⊗ₜ[ℂ] LinearMap.toSpanSingleton ℂ _ (₂deltaCap b' b) 1))) = x
```

lemma SUSY.N1.deltaContr$_2$_metric [source]

The ‘contr_metric‘ law (two-module): contracting the inner ‘M‘/‘N‘ legs of ‘deltaCap b ⊗ deltaCap b’‘ yields ‘deltaCap$_2$ b’ b‘.

```
lemma ₂deltaContr_metric (b : Basis ι ℂ M) (b' : Basis ι ℂ N) :
    (TensorProduct.comm ℂ M N ((TensorProduct.lid ℂ N).lTensor M
      (((₂deltaContr b b').rTensor N).lTensor M
        (((TensorProduct.assoc ℂ M N N).symm.toLinearMap.lTensor M)
          ((TensorProduct.assoc ℂ M M (N ⊗[ℂ] N))
            (LinearMap.toSpanSingleton ℂ _ (deltaCap b) 1 ⊗ₜ[ℂ]
              LinearMap.toSpanSingleton ℂ _ (deltaCap b') 1)))))) =
      LinearMap.toSpanSingleton ℂ _ (₂deltaCap b' b) 1
```

def SUSY.N1.chiralTensor [source]

The chiral-index tensor species, bundled with its conjugation. Its four colours ‘chiral‘/‘anti‘ × ‘up‘/‘down‘ carry the four distinct carriers of §C. ‘contr c‘ is the two-module δ pairing of a colour against its variance dual ‘τ c‘ (‘V c ⊗ V (τ c) → ℂ‘); ‘unit c‘ is the δ cap across those two carriers; ‘metric c‘ is the single-colour δ cap ‘∑_I b_I ⊗ b_I‘. Each ‘TensorSpecies‘ coherence law reduces, by case analysis on the colour, to the corresponding abstract two-module δ lemma of §D. The conjugation flips holomorphy (‘ChiralColor.bar‘) while preserving variance; every basis is indexed by ‘ι‘, so ‘barIdx_eq‘ is ‘rfl‘, and ‘conj_contrComm‘ is ‘star δ = δ‘. Instantiating ‘ConjTensorSpecies‘ this way gives the chiral sector both the framework’s generic tensor API and its conjugation API (‘conjT‘ and its laws) on one object.

```
def chiralTensor : ConjTensorSpecies ℂ ChiralColor Unit (chiralModule ι( := ι)) (fun _ => ι)
    (chiralRep ι( := ι)) (chiralBasis ι( := ι)) where
```

def SUSY.N1.conjScalar [source]

Conjugation of a scalar tensor, normalized back to the scalar colour list ‘![]‘.

```
def conjScalar (t : (chiralTensor ι( := ι)).Tensor ![]) :
    (chiralTensor ι( := ι)).Tensor ![]
```

def SUSY.N1.conjChiralCovector [source]

Conjugation of a holomorphic covector, normalized to the anti-holomorphic covector colour list ‘![antiDown]‘.

```
def conjChiralCovector
    (t : (chiralTensor ι( := ι)).Tensor ![chiralDown]) :
    (chiralTensor ι( := ι)).Tensor ![antiDown]
```

lemma SUSY.N1.toField_conjScalar [source]

For scalar tensors, ‘toField‘ of the normalized tensor conjugate is the complex conjugate of ‘toField‘.

```
lemma toField_conjScalar (t : (chiralTensor ι( := ι)).Tensor ![]) :
    (conjScalar t).toField = star t.toField
```

lemma SUSY.N1.repr_conjChiralCovector [source]

Component formula for the holomorphic covector conjugate: the ‘![I]‘ basis component of ‘conjChiralCovector t‘ is the complex conjugate of the ‘![I]‘ component of ‘t‘.

```
lemma repr_conjChiralCovector
    (t : (chiralTensor ι( := ι)).Tensor ![chiralDown]) (I : ι) :
    (basis (S := (chiralTensor ι( := ι)).toTensorSpecies) ![antiDown]).repr
        (conjChiralCovector t) ![I] =
      star ((basis (S := (chiralTensor ι( := ι)).toTensorSpecies) ![chiralDown]).repr t ![I])
```

lemma SUSY.N1.conjChiralCovector_add [source]

Conjugation of a holomorphic covector is additive.

```
@[simp]
lemma conjChiralCovector_add
    (₁t ₂t : (chiralTensor ι( := ι)).Tensor ![chiralDown]) :
    conjChiralCovector (₁t + ₂t) = conjChiralCovector ₁t + conjChiralCovector ₂t
```

lemma SUSY.N1.conjChiralCovector_smul [source]

Conjugation of a holomorphic covector is conjugate-linear: a scalar ‘r‘ pulls out as ‘star r‘.

```
@[simp]
lemma conjChiralCovector_smul (r : ℂ)
    (t : (chiralTensor ι( := ι)).Tensor ![chiralDown]) :
    conjChiralCovector (r • t) = star r • conjChiralCovector t
```

2.24 Physlib.Particles.SuperSymmetry.SU5.ChargeSpectrum.AllowsTerm

Physlib/Particles/SuperSymmetry/SU5/ChargeSpectrum/AllowsTerm.lean on GitHub.

instance SuperSymmetry.SU5.ChargeSpectrum.«instance@7b86df89» [source]

```
instance (x : ChargeSpectrum □) (q5 : □) (T : PotentialTerm) :
    Decidable (AllowsTermQ5 x q5 T)
```

instance SuperSymmetry.SU5.ChargeSpectrum.«instance@35144a57» [source]

```
instance (x : ChargeSpectrum □) (q10 : □) (T : PotentialTerm) :
    Decidable (AllowsTermQ10 x q10 T)
```

2.25 Physlib.Particles.SuperSymmetry.SU5.ChargeSpectrum.PhenoClosed

Physlib/Particles/SuperSymmetry/SU5/ChargeSpectrum/PhenoClosed.lean on GitHub.

instance SuperSymmetry.SU5.ChargeSpectrum.«instance@1aa8df08» [source]

```
instance {S5 S10 : Finset □} {charges : Multiset (ChargeSpectrum □)} :
    Decidable (ContainsPhenoCompletionsOfMinimallyAllows S5 S10 charges)
```

2.26 Physlib.QuantumMechanics.DDimensions.Operators.SpectralTheory.Basic

Physlib/QuantumMechanics/DDimensions/Operators/SpectralTheory/Basic.lean on GitHub.

abbrev LinearPMap.resolvent [source]

The resolvent, ‘(T - z • 1)⁻¹‘.

```
abbrev resolvent (T : H →ₗ[ℂ] H) (z : ℂ) : H →ₗ[ℂ] H
```

def LinearPMap.IsLowerBound [source]

*‘IsLowerBound T z c‘ is the property that ‘c * ‖x‖ ≤ ‖T x - z • x‖‘ for all ‘x : T.domain‘.*

```
def IsLowerBound (T : H →ₗ[ℂ] H) (z : ℂ) (c : ℝ) : Prop
```

lemma LinearPMap.isLowerBound_neg [source]

```
lemma isLowerBound_neg {T : H →ₗ[ℂ] H} {z : ℂ} {c : ℝ} (h : IsLowerBound T z c) :
    IsLowerBound (-T) (-z) c
```

lemma LinearPMap.isLowerBound_of_right_le [source]

```
lemma isLowerBound_of_right_le
    {T : H →ₗ[ℂ] H} {z : ℂ} {c₁ c₂ : ℝ} (hle : c₁ ≤ c₂) (h : IsLowerBound T z c₂) :
    IsLowerBound T z c₁
```

lemma LinearPMap.isLowerBound_of_left_le [source]

```
lemma isLowerBound_of_left_le
    {T₁ T₂ : H →ₗ[ℂ] H} (hle : T₁ ≤ T₂) {z : ℂ} {c : ℝ} (h : IsLowerBound T₂ z c) :
    IsLowerBound T₁ z c
```

lemma LinearPMap.isLowerBound_closure [source]

```
lemma isLowerBound_closure
    {T : H →ₗ[ℂ] H} {z : ℂ} {c : ℝ} (h : IsLowerBound T z c) : IsLowerBound T.closure z c
```

def LinearPMap.regularityDomain [source]

The regular points for ‘T‘.

*‘z : ℂ‘ is a regular point for ‘T‘ iff there exists a constant ‘c > 0‘ such that ‘c * ‖x‖ ≤ ‖(T - z • 1) x‖‘ for all ‘x ∈ T.domain‘.*

```
def regularityDomain (T : H →ₗ[ℂ] H) : Set ℂ
```

lemma LinearPMap.regularityDomain_neg [source]

```
@[simp]
lemma regularityDomain_neg (T : H →ₗ[ℂ] H) : (-T).regularityDomain = -T.regularityDomain
```

lemma LinearPMap.regularityDomain_smul [source]

```
@[simp]
lemma regularityDomain_smul (T : H →ₗ[ℂ] H) {w : ℂ} (hw : w ≠ 0) :
    (w • T).regularityDomain = w • T.regularityDomain
```

lemma LinearPMap.regularityDomain_antitone [source]

‘$T \leq T$’‘ implies ‘T’.regularityDomain $\subseteq$ T.regularityDomain‘.

```
lemma regularityDomain_antitone : Antitone (regularityDomain (H := H))
```

lemma LinearPMap.mem_regularityDomain_iff [source]

‘z‘ is a regular point for ‘T‘ iff ‘T - z • 1‘ has a continuous (equivalently, bounded) inverse.

```
lemma mem_regularityDomain_iff {T : H →ₗ[ℂ] H} {z : ℂ} :
    z ∈ T.regularityDomain ↔ (T - z • 1).toFun.ker = ⊥ ∧ Continuous ⇑( T z)
```

lemma LinearPMap.ball_subset_regularityDomain [source]

The regularity domain of ‘T‘ contains open balls with radii controlled by the lower bounds.

```
lemma ball_subset_regularityDomain
    {T : H →ₗ[ℂ] H} {z : ℂ} {c : ℝ} (h : IsLowerBound T z c) : ball z c ⊆ T.regularityDomain
```

lemma LinearPMap.regularityDomain_isOpen [source]

The regularity domain is an open set.

```
lemma regularityDomain_isOpen (T : H →ₗ[ℂ] H) : IsOpen T.regularityDomain
```

lemma LinearPMap.regularityDomain_closure [source]

‘T‘ and ‘T.closure‘ have the same regularity domain.

```
lemma regularityDomain_closure (T : H →ₗ[ℂ] H) :
    T.closure.regularityDomain = T.regularityDomain
```

lemma LinearPMap.IsClosable.closure_range_sub_eq_range_closure_sub [source]

```
lemma IsClosable.closure_range_sub_eq_range_closure_sub [CompleteSpace H]
    {T : H →ₗ[ℂ] H} (hT : T.IsClosable) {z : ℂ} (hz : z ∈ T.regularityDomain) :
```

```
    (T - z • 1).toFun.range.closure = (T.closure - z • 1).toFun.range
```

lemma LinearPMap.IsClosed.sub_range_isClosed [source]

```
lemma IsClosed.sub_range_isClosed [CompleteSpace H]
    {T : H →ₗ[ℂ] H} (hT : T.IsClosed) {z : ℂ} (hz : z ∈ T.regularityDomain) :
    _root_.IsClosed ((T - z • 1).toFun.range : Set H)
```

lemma LinearPMap.IsUnbounded.orthogonal_closure_sub_range [source]

‘(T.closure - z • 1).range$^{\perp}$ = (T† - conj z • 1).ker‘

```
lemma IsUnbounded.orthogonal_closure_sub_range [CompleteSpace H]
    {T : H →ₗ[ℂ] H} (hT : T.IsUnbounded) (z : ℂ) :
    (T.closure - z • 1).toFun.□range
      = (†T - conj z • 1).toFun.ker.map (†T - conj z • 1).domain.subtype
```

lemma LinearPMap.IsUnbounded.orthogonal_adjoint_sub_ker [source]

‘(T† - conj z • 1).ker$^{\perp}$ = (T.closure - z • 1).range‘

```
lemma IsUnbounded.orthogonal_adjoint_sub_ker [CompleteSpace H]
    {T : H →ₗ[ℂ] H} (hT : T.IsUnbounded) {z : ℂ} (hz : z ∈ T.regularityDomain) :
    ((†T - conj z • 1).toFun.ker.map (†T - conj z • 1).domain.subtype)□
      = (T.closure - z • 1).toFun.range
```

def LinearPMap.deficiencySubspace [source]

For a partial linear map ‘T‘ and any complex number ‘z‘, the closed submodule which is orthogonal to the range of ‘T - z • 1‘.

‘T.defectNumber z‘ is defined as the rank of this subspace.

```
def deficiencySubspace (T : H →ₗ[ℂ] H) (z : ℂ) : ClosedSubmodule ℂ H
```

lemma LinearPMap.deficiencySubspace_coe [source]

```
@[simp]
lemma deficiencySubspace_coe (T : H →ₗ[ℂ] H) (z : ℂ) :
    T.deficiencySubspace z = (T - z • 1).toFun.□range
```

def LinearPMap.defectNumber [source]

The rank of ‘T.deficiencySubspace z = (T - z • 1).range$^{\perp}$‘.

```
def defectNumber (T : H →ₗ[ℂ] H) (z : ℂ) : Cardinal
```

lemma LinearPMap.defectNumber_eq [source]

```
lemma defectNumber_eq (T : H →ₗ[ℂ] H) (z : ℂ) :
    T.defectNumber z = Module.rank ℂ (T.deficiencySubspace z)
```

lemma LinearPMap.IsClosed.defectNumber_eq_zero_iff [source]

```
lemma IsClosed.defectNumber_eq_zero_iff [CompleteSpace H]
    {T : H →ₗ[ℂ] H} (hT : T.IsClosed) {z : ℂ} (hz : z ∈ T.regularityDomain) :
    T.defectNumber z = 0 ↔ (T - z • 1).toFun.range = ⊤
```

lemma LinearPMap.defectNumber_closure [source]

'T' and 'T.closure' have the same defect number at points in their regularity domain.

```
lemma defectNumber_closure [CompleteSpace H]
    {T : H →ₗ[ℂ] H} {z : ℂ} (hz : z ∈ T.regularityDomain) :
    T.closure.defectNumber z = T.defectNumber z
```

lemma Submodule.inf_ne_bot_of_rank_lt [source]

```
lemma _root_.Submodule.inf_ne_bot_of_rank_lt
    {E F : Submodule ℂ H} [E.HasOrthogonalProjection] (h_rank : Module.rank ℂ E < Module.rank
    ℂ F) :
    E ⊓ F ≠ ⊥
```

lemma LinearPMap.IsClosed.exists_inner_eq_zero_of_defectNumber_lt [source]

```
lemma IsClosed.exists_inner_eq_zero_of_defectNumber_lt [CompleteSpace H]
    {T : H →ₗ[ℂ] H} (hT : T.IsClosed)
    {₁z ₂z : ℂ} (₁hz : ₁z ∈ T.regularityDomain) (h : T.defectNumber ₁z < T.defectNumber ₂z) :
    ∃ x : T.domain, x ≠ 0 ∧ ⟪T x - ₁z • x, T x - ₂z • ⟫_ℂx_ = 0
```

lemma LinearPMap.IsClosable.defectNumber_eq_of_mem_ball [source]

```
lemma IsClosable.defectNumber_eq_of_mem_ball [CompleteSpace H] {T : H →ₗ[ℂ] H} (hT : T.
    IsClosable)
    {₁z ₂z : ℂ} {c : ℝ} (h : IsLowerBound T ₁z c) (h_ball : ₂z ∈ ball ₁z c) :
    T.defectNumber ₁z = T.defectNumber ₂z
```

lemma LinearPMap.IsClosable.defectNumber_const [source]

The defect number is constant on each connected component of the regularity domain.

```
lemma IsClosable.defectNumber_const [CompleteSpace H]
    {T : H →ₗ[ℂ] H} (hT : T.IsClosable)
    {₁z ₂z : ℂ} (hz : ₂z ∈ connectedComponentIn T.regularityDomain ₁z) :
    T.defectNumber ₁z = T.defectNumber ₂z
```

def LinearPMap.numericalRange [source]

The set '$\{\langle\langle x,\ T\ x\rangle\rangle_{\mathbb{C}} \mid x \in T.domain \wedge \|x\| = 1\} \subseteq \mathbb{C}$'.

```
def numericalRange (T : H →ₗ[ℂ] H) : Set ℂ
```

lemma LinearPMap.numericalRange_eq [source]

```
lemma numericalRange_eq (T : H →ₗ[ℂ] H) : 0 T = (fun x ↦ ⟪↑x, T ⟫_ℂx_) '' {x | ‖x‖ = 1}
```

lemma LinearPMap.mem_numericalRange [source]

```
lemma mem_numericalRange {T : H →ₗ[ℂ] H} {x : T.domain} (hx : x ≠ 0) :
    ‖(‖x ^ 2)⁻¹ * ⟪↑x, T ⟪ℂ x_ ∈ 0 T
```

lemma LinearPMap.numericalRange_nonempty [source]

```
lemma numericalRange_nonempty {T : H →ₗ[ℂ] H} (hT : T.domain ≠ ⊥) : 0( T).Nonempty
```

lemma LinearPMap.numericalRange_neg [source]

```
@[simp]
lemma numericalRange_neg (T : H →ₗ[ℂ] H) : 0 (-T) = 0- T
```

lemma LinearPMap.numericalRange_smul [source]

```
@[simp]
lemma numericalRange_smul (T : H →ₗ[ℂ] H) (c : ℂ) : 0 (c • T) = c • 0 T
```

lemma LinearPMap.numericalRange_sub_const [source]

```
lemma numericalRange_sub_const (T : H →ₗ[ℂ] H) (c : ℂ) : 0 (T - c • 1) = 0 T - {c}
```

lemma LinearPMap.compl_closure_numericalRange_subset_regularityDomain [source]

The regularity domain contains the exterior of the numerical range.

```
lemma compl_closure_numericalRange_subset_regularityDomain (T : H →ₗ[ℂ] H) :
    (_root_.closure 0( T))ᶜ ⊆ T.regularityDomain
```

lemma LinearPMap.exists_phase_add_im_eq_zero [source]

```
private lemma exists_phase_add_im_eq_zero (₁z ₂z : ℂ) :
    ∃ θ : ℝ, (exp (I * θ) * ₁z + exp (-I * θ) * ₂z).im = 0
```

theorem LinearPMap.numericalRange_convex [source]

The Toeplitz-Hausdorff theorem.

```
theorem numericalRange_convex (T : H →ₗ[ℂ] H) : Convex ℝ 0( T)
```

def LinearPMap.resolventSet [source]

The resolvent set, 'ρ', of a partial linear map.

A complex number 'z' is in 'ρ T' iff the linear map 'T - z • 1' from 'T.domain' to 'H' is a bijection with continuous (equivalently, bounded) inverse.

```
def resolventSet (T : H →ₗ[ℂ] H) : Set ℂ
```

lemma LinearPMap.resolventSet_eq [source]

```
lemma resolventSet_eq (T : H →ₗ[ℂ] H) :
    ρ T = {z | (T - z • 1).toFun.ker = ⊥ ∧ (T - z • 1).toFun.range = ⊤ ∧ Continuous □( T z)}
```

lemma LinearPMap.mem_resolventSet_iff [source]

```
lemma mem_resolventSet_iff {T : H →ₗ[ℂ] H} {z : ℂ} :
    z ∈ ρ T ↔ (T - z • 1).toFun.ker = ⊥ ∧ (T - z • 1).toFun.range = ⊤ ∧ Continuous □( T z)
```

lemma LinearPMap.resolventSet_eq_empty [source]

If an operator is not closed then its resolvent set is empty.

```
lemma resolventSet_eq_empty [CompleteSpace H] {T : H →ₗ[ℂ] H} (h : ¬T.IsClosed) : ρ T = ∅
```

lemma LinearPMap.resolventSet_subset_regularityDomain [source]

```
lemma resolventSet_subset_regularityDomain (T : H →ₗ[ℂ] H) : ρ T ⊆ T.regularityDomain
```

lemma LinearPMap.IsClosed.resolventSet_eq [source]

For a closed operator the continuity of the resolvent is redundant in the definition of the resolvent set.

```
lemma IsClosed.resolventSet_eq [CompleteSpace H] {T : H →ₗ[ℂ] H} (hT : T.IsClosed) :
    ρ T = {z : ℂ | (T - z • 1).toFun.ker = ⊥ ∧ (T - z • 1).toFun.range = ⊤}
```

lemma LinearPMap.IsClosed.resolventSet_eq' [source]

For a closed operator the resolvent set consists of those regular points for which the defect number is zero.

```
lemma IsClosed.resolventSet_eq' [CompleteSpace H] {T : H →ₗ[ℂ] H} (hT : T.IsClosed) :
    ρ T = T.regularityDomain ∩ T.defectNumber ⁻¹' {0}
```

lemma LinearPMap.resolventSet_isOpen [source]

The resolvent set is an open subset of $\mathbb{C}$.

```
lemma resolventSet_isOpen [CompleteSpace H] (T : H →ₗ[ℂ] H) : IsOpen ρ( T)
```

def LinearPMap.spectrum [source]

The spectrum, 'σ', of a partial linear map.

'σ T' is the complement of 'ρ T'. A complex number 'z' is in 'σ T' iff the linear map 'T - z • 1' from 'T.domain' to 'H' fails to be bijective or '(T - z • 1)⁻¹' is not continuous (equivalently, is not bounded).

```
def spectrum (T : H →ₗ[ℂ] H) : Set ℂ
```

lemma LinearPMap.spectrum_eq [source]

```
lemma spectrum_eq (T : H →ₗ[ℂ] H) : σ T = ρ( T)ᶜ
```

lemma LinearPMap.mem_spectrum_iff [source]

```
lemma mem_spectrum_iff {T : H →ₗ[ℂ] H} {z : ℂ} :
    z ∈ σ T ↔ (T - z • 1).toFun.ker ≠ ⊥ ∨ (T - z • 1).toFun.range ≠ ⊤ ∨ ¬Continuous □( T z)
```

lemma LinearPMap.spectrum_eq_univ [source]

If an operator is not closed then its spectrum is all of $\mathbb{C}$.

```
lemma spectrum_eq_univ [CompleteSpace H] {T : H →ₗ[ℂ] H} (h : ¬T.IsClosed) : σ T = univ
```

lemma LinearPMap.spectrum_isClosed [source]

The spectrum is a closed subset of $\mathbb{C}$.

```
lemma spectrum_isClosed [CompleteSpace H] (T : H →ₗ[ℂ] H) : _root_.IsClosed σ( T)
```

def LinearPMap.pointSpectrum [source]

The point spectrum, 'σ^p', of a partial linear map.

A complex number 'z' is in 'σ^p T' iff 'T - z • 1' is not injective.

```
def pointSpectrum (T : H →ₗ[ℂ] H) : Set ℂ
```

lemma LinearPMap.pointSpectrum_eq [source]

```
lemma pointSpectrum_eq (T : H →ₗ[ℂ] H) : σₚ T = {z | (T - z • 1).toFun.ker ≠ ⊥}
```

lemma LinearPMap.mem_pointSpectrum_iff [source]

```
lemma mem_pointSpectrum_iff {T : H →ₗ[ℂ] H} {z : ℂ} : z ∈ σₚ T ↔ (T - z • 1).toFun.ker ≠ ⊥
```

lemma LinearPMap.pointSpectrum_subset_spectrum [source]

```
lemma pointSpectrum_subset_spectrum (T : H →ₗ[ℂ] H) : σₚ T ⊆ σ T
```

def LinearPMap.residualSpectrum [source]

The residual spectrum, 'σ^r', of a partial linear map.

A complex number 'z' is in 'σ^r T' iff 'T - z • 1' is injective but not surjective and '$(T - z \bullet 1)^{-1}$' is continuous (equivalently, bounded).

```
def residualSpectrum (T : H →ₗ[ℂ] H) : Set ℂ
```

lemma LinearPMap.residualSpectrum_eq [source]

```
lemma residualSpectrum_eq (T : H →ₗ[ℂ] H) :
    σ_r T = {z | (T - z • 1).toFun.ker = ⊥ ∧ (T - z • 1).toFun.range ≠ ⊤ ∧ Continuous ⇑(T z)}
```

lemma LinearPMap.mem_residualSpectrum_iff [source]

```
lemma mem_residualSpectrum_iff {T : H →ₗ[ℂ] H} {z : ℂ} :
    z ∈ σ_r T ↔ (T - z • 1).toFun.ker = ⊥ ∧ (T - z • 1).toFun.range ≠ ⊤ ∧ Continuous ⇑(T z)
```

lemma LinearPMap.residualSpectrum_subset_spectrum [source]

```
lemma residualSpectrum_subset_spectrum (T : H →ₗ[ℂ] H) : σ_r T ⊆ σ T
```

lemma LinearPMap.residualSpectrum_subset_regularityDomain [source]

```
lemma residualSpectrum_subset_regularityDomain (T : H →ₗ[ℂ] H) : σ_r T ⊆ T.regularityDomain
```

def LinearPMap.continuousSpectrum [source]

The continuous spectrum, ‘σ^c‘, of a partial linear map.

A complex number ‘z‘ is in ‘$\sigma^c\ T$‘ iff the range of ‘$T - z \bullet 1$‘ is not closed.

```
def continuousSpectrum (T : H →ₗ[ℂ] H) : Set ℂ
```

lemma LinearPMap.continuousSpectrum_eq [source]

```
lemma continuousSpectrum_eq (T : H →ₗ[ℂ] H) :
    σ_c T = {z | ¬_root_.IsClosed ((T - z • 1).toFun.range : Set H)}
```

lemma LinearPMap.mem_continuousSpectrum_iff [source]

```
lemma mem_continuousSpectrum_iff {T : H →ₗ[ℂ] H} {z : ℂ} :
    z ∈ σ_c T ↔ ¬_root_.IsClosed ((T - z • 1).toFun.range : Set H)
```

lemma LinearPMap.continuousSpectrum_subset_spectrum [source]

```
lemma continuousSpectrum_subset_spectrum (T : H →ₗ[ℂ] H) : σ_c T ⊆ σ T
```

lemma LinearPMap.IsClosed.spectrum_eq [source]

```
lemma IsClosed.spectrum_eq [CompleteSpace H] {T : H →ₗ[ℂ] H} (hT : T.IsClosed) :
    σ T = σ_p T ∪ σ_r T ∪ σ_c T
```

lemma LinearPMap.pointSpectrum_inter_residualSpectrum [source]

```
lemma pointSpectrum_inter_residualSpectrum (T : H →ₗ[ℂ] H) : σ_p T ∩ σ_r T = ∅
```

lemma LinearPMap.resolvent_sub [source]

```
lemma resolvent_sub
    {₁T ₂T : H →ₗ[ℂ] H} (hT : ₂T.domain ≤ ₁T.domain) {z : ℂ} (₁hz : z ∈ ρ ₁T) (₂hz : z ∈ ρ ₂T)
     :
    □ ₁T z - □ ₂T z = □ ₁T z * (₂T - ₁T) * □ ₂T z
```

lemma LinearPMap.resolvent_sub' [source]

```
lemma resolvent_sub' {T : H →ₗ[ℂ] H} (₁z ₂z : ℂ) (₁hz : ₁z ∈ ρ T) (₂hz : ₂z ∈ ρ T) :
    □ T ₁z - □ T ₂z = (₁z - ₂z) • □( T ₁z * □ T ₂z)
```

2.27 Physlib.QuantumMechanics.DDimensions.Operators.SpectralTheory.SelfAdjoint

Physlib/QuantumMechanics/DDimensions/Operators/SpectralTheory/SelfAdjoint.lean on GitHub.

lemma LinearPMap.IsSelfAdjoint.resolventSet_eq_regularityDomain [source]

```
lemma resolventSet_eq_regularityDomain : ρ T = T.regularityDomain
```

lemma LinearPMap.IsSelfAdjoint.mem_resolventSet_of_range_eq_top [source]

'(T - z • 1).range = ⊤' is a sufficient condition for 'z ∈ ρ T' (and it is a necessary condition by definition of 'ρ').

```
lemma mem_resolventSet_of_range_eq_top {z : ℂ} (h : (T - z • 1).toFun.range = ⊤) : z ∈ ρ T
```

lemma LinearPMap.IsSelfAdjoint.spectrum_real [source]

The spectrum of a self-adjoint operator is real.

```
lemma spectrum_real : σ T ⊆ range ofReal
```

lemma LinearPMap.IsSelfAdjoint.residualSpectrum_eq_empty [source]

The residual spectrum of a self-adjoint operator is empty.

```
lemma residualSpectrum_eq_empty : σʳ T = ∅
```

2.28 Physlib.QuantumMechanics.DDimensions.Operators.SpectralTheory.SpectralMeasure

Physlib/QuantumMechanics/DDimensions/Operators/SpectralTheory/SpectralMeasure.lean on GitHub.

instance «instance@38b2a7ba» [source]

```
instance (H : Type*) [SeminormedAddCommGroup H] [InnerProductSpace ℂ H] :
    IsAddTorsionFree (H →L[ℂ] H) where
```

structure SpectralMeasure [source]

A _spectral measure_ on a measurable space 'α' is a σ-additive function 'Set α → H →L[ℂ] H' such that each set is mapped to a star projection on 'H', the empty set and non-measurable sets are mapped to zero, and 'univ' is mapped to the identity.

```
structure SpectralMeasure
    α( : Type*) [MeasurableSpace α]
    (H : Type*) [NormedAddCommGroup H] [InnerProductSpace ℂ H] [CompleteSpace H]
    extends VectorMeasure α (H →L[ℂ] H) where
  isStarProjection' : ∀ A, IsStarProjection (measureOf' A)
  univ' : measureOf' univ = 1
```

instance SpectralMeasure.instCoeVectorMeasure [source]

```
instance instCoeVectorMeasure : Coe (SpectralMeasure α H) (VectorMeasure α (H →L[ℂ] H))
```

instance SpectralMeasure.instCoeFun [source]

```
instance instCoeFun : CoeFun (SpectralMeasure α H) fun _ ↦ Set α → H →L[ℂ] H
```

lemma SpectralMeasure.isStarProjection [source]

```
lemma isStarProjection (A : Set α) : IsStarProjection μ(S A)
```

lemma SpectralMeasure.univ [source]

```
@[simp]
lemma univ : μS univ = 1
```

lemma SpectralMeasure.comp_self [source]

```
@[simp]
lemma comp_self (A : Set α) : μS A ∘L μS A = μS A
```

lemma SpectralMeasure.comp_of_disjoint [source]

```
lemma comp_of_disjoint
    {A B : Set α} (h : Disjoint A B) (hA : MeasurableSet A) (hB : MeasurableSet B) :
    μS A ∘L μS B = 0
```

lemma SpectralMeasure.comp_eq_of_inter [source]

```
lemma comp_eq_of_inter {A B : Set α} (hA : MeasurableSet A) (hB : MeasurableSet B) :
    μS A ∘L μS B = μS (A ∩ B)
```

lemma SpectralMeasure.commute [source]

```
lemma commute (A B : Set α) : Commute μ(S A) μ(S B)
```

2.29 Physlib.QuantumMechanics.DDimensions.Operators.SpectralTheory.Symmetric

Physlib/QuantumMechanics/DDimensions/Operators/SpectralTheory/Symmetric.lean on GitHub.

def LinearPMap.IsSymmetric.realNumericalRange [source]

The projection of the numerical range onto the real axis.

```
def realNumericalRange (T : H →ₗ[ℂ] H) : Set ℝ
```

lemma LinearPMap.IsSymmetric.realNumericalRange_eq [source]

```
lemma realNumericalRange_eq (T : H →ₗ[ℂ] H) : 0ʳᵉ T = re '' 0 T
```

lemma LinearPMap.IsSymmetric.realNumericalRange_neg [source]

```
@[simp]
lemma realNumericalRange_neg (T : H →ₗ[ℂ] H) : 0ʳᵉ (-T) = 0ʳᵉ- T
```

lemma LinearPMap.IsSymmetric.im_eq_zero_of_mem_numericalRange [source]

The numerical range of a symmetric operator is contained in the real axis.

```
lemma im_eq_zero_of_mem_numericalRange {z : ℂ} (hz : z ∈ 0 T) : z.im = 0
```

lemma LinearPMap.IsSymmetric.numericalRange_subset [source]

The numerical range of a symmetric operator is contained in the real axis.

```
lemma numericalRange_subset : 0 T ⊆ range ofReal
```

lemma LinearPMap.IsSymmetric.numericalRange_eq [source]

The numerical range of a symmetric operator is equal to its projection onto the real axis.

```
lemma numericalRange_eq : 0 T = ofReal '' 0ʳᵉ T
```

lemma LinearPMap.IsSymmetric.closure_numericalRange_subset [source]

```
lemma closure_numericalRange_subset : _root_.closure 0( T) ⊆ range ofReal
```

lemma LinearPMap.IsSymmetric.mem_regularityDomain_of_im_ne_zero [source]

The regularity domain of a symmetric operator contains all complex numbers with non-zero imaginary part.

```
lemma mem_regularityDomain_of_im_ne_zero {z : ℂ} (hz : z.im ≠ 0) : z ∈ T.regularityDomain
```

lemma LinearPMap.IsSymmetric.compl_ofReal_subset_regularityDomain [source]

The regularity domain of a symmetric operator contains all complex numbers with non-zero imaginary part.

```
lemma compl_ofReal_subset_regularityDomain : (range ofReal)ᶜ ⊆ T.regularityDomain
```

lemma LinearPMap.IsSymmetric.Iio_subset_regularityDomain [source]

If ‘m‘ is a lower bound on the numerical range then the regularity domain contains ‘(-∞,m)‘.

```
lemma Iio_subset_regularityDomain {m : ℝ} (h : m ∈ lowerBounds θ_re( T)) :
    ofReal '' Iio m ⊆ T.regularityDomain
```

lemma LinearPMap.IsSymmetric.Ioi_subset_regularityDomain [source]

If ‘m‘ is an upper bound on the numerical range then the regularity domain contains ‘(m,∞)‘.

```
lemma Ioi_subset_regularityDomain {m : ℝ} (h : m ∈ upperBounds θ_re( T)) :
    ofReal '' Ioi m ⊆ T.regularityDomain
```

lemma LinearPMap.IsSymmetric.regularityDomain_isConnected_iff [source]

The regularity domain of a symmetric operator is connected iff it contains a real number.

```
lemma regularityDomain_isConnected_iff :
    IsConnected T.regularityDomain ↔ (ofReal ⁻¹' T.regularityDomain).Nonempty
```

lemma LinearPMap.IsSymmetric.regularityDomain_isConnected_of_bddBelow [source]

The regularity domain of a lower semibounded symmetric operator is connected.

```
lemma regularityDomain_isConnected_of_bddBelow (h : BddBelow θ_re( T)) :
    IsConnected T.regularityDomain
```

lemma LinearPMap.IsSymmetric.regularityDomain_isConnected_of_bddAbove [source]

The regularity domain of an upper semibounded symmetric operator is connected.

```
lemma regularityDomain_isConnected_of_bddAbove (h : BddAbove θ_re( T)) :
    IsConnected T.regularityDomain
```

lemma LinearPMap.IsSymmetric.pointSpectrum_real [source]

Eigenvalues of a symmetric unbounded operator are real.

```
lemma pointSpectrum_real : σ_p T ⊆ range ofReal
```

2.30 Physlib.QuantumMechanics.DDimensions.Operators.Unbounded

Physlib/QuantumMechanics/DDimensions/Operators/Unbounded.lean on GitHub.

lemma LinearPMap.HasDenseDomain.orthogonal_range [source]

‘U.range$^{\perp}$ = U†.ker‘

c.f. ‘LinearMap.orthogonal_range‘ and ‘ContinuousLinearMap.orthogonal_range‘

```
lemma HasDenseDomain.orthogonal_range [CompleteSpace H] (h : U.HasDenseDomain) :
    U.toFun.□range = †U.toFun.ker.map †U.domain.subtype
```

lemma LinearPMap.HasDenseDomain.orthogonal_adjoint_ker [source]

‘U†.ker$^{\perp}$ = U.range.closure‘

```
lemma HasDenseDomain.orthogonal_adjoint_ker [CompleteSpace H] [CompleteSpace H']
    (h : U.HasDenseDomain) :
    (†U.toFun.ker.map †U.domain.subtype)□ = U.toFun.range.closure
```

lemma LinearPMap.IsClosed.closure_eq [source]

```
lemma IsClosed.closure_eq (h : U.IsClosed) : U.closure = U
```

lemma LinearPMap.adjoint_zero [source]

```
@[simp]
lemma adjoint_zero [CompleteSpace H] : (0 : H →ₗ[ℂ] H')† = 0
```

lemma LinearPMap.adjoint_sub_le_sub_adjoint [source]

```
lemma adjoint_sub_le_sub_adjoint [CompleteSpace H]
    (₁U ₂U : H →ₗ[ℂ] H') (₁₂h : (₁U - ₂U).HasDenseDomain) : ₁†U - ₂†U ≤ (₁U - ₂U)†
```

lemma LinearMap.continuous_iff_bounded [source]

*‘f : E →ₗ[𝕜] F‘ is continuous iff there exists ‘M > 0‘ s.t. ‘‖f x‖ ≤ M * ‖x‖‘ for all ‘x : E‘.*

This is a (convenient) immediate consequence of ‘IsBoundedLinearMap.isLinearMap_and_continuous_iff

```
lemma _root_.LinearMap.continuous_iff_bounded □{ E F : Type*} [NontriviallyNormedField □]
    [SeminormedAddCommGroup E] [NormedSpace □ E] [SeminormedAddCommGroup F] [NormedSpace □ F]
    {f : E →ₗ□[] F} : Continuous f ↔ ∃ M, 0 < M ∧ ∀ x : E, ‖f ‖x ≤ M * ‖‖x
```

lemma LinearPMap.isClosable_of_continuous [source]

Continuous operators are closable.

```
lemma isClosable_of_continuous (h : Continuous U) : U.IsClosable
```

lemma LinearPMap.closure_domain_eq_domain_closure_of_continuous [source]

A strengthening of ‘closure_domain_le_domain_closure‘ for continuous operators.

```
lemma closure_domain_eq_domain_closure_of_continuous [CompleteSpace H'] (h : Continuous U) :
    U.closure.domain = U.domain.closure
```

lemma LinearPMap.isClosed_iff_isClosed_domain_of_continuous [source]

A continuous operator is closed iff its domain is closed.

```
lemma isClosed_iff_isClosed_domain_of_continuous [CompleteSpace H'] (h : Continuous U) :
    U.IsClosed ↔ _root_.IsClosed (U.domain : Set H)
```

lemma LinearPMap.IsClosed.isClosed_toFun_graph [source]

```
lemma IsClosed.isClosed_toFun_graph (hU : U.IsClosed) :
    _root_.IsClosed (U.toFun.graph : Set (U.domain × H'))
```

lemma LinearPMap.IsClosed.continuous_of_isClosed_domain [source]

*The **closed graph theorem** for partial linear maps: a closed operator with closed domain is continuous.*

This follows immediately from ‘LinearMap.continuous_of_isClosed_graph‘ and the completeness of ‘H‘ and ‘H'‘.

```
lemma IsClosed.continuous_of_isClosed_domain [CompleteSpace H] [CompleteSpace H']
    (hU : U.IsClosed) (h : _root_.IsClosed (U.domain : Set H)) :
    Continuous U
```

lemma LinearPMap.IsClosable.add_continuous [source]

Closability is preserved upon adding a continuous operator.

```
lemma IsClosable.add_continuous
    (₁h : ₁U.IsClosable) (₂h : Continuous ₂U) (h : ₁U.domain ≤ ₂U.domain) :
    (₁U + ₂U).IsClosable
```

lemma LinearPMap.IsClosable.sub_continuous [source]

Closability is preserved upon subtracting a continuous operator.

```
lemma IsClosable.sub_continuous
    (₁h : ₁U.IsClosable) (₂h : Continuous ₂U) (h : ₁U.domain ≤ ₂U.domain) : (₁U - ₂U).
    IsClosable
```

lemma LinearPMap.IsClosed.add_continuous [source]

Closedness is preserved upon adding a continuous operator.

```
lemma IsClosed.add_continuous [CompleteSpace H']
    (₁h : ₁U.IsClosed) (₂h : Continuous ₂U) (h : ₁U.domain ≤ ₂U.domain) : (₁U + ₂U).IsClosed
```

lemma LinearPMap.IsClosed.sub_continuous [source]

Closedness is preserved upon subtracting a continuous operator.

```
lemma IsClosed.sub_continuous [CompleteSpace H']
    (₁h : ₁U.IsClosed) (₂h : Continuous ₂U) (h : ₁U.domain ≤ ₂U.domain) : (₁U - ₂U).IsClosed
```

lemma LinearPMap.IsUnbounded.orthogonal_adjoint_range [source]

‘$U\dagger.\mathrm{range}^{\perp} = U.\mathrm{closure}.\ker$‘

```
lemma IsUnbounded.orthogonal_adjoint_range [CompleteSpace H] [CompleteSpace H']
    (h : U.IsUnbounded) : †U.toFun.□range = U.closure.toFun.ker.map U.closure.domain.subtype
```

lemma LinearPMap.IsUnbounded.orthogonal_closure_ker [source]

'$U.closure.ker^{\perp} = U\dagger.range$'

```
lemma IsUnbounded.orthogonal_closure_ker [CompleteSpace H] [CompleteSpace H'] (h : U.
    IsUnbounded) :
    (U.closure.toFun.ker.map U.closure.domain.subtype)ᗮ = †U.toFun.range.closure
```

2.31 Physlib.QuantumMechanics.FiniteTarget.HilbertSpace

Physlib/QuantumMechanics/FiniteTarget/HilbertSpace.lean on GitHub.

def QuantumMechanics.FiniteHilbertSpace.equivEuclidean [source]

The equivalence between 'FiniteHilbertSpace d' and 'EuclideanSpace ℂ d' given by 'val'.

```
def equivEuclidean : FiniteHilbertSpace d ≃ EuclideanSpace ℂ d where
```

instance QuantumMechanics.FiniteHilbertSpace.«instance@89dd6b4e» [source]

```
noncomputable instance : AddCommGroup (FiniteHilbertSpace d)
```

instance QuantumMechanics.FiniteHilbertSpace.«instance@ba152d22» [source]

```
noncomputable instance : Module ℂ (FiniteHilbertSpace d)
```

lemma QuantumMechanics.FiniteHilbertSpace.val_add [source]

```
@[simp]
lemma val_add ψ( φ : FiniteHilbertSpace d) : ψ( + φ).val = ψ.val + φ.val
```

lemma QuantumMechanics.FiniteHilbertSpace.val_smul [source]

```
@[simp]
lemma val_smul (c : ℂ) ψ( : FiniteHilbertSpace d) : (c • ψ).val = c • ψ.val
```

lemma QuantumMechanics.FiniteHilbertSpace.val_zero [source]

```
@[simp]
lemma val_zero : (0 : FiniteHilbertSpace d).val = 0
```

def QuantumMechanics.FiniteHilbertSpace.linearEquivEuclidean [source]

The equivalence between 'FiniteHilbertSpace d' and 'EuclideanSpace ℂ d' as a 'ℂ'-linear equivalence, upgrading 'equivEuclidean'.

```
noncomputable def linearEquivEuclidean : FiniteHilbertSpace d ≃ₗ[ℂ] EuclideanSpace ℂ d
```

instance QuantumMechanics.FiniteHilbertSpace.«instance@a3aa9745» [source]

```
noncomputable instance : NormedAddCommGroup (FiniteHilbertSpace d)
```

lemma QuantumMechanics.FiniteHilbertSpace.norm_eq_val [source]

```
@[simp]
lemma norm_eq_val ψ( : FiniteHilbertSpace d) : ‖ψ‖ = ‖ψ.‖val
```

instance QuantumMechanics.FiniteHilbertSpace.«instance@d6d3ba74» [source]

```
noncomputable instance : InnerProductSpace ℂ (FiniteHilbertSpace d)
```

lemma QuantumMechanics.FiniteHilbertSpace.inner_eq_val [source]

```
@[simp]
lemma inner_eq_val ψ( φ : FiniteHilbertSpace d) : inner ℂ ψ φ = inner ℂ ψ.val φ.val
```

instance QuantumMechanics.FiniteHilbertSpace.«instance@df6e240e» [source]

```
instance : FiniteDimensional ℂ (FiniteHilbertSpace d)
```

instance QuantumMechanics.FiniteHilbertSpace.«instance@25da29e8» [source]

```
instance : CompleteSpace (FiniteHilbertSpace d)
```

def QuantumMechanics.FiniteHilbertSpace.isometryEquivEuclidean [source]

The equivalence between 'FiniteHilbertSpace d' and 'EuclideanSpace ℂ d' as a linear isometry equivalence, upgrading 'linearEquivEuclidean'.

```
noncomputable def isometryEquivEuclidean : FiniteHilbertSpace d ≃ₗᵢ[ℂ] EuclideanSpace ℂ d where
```

def QuantumMechanics.FiniteHilbertSpace.basisFun [source]

The standard orthonormal basis of 'FiniteHilbertSpace d', indexed by 'd'.

```
noncomputable def basisFun (d : Type*) [Fintype d] [DecidableEq d] :
    OrthonormalBasis d ℂ (FiniteHilbertSpace d)
```

lemma QuantumMechanics.FiniteHilbertSpace.basisFun_apply [source]

```
lemma basisFun_apply (i : d) : basisFun d i = ⟨EuclideanSpace.single i ⟩1
```

2.32 Physlib.Relativity.LorentzGroup.Basic

Physlib/Relativity/LorentzGroup/Basic.lean on GitHub.

lemma LorentzGroup.continuous_self_mul_dual [source]

```
lemma continuous_self_mul_dual {d : ℕ} :
    Continuous (fun Λ : Matrix (Fin 1 ⊕ Fin d) (Fin 1 ⊕ Fin d) ℝ => Λ * dual Λ)
```

lemma LorentzGroup.isClosed [source]

The Lorentz group is closed in the ambient matrix space.

```
lemma isClosed (d : ℕ) :
    IsClosed (LorentzGroup d : Set (Matrix (Fin 1 ⊕ Fin d) (Fin 1 ⊕ Fin d) ℝ))
```

lemma LorentzGroup.isClosedEmbedding_val [source]

The coercion from the Lorentz group to its ambient matrix space is a closed embedding.

```
lemma isClosedEmbedding_val {d : ℕ} :
    Topology.IsClosedEmbedding
      (Subtype.val : LorentzGroup d → Matrix (Fin 1 ⊕ Fin d) (Fin 1 ⊕ Fin d) ℝ)
```

2.33 Physlib.Relativity.Tensors.Basic

Physlib/Relativity/Tensors/Basic.lean on GitHub.

lemma TensorSpecies.Tensor.Pure.update_eq_function_update [source]

```
lemma update_eq_function_update {n : ℕ} {c : Fin n → C} [inst : DecidableEq (Fin n)]
    (p : Pure S c) (i : Fin n)
    (x : V (c i)) : update p i x = Function.update p i x
```

lemma TensorSpecies.Tensor.Pure.update_diff [source]

```
lemma update_diff {n : ℕ} {c : Fin n → C} [inst : DecidableEq (Fin n)] (p : Pure S c) (i j :
    Fin n)
    (x : V (c i)) (hij : i ≠ j) : (update p i x) j = p j
```

lemma TensorSpecies.Tensor.componentMap_eq_repr [source]

'componentMap' is the basis representation '(basis c).repr' definitionally; this bridges the two notations wherever a component computation meets a 'repr' rewrite.

```
lemma componentMap_eq_repr {n : ℕ} (c : Fin n → C) (t : S.Tensor c)
    ψ( : ComponentIdx (S := S) c) : componentMap c t ψ = (basis c).repr t ψ
```

lemma TensorSpecies.Tensor.componentMap_ext [source]

Two tensors with the same colour sequence are equal when all their components agree.

```
lemma componentMap_ext {n : ℕ} {c : Fin n → C} {₁t ₂t : S.Tensor c}
    (h : ∀ b, componentMap c ₁t b = componentMap c ₂t b) : ₁t = ₂t
```

instance TensorSpecies.Tensor.«instance@180d6a48» [source]

```
instance {k : Type} [Field k] {C : Type} {G : Type} [Group G]
    {V : C → Type} ∀[ c, AddCommGroup (V c)] ∀[ c, Module k (V c)]
    {basisIdx : C → Type} ∀[ c, Fintype (basisIdx c)] ∀[ c, DecidableEq (basisIdx c)]
    {rep : (c : C) → Representation k G (V c)} {b : (c : C) → Basis (basisIdx c) k (V c)}
    (S : TensorSpecies k C G V basisIdx rep b)
    {c : Fin n → C} : FiniteDimensional k (S.Tensor c)
```

instance TensorSpecies.Tensor.«instance@357a4078» [source]

```
noncomputable instance {k : Type} [RCLike k] {C : Type} {G : Type} [Group G]
    {V : C → Type} ∀[ c, AddCommGroup (V c)] ∀[ c, Module k (V c)]
    {basisIdx : C → Type} ∀[ c, Fintype (basisIdx c)] ∀[ c, DecidableEq (basisIdx c)]
    {rep : (c : C) → Representation k G (V c)} {b : (c : C) → Basis (basisIdx c) k (V c)}
    (S : TensorSpecies k C G V basisIdx rep b)
    {c : Fin n → C} : TopologicalSpace (S.Tensor c)
```

instance TensorSpecies.Tensor.«instance@958788b8» [source]

```
instance {k : Type} [RCLike k] {C : Type} {G : Type} [Group G]
    {V : C → Type} ∀[ c, AddCommGroup (V c)] ∀[ c, Module k (V c)]
    {basisIdx : C → Type} ∀[ c, Fintype (basisIdx c)] ∀[ c, DecidableEq (basisIdx c)]
    {rep : (c : C) → Representation k G (V c)} {b : (c : C) → Basis (basisIdx c) k (V c)}
    (S : TensorSpecies k C G V basisIdx rep b)
    {c : Fin n → C} : IsTopologicalAddGroup (S.Tensor c)
```

lemma TensorSpecies.Tensor.Pure.rep_cast [source]

```
lemma rep_cast {g : G} {c c1 : C} (p : V c) (h : c = c1) :
    LinearEquiv.cast (R := k) h (rep c g p) = rep c1 g (LinearEquiv.cast (R := k) h p)
```

lemma TensorSpecies.Tensor.Pure.actionP_cast [source]

```
lemma actionP_cast {g : G} {p : Pure S c} (h : c = c1) :
    LinearEquiv.cast (R := k) h (g • p) = g • LinearEquiv.cast (R := k) h p
```

instance TensorSpecies.Tensor.«instance@713c1a5c» [source]

```
noncomputable instance : SMul G (S.Tensor c) where
```

instance TensorSpecies.Tensor.«instance@62e8adcb» [source]

```
noncomputable instance : DistribMulAction G (S.Tensor c) where
```

lemma TensorSpecies.Tensor.Pure.permP_equivariant [source]

```
lemma Pure.permP_equivariant {n m : ℕ} {c : Fin n → C} {c1 : Fin m → C}
    σ{ : Fin m → Fin n} (h : IsReindexing c c1 σ) (g : G) (p : Pure S c) :
    Pure.permP σ h (g • p) = g • Pure.permP σ h p
```

lemma TensorSpecies.Tensor.toField_injective [source]

```
lemma toField_injective {c : Fin 0 → C} :
    Function.Injective (toField : S.Tensor c → k)
```

lemma TensorSpecies.Tensor.toField_basis [source]

```
lemma toField_basis {c : Fin 0 → C} (b : ComponentIdx (S := S) c) :
    toField (basis c b) = 1
```

lemma TensorSpecies.Tensor.eq_smul_toField [source]

```
lemma eq_smul_toField {c : Fin 0 → C} (t : Tensor S c) :
    t = toField t • (basis c (@default (ComponentIdx (S := S) c) Unique.instInhabited))
```

2.34 Physlib.Relativity.Tensors.ComplexTensor.Basic

Physlib/Relativity/Tensors/ComplexTensor/Basic.lean on GitHub.

abbrev complexLorentzTensor.modules [source]

The modules associated with each of the different types of complex Lorentz vector space.

```
abbrev modules : Color → Type
  | Color.upL => Fermion.LeftHandedWeyl
  | Color.downL => Fermion.DualLeftHandedWeyl
  | Color.upR => Fermion.RightHandedWeyl
  | Color.downR => Fermion.DualRightHandedWeyl
  | Color.up => Lorentz.ℂContrModule
  | Color.down => Lorentz.ℂCoModule
```

instance complexLorentzTensor.modulesAddCommGroup [source]

```
instance modulesAddCommGroup : ∀ c, AddCommGroup (modules c)
  | Color.upL => inferInstance
  | Color.downL => inferInstance
  | Color.upR => inferInstance
  | Color.downR => inferInstance
  | Color.up => inferInstance
  | Color.down => inferInstance
```

instance complexLorentzTensor.modulesModule [source]

```
noncomputable instance modulesModule : ∀ c, Module ℂ (modules c)
  | Color.upL => inferInstance
  | Color.downL => inferInstance
  | Color.upR => inferInstance
  | Color.downR => inferInstance
  | Color.up => inferInstance
  | Color.down => inferInstance
```

abbrev complexLorentzTensor.basis [source]

The basis associated with each of the different types of complex Lorentz vector space.

```
abbrev basis (c : Color) : Module.Basis (Fin (repDim c)) ℂ (modules c)
```

abbrev complexLorentzTensor.rep [source]

The reps associated with each of the different types of complex Lorentz vector space.

```
abbrev rep (c : Color) : Representation ℂ SL(2, ℂ) (modules c)
```

lemma complexLorentzTensor.contrPCoeff_basis [source]

```
lemma contrPCoeff_basis {n : ℕ} {c : Fin n → complexLorentzTensor.Color} (i j : Fin n)
    (hij : i ≠ j ∧ (complexLorentzTensorτ. (c i) = c j))
    (b : ComponentIdx (S := complexLorentzTensor) c) :
    Pure.contrPCoeff i j hij (Pure.basisVector c b) = if b i =
      Fin.cast (by simp ←[ hij.2, repDim_tau]) (b j)
    then 1 else 0
```

2.35 Physlib.Relativity.Tensors.ComplexTensor.Metrics.Basic

Physlib/Relativity/Tensors/ComplexTensor/Metrics/Basic.lean on GitHub.

abbrev complexLorentzTensor.dualLeftMetric [source]

The metric ‘ε_{aa}‘ as a complex Lorentz tensor.

```
abbrev dualLeftMetric : ℂT[.downL, .downL]
```

abbrev complexLorentzTensor.dualRightMetric [source]

The metric ‘ε_{dot a}_{dot a}‘ as a complex Lorentz tensor.

```
abbrev dualRightMetric : ℂT[.downR, .downR]
```

lemma complexLorentzTensor.dualLeftMetric_eq_fromConstPair [source]

```
lemma dualLeftMetric_eq_fromConstPair : εL' = fromConstPair Fermion.dualLeftMetric
```

lemma complexLorentzTensor.dualRightMetric_eq_fromConstPair [source]

```
lemma dualRightMetric_eq_fromConstPair : εR' = fromConstPair Fermion.dualRightMetric
```

lemma complexLorentzTensor.dualLeftMetric_eq_fromPairT [source]

```
lemma dualLeftMetric_eq_fromPairT : εL' = fromPairT (Fermion.dualLeftMetricVal)
```

lemma complexLorentzTensor.dualRightMetric_eq_fromPairT [source]

```
lemma dualRightMetric_eq_fromPairT : εR' = fromPairT (Fermion.dualRightMetricVal)
```

lemma complexLorentzTensor.dualLeftMetric_eq_dualLeftBasis [source]

```
lemma dualLeftMetric_eq_dualLeftBasis : εL' =
    fromPairT (dualLeftBasis 0 ⊗ₜ[ℂ] dualLeftBasis 1)
    - fromPairT (dualLeftBasis 1 ⊗ₜ[ℂ] dualLeftBasis 0)
```

lemma complexLorentzTensor.dualRightMetric_eq_dualRightBasis [source]

```
lemma dualRightMetric_eq_dualRightBasis : εR' =
    fromPairT (dualRightBasis 0 ⊗ₜ[ℂ] dualRightBasis 1)
    - fromPairT (dualRightBasis 1 ⊗ₜ[ℂ] dualRightBasis 0)
```

lemma complexLorentzTensor.dualLeftMetric_eq_basis [source]

```
lemma dualLeftMetric_eq_basis : εL' =
    (Tensor.basis (S := complexLorentzTensor) ![Color.downL, Color.downL]
      (fun | 0 => (0 : Fin 2) | 1 => (1 : Fin 2)))
    - (Tensor.basis (S := complexLorentzTensor)
      ![Color.downL, Color.downL] (fun | 0 => (1 : Fin 2) | 1 => (0 : Fin 2)))
```

lemma complexLorentzTensor.dualRightMetric_eq_basis [source]

```
lemma dualRightMetric_eq_basis : εR' =
    (Tensor.basis (S := complexLorentzTensor)
      ![Color.downR, Color.downR] (fun | 0 => (0 : Fin 2) | 1 => (1 : Fin 2)))
    - (Tensor.basis (S := complexLorentzTensor)
      ![Color.downR, Color.downR] (fun | 0 => (1 : Fin 2) | 1 => (0 : Fin 2)))
```

lemma complexLorentzTensor.dualLeftMetric_eq_ofRat [source]

```
lemma dualLeftMetric_eq_ofRat : εL' = ofRat fun f =>
    if f 0 = Fin.cast (by rfl) (0 : Fin 2) ∧ f 1 = Fin.cast (by rfl) (1 : Fin 2) then 1 else
    if f 1 = Fin.cast (by rfl) (0 : Fin 2) ∧ f 0 = Fin.cast (by rfl) (1 : Fin 2) then
      - 1 else 0
```

lemma complexLorentzTensor.dualRightMetric_eq_ofRat [source]

```
lemma dualRightMetric_eq_ofRat : εR' = ofRat fun f =>
    if f 0 = Fin.cast (by rfl) (0 : Fin 2) ∧ f 1 = Fin.cast (by rfl) (1 : Fin 2) then 1 else
    if f 1 = Fin.cast (by rfl) (0 : Fin 2) ∧ f 0 = Fin.cast (by rfl) (1 : Fin 2) then
      - 1 else 0
```

lemma complexLorentzTensor.actionT_dualLeftMetric [source]

The tensor 'dualLeftMetric' is invariant under the action of 'SL(2,ℂ)'.

```
lemma actionT_dualLeftMetric (g : SL℃(2,)) : g • εL' = εL'
```

lemma complexLorentzTensor.actionT_dualRightMetric [source]

The tensor 'dualRightMetric' is invariant under the action of 'SL(2,ℂ)'.

```
lemma actionT_dualRightMetric (g : SL℃(2,)) : g • εR' = εR'
```

2.36 Physlib.Relativity.Tensors.ComplexTensor.Metrics.Lemmas

Physlib/Relativity/Tensors/ComplexTensor/Metrics/Lemmas.lean on GitHub.

lemma complexLorentzTensor.dualLeftMetric_antisymm [source]

The dual-left metric is antisymmetric '$\{\varepsilon L' \mid \alpha\ \alpha' = -\varepsilon L' \mid \alpha'\ \alpha\}^T$'.

```
lemma dualLeftMetric_antisymm : ε{L' | α α' = - ε(L' | α' α)ᵀ}
```

lemma complexLorentzTensor.dualRightMetric_antisymm [source]

The dual-right metric is antisymmetric ‘{εR’ | β β’ = - εR’ | β’ β}^{T}‘.

```
lemma dualRightMetric_antisymm : ε{R' | α α' = - ε(R' | α' α)ᵀ}
```

lemma complexLorentzTensor.leftMetric_contr_dualLeftMetric [source]

The contraction of the left metric with the dual-left metric is the unit ‘{εL | α β ⊗ εL’ | β γ = δL | α γ}^{T}‘.

```
lemma leftMetric_contr_dualLeftMetric : ε{L | α β ⊗ εL' | β γ = δL | α γᵀ}
```

lemma complexLorentzTensor.rightMetric_contr_dualRightMetric [source]

The contraction of the right metric with the dual-right metric is the unit ‘{εR | α β ⊗ εR’ | β γ = δR | α γ}^{T}‘.

```
lemma rightMetric_contr_dualRightMetric : ε{R | α β ⊗ εR' | β γ = δR | α γᵀ}
```

lemma complexLorentzTensor.dualLeftMetric_contr_leftMetric [source]

The contraction of the dual-left metric with the left metric is the unit ‘{εL’ | α β ⊗ εL | β γ = δL’ | α γ}^{T}‘.

```
lemma dualLeftMetric_contr_leftMetric : ε{L' | α β ⊗ εL | β γ = δL' | α γᵀ}
```

lemma complexLorentzTensor.dualRightMetric_contr_rightMetric [source]

The contraction of the dual-right metric with the right metric is the unit ‘{εR’ | α β ⊗ εR | β γ = δR’ | α γ}^{T}‘.

```
lemma dualRightMetric_contr_rightMetric : ε{R' | α β ⊗ εR | β γ = δR' | α γᵀ}
```

2.37 Physlib.Relativity.Tensors.ComplexTensor.Units.Basic

Physlib/Relativity/Tensors/ComplexTensor/Units/Basic.lean on GitHub.

abbrev complexLorentzTensor.dualLeftLeftUnit [source]

The unit ‘$\delta_a{}^a$‘ as a complex Lorentz tensor.

```
abbrev dualLeftLeftUnit : ℂT[.downL, .upL]
```

abbrev complexLorentzTensor.leftDualLeftUnit [source]

The unit ‘$\delta^a{}_a$‘ as a complex Lorentz tensor.

```
abbrev leftDualLeftUnit : ℂT[.upL, .downL]
```

abbrev complexLorentzTensor.dualRightRightUnit [source]

The unit ‘δ_{dot a}^{dot a}‘ as a complex Lorentz tensor.

```
abbrev dualRightRightUnit : ℂT[.downR, .upR]
```

abbrev complexLorentzTensor.rightDualRightUnit [source]

The unit ‘δ^{dot a}_{dot a}‘ as a complex Lorentz tensor.

```
abbrev rightDualRightUnit : ℂT[.upR, .downR]
```

lemma complexLorentzTensor.dualLeftLeftUnit_eq_fromConstPair [source]

```
lemma dualLeftLeftUnit_eq_fromConstPair : δL' = fromConstPair Fermion.dualLeftLeftUnit
```

lemma complexLorentzTensor.leftDualLeftUnit_eq_fromConstPair [source]

```
lemma leftDualLeftUnit_eq_fromConstPair : δL = fromConstPair Fermion.leftDualLeftUnit
```

lemma complexLorentzTensor.dualRightRightUnit_eq_fromConstPair [source]

```
lemma dualRightRightUnit_eq_fromConstPair : δR' = fromConstPair Fermion.dualRightRightUnit
```

lemma complexLorentzTensor.rightDualRightUnit_eq_fromConstPair [source]

```
lemma rightDualRightUnit_eq_fromConstPair : δR = fromConstPair Fermion.rightDualRightUnit
```

lemma complexLorentzTensor.dualLeftLeftUnit_eq_fromPairT [source]

```
lemma dualLeftLeftUnit_eq_fromPairT : δL' = fromPairT (Fermion.dualLeftLeftUnitVal)
```

lemma complexLorentzTensor.leftDualLeftUnit_eq_fromPairT [source]

```
lemma leftDualLeftUnit_eq_fromPairT : δL = fromPairT (Fermion.leftDualLeftUnitVal)
```

lemma complexLorentzTensor.dualRightRightUnit_eq_fromPairT [source]

```
lemma dualRightRightUnit_eq_fromPairT : δR' = fromPairT (Fermion.dualRightRightUnitVal)
```

lemma complexLorentzTensor.rightDualRightUnit_eq_fromPairT [source]

```
lemma rightDualRightUnit_eq_fromPairT : δR = fromPairT (Fermion.rightDualRightUnitVal)
```

lemma complexLorentzTensor.dualLeftLeftUnit_eq_dualLeftBasis_leftBasis [source]

```
lemma dualLeftLeftUnit_eq_dualLeftBasis_leftBasis : δL' =
    ∑ i, fromPairT (dualLeftBasis i ⊗ₜ[ℂ] leftBasis i)
```

lemma complexLorentzTensor.leftDualLeftUnit_eq_leftBasis_dualLeftBasis [source]

```
lemma leftDualLeftUnit_eq_leftBasis_dualLeftBasis : δL =
    ∑ i, fromPairT (leftBasis i ⊗ₜ[ℂ] dualLeftBasis i)
```

lemma complexLorentzTensor.dualRightRightUnit_eq_dualRightBasis_rightBasis [source]

```
lemma dualRightRightUnit_eq_dualRightBasis_rightBasis : δR' =
    ∑ i, fromPairT (dualRightBasis i ⊗ₜℂ[] rightBasis i)
```

lemma complexLorentzTensor.rightDualRightUnit_eq_rightBasis_dualRightBasis [source]

```
lemma rightDualRightUnit_eq_rightBasis_dualRightBasis : δR =
    ∑ i, fromPairT (rightBasis i ⊗ₜℂ[] dualRightBasis i)
```

lemma complexLorentzTensor.dualLeftLeftUnit_eq_basis [source]

```
lemma dualLeftLeftUnit_eq_basis : δL' =
    ∑ i, Tensor.basis (S := complexLorentzTensor)
      ![Color.downL, Color.upL] (fun | 0 => i | 1 => i)
```

lemma complexLorentzTensor.leftDualLeftUnit_eq_basis [source]

```
lemma leftDualLeftUnit_eq_basis : δL =
    ∑ i, Tensor.basis (S := complexLorentzTensor)
      ![Color.upL, Color.downL] (fun | 0 => i | 1 => i)
```

lemma complexLorentzTensor.dualRightRightUnit_eq_basis [source]

```
lemma dualRightRightUnit_eq_basis : δR' =
    ∑ i, Tensor.basis (S := complexLorentzTensor)
      ![Color.downR, Color.upR] (fun | 0 => i | 1 => i)
```

lemma complexLorentzTensor.rightDualRightUnit_eq_basis [source]

```
lemma rightDualRightUnit_eq_basis : δR =
    ∑ i, Tensor.basis (S := complexLorentzTensor)
      ![Color.upR, Color.downR] (fun | 0 => i | 1 => i)
```

lemma complexLorentzTensor.dualLeftLeftUnit_eq_ofRat [source]

```
lemma dualLeftLeftUnit_eq_ofRat : δL' = ofRat fun f =>
    if f 0 = f 1 then 1 else 0
```

lemma complexLorentzTensor.leftDualLeftUnit_eq_ofRat [source]

```
lemma leftDualLeftUnit_eq_ofRat : δL = ofRat fun f =>
    if f 0 = f 1 then 1 else 0
```

lemma complexLorentzTensor.dualRightRightUnit_eq_ofRat [source]

```
lemma dualRightRightUnit_eq_ofRat : δR' = ofRat fun f =>
    if f 0 = f 1 then 1 else 0
```

lemma complexLorentzTensor.rightDualRightUnit_eq_ofRat [source]

```
lemma rightDualRightUnit_eq_ofRat : δR = ofRat fun f =>
    if f 0 = f 1 then 1 else 0
```

lemma complexLorentzTensor.actionT_dualLeftLeftUnit [source]

The tensor ‘dualLeftLeftUnit‘ is invariant under the action of ‘SL(2,ℂ)‘.

```
lemma actionT_dualLeftLeftUnit (g : SL℃(2,)) : g • δL' = δL'
```

lemma complexLorentzTensor.actionT_leftDualLeftUnit [source]

The tensor ‘leftDualLeftUnit‘ is invariant under the action of ‘SL(2,ℂ)‘.

```
lemma actionT_leftDualLeftUnit (g : SL℃(2,)) : g • δL = δL
```

lemma complexLorentzTensor.actionT_dualRightRightUnit [source]

The tensor ‘dualRightRightUnit‘ is invariant under the action of ‘SL(2,ℂ)‘.

```
lemma actionT_dualRightRightUnit (g : SL℃(2,)) : g • δR' = δR'
```

lemma complexLorentzTensor.actionT_rightDualRightUnit [source]

The tensor ‘rightDualRightUnit‘ is invariant under the action of ‘SL(2,ℂ)‘.

```
lemma actionT_rightDualRightUnit (g : SL℃(2,)) : g • δR = δR
```

2.38 Physlib.Relativity.Tensors.ComplexTensor.Units.Symm

Physlib/Relativity/Tensors/ComplexTensor/Units/Symm.lean on GitHub.

lemma complexLorentzTensor.dualLeftLeftUnit_symm [source]

Swapping indices of ‘dualLeftLeftUnit‘ returns ‘leftDualLeftUnit‘: ‘$\{\delta L' \mid \alpha\ \alpha' = \delta L \mid \alpha'\ \alpha\}^T$‘.

```
lemma dualLeftLeftUnit_symm : δ{L' | α α' = δL | α' α^ᵀ}
```

lemma complexLorentzTensor.leftDualLeftUnit_symm [source]

Swapping indices of ‘leftDualLeftUnit‘ returns ‘dualLeftLeftUnit‘: ‘$\{\delta L \mid \alpha\ \alpha' = \delta L' \mid \alpha'\ \alpha\}^T$‘.

```
lemma leftDualLeftUnit_symm : δ{L | α α' = δL' | α' α^ᵀ}
```

lemma complexLorentzTensor.dualRightRightUnit_symm [source]

Swapping indices of ‘dualRightRightUnit‘ returns ‘rightDualRightUnit‘: ‘$\{\delta R' \mid \beta\ \beta' = \delta R \mid \beta'\ \beta\}^T$‘.

```
lemma dualRightRightUnit_symm : δ{R' | β β' = δR | β' β^ᵀ}
```

lemma complexLorentzTensor.rightDualRightUnit_symm [source]

Swapping indices of 'rightDualRightUnit' returns 'dualRightRightUnit': '{δR | β β' = δR' | β' β}ᵀ'.

```
lemma rightDualRightUnit_symm : δ{R | β β' = δR' | β' β}ᵀ
```

2.39 Physlib.Relativity.Tensors.ComplexTensor.Vector.Pre.Basic

Physlib/Relativity/Tensors/ComplexTensor/Vector/Pre/Basic.lean on GitHub.

lemma Lorentz.CoℂModule.SL2CRep_val [source]

```
lemma ℂCoModule.SL2CRep_val (M : SL℃(2,)) (v : ℂCoModule) :
    ((ℂCoModule.SL2CRep M) v).val =
    (LorentzGroup.toComplex (SL2C.toLorentzGroup M))⁻ᵀ¹ ᵥ* v.val
```

def Lorentz.inclCoRealLorentz [source]

The semilinear map including real Lorentz co-vectors into complex covariant Lorentz vectors.

```
def inclCoRealLorentz : CoMod 3 →ₛₗ[Complex.ofRealHom] ℂCoModule where
```

lemma Lorentz.inclCoRealLorentz_val [source]

```
lemma inclCoRealLorentz_val (v : CoMod 3) :
    (inclCoRealLorentz v).val = ofRealHom ∘ v.ℝtoFin1d
```

lemma Lorentz.complexCoBasis_of_real [source]

```
lemma complexCoBasis_of_real (i : Fin 1 ⊕ Fin 3) :
    (complexCoBasis i) = inclCoRealLorentz (CoMod.stdBasis i)
```

lemma Lorentz.inclCoRealLorentz_ρ [source]

```
lemma ρinclCoRealLorentz_ (M : SL(2, ℂ)) (v : CoMod 3) :
    (ℂCoModule.SL2CRep M) (inclCoRealLorentz v) =
    inclCoRealLorentz ((Co 3)ρ. (SL2C.toLorentzGroup M) v)
```

2.40 Physlib.Relativity.Tensors.ComplexTensor.Vector.Pre.Modules

Physlib/Relativity/Tensors/ComplexTensor/Vector/Pre/Modules.lean on GitHub.

lemma Lorentz.CoℂModule.ext [source]

```
@[ext]
lemma ext ψ( ψ' : ℂCoModule) (h : ψ.val = ψ'.val) : ψ = ψ'
```

lemma Lorentz.CoℂModule.val_add [source]

```
@[simp]
lemma val_add ψ( ψ' : ℂCoModule) : ψ( + ψ').val = ψ.val + ψ'.val
```

lemma Lorentz.Co$\mathbb{C}$Module.val_smul [source]

```
@[simp]
lemma val_smul (r : ℂ) ψ( : ℂCoModule) : (r • ψ).val = r • ψ.val
```

2.41 Physlib.Relativity.Tensors.ComplexTensor.Weyl.Basic

Physlib/Relativity/Tensors/ComplexTensor/Weyl/Basic.lean on GitHub.

def Fermion.leftHandedRep [source]

The vector space $\mathbb{C}\hat{}2$ carrying the fundamental representation of SL(2,C). In index notation corresponds to a Weyl fermion with indices $\psi\hat{}a$.

```
def leftHandedRep : Representation ℂ SL(2,) LeftHandedWeyl where
```

def Fermion.dualLeftHandedRep [source]

The vector space $\mathbb{C}\hat{}2$ carrying the representation of SL(2,C) given by $M \to (M^{-1})^T$. In index notation corresponds to a Weyl fermion with indices ψ_a.

```
def dualLeftHandedRep : Representation ℂ SL(2,) DualLeftHandedWeyl where
```

def Fermion.dualLeftBasis [source]

The standard basis on dual-left-handed Weyl fermions.

```
def dualLeftBasis : Basis (Fin 2) ℂ DualLeftHandedWeyl
```

lemma Fermion.dualLeftBasis_toFin2$\mathbb{C}$ [source]

```
@[simp]
lemma ℂdualLeftBasis_toFin2 (i : Fin 2) : (dualLeftBasis i).ℂtoFin2 = Pi.single i 1
```

lemma Fermion.dualLeftBasis_ρ_apply [source]

```
@[simp]
lemma ρdualLeftBasis_apply (M : SL(2,)) (i j : Fin 2) :
    (LinearMap.toMatrix dualLeftBasis dualLeftBasis) (dualLeftHandedRep M) i j = (M⁻.1¹)ᵀ i j
```

def Fermion.rightHandedRep [source]

The vector space $\mathbb{C}\hat{}2$ carrying the conjugate representation of SL(2,C). In index notation corresponds to a Weyl fermion with indices $\psi\hat{}\{dot\ a\}$.

```
def rightHandedRep : Representation ℂ SL(2,) RightHandedWeyl where
```

def Fermion.dualRightHandedRep [source]

The vector space $\mathbb{C}\hat{}2$ carrying the representation of SL(2,C) given by $M \to (M^{-1})\hat{}\dagger$. In index notation this corresponds to a Weyl fermion with index '$\psi\_\{dot\ a\}$'.

```
def dualRightHandedRep : Representation ℂ SL(2,) DualRightHandedWeyl where
```

def Fermion.dualRightBasis [source]

The standard basis on dual-right-handed Weyl fermions.

```
def dualRightBasis : Basis (Fin 2) ℂ DualRightHandedWeyl
```

lemma Fermion.dualRightBasis_toFin2ℂ [source]

```
@[simp]
lemma ℂdualRightBasis_toFin2 (i : Fin 2) : (dualRightBasis i).ℂtoFin2 = Pi.single i 1
```

lemma Fermion.dualRightBasis_ρ_apply [source]

```
@[simp]
lemma ρdualRightBasis_apply (M : SLℂ(2,)) (i j : Fin 2) :
    (LinearMap.toMatrix dualRightBasis dualRightBasis) (dualRightHandedRep M) i j =
    ((M⁻.1¹).conjTranspose) i j
```

def Fermion.leftHandedToDual [source]

The morphism between the representation 'leftHanded' and the representation 'dualLeftHanded' defined by multiplying an element of 'leftHanded' by the matrix 'ε^{a0a1} = !![0, 1; -1, 0]]'.

```
def leftHandedToDual : leftHandedRep.IntertwiningMap dualLeftHandedRep where
```

lemma Fermion.leftHandedToDual_hom_apply [source]

```
lemma leftHandedToDual_hom_apply ψ( : LeftHandedWeyl) :
    leftHandedToDual ψ =
    DualLeftHandedWeyl.ℂtoFin2Equiv.symm (!![0, 1; -1, 0] ᵥ* ψ.ℂtoFin2)
```

def Fermion.leftHandedDualTo [source]

The morphism from 'dualLeftHanded' to 'leftHanded' defined by multiplying an element of DualLeftHandedWeyl by the matrix 'ε_{a1a2} = !![0, -1; 1, 0]'.

```
def leftHandedDualTo : dualLeftHandedRep.IntertwiningMap leftHandedRep where
```

lemma Fermion.leftHandedDualTo_hom_apply [source]

```
lemma leftHandedDualTo_hom_apply ψ( : DualLeftHandedWeyl) :
    leftHandedDualTo ψ =
    LeftHandedWeyl.ℂtoFin2Equiv.symm (!![0, -1; 1, 0] ᵥ* ψ.ℂtoFin2)
```

def Fermion.leftHandedDualEquiv [source]

The equivalence between the representation 'leftHanded' and the representation 'dualLeftHanded' defined by multiplying an element of 'leftHanded' by the matrix 'ε^{a0a1} = !![0, 1; -1, 0]]'.

```
def leftHandedDualEquiv : leftHandedRep.Equiv dualLeftHandedRep
```

lemma Fermion.leftHandedDualEquiv_hom_hom_apply [source]

'leftHandedDualEquiv' acting on an element 'ψ : leftHanded' corresponds to multiplying 'ψ' by the matrix '!![0, 1; -1, 0]'.

```
lemma leftHandedDualEquiv_hom_hom_apply ψ( : LeftHandedWeyl) :
    leftHandedDualEquiv ψ =
    DualLeftHandedWeyl.ℂtoFin2Equiv.symm (!![0, 1; -1, 0] ᵥ* ψ.ℂtoFin2)
```

lemma Fermion.leftHandedDualEquiv_inv_hom_apply [source]

The inverse of 'leftHandedDualEquiv' acting on an element'ψ : dualLeftHanded' corresponds to multiplying 'ψ' by the matrix '!![0, -1; 1, 0]'.

```
lemma leftHandedDualEquiv_inv_hom_apply ψ( : DualLeftHandedWeyl) :
    leftHandedDualEquiv.symm ψ =
    LeftHandedWeyl.ℂtoFin2Equiv.symm (!![0, -1; 1, 0] ᵥ* ψ.ℂtoFin2)
```

informal_definition Fermion.rightHandedWeylDualEquiv [source]

The linear equivalence between 'rightHandedWeyl' and 'DualRightHandedWeyl' given by multiplying an element of 'rightHandedWeyl' by the matrix 'ε^{a0a1} = !![0, 1; -1, 0]]'.

```
informal_definition rightHandedWeylDualEquiv where
  deps := [``rightHandedRep, ``dualRightHandedRep]
  tag := "6VZR4"
```

informal_lemma Fermion.rightHandedWeylDualEquiv_equivariant [source]

The linear equivalence 'rightHandedWeylDualEquiv' is equivariant with respect to the action of 'SL(2,C)' on 'rightHandedWeyl' and 'DualRightHandedWeyl'.

```
informal_lemma rightHandedWeylDualEquiv_equivariant where
  deps := [``rightHandedWeylDualEquiv]
  tag := "6VZSG"
```

2.42 Physlib.Relativity.Tensors.ComplexTensor.Weyl.Contraction

Physlib/Relativity/Tensors/ComplexTensor/Weyl/Contraction.lean on GitHub.

def Fermion.leftDualBi [source]

The bi-linear map corresponding to contraction of a left-handed Weyl fermion with a dual-left-handed Weyl fermion.

```
def leftDualBi : LeftHandedWeyl →ₗ[ℂ] DualLeftHandedWeyl →ₗ[ℂ] ℂ where
```

def Fermion.dualLeftBi [source]

The bi-linear map corresponding to contraction of a dual-left-handed Weyl fermion with a left-handed Weyl fermion.

```
def dualLeftBi : DualLeftHandedWeyl →ₗ[ℂ] LeftHandedWeyl →ₗ[ℂ] ℂ where
```

def Fermion.rightDualBi [source]

The bi-linear map corresponding to contraction of a right-handed Weyl fermion with a dual-right-handed Weyl fermion.

```
def rightDualBi : RightHandedWeyl →ₗ[ℂ] DualRightHandedWeyl →ₗ[ℂ] ℂ where
```

def Fermion.dualRightBi [source]

The bi-linear map corresponding to contraction of a dual-right-handed Weyl fermion with a right-handed Weyl fermion.

```
def dualRightBi : DualRightHandedWeyl →ₗ[ℂ] RightHandedWeyl →ₗ[ℂ] ℂ where
```

def Fermion.leftDualContraction [source]

The linear map from leftHandedWeyl ⊗ DualLeftHandedWeyl to ℂ given by summing over components of leftHandedWeyl and DualLeftHandedWeyl in the standard basis (i.e. the dot product). Physically, the contraction of a left-handed Weyl fermion with a dual-left-handed Weyl fermion. In index notation this is ψ^a φ_a.

```
def leftDualContraction : (leftHandedRep.tprod dualLeftHandedRep).IntertwiningMap
    (Representation.trivial ℂ SL(2,) ℂ) where
```

lemma Fermion.leftDualContraction_hom_tmul [source]

```
lemma leftDualContraction_hom_tmul ψ( : LeftHandedWeyl)
    φ( : DualLeftHandedWeyl) :
    leftDualContraction ψ( ⊗ₜ φ) = ψ.ℂtoFin2 ⬝ᵥ φ.ℂtoFin2
```

lemma Fermion.leftDualContraction_basis [source]

```
lemma leftDualContraction_basis (i j : Fin 2) :
    leftDualContraction (leftBasis i ⊗ₜ dualLeftBasis j) = if i.1 = j.1 then (1 : ℂ) else 0
```

def Fermion.dualLeftContraction [source]

The linear map from DualLeftHandedWeyl ⊗ leftHandedWeyl to ℂ given by summing over components of DualLeftHandedWeyl and leftHandedWeyl in the standard basis (i.e. the dot product). Physically, the contraction of a dual-left-handed Weyl fermion with a left-handed Weyl fermion. In index notation this is φ_a ψ^a.

```
def dualLeftContraction : (dualLeftHandedRep.tprod leftHandedRep).IntertwiningMap
    (Representation.trivial ℂ SL(2,) ℂ) where
```

lemma Fermion.dualLeftContraction_hom_tmul [source]

```
lemma dualLeftContraction_hom_tmul φ( : DualLeftHandedWeyl) ψ( : LeftHandedWeyl) :
    dualLeftContraction φ( ⊗ₜ ψ) = φ.ℂtoFin2 ⬝ᵥ ψ.ℂtoFin2
```

lemma Fermion.dualLeftContraction_basis [source]

```
lemma dualLeftContraction_basis (i j : Fin 2) :
    dualLeftContraction (dualLeftBasis i ⊗ₜ leftBasis j) = if i.1 = j.1 then (1 : ℂ) else 0
```

def Fermion.rightDualContraction [source]

The linear map from 'rightHandedWeyl ⊗ DualRightHandedWeyl' to 'ℂ' given by summing over components of 'rightHandedWeyl' and 'DualRightHandedWeyl' in the standard basis (i.e. the dot product). The contraction of a right-handed Weyl fermion with a left-handed Weyl fermion. In index notation this is 'ψ^{dot a} φ_{dot a}'.

```
def rightDualContraction : (rightHandedRep.tprod dualRightHandedRep).IntertwiningMap
    (Representation.trivial ℂ SL(2,) ℂ) where
```

lemma Fermion.rightDualContraction_hom_tmul [source]

```
lemma rightDualContraction_hom_tmul ψ( : RightHandedWeyl)
    φ( : DualRightHandedWeyl) :
    rightDualContraction ψ( ⊗ₜ φ) = ψ.ℂtoFin2 ⬝ᵥ φ.ℂtoFin2
```

lemma Fermion.rightDualContraction_basis [source]

```
lemma rightDualContraction_basis (i j : Fin 2) :
    rightDualContraction (rightBasis i ⊗ₜ dualRightBasis j) =
    if i.1 = j.1 then (1 : ℂ) else 0
```

def Fermion.dualRightContraction [source]

The linear map from DualRightHandedWeyl ⊗ rightHandedWeyl to ℂ given by summing over components of DualRightHandedWeyl and rightHandedWeyl in the standard basis (i.e. the dot product). The contraction of a right-handed Weyl fermion with a left-handed Weyl fermion. In index notation this is φ_{dot a} ψ^{dot a}.

```
def dualRightContraction : (dualRightHandedRep.tprod rightHandedRep).IntertwiningMap
    (Representation.trivial ℂ SL(2,) ℂ) where
```

lemma Fermion.dualRightContraction_hom_tmul [source]

```
lemma dualRightContraction_hom_tmul φ( : DualRightHandedWeyl)
    ψ( : RightHandedWeyl) :
    dualRightContraction φ( ⊗ₜ ψ) = φ.ℂtoFin2 ⬝ᵥ ψ.ℂtoFin2
```

lemma Fermion.dualRightContraction_basis [source]

```
lemma dualRightContraction_basis (i j : Fin 2) :
    dualRightContraction (dualRightBasis i ⊗ₜ rightBasis j) =
    if i.1 = j.1 then (1 : ℂ) else 0
```

lemma Fermion.leftDualContraction_tmul_symm [source]

```
lemma leftDualContraction_tmul_symm ψ( : LeftHandedWeyl) φ( : DualLeftHandedWeyl) :
    leftDualContraction ψ( ⊗ₜ ℂ[] φ) = dualLeftContraction φ( ⊗ₜ ℂ[] ψ)
```

lemma Fermion.dualLeftContraction_tmul_symm [source]

```
lemma dualLeftContraction_tmul_symm φ( : DualLeftHandedWeyl) ψ( : LeftHandedWeyl) :
    dualLeftContraction φ( ⊗ₜ ℂ[] ψ) = leftDualContraction ψ( ⊗ₜ ℂ[] φ)
```

lemma Fermion.rightDualContraction_tmul_symm [source]

```
lemma rightDualContraction_tmul_symm ψ( : RightHandedWeyl) φ( : DualRightHandedWeyl) :
    rightDualContraction ψ( ⊗ₜ ℂ[] φ) = dualRightContraction φ( ⊗ₜ ℂ[] ψ)
```

lemma Fermion.dualRightContraction_tmul_symm [source]

```
lemma dualRightContraction_tmul_symm φ( : DualRightHandedWeyl) ψ( : RightHandedWeyl) :
    dualRightContraction φ( ⊗ₜ ℂ[] ψ) = rightDualContraction ψ( ⊗ₜ ℂ[] φ)
```

2.43 Physlib.Relativity.Tensors.ComplexTensor.Weyl.Metric

Physlib/Relativity/Tensors/ComplexTensor/Weyl/Metric.lean on GitHub.

lemma Fermion.leftMetricVal_expand_tmul' [source]

```
lemma leftMetricVal_expand_tmul' : leftMetricVal = leftBasis 1 ⊗ₜ ℂ[] leftBasis 0
    - leftBasis 0 ⊗ₜ ℂ[] leftBasis 1
```

def Fermion.dualLeftMetricVal [source]

The metric ‘ε_{aa}‘ as an element of ‘(dualLeftHanded ⊗ dualLeftHanded).V‘.

```
def dualLeftMetricVal : (DualLeftHandedWeyl ⊗ℂ[] DualLeftHandedWeyl)
```

lemma Fermion.dualLeftMetricVal_expand_tmul [source]

Expansion of ‘dualLeftMetricVal‘ into the left basis.

```
lemma dualLeftMetricVal_expand_tmul : dualLeftMetricVal =
    dualLeftBasis 0 ⊗ₜ ℂ[] dualLeftBasis 1 - dualLeftBasis 1 ⊗ₜ ℂ[] dualLeftBasis 0
```

def Fermion.dualLeftMetric [source]

The metric ‘ε_{aa}‘ as a morphism ↾_ (Rep ℂ SL(2,ℂ)) ⟶ dualLeftHanded ⊗ dualLeftHanded‘, making manifest its invariance under the action of ‘SL(2,ℂ)‘.

```
def dualLeftMetric : (Representation.trivial ℂ SL(2,) ℂ).IntertwiningMap
    (dualLeftHandedRep.tprod dualLeftHandedRep) where
```

lemma Fermion.dualLeftMetric_apply_one [source]

```
lemma dualLeftMetric_apply_one : dualLeftMetric (1 : ℂ) = dualLeftMetricVal
```

lemma Fermion.rightMetricVal_expand_tmul' [source]

```
lemma rightMetricVal_expand_tmul' : rightMetricVal = rightBasis 1 ⊗ₜ[ℂ] rightBasis 0
    - rightBasis 0 ⊗ₜ[ℂ] rightBasis 1
```

def Fermion.dualRightMetricVal [source]

The metric ‘ε_{dot a}_{dot a}‘ as an element of ‘(dualRightHanded ⊗ dualRightHanded).V‘.

```
def dualRightMetricVal : DualRightHandedWeyl ⊗[ℂ] DualRightHandedWeyl
```

lemma Fermion.dualRightMetricVal_expand_tmul [source]

Expansion of ‘rightMetricVal‘ into the left basis.

```
lemma dualRightMetricVal_expand_tmul : dualRightMetricVal =
    dualRightBasis 0 ⊗ₜ[ℂ] dualRightBasis 1 - dualRightBasis 1 ⊗ₜ[ℂ] dualRightBasis 0
```

def Fermion.dualRightMetric [source]

The metric ‘ε_{dot a}_{dot a}‘ as a morphism ‘𝓘_ (Rep ℂ SL(2,ℂ)) ⟶ dualRightHanded ⊗ dualRightHanded‘, making manifest its invariance under the action of ‘SL(2,ℂ)‘.

```
def dualRightMetric : (Representation.trivial ℂ SL(2,) ℂ).IntertwiningMap
    (dualRightHandedRep.tprod dualRightHandedRep) where
```

lemma Fermion.dualRightMetric_apply_one [source]

```
lemma dualRightMetric_apply_one : dualRightMetric (1 : ℂ) = dualRightMetricVal
```

lemma Fermion.leftDualContraction_apply_metric [source]

```
lemma leftDualContraction_apply_metric :
    (TensorProduct.comm ℂ _ _ ∘ₗ
      (TensorProduct.lid ℂ _).lTensor _ ∘ₗ
      (leftDualContraction.toLinearMap.rTensor _).lTensor _ ∘ₗ
      (TensorProduct.assoc ℂ _ _ _).symm.toLinearMap.lTensor _∘ₗ
      TensorProduct.assoc ℂ _ _ (_ ⊗ₜ[ℂ] _) ∘ₗ
      (leftMetric 1) ⊗ₜ[ℂ] (dualLeftMetric 1)) = dualLeftLeftUnit (1 : ℂ)
```

lemma Fermion.dualLeftContraction_apply_metric [source]

```
lemma dualLeftContraction_apply_metric :
    (TensorProduct.comm ℂ _ _ ∘ₗ
    (TensorProduct.lid ℂ _).lTensor _ ∘ₗ
    (dualLeftContraction.toLinearMap.rTensor _).lTensor _ ∘ₗ
    (TensorProduct.assoc ℂ _ _ _).symm.toLinearMap.lTensor _∘ₗ
    TensorProduct.assoc ℂ _ _ (_ ⊗ₜ[ℂ] _) ∘ₗ
    (dualLeftMetric 1) ⊗ₜ[ℂ] (leftMetric 1)) = leftDualLeftUnit (1 : ℂ)
```

lemma Fermion.rightDualContraction_apply_metric [source]

```
lemma rightDualContraction_apply_metric :
    (TensorProduct.comm ℂ _ _ <|
    (TensorProduct.lid ℂ _).lTensor _ <|
    (rightDualContraction.toLinearMap.rTensor _).lTensor _ <|
    (TensorProduct.assoc ℂ _ _ _).symm.toLinearMap.lTensor _<|
    TensorProduct.assoc ℂ _ _ (_ ⊗ℂ[] _) <|
    (rightMetric 1) ⊗ₜℂ[] (dualRightMetric 1)) = dualRightRightUnit (1 : ℂ)
```

lemma Fermion.dualRightContraction_apply_metric [source]

```
lemma dualRightContraction_apply_metric :
    (TensorProduct.comm ℂ _ _ <|
      (TensorProduct.lid ℂ _).lTensor _ <|
      (dualRightContraction.toLinearMap.rTensor _).lTensor _ <|
      (TensorProduct.assoc ℂ _ _ _).symm.toLinearMap.lTensor _<|
      TensorProduct.assoc ℂ _ _ (_ ⊗ℂ[] _) <|
      (dualRightMetric 1) ⊗ₜℂ[] (rightMetric 1)) = rightDualRightUnit (1 : ℂ)
```

2.44 Physlib.Relativity.Tensors.ComplexTensor.Weyl.Modules

Physlib/Relativity/Tensors/ComplexTensor/Weyl/Modules.lean on GitHub.

structure Fermion.LeftHandedWeyl [source]

The module in which left handed fermions live. This is equivalent to ‘Fin 2 → ℂ‘.

```
structure LeftHandedWeyl where
  val : Fin 2 → ℂ
```

def Fermion.LeftHandedWeyl.toFin2ℂFun [source]

The equivalence between ‘LeftHandedWeyl‘ and ‘Fin 2 → ℂ‘.

```
def ℂtoFin2Fun : LeftHandedWeyl ≃ (Fin 2 → ℂ) where
```

instance Fermion.LeftHandedWeyl.«instance@c5c29288» [source]

The instance of ‘AddCommMonoid‘ on ‘LeftHandedWeyl‘ defined via its equivalence with ‘Fin 2 → ℂ‘.

```
instance : AddCommMonoid LeftHandedWeyl
```

instance Fermion.LeftHandedWeyl.«instance@c3de7b53» [source]

The instance of ‘AddCommGroup‘ on ‘LeftHandedWeyl‘ defined via its equivalence with ‘Fin 2 → ℂ‘.

```
instance : AddCommGroup LeftHandedWeyl
```

instance Fermion.LeftHandedWeyl.«instance@db8c5fc4» [source]

The instance of ‘Module‘ on ‘LeftHandedWeyl‘ defined via its equivalence with ‘Fin 2 → ℂ‘.

```
instance : Module ℂ LeftHandedWeyl
```

def Fermion.LeftHandedWeyl.toFin2ℂEquiv [source]

The linear equivalence between ‘LeftHandedWeyl‘ and ‘(Fin 2 → ℂ)‘.

```
@[simps!]
def ℂtoFin2Equiv : LeftHandedWeyl ≃ₗ[ℂ] (Fin 2 → ℂ) where
```

abbrev Fermion.LeftHandedWeyl.toFin2ℂ [source]

The underlying element of ‘Fin 2 → ℂ‘ of a element in ‘LeftHandedWeyl‘ defined through the linear equivalence ‘toFin2ℂEquiv‘.

```
abbrev ℂtoFin2 ψ( : LeftHandedWeyl)
```

structure Fermion.DualLeftHandedWeyl [source]

The module in which dual-left handed fermions live. This is equivalent to ‘Fin 2 → ℂ‘.

```
structure DualLeftHandedWeyl where
  val : Fin 2 → ℂ
```

def Fermion.DualLeftHandedWeyl.toFin2ℂFun [source]

The equivalence between ‘DualLeftHandedWeyl‘ and ‘Fin 2 → ℂ‘.

```
def ℂtoFin2Fun : DualLeftHandedWeyl ≃ (Fin 2 → ℂ) where
```

instance Fermion.DualLeftHandedWeyl.«instance@ed87b73f» [source]

The instance of ‘AddCommMonoid‘ on ‘DualLeftHandedWeyl‘ defined via its equivalence with ‘Fin 2 → ℂ‘.

```
instance : AddCommMonoid DualLeftHandedWeyl
```

instance Fermion.DualLeftHandedWeyl.«instance@61470bec» [source]

The instance of ‘AddCommGroup‘ on ‘DualLeftHandedWeyl‘ defined via its equivalence with ‘Fin 2 → ℂ‘.

```
instance : AddCommGroup DualLeftHandedWeyl
```

instance Fermion.DualLeftHandedWeyl.«instance@c5aed84d» [source]

The instance of ‘Module‘ on ‘DualLeftHandedWeyl‘ defined via its equivalence with ‘Fin 2 → ℂ‘.

```
instance : Module ℂ DualLeftHandedWeyl
```

def Fermion.DualLeftHandedWeyl.toFin2ℂEquiv [source]

The linear equivalence between 'DualLeftHandedWeyl' and '(Fin 2 → ℂ)'.

```
@[simps!]
def ℂtoFin2Equiv : DualLeftHandedWeyl ≃ₗ[ℂ] (Fin 2 → ℂ) where
```

abbrev Fermion.DualLeftHandedWeyl.toFin2ℂ [source]

The underlying element of 'Fin 2 → ℂ' of a element in 'DualLeftHandedWeyl' defined through the linear equivalence 'toFin2ℂEquiv'.

```
abbrev ℂtoFin2 ψ( : DualLeftHandedWeyl)
```

structure Fermion.RightHandedWeyl [source]

The module in which right handed fermions live. This is equivalent to 'Fin 2 → ℂ'.

```
structure RightHandedWeyl where
  val : Fin 2 → ℂ
```

def Fermion.RightHandedWeyl.toFin2ℂFun [source]

The equivalence between 'RightHandedWeyl' and 'Fin 2 → ℂ'.

```
def ℂtoFin2Fun : RightHandedWeyl ≃ (Fin 2 → ℂ) where
```

instance Fermion.RightHandedWeyl.«instance@97ccc5aa» [source]

The instance of 'AddCommMonoid' on 'RightHandedWeyl' defined via its equivalence with 'Fin 2 → ℂ'.

```
instance : AddCommMonoid RightHandedWeyl
```

instance Fermion.RightHandedWeyl.«instance@4a60c666» [source]

The instance of 'AddCommGroup' on 'RightHandedWeyl' defined via its equivalence with 'Fin 2 → ℂ'.

```
instance : AddCommGroup RightHandedWeyl
```

instance Fermion.RightHandedWeyl.«instance@753cf914» [source]

The instance of 'Module' on 'RightHandedWeyl' defined via its equivalence with 'Fin 2 → ℂ'.

```
instance : Module ℂ RightHandedWeyl
```

def Fermion.RightHandedWeyl.toFin2ℂEquiv [source]

The linear equivalence between 'RightHandedWeyl' and '(Fin 2 → ℂ)'.

```
@[simps!]
def ℂtoFin2Equiv : RightHandedWeyl ≃ₗ[ℂ] (Fin 2 → ℂ) where
```

abbrev Fermion.RightHandedWeyl.toFin2ℂ [source]

The underlying element of ‘Fin 2 → ℂ‘ of a element in ‘RightHandedWeyl‘ defined through the linear equivalence ‘toFin2ℂEquiv‘.

```
abbrev ℂtoFin2 ψ( : RightHandedWeyl)
```

structure Fermion.DualRightHandedWeyl [source]

The module in which dual-right handed fermions live. This is equivalent to ‘Fin 2 → ℂ‘.

```
structure DualRightHandedWeyl where
  val : Fin 2 → ℂ
```

def Fermion.DualRightHandedWeyl.toFin2ℂFun [source]

The equivalence between ‘DualRightHandedWeyl‘ and ‘Fin 2 → ℂ‘.

```
def ℂtoFin2Fun : DualRightHandedWeyl ≃ (Fin 2 → ℂ) where
```

instance Fermion.DualRightHandedWeyl.«instance@c7559440» [source]

The instance of ‘AddCommMonoid‘ on ‘DualRightHandedWeyl‘ defined via its equivalence with ‘Fin 2 → ℂ‘.

```
instance : AddCommMonoid DualRightHandedWeyl
```

instance Fermion.DualRightHandedWeyl.«instance@a8cda771» [source]

The instance of ‘AddCommGroup‘ on ‘DualRightHandedWeyl‘ defined via its equivalence with ‘Fin 2 → ℂ‘.

```
instance : AddCommGroup DualRightHandedWeyl
```

instance Fermion.DualRightHandedWeyl.«instance@b26a3288» [source]

The instance of ‘Module‘ on ‘DualRightHandedWeyl‘ defined via its equivalence with ‘Fin 2 → ℂ‘.

```
instance : Module ℂ DualRightHandedWeyl
```

def Fermion.DualRightHandedWeyl.toFin2ℂEquiv [source]

The linear equivalence between ‘DualRightHandedWeyl‘ and ‘(Fin 2 → ℂ)‘.

```
@[simps!]
def ℂtoFin2Equiv : DualRightHandedWeyl ≃ₗ[ℂ] (Fin 2 → ℂ) where
```

abbrev Fermion.DualRightHandedWeyl.toFin2ℂ [source]

The underlying element of ‘Fin 2 → ℂ‘ of a element in ‘DualRightHandedWeyl‘ defined through the linear equivalence ‘toFin2ℂEquiv‘.

```
abbrev ℂtoFin2 ψ( : DualRightHandedWeyl)
```

2.45 Physlib.Relativity.Tensors.ComplexTensor.Weyl.Two

Physlib/Relativity/Tensors/ComplexTensor/Weyl/Two.lean on GitHub.

def Fermion.dualLeftdualLeftToMatrix [source]

Equivalence of 'dualLeftHanded ⊗ dualLeftHanded' to '2 x 2' complex matrices.

```
def dualLeftdualLeftToMatrix : (DualLeftHandedWeyl ⊗ℂ[] DualLeftHandedWeyl) ≃ₗ[ℂ]
    Matrix (Fin 2) (Fin 2) ℂ
```

lemma Fermion.dualLeftdualLeftToMatrix_symm_expand_tmul [source]

Expanding 'dualLeftdualLeftToMatrix' in terms of the standard basis.

```
lemma dualLeftdualLeftToMatrix_symm_expand_tmul (M : Matrix (Fin 2) (Fin 2) ℂ) :
    dualLeftdualLeftToMatrix.symm M = ∑ i, ∑ j, M i j •
      (dualLeftBasis i ⊗ₜ[ℂ] dualLeftBasis j)
```

def Fermion.leftDualLeftToMatrix [source]

Equivalence of 'leftHanded ⊗ dualLeftHanded' to '2 x 2' complex matrices.

```
def leftDualLeftToMatrix : (LeftHandedWeyl ⊗ℂ[] DualLeftHandedWeyl) ≃ₗ[ℂ]
    Matrix (Fin 2) (Fin 2) ℂ
```

lemma Fermion.leftDualLeftToMatrix_symm_expand_tmul [source]

Expanding 'leftDualLeftToMatrix' in terms of the standard basis.

```
lemma leftDualLeftToMatrix_symm_expand_tmul (M : Matrix (Fin 2) (Fin 2) ℂ) :
    leftDualLeftToMatrix.symm M = ∑ i, ∑ j, M i j • (leftBasis i ⊗ₜ[ℂ] dualLeftBasis j)
```

def Fermion.dualLeftLeftToMatrix [source]

Equivalence of 'dualLeftHanded ⊗ leftHanded' to '2 x 2' complex matrices.

```
def dualLeftLeftToMatrix : (DualLeftHandedWeyl ⊗ℂ[] LeftHandedWeyl) ≃ₗ[ℂ]
    Matrix (Fin 2) (Fin 2) ℂ
```

lemma Fermion.dualLeftLeftToMatrix_symm_expand_tmul [source]

Expanding 'dualLeftLeftToMatrix' in terms of the standard basis.

```
lemma dualLeftLeftToMatrix_symm_expand_tmul (M : Matrix (Fin 2) (Fin 2) ℂ) :
    dualLeftLeftToMatrix.symm M = ∑ i, ∑ j, M i j • (dualLeftBasis i ⊗ₜ[ℂ] leftBasis j)
```

def Fermion.dualRightDualRightToMatrix [source]

Equivalence of 'dualRightHanded ⊗ dualRightHanded' to '2 x 2' complex matrices.

```
def dualRightDualRightToMatrix : (DualRightHandedWeyl ⊗ℂ[] DualRightHandedWeyl) ≃ₗ[ℂ]
    Matrix (Fin 2) (Fin 2) ℂ
```

lemma Fermion.dualRightDualRightToMatrix_symm_expand_tmul [source]

Expanding 'dualRightDualRightToMatrix' in terms of the standard basis.

```
lemma dualRightDualRightToMatrix_symm_expand_tmul (M : Matrix (Fin 2) (Fin 2) ℂ) :
    dualRightDualRightToMatrix.symm M =
    ∑ i, ∑ j, M i j • (dualRightBasis i ⊗ₜ[ℂ] dualRightBasis j)
```

def Fermion.rightDualRightToMatrix [source]

Equivalence of 'rightHanded ⊗ dualRightHanded' to '2 x 2' complex matrices.

```
def rightDualRightToMatrix : (RightHandedWeyl ⊗ℂ[] DualRightHandedWeyl) ≃ₗ[ℂ]
    Matrix (Fin 2) (Fin 2) ℂ
```

lemma Fermion.rightDualRightToMatrix_symm_expand_tmul [source]

Expanding 'rightDualRightToMatrix' in terms of the standard basis.

```
lemma rightDualRightToMatrix_symm_expand_tmul (M : Matrix (Fin 2) (Fin 2) ℂ) :
    rightDualRightToMatrix.symm M = ∑ i, ∑ j, M i j • (rightBasis i ⊗ₜ[ℂ] dualRightBasis j)
```

def Fermion.dualRightRightToMatrix [source]

Equivalence of 'dualRightHanded ⊗ rightHanded' to '2 x 2' complex matrices.

```
def dualRightRightToMatrix : (DualRightHandedWeyl ⊗ℂ[] RightHandedWeyl) ≃ₗ[ℂ]
    Matrix (Fin 2) (Fin 2) ℂ
```

lemma Fermion.dualRightRightToMatrix_symm_expand_tmul [source]

Expanding 'dualRightRightToMatrix' in terms of the standard basis.

```
lemma dualRightRightToMatrix_symm_expand_tmul (M : Matrix (Fin 2) (Fin 2) ℂ) :
    dualRightRightToMatrix.symm M = ∑ i, ∑ j, M i j • (dualRightBasis i ⊗ₜ[ℂ] rightBasis j)
```

def Fermion.dualLeftDualRightToMatrix [source]

Equivalence of 'dualLeftHanded ⊗ dualRightHanded' to '2 x 2' complex matrices.

```
def dualLeftDualRightToMatrix : (DualLeftHandedWeyl ⊗ℂ[] DualRightHandedWeyl) ≃ₗ[ℂ]
    Matrix (Fin 2) (Fin 2) ℂ
```

lemma Fermion.dualLeftDualRightToMatrix_symm_expand_tmul [source]

Expanding 'dualLeftDualRightToMatrix' in terms of the standard basis.

```
lemma dualLeftDualRightToMatrix_symm_expand_tmul (M : Matrix (Fin 2) (Fin 2) ℂ) :
    dualLeftDualRightToMatrix.symm M = ∑ i, ∑ j, M i j •
      (dualLeftBasis i ⊗ₜ[ℂ] dualRightBasis j)
```

lemma Fermion.coe_linearEquivFunOnFinite [source]

The coercion of ‘Finsupp.linearEquivFunOnFinite‘ to a function is the underlying finitely-supported function, used to bridge it with ‘Matrix.mulVec‘.

```
private lemma coe_linearEquivFunOnFinite (g : (Fin 2 × Fin 2) →₀ ℂ) :
    Finsupp.linearEquivFunOnFinite ℂ ℂ (Fin 2 × Fin 2) g = ⇑g
```

lemma Fermion.dualLeftdualLeftToMatrix_ρ [source]

*The group action of ‘SL(2,ℂ)‘ on ‘dualLeftHanded ⊗ dualLeftHanded‘ is equivalent to ‘$(M.1^{-1})^T$ * leftLeftToMatrix v * $(M.1^{-1})$‘.*

```
lemma pdualLeftdualLeftToMatrix_ (v : (DualLeftHandedWeyl ⊗[ℂ] DualLeftHandedWeyl)) (M : SL
    ℂ(2,)) :
    dualLeftdualLeftToMatrix (TensorProduct.map (dualLeftHandedRep M) (dualLeftHandedRep M) v)
    =
    (M⁻.1¹)ᵀ * dualLeftdualLeftToMatrix v * (M⁻.1¹)
```

lemma Fermion.leftDualLeftToMatrix_ρ [source]

*The group action of ‘SL(2,ℂ)‘ on ‘leftHanded ⊗ dualLeftHanded‘ is equivalent to ‘M.1 * leftDualLeftToMatrix v * $(M.1^{-1})$‘.*

```
lemma pleftDualLeftToMatrix_ (v : (LeftHandedWeyl ⊗[ℂ] DualLeftHandedWeyl)) (M : SL℃(2,)) :
    leftDualLeftToMatrix (TensorProduct.map (leftHandedRep M) (dualLeftHandedRep M) v) =
    M.1 * leftDualLeftToMatrix v * (M⁻.1¹)
```

lemma Fermion.dualLeftLeftToMatrix_ρ [source]

*The group action of ‘SL(2,ℂ)‘ on ‘dualLeftHanded ⊗ leftHanded‘ is equivalent to ‘$(M.1^{-1})^T$ * leftDualLeftToMatrix v * $(M.1)^T$‘.*

```
lemma pdualLeftLeftToMatrix_ (v : (DualLeftHandedWeyl ⊗[ℂ] LeftHandedWeyl)) (M : SL℃(2,)) :
    dualLeftLeftToMatrix (TensorProduct.map (dualLeftHandedRep M) (leftHandedRep M) v) =
    (M⁻.1¹)ᵀ * dualLeftLeftToMatrix v * (M.1)ᵀ
```

lemma Fermion.dualRightDualRightToMatrix_ρ [source]

*The group action of ‘SL(2,ℂ)‘ on ‘dualRightHanded ⊗ dualRightHanded‘ is equivalent to ‘$((M.1^{-1}).conjTranspose$ * rightRightToMatrix v * $(((M.1^{-1}).conjTranspose)^T$‘.*

```
lemma pdualRightDualRightToMatrix_ (v : (DualRightHandedWeyl ⊗[ℂ] DualRightHandedWeyl))
    (M : SL℃(2,)) :
    dualRightDualRightToMatrix (TensorProduct.map (dualRightHandedRep M) (dualRightHandedRep M)
     v) =
    ((M⁻.1¹).conjTranspose) * dualRightDualRightToMatrix v * (((M⁻.1¹).conjTranspose)ᵀ)
```

lemma Fermion.rightDualRightToMatrix_ρ [source]

*The group action of ‘SL(2,ℂ)‘ on ‘rightHanded ⊗ dualRightHanded‘ is equivalent to ‘(M.1.map star) * rightDualRightToMatrix v * $(((M.1^{-1}).conjTranspose)^T$‘.*

```
lemma prightDualRightToMatrix_ (v : (RightHandedWeyl ⊗[ℂ] DualRightHandedWeyl)) (M : SL℃(2,)) :
    rightDualRightToMatrix (TensorProduct.map (rightHandedRep M) (dualRightHandedRep M) v) =
    (M.1.map star) * rightDualRightToMatrix v * (((M⁻.1¹).conjTranspose)ᵀ)
```

lemma Fermion.dualRightRightToMatrix_ρ [source]

*The group action of 'SL(2,ℂ)' on 'dualRightHanded ⊗ rightHanded' is equivalent to '((M.1⁻¹).conjTranspose * rightDualRightToMatrix v * ((M.1.map star)).T'.*

```
lemma ρdualRightRightToMatrix_ (v : (DualRightHandedWeyl ⊗ℂ[] RightHandedWeyl)) (M : SL℃(2,)) :
    dualRightRightToMatrix (TensorProduct.map (dualRightHandedRep M) (rightHandedRep M) v) =
    ((M⁻.1¹).conjTranspose) * dualRightRightToMatrix v * (M.1.map star)ᵀ
```

lemma Fermion.dualLeftDualRightToMatrix_ρ [source]

```
lemma ρdualLeftDualRightToMatrix_ (v : (DualLeftHandedWeyl ⊗ℂ[] DualRightHandedWeyl))
    (M : SL℃(2,)) :
    dualLeftDualRightToMatrix (TensorProduct.map (dualLeftHandedRep M) (dualRightHandedRep M) v
    ) =
    (M⁻.1¹)ᵀ * dualLeftDualRightToMatrix v * ((M⁻.1¹).conjTranspose)ᵀ
```

lemma Fermion.dualLeftdualLeftToMatrix_ρ_symm [source]

```
lemma ρdualLeftdualLeftToMatrix_symm (v : Matrix (Fin 2) (Fin 2) ℂ) (M : SL℃(2,)) :
    TensorProduct.map (dualLeftHandedRep M) (dualLeftHandedRep M)
      (dualLeftdualLeftToMatrix.symm v) =
    dualLeftdualLeftToMatrix.symm ((M⁻.1¹)ᵀ * v * (M⁻.1¹))
```

lemma Fermion.leftDualLeftToMatrix_ρ_symm [source]

```
lemma ρleftDualLeftToMatrix_symm (v : Matrix (Fin 2) (Fin 2) ℂ) (M : SL℃(2,)) :
    TensorProduct.map (leftHandedRep M) (dualLeftHandedRep M) (leftDualLeftToMatrix.symm v) =
    leftDualLeftToMatrix.symm (M.1 * v * (M⁻.1¹))
```

lemma Fermion.dualLeftLeftToMatrix_ρ_symm [source]

```
lemma ρdualLeftLeftToMatrix_symm (v : Matrix (Fin 2) (Fin 2) ℂ) (M : SL℃(2,)) :
    TensorProduct.map (dualLeftHandedRep M) (leftHandedRep M) (dualLeftLeftToMatrix.symm v) =
    dualLeftLeftToMatrix.symm ((M⁻.1¹)ᵀ * v * (M.1)ᵀ)
```

lemma Fermion.dualRightDualRightToMatrix_ρ_symm [source]

```
lemma ρdualRightDualRightToMatrix_symm (v : Matrix (Fin 2) (Fin 2) ℂ) (M : SL℃(2,)) :
    TensorProduct.map (dualRightHandedRep M) (dualRightHandedRep M)
      (dualRightDualRightToMatrix.symm v) =
    dualRightDualRightToMatrix.symm (((M⁻.1¹).conjTranspose) * v * ((M⁻.1¹).conjTranspose)ᵀ)
```

lemma Fermion.rightDualRightToMatrix_ρ_symm [source]

```
lemma ρrightDualRightToMatrix_symm (v : Matrix (Fin 2) (Fin 2) ℂ) (M : SL℃(2,)) :
    TensorProduct.map (rightHandedRep M) (dualRightHandedRep M) (rightDualRightToMatrix.symm v)
     =
    rightDualRightToMatrix.symm ((M.1.map star) * v * (((M⁻.1¹).conjTranspose)ᵀ))
```

lemma Fermion.dualRightRightToMatrix_ρ_symm [source]

```
lemma pdualRightRightToMatrix_symm (v : Matrix (Fin 2) (Fin 2) ℂ) (M : SL(2,)) :
    TensorProduct.map (dualRightHandedRep M) (rightHandedRep M) (dualRightRightToMatrix.symm v)
     =
    dualRightRightToMatrix.symm (((M⁻.1¹).conjTranspose) * v * (M.1.map star)ᵀ)
```

lemma Fermion.dualLeftDualRightToMatrix_ρ_symm [source]

```
lemma pdualLeftDualRightToMatrix_symm (v : Matrix (Fin 2) (Fin 2) ℂ) (M : SL(2,)) :
    TensorProduct.map (dualLeftHandedRep M) (dualRightHandedRep M)
      (dualLeftDualRightToMatrix.symm v) =
    dualLeftDualRightToMatrix.symm ((M⁻.1¹)ᵀ * v * ((M⁻.1¹).conjTranspose)ᵀ)
```

lemma Fermion.dualLeftDualRightToMatrix_ρ_symm_selfAdjoint [source]

```
lemma pdualLeftDualRightToMatrix_symm_selfAdjoint (v : Matrix (Fin 2) (Fin 2) ℂ)
    (hv : IsSelfAdjoint v) (M : SL(2,)) :
    TensorProduct.map (dualLeftHandedRep M) (dualRightHandedRep M)
      (dualLeftDualRightToMatrix.symm v) =
    dualLeftDualRightToMatrix.symm (SL2C.toSelfAdjointMap (M.⁻transpose¹) ⟨v, ⟩hv)
```

2.46 Physlib.Relativity.Tensors.ComplexTensor.Weyl.Unit

Physlib/Relativity/Tensors/ComplexTensor/Weyl/Unit.lean on GitHub.

def Fermion.leftDualLeftUnitVal [source]

The left-dual-left unit ‘$\delta^a{}_a$‘ as an element of ‘(leftHanded ⊗ dualLeftHanded).V‘.

```
def leftDualLeftUnitVal : (LeftHandedWeyl ⊗ℂ[] DualLeftHandedWeyl)
```

lemma Fermion.leftDualLeftUnitVal_expand_tmul [source]

Expansion of ‘leftDualLeftUnitVal‘ into the basis.

```
lemma leftDualLeftUnitVal_expand_tmul : leftDualLeftUnitVal =
    leftBasis 0 ⊗ₜℂ[] dualLeftBasis 0 + leftBasis 1 ⊗ₜℂ[] dualLeftBasis 1
```

def Fermion.leftDualLeftUnit [source]

The left-dual-left unit ‘$\delta^a{}_a$‘ as a morphism ‘𝟙_ (Rep ℂ SL(2,ℂ)) ⟶ leftHanded ⊗ dualLeftHanded ‘, manifesting the invariance under the ‘SL(2,ℂ)‘ action.

```
def leftDualLeftUnit : (Representation.trivial ℂ SL(2,) ℂ).IntertwiningMap
    (leftHandedRep.tprod dualLeftHandedRep) where
```

lemma Fermion.leftDualLeftUnit_apply_one [source]

```
lemma leftDualLeftUnit_apply_one : leftDualLeftUnit (1 : ℂ) = leftDualLeftUnitVal
```

def Fermion.dualLeftLeftUnitVal [source]

The dual-left-left unit ‘$\delta_a{}^a$‘ as an element of ‘(dualLeftHanded ⊗ leftHanded).V‘.

```
def dualLeftLeftUnitVal : (DualLeftHandedWeyl ⊗[ℂ] LeftHandedWeyl)
```

lemma Fermion.dualLeftLeftUnitVal_expand_tmul [source]

Expansion of 'dualLeftLeftUnitVal' into the basis.

```
lemma dualLeftLeftUnitVal_expand_tmul : dualLeftLeftUnitVal =
    dualLeftBasis 0 ⊗ₜ[ℂ] leftBasis 0 + dualLeftBasis 1 ⊗ₜ[ℂ] leftBasis 1
```

def Fermion.dualLeftLeftUnit [source]

The dual-left-left unit '$\delta_a{}^a$' as a morphism '𝓚_ (Rep ℂ SL(2,ℂ)) ⟶ dualLeftHanded ⊗ leftHanded', manifesting the invariance under the 'SL(2,ℂ)' action.

```
def dualLeftLeftUnit :
    (Representation.trivial ℂ SL(2,) ℂ).IntertwiningMap
      (dualLeftHandedRep.tprod leftHandedRep) where
```

lemma Fermion.dualLeftLeftUnit_apply_one [source]

Applying the morphism 'dualLeftLeftUnit' to '1' returns 'dualLeftLeftUnitVal'.

```
lemma dualLeftLeftUnit_apply_one : dualLeftLeftUnit (1 : ℂ) = dualLeftLeftUnitVal
```

def Fermion.rightDualRightUnitVal [source]

The right-dual-right unit 'δ^{dot a}_{dot a}' as an element of '(rightHanded ⊗ dualRightHanded).V'.

```
def rightDualRightUnitVal : RightHandedWeyl ⊗[ℂ] DualRightHandedWeyl
```

lemma Fermion.rightDualRightUnitVal_expand_tmul [source]

Expansion of 'rightDualRightUnitVal' into the basis.

```
lemma rightDualRightUnitVal_expand_tmul : rightDualRightUnitVal =
    rightBasis 0 ⊗ₜ[ℂ] dualRightBasis 0 + rightBasis 1 ⊗ₜ[ℂ] dualRightBasis 1
```

def Fermion.rightDualRightUnit [source]

The right-dual-right unit 'δ^{dot a}_{dot a}' as a morphism '𝓚_ (Rep ℂ SL(2,ℂ)) ⟶ rightHanded ⊗ dualRightHanded', manifesting the invariance under the 'SL(2,ℂ)' action.

```
def rightDualRightUnit : (Representation.trivial ℂ SL(2,) ℂ).IntertwiningMap
    (rightHandedRep.tprod dualRightHandedRep) where
```

lemma Fermion.rightDualRightUnit_apply_one [source]

```
lemma rightDualRightUnit_apply_one : rightDualRightUnit (1 : ℂ) = rightDualRightUnitVal
```

def Fermion.dualRightRightUnitVal [source]

The dual-right-right unit ‘δ_{dot a}^{dot a}‘ as an element of ‘(rightHanded ⊗ dualRightHanded).V‘.

```
def dualRightRightUnitVal : (DualRightHandedWeyl ⊗[ℂ] RightHandedWeyl)
```

lemma Fermion.dualRightRightUnitVal_expand_tmul [source]

Expansion of ‘dualRightRightUnitVal‘ into the basis.

```
lemma dualRightRightUnitVal_expand_tmul : dualRightRightUnitVal =
    dualRightBasis 0 ⊗ₜ[ℂ] rightBasis 0 + dualRightBasis 1 ⊗ₜ[ℂ] rightBasis 1
```

def Fermion.dualRightRightUnit [source]

The dual-right-right unit ‘δ_{dot a}^{dot a}‘ as a morphism ‘𝓘_ (Rep ℂ SL(2,ℂ)) ⟶ dualRightHanded ⊗ rightHanded‘, manifesting the invariance under the ‘SL(2,ℂ)‘ action.

```
def dualRightRightUnit : (Representation.trivial ℂ SL(2,) ℂ).IntertwiningMap
    (dualRightHandedRep.tprod rightHandedRep) where
```

lemma Fermion.dualRightRightUnit_apply_one [source]

```
lemma dualRightRightUnit_apply_one : dualRightRightUnit (1 : ℂ) = dualRightRightUnitVal
```

lemma Fermion.contr_dualLeftLeftUnit [source]

Contraction on the right with ‘dualLeftLeftUnit‘ does nothing.

```
lemma contr_dualLeftLeftUnit (x : LeftHandedWeyl) :
    (TensorProduct.lid ℂ _ <|
    leftDualContraction.toLinearMap.rTensor _ <|
    (TensorProduct.assoc ℂ _ _ _).symm <|
    x ⊗ₜ[ℂ] (dualLeftLeftUnit (1 : ℂ))) = x
```

lemma Fermion.contr_leftDualLeftUnit [source]

Contraction on the right with ‘leftDualLeftUnit‘ does nothing.

```
lemma contr_leftDualLeftUnit (x : DualLeftHandedWeyl) :
    (TensorProduct.lid ℂ _ <|
    dualLeftContraction.toLinearMap.rTensor _ <|
    (TensorProduct.assoc ℂ _ _ _).symm <|
    x ⊗ₜ[ℂ] (leftDualLeftUnit (1 : ℂ))) = x
```

lemma Fermion.contr_dualRightRightUnit [source]

Contraction on the right with ‘dualRightRightUnit‘ does nothing.

```
lemma contr_dualRightRightUnit (x : RightHandedWeyl) :
    (TensorProduct.lid ℂ _ <|
    rightDualContraction.toLinearMap.rTensor _ <|
    (TensorProduct.assoc ℂ _ _ _).symm <|
```

```
    x ⊗ₜ ℂ[] (dualRightRightUnit (1 : ℂ)) = x
```

lemma Fermion.contr_rightDualRightUnit [source]

Contraction on the right with ‘rightDualRightUnit‘ does nothing.

```
lemma contr_rightDualRightUnit (x : DualRightHandedWeyl) :
    (TensorProduct.lid ℂ _ <|
    dualRightContraction.toLinearMap.rTensor _ <|
    (TensorProduct.assoc ℂ _ _ _).symm <|
    x ⊗ₜ ℂ[] (rightDualRightUnit (1 : ℂ))) = x
```

lemma Fermion.dualLeftLeftUnit_symm [source]

```
lemma dualLeftLeftUnit_symm :
    dualLeftLeftUnit (1 : ℂ) = LinearMap.lTensor _ (LinearEquiv.refl _ _).toLinearMap
    (TensorProduct.comm ℂ _ _ (leftDualLeftUnit (1 : ℂ)))
```

lemma Fermion.leftDualLeftUnit_symm [source]

```
lemma leftDualLeftUnit_symm :
    leftDualLeftUnit (1 : ℂ) = LinearMap.lTensor _ (LinearEquiv.refl _ _).toLinearMap
      (TensorProduct.comm ℂ _ _ (dualLeftLeftUnit (1 : ℂ)))
```

lemma Fermion.dualRightRightUnit_symm [source]

```
lemma dualRightRightUnit_symm :
    dualRightRightUnit (1 : ℂ) = LinearMap.lTensor _ (LinearEquiv.refl _ _).toLinearMap
      (TensorProduct.comm ℂ _ _ (rightDualRightUnit (1 : ℂ)))
```

lemma Fermion.rightDualRightUnit_symm [source]

```
lemma rightDualRightUnit_symm :
    rightDualRightUnit (1 : ℂ) = LinearMap.lTensor _ (LinearEquiv.refl _ _).toLinearMap
      (TensorProduct.comm ℂ _ _ (dualRightRightUnit (1 : ℂ)))
```

2.47 Physlib.Relativity.Tensors.ComponentIdx.Basic

Physlib/Relativity/Tensors/ComponentIdx/Basic.lean on GitHub.

abbrev TensorSpecies.Tensor.ComponentIdx [source]

Given a list of indices ‘c : Fin n → C‘, the type ‘ComponentIdx c‘ is the type of component indices of a tensor with those colors. For instance, if ‘c = ![.up, .down]‘, then an element of ‘ComponentIdx c‘ specifies one basis index for ‘.up‘ and one for ‘.down‘.

```
@[nolint unusedArguments]
abbrev ComponentIdx {n : ℕ} {S : TensorSpecies k C G V basisIdx rep b}
    (c : Fin n → C) : Type
```

lemma TensorSpecies.Tensor.ComponentIdx.congr_right [source]

```
lemma ComponentIdx.congr_right {n : ℕ} {c : Fin n → C} (b : ComponentIdx (S := S) c)
    (i j : Fin n) (h : i = j) : b i = basisIdxCongr (by simp [h]) (b j)
```

def TensorSpecies.Tensor.ComponentIdx.cast [source]

Casting of a 'ComponentIdx' through equivalent color maps.

```
def ComponentIdx.cast {n m : ℕ} {c : Fin n → C} {cm : Fin m → C}
    (h : n = m) (hc : c = cm ∘ Fin.cast h) (b : ComponentIdx (S := S) c) :
    ComponentIdx (S := S) cm
```

2.48 Physlib.Relativity.Tensors.ComponentIdx.Contraction

Physlib/Relativity/Tensors/ComponentIdx/Contraction.lean on GitHub.

def TensorSpecies.Tensor.ComponentIdx.dropPair [source]

The 'ComponentIdx' obtained by dropping two components.

```
def dropPair {n : ℕ} {c : Fin (n + 1 + 1) → C}
    (i j : Fin (n + 1 + 1)) (b : ComponentIdx (S := S) c) :
    ComponentIdx (S := S) (c ∘ Fin.succSuccAbove i j)
```

def TensorSpecies.Tensor.ComponentIdx.DropPairSection [source]

Given a coordinate parameter 'b : Π k, Fin (S.repDim ((c ∘ i.succAbove ∘ j.succAbove) k))', the coordinate parameter 'Π k, Fin (S.repDim (c k))' which map down to 'b'.

```
def DropPairSection {n : ℕ} {c : Fin (n + 1 + 1) → C}
    {i : Fin (n + 1 + 1)} {j : Fin (n + 1 + 1)}
    (b : ComponentIdx (S := S) (c ∘ Fin.succSuccAbove i j)) :
    Finset (ComponentIdx (S := S) c)
```

lemma TensorSpecies.Tensor.ComponentIdx.DropPairSection.mem_iff_apply_succSuccAbove_eq [source]

```
lemma mem_iff_apply_succSuccAbove_eq {n : ℕ} {c : Fin (n + 1 + 1) → C}
    {i j : Fin (n + 1 + 1)}
    (b : ComponentIdx (c ∘ Fin.succSuccAbove i j))
    (b' : ComponentIdx c) :
    b' ∈ DropPairSection (S := S) b ↔
      ∀ m, b' (Fin.succSuccAbove i j m) = b m
```

lemma TensorSpecies.Tensor.ComponentIdx.DropPairSection.mem_self_of_dropPair [source]

```
@[simp]
lemma mem_self_of_dropPair {n : ℕ} {c : Fin (n + 1 + 1) → C}
    {i j : Fin (n + 1 + 1)}
    (b : ComponentIdx (c)) :
    b ∈ DropPairSection (S := S) (b.dropPair i j)
```

def TensorSpecies.Tensor.ComponentIdx.DropPairSection.ofFin [source]

Given a ‘b‘ in ‘ComponentIdx (c ∘ Fin.succSuccAbove i j))‘ and an ‘x‘ in ‘Fin (S.repDim (c i)) × Fin (S.repDim (c j))‘, the corresponding coordinate parameter in ‘ComponentIdx c‘.

```
def ofFin {n : ℕ} {c : Fin (n + 1 + 1) → C}
    {i j : Fin (n + 1 + 1)} (hij : i ≠ j) (b : ComponentIdx (S := S) (c ∘ Fin.succSuccAbove i j
    ))
    (x : basisIdx (c i) × basisIdx (c j)) :
    ComponentIdx (S := S) c
```

lemma TensorSpecies.Tensor.ComponentIdx.DropPairSection.ofFin_apply_fst [source]

```
@[simp]
lemma ofFin_apply_fst {n : ℕ} {c : Fin (n + 1 + 1) → C}
    {i j : Fin (n + 1 + 1)} (hij : i ≠ j) (b : ComponentIdx (c ∘ Fin.succSuccAbove i j))
    (x : basisIdx (c i) × basisIdx (c j)) :
    ofFin (S := S) hij b x i = x.1
```

lemma TensorSpecies.Tensor.ComponentIdx.DropPairSection.ofFin_apply_snd [source]

```
@[simp]
lemma ofFin_apply_snd {n : ℕ} {c : Fin (n + 1 + 1) → C}
    {i j : Fin (n + 1 + 1)} (hij : i ≠ j) (b : ComponentIdx (c ∘ Fin.succSuccAbove i j))
    (x : basisIdx (c i) × basisIdx (c j)) :
    ofFin (S := S) hij b x j = x.2
```

lemma TensorSpecies.Tensor.ComponentIdx.DropPairSection.ofFin_mem_succSuccAboveSection [source]

```
lemma ofFin_mem_succSuccAboveSection {n : ℕ} {c : Fin (n + 1 + 1) → C}
    {i j : Fin (n + 1 + 1)} (hij : i ≠ j) (b : ComponentIdx (c ∘ Fin.succSuccAbove i j))
    (x : basisIdx (c i) × basisIdx (c j)) :
    ofFin (S := S) hij b x ∈ DropPairSection b
```

def TensorSpecies.Tensor.ComponentIdx.DropPairSection.ofFinEquiv [source]

The equivalence between ‘ContrSection b‘ and ‘basisIdx (c i) × basisIdx (c j)‘.

```
def ofFinEquiv {n : ℕ} {c : Fin n.succ.succ → C}
    {i j : Fin (n + 1 + 1)} (hij : i ≠ j)
    (b : ComponentIdx (c ∘ Fin.succSuccAbove i j)) :
    basisIdx (c i) × basisIdx (c j) ≃ DropPairSection (S := S) b where
```

lemma TensorSpecies.Tensor.ComponentIdx.DropPairSection.ofFinEquiv_apply_fst [source]

```
@[simp]
lemma ofFinEquiv_apply_fst {n : ℕ} {c : Fin (n + 1 + 1) → C}
    {i j : Fin (n + 1 + 1)} (hij : i ≠ j) (b : ComponentIdx (c ∘ Fin.succSuccAbove i j))
    (x : basisIdx (c i) × basisIdx (c j)) :
    (ofFinEquiv (S := S) hij b x).1 i = x.1
```

lemma TensorSpecies.Tensor.ComponentIdx.DropPairSection.ofFinEquiv_apply_snd [source]

```
@[simp]
lemma ofFinEquiv_apply_snd {n : ℕ} {c : Fin (n + 1 + 1) → C}
    {i j : Fin (n + 1 + 1)} (hij : i ≠ j) (b : ComponentIdx (c ∘ Fin.succSuccAbove i j))
    (x : basisIdx (c i) × basisIdx (c j)) :
    (ofFinEquiv (S := S) hij b x).1 j = x.2
```

2.49 Physlib.Relativity.Tensors.ComponentIdx.Product

Physlib/Relativity/Tensors/ComponentIdx/Product.lean on GitHub.

def TensorSpecies.Tensor.ComponentIdx.prod [source]

The equivalence between 'ComponentIdx (Fin.append c c1)' and 'ComponentIdx c × ComponentIdx c1' formed by products.

```
def ComponentIdx.prod {n1 n2 : ℕ} {c : Fin n1 → C} {c1 : Fin n2 → C} :
    ComponentIdx (S := S) (Fin.append c c1) ≃
      ComponentIdx (S := S) c × ComponentIdx (S := S) c1 where
```

lemma TensorSpecies.Tensor.ComponentIdx.prod_symm_natAdd [source]

```
@[simp]
lemma ComponentIdx.prod_symm_natAdd {n1 n2 : ℕ} {c : Fin n1 → C} {c1 : Fin n2 → C}
    (p : ComponentIdx (S := S) c) (q : ComponentIdx (S := S) c1) (i : Fin n2) :
    ComponentIdx.prod.symm (p, q) (Fin.natAdd n1 i) =
      basisIdxCongr (by simp) (q i)
```

lemma TensorSpecies.Tensor.ComponentIdx.prod_symm_castAdd [source]

```
@[simp]
lemma ComponentIdx.prod_symm_castAdd {n1 n2 : ℕ} {c : Fin n1 → C} {c1 : Fin n2 → C}
    (p : ComponentIdx (S := S) c) (q : ComponentIdx (S := S) c1) (i : Fin n1) :
    ComponentIdx.prod.symm (p, q) (Fin.castAdd n2 i) =
      basisIdxCongr (by simp) (p i)
```

2.50 Physlib.Relativity.Tensors.ComponentIdx.Single

Physlib/Relativity/Tensors/ComponentIdx/Single.lean on GitHub.

def TensorSpecies.Tensor.ComponentIdx.single [source]

The equivalence between component indices for a single color and the basis indices of that color.

```
def ComponentIdx.single {c : C} :
    ComponentIdx (S := S) ![c] ≃ basisIdx c where
```

lemma TensorSpecies.Tensor.ComponentIdx.single_apply [source]

```
@[simp]
lemma ComponentIdx.single_apply {c : C} (b : ComponentIdx (S := S) ![c]) :
    ComponentIdx.single (S := S) b = basisIdxCongr (by simp) (b 0)
```

lemma TensorSpecies.Tensor.ComponentIdx.single_symm_apply [source]

```
@[simp]
lemma ComponentIdx.single_symm_apply {c : C} (b : basisIdx c) (i : Fin 1) :
    (ComponentIdx.single (S := S) (c := c)).symm b i = basisIdxCongr (by simp) b
```

2.51 Physlib.Relativity.Tensors.Conjugation.Basic

Physlib/Relativity/Tensors/Conjugation/Basic.lean on GitHub.

structure ConjTensorSpecies [source]

A tensor species bundled with a conjugation. Beyond the ‘TensorSpecies‘ it ‘extends‘, it carries the conjugate-colour involution ‘bar‘, the index-set identification ‘barIdx_eq‘, and the contraction-coherence ‘conj_contrComm‘. Also carries ‘bar_involution‘ and ‘bar_tau‘ (that ‘bar‘ is involutive and commutes with the variance dual).

```
structure ConjTensorSpecies (k : Type) [CommRing k] [StarRing k] (C : Type) (G : Type) [Group G
    ]
    (V : C → Type) ∀[ c, AddCommGroup (V c)] ∀[ c, Module k (V c)]
    (basisIdx : C → Type) ∀[ c, Fintype (basisIdx c)] ∀[ c, DecidableEq (basisIdx c)]
    (rep : (c : C) → Representation k G (V c)) (b : (c : C) → Basis (basisIdx c) k (V c))
    extends TensorSpecies k C G V basisIdx rep b where
  bar : C → C
  bar_involution : Function.Involutive bar
  bar_tau : ∀ c, bar (toTensorSpeciesτ. c) = toTensorSpeciesτ. (bar c)
  barIdx_eq : ∀ c, basisIdx (bar c) = basisIdx c
  conj_contrComm : ∀ (d : C) (₁x : basisIdx d) (₂x : basisIdx (toTensorSpeciesτ. d)),
      star (toTensorSpecies.contr d (b d ₁x ⊗ₜ[k] b (toTensorSpeciesτ. d) ₂x))
        = toTensorSpecies.contr (bar d) (b (bar d) ((Equiv.cast (barIdx_eq d)).symm ₁x) ⊗ₜ[k]
            b (toTensorSpeciesτ. (bar d)) (basisIdxCongr (bar_tau d)
              ((Equiv.cast (barIdx_eq (toTensorSpeciesτ. d))).symm ₂x)))
```

def ConjTensorSpecies.componentReindex [source]

Reindex component labels of ‘bar ∘ c‘ back to ‘c‘, slotwise via the cast ‘barIdx_eq‘.

```
def componentReindex {n : ℕ} (c : Fin n → C) :
    ComponentIdx (S := S.toTensorSpecies) (S.bar ∘ c) ≃ ComponentIdx (S := S.toTensorSpecies) c
```

def ConjTensorSpecies.conjT [source]

Conjugation of a tensor: conjugate the components and reindex the basis to the conjugate colours. ‘conj‘-semilinear by construction (see ‘conjT_smul‘).

```
def conjT {n : ℕ} {c : Fin n → C} (t : S.Tensor c) : S.Tensor (S.bar ∘ c)
```

lemma ConjTensorSpecies.conjT_eq_permT_iff [source]

Componentwise criterion for ‘conjT t = permT σ h t’‘. The conjugate of ‘t‘ equals the recolouring ‘permT σ h t’‘ exactly when, at every component, the ‘star‘-conjugated reindexed component of ‘t‘ matches the corresponding component of ‘permT σ h t’‘. This packages the ‘componentMap_conjT‘ expansion and the ‘repr‘/‘componentMap‘ bridge that the reality and Hermiticity proofs downstream would otherwise repeat by

hand; the caller is left only with the species-specific permutation bookkeeping on the right-hand side.

```
lemma conjT_eq_permT_iff {n m : ℕ} {c : Fin n → C} {c' : Fin m → C}
    σ{ : Fin n → Fin m} (h : IsReindexing c' (S.bar ∘ c) σ)
    (t : S.Tensor c) (t' : S.Tensor c') :
    S.conjT t = permT σ h t' ↔
      ∀ φ : ComponentIdx (S := S.toTensorSpecies) (S.bar ∘ c),
        star (componentMap c t (S.componentReindex c φ))
          = (Tensor.basis (S.bar ∘ c)).repr (permT σ h t') φ
```

lemma ConjTensorSpecies.barIdx_involutive_symm [source]

Undoing the two cast reindexings (at ‘bar c‘ then at ‘c‘) on a label of the doubly-conjugated colour returns the ‘bar_involution‘ cast. Free from ‘barIdx_eq‘ by ‘cast_cast‘ + proof irrelevance, with no coherence field needed.

```
private lemma barIdx_involutive_symm (c : C) (y : basisIdx (S.bar (S.bar c))) :
    Equiv.cast (S.barIdx_eq c) (Equiv.cast (S.barIdx_eq (S.bar c)) y)
      = basisIdxCongr (S.bar_involution c) y
```

lemma ConjTensorSpecies.isReindexing_bar_bar [source]

The identity permutation satisfies ‘IsReindexing‘ from ‘c‘ to ‘bar ∘ bar ∘ c‘, as ‘bar‘ is an involution.

```
lemma isReindexing_bar_bar {n : ℕ} (c : Fin n → C) :
    IsReindexing c (S.bar ∘ S.bar ∘ c) (id : Fin n → Fin n)
```

lemma ConjTensorSpecies.conjT_conjT [source]

Conjugation is an involution: conjugating twice returns the original tensor, up to the ‘bar_involution‘ recolouring (the identity permutation ‘permT‘).

```
lemma conjT_conjT {n : ℕ} {c : Fin n → C} (t : S.Tensor c) :
    S.conjT (S.conjT t)
      = permT id (S.isReindexing_bar_bar c) t
```

lemma ConjTensorSpecies.contrCond_bar [source]

Slots with dual colour in ‘c‘ have dual colour in ‘bar ∘ c‘, as ‘bar‘ commutes with ‘τ‘.

```
lemma contrCond_bar {n : ℕ} {c : Fin (n + 1 + 1) → C} {i j : Fin (n + 1 + 1)}
    (h : i ≠ j ∧ Sτ. (c i) = c j) :
    i ≠ j ∧ Sτ. ((S.bar ∘ c) i) = (S.bar ∘ c) j
```

lemma ConjTensorSpecies.conjT_contrT [source]

Conjugation commutes with contraction: conjugating a contracted tensor equals contracting the conjugate on the ‘bar‘-images of the same two slots.

```
lemma conjT_contrT {n : ℕ} {c : Fin (n + 1 + 1) → C} (i j : Fin (n + 1 + 1))
    (h : i ≠ j ∧ Sτ. (c i) = c j) (t : S.Tensor c) :
    S.conjT (contrT n i j h t)
      = contrT n i j (S.contrCond_bar h) (S.conjT t)
```

def ConjTensorSpecies.conjEquiv [source]

The conjugate-linear slot isomorphism ‘V c ≃ₛₗ[starRingEnd k] V (bar c)‘: conjugate the coordinates in the species basis and relabel to the conjugate colour via ‘barIdx_eq‘.

```
def conjEquiv {c : C} : V c ≃ₛₗ[starRingEnd k] V (S.bar c)
```

def ConjTensorSpecies.IsHermitian [source]

A pairing ‘g : V c ⊗ V (bar c) → k‘ is Hermitian when conjugating and swapping its two slots via ‘conjEquiv‘ returns its ‘star‘: ‘g (x ⊗ y) = star (g (conjEquiv.symm y ⊗ conjEquiv x))‘.

```
def IsHermitian {c : C} (g : V c ⊗[k] V (S.bar c) →ₗ[k] k) : Prop
```

2.52 Physlib.Relativity.Tensors.Constructors

Physlib/Relativity/Tensors/Constructors.lean on GitHub.

lemma TensorSpecies.Tensor.fromPairT_eq_pure [source]

```
lemma fromPairT_eq_pure {c1 c2 : C} (x : V c1) (y : V c2) :
    fromPairT (S := S) (x ⊗ₜ[k] y) = Pure.toTensor (fun | 0 => x | 1 => y)
```

2.53 Physlib.Relativity.Tensors.Contraction.SuccSuccAbove

Physlib/Relativity/Tensors/Contraction/SuccSuccAbove.lean on GitHub.

lemma Fin.apply_succSuccAbove_symm [source]

```
@[simp, nolint simpVarHead]
lemma apply_succSuccAbove_symm {c : Fin (n + 1 + 1) → C} (i j : Fin (n + 1 + 1))
    (k : Fin n) : c (succSuccAbove i j k) = c (succSuccAbove j i k)
```

2.54 Physlib.Relativity.Tensors.Evaluation

Physlib/Relativity/Tensors/Evaluation.lean on GitHub.

lemma TensorSpecies.Tensor.evalT_basis_single [source]

Evaluating the single-index basis tensor ‘basis ![c] (single.symm b)‘ at the index ‘x‘ yields the field element ‘1‘ if ‘b = x‘ (transported across ‘![c] 0 = c‘) and ‘0‘ otherwise: evaluation of a one-index basis tensor is the Kronecker delta.

```
lemma evalT_basis_single {c : C} (b : basisIdx c) (x : basisIdx (![c] 0)) :
    (evalT 0 x (basis (S := S) ![c] (ComponentIdx.single.symm b))).toField =
    if basisIdxCongr (by simp) b =  x then 1 else 0
```

lemma TensorSpecies.Tensor.eq_sum_evalT_of_single_tensor_basis [source]

Basis expansion of a one-index tensor: every ‘t : Tensor S ![c]‘ is the sum over basis indices ‘i‘ of its evaluation coefficient ‘toField (evalT 0 i t)‘ times the corresponding basis tensor.

```
lemma eq_sum_evalT_of_single_tensor_basis {c : C} (t : Tensor S ![c]) :
    t = ∑ i, toField (evalT 0 i t) • basis ![c] (ComponentIdx.single.symm
      (basisIdxCongr (by simp) i))
```

lemma TensorSpecies.Tensor.eq_sum_evalT [source]

Reconstruction of a tensor from the evaluations of its last index: every ‘t : Tensor S c‘ is the sum over basis indices ‘i‘ of the evaluation ‘evalT (Fin.last n) i t‘ tensored with the basis covector ‘basis ![c (Fin.last n)] (single.symm i)‘, with the appended index permuted back into the last slot.

```
lemma eq_sum_evalT {n : ℕ} {c : Fin (n + 1) → C} (t : Tensor S c) :
    t = ∑ i, permT id (IsReindexing.append_succ_last c) (prodT (evalT (Fin.last n) i t)
      (basis ![c (Fin.last n)] (ComponentIdx.single.symm i)))
```

lemma TensorSpecies.Tensor.eq_sum_evalT_zero [source]

Reconstruction of a tensor from the evaluations of its first index: every ‘t : Tensor S c‘ is the sum over basis indices ‘i‘ of the basis covector ‘basis ![c 0] (single.symm i)‘ tensored with the evaluation ‘evalT 0 i t‘, with the prepended index permuted back into the first slot. This is the first-index analogue of ‘eq_sum_evalT‘.

```
lemma eq_sum_evalT_zero {n : ℕ} {c : Fin (n + 1) → C} (t : Tensor S c) :
    t = ∑ i, permT _ (IsReindexing.append_of_first c)
    (prodT (basis ![c 0] (ComponentIdx.single.symm i)) (evalT 0 i t))
```

2.55 Physlib.Relativity.Tensors.LeviCivita.Basic

Physlib/Relativity/Tensors/LeviCivita/Basic.lean on GitHub.

def realLorentzTensor.leviCivita [source]

The Levi-Civita tensor ‘$\varepsilon^{\mu\nu\rho\delta}$‘ as a real Lorentz tensor in ‘d = 3‘, with ‘$\varepsilon^{0123} = 1$‘.

The component on a multi-index ‘f‘ is the generalized Kronecker delta of ‘f‘ against the identity, i.e. the sign of ‘f‘ when ‘f‘ is a permutation and ‘0‘ otherwise.

```
noncomputable def leviCivita : ℝT[3, .up, .up, .up, .up]
```

lemma realLorentzTensor.leviCivita_eq_ofInt [source]

The ‘TensorInt.toTensor‘ form of the Levi-Civita tensor.

```
lemma leviCivita_eq_ofInt : ε4 =
    TensorInt.toTensor (S := realLorentzTensor 3)
    (c := ![Color.up, Color.up, Color.up, Color.up]) fun f =>
    generalizedKroneckerDelta (fun i => finSumFinEquiv (f i)) (id : Fin 4 → Fin 4)
```

lemma realLorentzTensor.leviCivita_basis_repr_apply [source]

The components of the Levi-Civita tensor in the standard basis are the generalized Kronecker delta of the multi-index against the identity.

```
lemma leviCivita_basis_repr_apply
    (b : ComponentIdx (S := realLorentzTensor 3) ![Color.up, Color.up, Color.up, Color.up]) :
    (Tensor.basis _).repr ε4 b
      = (generalizedKroneckerDelta (fun i => finSumFinEquiv (b i)) (id : Fin 4 → Fin 4) : ℝ)
```

lemma realLorentzTensor.leviCivita_basis_repr_eq_zero_of_eq [source]

The Levi-Civita tensor vanishes on any multi-index with a repeated value: if two distinct index positions ‘i ≠ j‘ carry the same basis index, the component is zero.

```
lemma leviCivita_basis_repr_eq_zero_of_eq
    {b : ComponentIdx (S := realLorentzTensor 3) ![Color.up, Color.up, Color.up, Color.up]}
    {i j : Fin 4} (hij : i ≠ j) (h : b i = b j) :
    (Tensor.basis _).repr ε4 b = 0
```

lemma realLorentzTensor.leviCivita_antisymm [source]

The Levi-Civita tensor is antisymmetric in its first two indices ‘{ε4 | μ ν ρ σ = - ε4 | ν μ ρ σ}ᵀ‘.

```
lemma leviCivita_antisymm : ε{4 | μ ν ρ σ = - ε(4 | ν μ ρ σ)ᵀ}
```

lemma realLorentzTensor.leviCivita_antisymm_mid [source]

The Levi-Civita tensor is antisymmetric in its middle two indices ‘{ε4 | μ ν ρ σ = - ε4 | μ ρ ν σ}ᵀ‘.

```
lemma leviCivita_antisymm_mid : ε{4 | μ ν ρ σ = - ε(4 | μ ρ ν σ)ᵀ}
```

lemma realLorentzTensor.leviCivita_antisymm_last [source]

The Levi-Civita tensor is antisymmetric in its last two indices ‘{ε4 | μ ν ρ σ = - ε4 | μ ν σ ρ}ᵀ‘.

```
lemma leviCivita_antisymm_last : ε{4 | μ ν ρ σ = - ε(4 | μ ν σ ρ)ᵀ}
```

2.56 Physlib.Relativity.Tensors.Product

Physlib/Relativity/Tensors/Product.lean on GitHub.

lemma TensorSpecies.Tensor.Pure.prodP_update_left [source]

```
@[simp]
lemma Pure.prodP_update_left {n1 n2} {c : Fin n1 → C} {c1 : Fin n2 → C}
    (p1 : Pure S c) (p2 : Pure S c1) (i : Fin n1) (x : V (c i)) :
    Pure.prodP (Pure.update p1 i x) p2
    = Pure.update (Pure.prodP p1 p2) (Fin.castAdd n2 i)
      (LinearEquiv.cast (R := k) (by simp) x)
```

lemma TensorSpecies.Tensor.Pure.prodP_update_right [source]

```
@[simp]
lemma Pure.prodP_update_right {n1 n2} {c : Fin n1 → C} {c1 : Fin n2 → C}
```

```
    (p1 : Pure S c) (p2 : Pure S c1) (i : Fin n2) (x : V (c1 i)) :
    Pure.prodP p1 (Pure.update p2 i x)
    = Pure.update (Pure.prodP p1 p2) (Fin.natAdd n1 i)
      (LinearEquiv.cast (R := k) (by simp) x)
```

lemma TensorSpecies.Tensor.Pure.prodP_update_add_toTensor_left [source]

```
lemma Pure.prodP_update_add_toTensor_left {n1 n2} {c : Fin n1 → C} {c1 : Fin n2 → C}
    (p1 : Pure S c) (p2 : Pure S c1) (i : Fin n2) (x y : V (c1 i)) :
    (Pure.prodP p1 (Pure.update p2 i (x + y))).toTensor
    = (Pure.prodP p1 (Pure.update p2 i x)).toTensor
    + (Pure.prodP p1 (Pure.update p2 i y)).toTensor
```

lemma TensorSpecies.Tensor.prodT_basis' [source]

```
lemma prodT_basis' {n1 n2} {c : Fin n1 → C} {c1 : Fin n2 → C}
    (b : ComponentIdx c) (b1 : ComponentIdx (S := S) c1) :
    (basis c b).prodT (basis c1 b1) =
    basis (Fin.append c c1) (ComponentIdx.prod.symm (b, b1))
```

lemma TensorSpecies.Tensor.prodT_zero_right [source]

```
lemma prodT_zero_right {n} {c : Fin n → C}
    {c1 : Fin 0 → C} (t : S.Tensor c) (t1 : S.Tensor c1) :
    prodT t t1 = (toField t1) • permT id (IsReindexing.append_zero_right) t
```

2.57 Physlib.Relativity.Tensors.RealTensor.Basic

Physlib/Relativity/Tensors/RealTensor/Basic.lean on GitHub.

abbrev realLorentzTensor.modules [source]

The modules associated with each of the different types of real Lorentz vector space.

```
abbrev modules (d : ℕ) : Color → Type
  | Color.up => Lorentz.ContrMod d
  | Color.down => Lorentz.CoMod d
```

instance realLorentzTensor.modulesAddCommGroup [source]

```
instance modulesAddCommGroup (d : ℕ) : ∀ c, AddCommGroup (modules d c)
  | Color.up => inferInstance
  | Color.down => inferInstance
```

instance realLorentzTensor.modulesModule [source]

```
instance modulesModule (d : ℕ) : ∀ c, Module ℝ (modules d c)
  | Color.up => inferInstance
  | Color.down => inferInstance
```

2.58 Physlib.Relativity.Tensors.RealTensor.CoVector.Basic

Physlib/Relativity/Tensors/RealTensor/CoVector/Basic.lean on GitHub.

lemma Lorentz.CoVector.eq_of_apply_eq [source]

```
@[ext]
lemma eq_of_apply_eq {d : ℕ} {v w : CoVector d} (h : ∀ i, v i = w i) : v = w
```

lemma Lorentz.CoVector.apply_sum [source]

```
@[simp]
lemma apply_sum {d : ℕ} ι{ : Type} [Fintype ι] (f : ι → CoVector d) (i : Fin 1 ⊕ Fin d) :
    ∑( j, f j) i = ∑ j, f j i
```

2.59 Physlib.Relativity.Tensors.RealTensor.CoVector.Representation

Physlib/Relativity/Tensors/RealTensor/CoVector/Representation.lean on GitHub.

def Lorentz.CoVector.rep [source]

The representation of the Lorentz group on Lorentz vectors.

```
def rep {d : ℕ} : Representation ℝ (LorentzGroup d) (CoVector d) where
```

lemma Lorentz.CoVector.rep_apply_eq_mulVec [source]

```
lemma rep_apply_eq_mulVec (d : ℕ) Λ( : LorentzGroup d) (v : CoVector d) :
    rep Λ v = (LorentzGroup.transpose Λ⁻¹) ᵥ* v
```

lemma Lorentz.CoVector.rep_apply_eq_sum [source]

```
lemma rep_apply_eq_sum (d : ℕ) Λ( : LorentzGroup d) (v : CoVector d) (k : Fin 1 ⊕ Fin d) :
    rep Λ v k = ∑ j, Λ⁻(¹).1 j k • v j
```

lemma Lorentz.CoVector.rep_apply_eq_sum_coe [source]

```
lemma rep_apply_eq_sum_coe (d : ℕ) Λ( : LorentzGroup d) (v : CoVector d) (k : Fin 1 ⊕ Fin d) :
    rep Λ v k = ∑ j, Λ⁻.1¹ j k • v j
```

lemma Lorentz.CoVector.rep_apply_basis [source]

```
lemma rep_apply_basis {d} μ( : Fin 1 ⊕ Fin d) Λ( : LorentzGroup d) :
    rep Λ (basis μ) = ∑ j, Λ⁻.1¹ μ j • basis j
```

lemma Lorentz.CoVector.rep_toMatrix [source]

```
lemma rep_toMatrix (d : ℕ) Λ( : LorentzGroup d) :
    LinearMap.toMatrix basis basis (rep Λ) = Λ⁻.1ᵀ¹
```

lemma Lorentz.CoVector.rep_injective [source]

```
lemma rep_injective (d : ℕ) Λ( : LorentzGroup d) : Function.Injective (rep Λ)
```

lemma Lorentz.CoVector.rep_surjective [source]

```
lemma rep_surjective (d : ℕ) Λ( : LorentzGroup d) : Function.Surjective (rep Λ)
```

lemma Lorentz.CoVector.rep_bijective [source]

```
lemma rep_bijective (d : ℕ) Λ( : LorentzGroup d) : Function.Bijective (rep Λ)
```

2.60 Physlib.Relativity.Tensors.RealTensor.CoVector.Tensorial

Physlib/Relativity/Tensors/RealTensor/CoVector/Tensorial.lean on GitHub.

def Lorentz.CoVector.indexEquiv [source]

The equivalence between the type of indices of a Lorentz vector and 'Fin 1 ⊕ Fin d'.

```
def indexEquiv {d : ℕ} :
    ComponentIdx (S := (realLorentzTensor d)) ![Color.down] ≃ Fin 1 ⊕ Fin d
```

instance Lorentz.CoVector.tensorial [source]

```
instance tensorial {d : ℕ} : Tensorial (realLorentzTensor d) ![.down] (CoVector d) where
```

lemma Lorentz.CoVector.toTensor_symm_apply [source]

```
lemma toTensor_symm_apply {d : ℕ} (p : ℝT[d, .down]) :
    (toTensor (self := tensorial)).symm p =
    (Equiv.piCongrLeft' _ indexEquiv <|
    Finsupp.equivFunOnFinite <|
    (Tensor.basis (S := (realLorentzTensor d)) _).repr p)
```

lemma Lorentz.CoVector.toTensor_symm_pure [source]

```
lemma toTensor_symm_pure {d : ℕ} (p : Pure (realLorentzTensor d) ![.down]) (i : Fin 1 ⊕ Fin d)
    :
    (toTensor (self := tensorial)).symm p.toTensor i =
    ((Lorentz.coBasis d).repr (p 0)) (indexEquiv.symm i 0)
```

lemma Lorentz.CoVector.toTensor_symm_basis [source]

```
lemma toTensor_symm_basis {d : ℕ} μ( : Fin 1 ⊕ Fin d) :
    (toTensor (self := tensorial)).symm (Tensor.basis ![Color.down] (indexEquiv.symm μ)) =
    basis μ
```

lemma Lorentz.CoVector.toTensor_basis_eq_tensor_basis [source]

```
lemma toTensor_basis_eq_tensor_basis {d : ℕ} μ( : Fin 1 ⊕ Fin d) :
    toTensor (basis μ) = Tensor.basis ![Color.down] (indexEquiv.symm μ)
```

lemma Lorentz.CoVector.basis_eq_map_tensor_basis [source]

```
lemma basis_eq_map_tensor_basis {d} : basis =
    ((Tensor.basis (S := realLorentzTensor d) ![Color.down]).map
    toTensor.symm).reindex indexEquiv
```

lemma Lorentz.CoVector.tensor_basis_map_eq_basis_reindex [source]

```
lemma tensor_basis_map_eq_basis_reindex {d} :
    (Tensor.basis (S := realLorentzTensor d) ![Color.down]).map toTensor.symm =
    basis.reindex indexEquiv.symm
```

lemma Lorentz.CoVector.tensor_basis_repr_toTensor_apply [source]

```
lemma tensor_basis_repr_toTensor_apply {d : ℕ} (p : CoVector d) μ( : ComponentIdx ![Color.down
    ]) :
    (Tensor.basis ![Color.down]).repr (toTensor p) μ =
    p (indexEquiv μ)
```

lemma Lorentz.CoVector.smul_eq_sum [source]

```
lemma smul_eq_sum {d : ℕ} (i : Fin 1 ⊕ Fin d) Λ( : LorentzGroup d) (p : CoVector d) :
    Λ( • p) i = ∑ j, Λ⁻¹.1 j i * p j
```

lemma Lorentz.CoVector.smul_eq_mulVec [source]

```
lemma smul_eq_mulVec {d} Λ( : LorentzGroup d) (p : CoVector d) :
    Λ • p = (LorentzGroup.transpose Λ⁻¹).1 ᵥ* p
```

lemma Lorentz.CoVector.smul_add [source]

```
lemma smul_add {d : ℕ} Λ( : LorentzGroup d) (p q : CoVector d) :
    Λ • (p + q) = Λ • p + Λ • q
```

lemma Lorentz.CoVector.smul_sub [source]

```
@[simp]
lemma smul_sub {d : ℕ} Λ( : LorentzGroup d) (p q : CoVector d) :
    Λ • (p - q) = Λ • p - Λ • q
```

lemma Lorentz.CoVector.smul_zero [source]

```
lemma smul_zero {d : ℕ} Λ( : LorentzGroup d) :
    Λ • (0 : CoVector d) = 0
```

lemma Lorentz.CoVector.smul_neg [source]

```
lemma smul_neg {d : ℕ} Λ( : LorentzGroup d) (p : CoVector d) :
    Λ • (-p) = - Λ( • p)
```

def Lorentz.CoVector.actionCLM [source]

The Lorentz action on vectors as a continuous linear map.

```
def actionCLM {d : ℕ} Λ( : LorentzGroup d) :
    CoVector d →L[ℝ] CoVector d
```

lemma Lorentz.CoVector.actionCLM_apply [source]

```
lemma actionCLM_apply {d : ℕ} Λ( : LorentzGroup d) (p : CoVector d) :
    actionCLM Λ p = Λ • p
```

lemma Lorentz.CoVector.smul_basis [source]

```
lemma smul_basis {d : ℕ} Λ( : LorentzGroup d) μ( : Fin 1 ⊕ Fin d) :
    Λ • basis μ = ∑ ν, Λ⁻¹.1 μ ν • basis ν
```

2.61 Physlib.Relativity.Tensors.RealTensor.ToComplex

Physlib/Relativity/Tensors/RealTensor/ToComplex.lean on GitHub.

def realLorentzTensor.toComplexVector [source]

For a given color, the map turning a real Lorentz vector into a complex one.

```
noncomputable def toComplexVector (c : realLorentzTensor.Color) :
  realLorentzTensor.modules 3 c →ₛₗ[Complex.ofRealHom] complexLorentzTensor.modules
    (colorToComplex c) where
```

lemma realLorentzTensor.toComplexVector_up_eq_inclCongrRealLorentz [source]

```
lemma toComplexVector_up_eq_inclCongrRealLorentz (v : Lorentz.ContrMod 3) :
    toComplexVector Color.up v = Lorentz.inclCongrRealLorentz v
```

lemma realLorentzTensor.toComplexVector_down_eq_inclCoRealLorentz [source]

```
lemma toComplexVector_down_eq_inclCoRealLorentz (v : Lorentz.CoMod 3) :
    toComplexVector Color.down v = Lorentz.inclCoRealLorentz v
```

def realLorentzTensor.toComplexPure [source]

The function which turns a real pure tensor into a complex one.

```
noncomputable def toComplexPure {c : Fin n → Color} (p : Pure realLorentzTensor c) :
    Pure complexLorentzTensor (colorToComplex ∘ c)
```

lemma realLorentzTensor.toComplexPure_component [source]

```
lemma toComplexPure_component {c : Fin n → Color} (p : Pure realLorentzTensor c)
    φ( : ComponentIdx c) : (toComplexPure p).component (ComponentIdx.complexify φ) =
      p.component φ
```

lemma realLorentzTensor.actionP_toComplexPure [source]

```
lemma actionP_toComplexPure {n : ℕ} (c : Fin n → Color) (p : Pure realLorentzTensor c)
    Λ( : SL(2, ℂ)) :
    Λ • toComplexPure p = toComplexPure (toLorentzGroup Λ • p)
```

lemma realLorentzTensor.toComplex_pure [source]

```
lemma toComplex_pure {n : ℕ} (c : Fin n → Color) (p : Pure realLorentzTensor c) :
    toComplex p.toTensor = (toComplexPure p).toTensor
```

lemma realLorentzTensor.isReindexing_prodTColorToComplex [source]

```
lemma isReindexing_prodTColorToComplex {n m : ℕ}
    {c : Fin n → realLorentzTensor.Color} {c1 : Fin m → realLorentzTensor.Color} :
    IsReindexing (Fin.append (colorToComplex ∘ c) (colorToComplex ∘ c1))
      (colorToComplex ∘ Fin.append c c1)
      (id : Fin (n + m) → Fin (n + m))
```

2.62 Physlib.Relativity.Tensors.RealTensor.Vector.Basic

Physlib/Relativity/Tensors/RealTensor/Vector/Basic.lean on GitHub.

lemma Lorentz.Vector.eq_of_apply_eq [source]

```
@[ext]
lemma eq_of_apply_eq {d : ℕ} {v w : Vector d} (h : ∀ i, v i = w i) : v = w
```

def Lorentz.Vector.ofSpatialComponent [source]

The continuous linear map corresponding to the creation of a Lorentz Vector with only non-zero spatial components.

```
def ofSpatialComponent {d : ℕ} : EuclideanSpace ℝ (Fin d) →L[ℝ] Vector d where
```

2.63 Physlib.Relativity.Tensors.RealTensor.Vector.Representation

Physlib/Relativity/Tensors/RealTensor/Vector/Representation.lean on GitHub.

def Lorentz.Vector.rep [source]

The representation of the Lorentz group on Lorentz vectors.

```
def rep {d : ℕ} : Representation ℝ (LorentzGroup d) (Vector d) where
```

lemma Lorentz.Vector.rep_apply_eq_mulVec [source]

```
lemma rep_apply_eq_mulVec (d : ℕ) Λ( : LorentzGroup d) (v : Vector d) :
    rep Λ v = Λ ᵥ* v
```

lemma Lorentz.Vector.rep_apply_eq_sum [source]

```
lemma rep_apply_eq_sum (d : ℕ) Λ( : LorentzGroup d) (v : Vector d) (k : Fin 1 ⊕ Fin d) :
    rep Λ v k = ∑ j, Λ.1 k j • v j
```

lemma Lorentz.Vector.rep_apply_basis [source]

```
lemma rep_apply_basis {d} μ( : Fin 1 ⊕ Fin d) Λ( : LorentzGroup d) :
    rep Λ (basis μ) = ∑ j, Λ.1 j μ • basis j
```

lemma Lorentz.Vector.rep_toMatrix [source]

```
lemma rep_toMatrix (d : ℕ) Λ( : LorentzGroup d) :
    LinearMap.toMatrix basis basis (rep Λ) = Λ.1
```

lemma Lorentz.Vector.rep_injective [source]

```
lemma rep_injective (d : ℕ) Λ( : LorentzGroup d) : Function.Injective (rep Λ)
```

lemma Lorentz.Vector.rep_surjective [source]

```
lemma rep_surjective (d : ℕ) Λ( : LorentzGroup d) : Function.Surjective (rep Λ)
```

lemma Lorentz.Vector.rep_bijective [source]

```
lemma rep_bijective (d : ℕ) Λ( : LorentzGroup d) : Function.Bijective (rep Λ)
```

lemma Lorentz.Vector.rep_contDiff [source]

```
@[fun_prop]
lemma rep_contDiff (d : ℕ) {n} Λ( : LorentzGroup d) : ContDiff ℝ n (rep Λ)
```

lemma Lorentz.Vector.rep_left_injective [source]

```
lemma rep_left_injective (d : ℕ) : Function.Injective (rep (d := d))
```

2.64 Physlib.Relativity.Tensors.RealTensor.Vector.Tensorial

Physlib/Relativity/Tensors/RealTensor/Vector/Tensorial.lean on GitHub.

def Lorentz.Vector.indexEquiv [source]

The equivalence between the type of indices of a Lorentz vector and 'Fin 1 ⊕ Fin d'.

```
def indexEquiv {d : ℕ} :
    ComponentIdx (S := (realLorentzTensor d)) ![Color.up] ≃ Fin 1 ⊕ Fin d
```

instance Lorentz.Vector.tensorial [source]

```
instance tensorial {d : ℕ} : Tensorial (realLorentzTensor d) ![.up] (Vector d) where
```

lemma Lorentz.Vector.toTensor_symm_apply [source]

```
lemma toTensor_symm_apply {d : ℕ} (p : ℝT[d, .up]) :
    (toTensor (self := tensorial)).symm p =
    (Equiv.piCongrLeft' _ indexEquiv <|
    Finsupp.equivFunOnFinite <|
    (Tensor.basis (S := (realLorentzTensor d)) _).repr p)
```

lemma Lorentz.Vector.toTensor_symm_pure [source]

```
lemma toTensor_symm_pure {d : ℕ} (p : Pure (realLorentzTensor d) ![.up]) (i : Fin 1 ⊕ Fin d) :
    (toTensor (self := tensorial)).symm p.toTensor i =
    ((Lorentz.contrBasis d).repr (p 0)) (indexEquiv.symm i 0)
```

lemma Lorentz.Vector.toTensor_symm_basis [source]

```
lemma toTensor_symm_basis {d : ℕ} μ( : Fin 1 ⊕ Fin d) :
    (toTensor (self := tensorial)).symm (Tensor.basis ![Color.up] (indexEquiv.symm μ)) =
    basis μ
```

lemma Lorentz.Vector.toTensor_basis_eq_tensor_basis [source]

```
lemma toTensor_basis_eq_tensor_basis {d : ℕ} μ( : Fin 1 ⊕ Fin d) :
    toTensor (basis μ) = Tensor.basis ![Color.up] (indexEquiv.symm μ)
```

lemma Lorentz.Vector.basis_eq_map_tensor_basis [source]

```
lemma basis_eq_map_tensor_basis {d} : basis =
    ((Tensor.basis
    (S := realLorentzTensor d) ![Color.up]).map toTensor.symm).reindex indexEquiv
```

lemma Lorentz.Vector.tensor_basis_map_eq_basis_reindex [source]

```
lemma tensor_basis_map_eq_basis_reindex {d} :
    (Tensor.basis (S := realLorentzTensor d) ![Color.up]).map toTensor.symm =
    basis.reindex indexEquiv.symm
```

lemma Lorentz.Vector.tensor_basis_repr_toTensor_apply [source]

```
lemma tensor_basis_repr_toTensor_apply {d : ℕ} (p : Vector d) μ( : ComponentIdx ![Color.up]) :
    (Tensor.basis ![Color.up]).repr (toTensor p) μ =
    p (indexEquiv μ)
```

lemma Lorentz.Vector.smul_eq_sum [source]

```
lemma smul_eq_sum {d : ℕ} (i : Fin 1 ⊕ Fin d) Λ( : LorentzGroup d) (p : Vector d) :
    Λ( • p) i = ∑ j, Λ.1 i j * p j
```

lemma Lorentz.Vector.smul_eq_mulVec [source]

```
lemma smul_eq_mulVec {d} Λ( : LorentzGroup d) (p : Vector d) :
    Λ • p = Λ.1 ᵥ* p
```

lemma Lorentz.Vector.smul_add [source]

```
lemma smul_add {d : ℕ} Λ( : LorentzGroup d) (p q : Vector d) :
    Λ • (p + q) = Λ • p + Λ • q
```

lemma Lorentz.Vector.smul_sub [source]

```
@[simp]
lemma smul_sub {d : ℕ} Λ( : LorentzGroup d) (p q : Vector d) :
    Λ • (p - q) = Λ • p - Λ • q
```

lemma Lorentz.Vector.smul_zero [source]

```
lemma smul_zero {d : ℕ} Λ( : LorentzGroup d) :
    Λ • (0 : Vector d) = 0
```

lemma Lorentz.Vector.smul_neg [source]

```
lemma smul_neg {d : ℕ} Λ( : LorentzGroup d) (p : Vector d) :
    Λ • (-p) = - Λ( • p)
```

lemma Lorentz.Vector.neg_smul [source]

```
lemma neg_smul {d} Λ( : LorentzGroup d) (p : Vector d) :
    Λ(-) • p = - Λ( • p)
```

lemma LorentzGroup.eq_of_action_vector_eq [source]

```
lemma _root_.LorentzGroup.eq_of_action_vector_eq {d : ℕ}
    Λ{ Λ' : LorentzGroup d} (h : ∀ p : Vector d, Λ • p = Λ' • p) :
    Λ = Λ'
```

def Lorentz.Vector.actionCLM [source]

The Lorentz action on vectors as a continuous linear map.

```
def actionCLM {d : ℕ} Λ( : LorentzGroup d) :
    Vector d →L[ℝ] Vector d
```

lemma Lorentz.Vector.actionCLM_apply [source]

```
lemma actionCLM_apply {d : ℕ} Λ( : LorentzGroup d) (p : Vector d) :
    actionCLM Λ p = Λ • p
```

lemma Lorentz.Vector.actionCLM_injective [source]

```
lemma actionCLM_injective {d : ℕ} Λ( : LorentzGroup d) :
    Function.Injective (actionCLM Λ)
```

lemma Lorentz.Vector.actionCLM_surjective [source]

```
lemma actionCLM_surjective {d : ℕ} Λ( : LorentzGroup d) :
    Function.Surjective (actionCLM Λ)
```

lemma Lorentz.Vector.smul_basis [source]

```
lemma smul_basis {d : ℕ} Λ( : LorentzGroup d) μ( : Fin 1 ⊕ Fin d) :
    Λ • basis μ = ∑ ν, Λ.1 ν μ • basis ν
```

2.65 Physlib.Relativity.Tensors.Reindexing

Physlib/Relativity/Tensors/Reindexing.lean on GitHub.

def TensorSpecies.Tensor.IsReindexing [source]

Given two lists of indices ‘c : Fin n → C‘ and ‘c1 : Fin m → C‘ a map ‘σ : Fin m → Fin n‘ satisfies the condition ‘IsReindexing c c1 σ‘ if it is: - A bijection - Forms a commutative triangle with ‘c‘ and ‘c1‘.

```
def IsReindexing {n m : ℕ} (c : Fin n → C) (c1 : Fin m → C)
    σ( : Fin m → Fin n) : Prop
```

lemma TensorSpecies.Tensor.IsReindexing.injective [source]

```
lemma injective {n m : ℕ} {c : Fin n → C} {c1 : Fin m → C}
    σ{ : Fin m → Fin n} (h : IsReindexing c c1 σ) : Function.Injective σ
```

lemma TensorSpecies.Tensor.IsReindexing.surjective [source]

```
lemma surjective {n m : ℕ} {c : Fin n → C} {c1 : Fin m → C}
    σ{ : Fin m → Fin n} (h : IsReindexing c c1 σ) : Function.Surjective σ
```

lemma TensorSpecies.Tensor.IsReindexing.auto [source]

```
lemma auto {n m : ℕ} {c : Fin n → C} {c1 : Fin m → C}
    σ{ : Fin m → Fin n} (h : IsReindexing c c1 σ := by {simp [IsReindexing]; try decide}) :
    IsReindexing c c1 σ
```

lemma TensorSpecies.Tensor.IsReindexing.on_id [source]

```
@[simp]
lemma on_id {n : ℕ} {c c1 : Fin n → C} :
    IsReindexing c c1 (id : Fin n → Fin n) ↔ ∀ i, c i = c1 i
```

lemma TensorSpecies.Tensor.IsReindexing.on_id_symm [source]

```
lemma on_id_symm {n : ℕ} {c c1 : Fin n → C} (h : IsReindexing c1 c id) :
    IsReindexing c c1 (id : Fin n → Fin n)
```

def TensorSpecies.Tensor.IsReindexing.inv [source]

For a map ‘σ‘ satisfying ‘IsReindexing c c1 σ‘, the inverse of that map.

```
def inv {n m : ℕ} {c : Fin n → C} {c1 : Fin m → C}
    σ( : Fin m → Fin n) (h : IsReindexing c c1 σ) : Fin n → Fin m
```

def TensorSpecies.Tensor.IsReindexing.toEquiv [source]

For a map ‘σ : Fin m → Fin n‘ satisfying ‘IsReindexing c c1 σ‘, that map lifted to an equivalence between ‘Fin n‘ and ‘Fin m‘.

```
def toEquiv {n m : ℕ} {c : Fin n → C} {c1 : Fin m → C}
    σ{ : Fin m → Fin n} (h : IsReindexing c c1 σ) :
    Fin n ≃ Fin m where
```

lemma TensorSpecies.Tensor.IsReindexing.apply_inv_apply [source]

```
lemma apply_inv_apply {n m : ℕ} {c : Fin n → C} {c1 : Fin m → C}
    σ( : Fin m → Fin n) (h : IsReindexing c c1 σ) (x : Fin m) :
    h.inv σ σ( x) = x
```

lemma TensorSpecies.Tensor.IsReindexing.inv_apply_apply [source]

```
lemma inv_apply_apply {n m : ℕ} {c : Fin n → C} {c1 : Fin m → C}
    σ( : Fin m → Fin n) (h : IsReindexing c c1 σ) (x : Fin n) :
    σ (h.inv σ x) = x
```

lemma TensorSpecies.Tensor.IsReindexing.preserve_color [source]

```
lemma preserve_color {n m : ℕ} {c : Fin n → C} {c1 : Fin m → C}
    σ{ : Fin m → Fin n} (h : IsReindexing c c1 σ) :
    ∀ (x : Fin m), c1 x = (c ∘ σ) x
```

lemma TensorSpecies.Tensor.IsReindexing.inv_perserve_color [source]

```
@[simp, nolint simpVarHead]
lemma inv_perserve_color {n m : ℕ} {c : Fin n → C} {c1 : Fin m → C}
    σ{ : Fin m → Fin n} (h : IsReindexing c c1 σ) (x : Fin n) :
    c1 (h.inv σ x) = c x
```

lemma TensorSpecies.Tensor.IsReindexing.toEquiv_symm_perserve_color [source]

```
@[simp, nolint simpVarHead]
lemma toEquiv_symm_perserve_color {n m : ℕ} {c : Fin n → C} {c1 : Fin m → C}
    σ{ : Fin m → Fin n} (h : IsReindexing c c1 σ) (x : Fin m) :
    c (h.toEquiv.symm x) = c1 x
```

lemma TensorSpecies.Tensor.IsReindexing.symm [source]

The inverse of a map satisfying ‘IsReindexing c c1 σ‘ is a reindexing of ‘c1‘ by ‘c‘.

```
lemma symm {n m : ℕ} {c : Fin n → C} {c1 : Fin m → C}
    σ{ : Fin m → Fin n} (h : IsReindexing c c1 σ) : IsReindexing c1 c (h.inv σ)
```

lemma TensorSpecies.Tensor.IsReindexing.symm_of_id [source]

```
lemma symm_of_id {n : ℕ} {c c1 : Fin n → C} (h : IsReindexing c c1 id) :
    IsReindexing c1 c id
```

lemma TensorSpecies.Tensor.IsReindexing.comp [source]

The composition of two maps satisfying 'IsReindexing' also satisfies the 'IsReindexing'.

```
lemma comp {n n1 n2 : ℕ} {c : Fin n → C} {c1 : Fin n1 → C}
    {c2 : Fin n2 → C} σ{ : Fin n1 → Fin n} σ{2 : Fin n2 → Fin n1}
    (h : IsReindexing c c1 σ) (h2 : IsReindexing c1 c2 σ2) : IsReindexing c c2 σ( ∘ σ2)
```

lemma TensorSpecies.Tensor.IsReindexing.append_congr_left [source]

```
lemma append_congr_left {n n' n2 : ℕ} {c : Fin n → C} {c' : Fin n' → C}
    σ{ : Fin n' → Fin n} (c2 : Fin n2 → C) (h : IsReindexing c c' σ) :
    IsReindexing (Fin.append c c2) (Fin.append c' c2)
      (Fin.append (Fin.castAdd n2 ∘ σ) (Fin.natAdd n))
```

lemma TensorSpecies.Tensor.IsReindexing.append_congr_right [source]

```
lemma append_congr_right {n n' n2 : ℕ} {c : Fin n → C} {c' : Fin n' → C}
    σ{ : Fin n' → Fin n} (c2 : Fin n2 → C) (h : IsReindexing c c' σ) :
    IsReindexing (Fin.append c2 c) (Fin.append c2 c')
      (Fin.append (Fin.castAdd n) (Fin.natAdd n2 ∘ σ))
```

lemma TensorSpecies.Tensor.IsReindexing.append_zero_right [source]

```
lemma append_zero_right {n} {c : Fin n → C}
    {c1 : Fin 0 → C} : IsReindexing c (Fin.append c c1) id
```

lemma TensorSpecies.Tensor.IsReindexing.append_swap [source]

```
lemma append_swap {n n2 : ℕ} {c : Fin n → C} {c2 : Fin n2 → C} :
    IsReindexing (Fin.append c c2) (Fin.append c2 c)
      (Fin.append (Fin.natAdd n) (Fin.castAdd n2))
```

lemma TensorSpecies.Tensor.IsReindexing.append_assoc_right [source]

```
lemma append_assoc_right   {n1 n2 n3 : ℕ} {c : Fin n1 → C} {c2 : Fin n2 → C} {c3 : Fin n3 → C}
    :
    IsReindexing (Fin.append c (Fin.append c2 c3)) (Fin.append (Fin.append c c2) c3)
      (Fin.cast (by grind))
```

lemma TensorSpecies.Tensor.IsReindexing.append_assoc_left [source]

```
lemma append_assoc_left {n1 n2 n3 : ℕ} {c : Fin n1 → C} {c2 : Fin n2 → C} {c3 : Fin n3 → C} :
    IsReindexing (Fin.append (Fin.append c c2) c3) (Fin.append c (Fin.append c2 c3))
      (Fin.cast (by grind))
```

lemma TensorSpecies.Tensor.IsReindexing.append_succAbove_natAdd [source]

```
lemma append_succAbove_natAdd {n n1 : ℕ} {c : Fin n → C} {c1 : Fin (n1 + 1) → C}
    (i : Fin (n1 + 1)) :
    IsReindexing (Fin.append c c1 ∘ (Fin.natAdd n i).succAbove)
      (Fin.append c (c1 ∘ i.succAbove)) id
```

lemma TensorSpecies.Tensor.IsReindexing.append_succAbove_castAdd [source]

```
lemma append_succAbove_castAdd {n n1 : ℕ} {c : Fin (n + 1) → C} {c1 : Fin (n1 + 1) → C}
    (i : Fin (n + 1)) :
    IsReindexing (Fin.append c c1 ∘ (Fin.castAdd (n1 + 1) i).succAbove)
      (Fin.append (c ∘ i.succAbove) c1) (Fin.cast (by grind))
```

lemma TensorSpecies.Tensor.IsReindexing.append_succSuccAbove_natAdd [source]

*Removing two entries from the right component of ‘Fin.append c1 c‘ commutes with the append: removing the ‘i‘-th and ‘j‘-th entries of ‘c‘ and then appending ‘c1‘ matches removing the corresponding entries of ‘Fin.append c1 c‘, via the identity permutation. This is used for the commutation of taking a *product* of tensors with *contraction* of indices.*

```
lemma append_succSuccAbove_natAdd {n n1 : ℕ} {c : Fin (n + 1 + 1) → C}
    {c1 : Fin n1 → C} (i j : Fin (n + 1 + 1)) :
    IsReindexing (Fin.append c1 c ∘ (Fin.natAdd n1 i).succSuccAbove (Fin.natAdd n1 j))
      (Fin.append c1 (c ∘ i.succSuccAbove j)) id
```

lemma TensorSpecies.Tensor.IsReindexing.succAbove_of_eq_zero [source]

Given a reindexing of ‘c‘ by ‘c1‘ via ‘σ‘ for which the index ‘i‘ is sent to ‘0‘, removing the ‘i‘-th entry of ‘c1‘ and the first entry of ‘c‘ yields a reindexing of ‘c ∘ Fin.succ‘ by ‘c1 ∘ i.succAbove‘ via the map sending ‘j‘ to the predecessor of ‘σ (i.succAbove j)‘.

```
lemma succAbove_of_eq_zero {n n1 : ℕ} {c : Fin (n + 1) → C} {c1 : Fin (n1 + 1) → C}
    σ{ : Fin (n1 + 1) → Fin (n + 1)} (i : Fin (n1 + 1))
    (h : IsReindexing c c1 σ) (hi : σ i = 0) :
    IsReindexing (c ∘ Fin.succ) (c1 ∘ i.succAbove)
      (fun j => σ( (i.succAbove j)).pred (by simp ←[ hi, h.injective.eq_iff]))
```

lemma TensorSpecies.Tensor.IsReindexing.succAbove_of_neq_zero [source]

Given a reindexing of ‘c‘ by ‘c1‘ via ‘σ‘ for which the index ‘i‘ is not sent to ‘0‘, removing the ‘i‘-th entry of ‘c1‘ and the ‘(σ i)‘-th entry of ‘c‘ yields a reindexing of ‘c ∘ (σ i).succAbove‘ by ‘c1 ∘ i.succAbove‘ via the map ‘(σ i).pred.predAbove ∘ σ ∘ i.succAbove‘.

```
lemma succAbove_of_neq_zero {n n1 : ℕ} {c : Fin (n + 1) → C} {c1 : Fin (n1 + 1) → C}
    σ{ : Fin (n1 + 1) → Fin (n + 1)} (i : Fin (n1 + 1))
    (h : IsReindexing c c1 σ) (hi : σ i ≠ 0) :
    IsReindexing (c ∘ σ( i).succAbove) (c1 ∘ i.succAbove)
      ((Fin.pred σ( i) hi).predAbove  ∘ σ ∘ i.succAbove)
```

lemma TensorSpecies.Tensor.IsReindexing.succAbove [source]

*Given a reindexing of ‘c‘ by ‘c1‘ via ‘σ‘, removing the ‘i‘-th entry of ‘c1‘ and the ‘(σ i)‘-th entry of ‘c‘ yields a reindexing of ‘c ∘ (σ i).succAbove‘ by ‘c1 ∘ i.succAbove‘. This unifies ‘succAbove_of_eq_zero‘ and ‘succAbove_of_neq_zero‘ via a case split on whether ‘σ i = 0‘. This is used for the commutation of *permutation* of indices with *evaluation* of indices.*

```
lemma succAbove {n n1 : ℕ} {c : Fin (n + 1) → C} {c1 : Fin (n1 + 1) → C}
    σ{ : Fin (n1 + 1) → Fin (n + 1)} (i : Fin (n1 + 1))
    (h : IsReindexing c c1 σ) :
    IsReindexing (c ∘ σ( i).succAbove) (c1 ∘ i.succAbove)
      (if hi : σ i = 0 then fun j => σ( (i.succAbove j)).pred (by simp ←[ hi, h.injective.
    eq_iff])
      else (Fin.pred σ( i) hi).predAbove  ∘ σ ∘ i.succAbove)
```

lemma TensorSpecies.Tensor.IsReindexing.succSuccAbove [source]

*Given a reindexing of ‘c‘ by ‘c1‘ via ‘σ‘ and two distinct indices ‘i ≠ j‘, removing the ‘i‘-th and ‘j‘-th entries of ‘c1‘ and the ‘(σ i)‘-th and ‘(σ j)‘-th entries of ‘c‘ yields a reindexing of ‘c ∘ (σ i).succSuccAbove (σ j)‘ by ‘c1 ∘ i.succSuccAbove j‘. This is used for the commutation of *permutation* of indices with *contraction* of indices.*

```
lemma succSuccAbove {n n1 : ℕ} {c : Fin (n + 1 + 1) → C}
    {c1 : Fin (n1 + 1 + 1) → C}
    (i j : Fin (n1 + 1 + 1)) (hij : i ≠ j)
    σ{ : Fin (n1 + 1 + 1) → Fin (n + 1 + 1)} (σh : IsReindexing c c1 σ) :
    IsReindexing (c ∘  σ( i).succSuccAbove σ( j))
      (c1 ∘ i.succSuccAbove j) (i.funPredPredAbove j hij σ σh.1)
```

lemma TensorSpecies.Tensor.IsReindexing.succSuccAbove_comm [source]

*Removing two pairs of entries from ‘c‘ in either order gives the same colour list: removing the ‘i1‘-th and ‘j1‘-th entries and then the (shifted) ‘i2‘-th and ‘j2‘-th entries matches removing the ‘i2‘-th and ‘j2‘-th entries first and then the (shifted) ‘i1‘-th and ‘j1‘-th entries, via the identity permutation. This is used for the commutation of two *contractions*.*

```
lemma succSuccAbove_comm {n : ℕ} {c : Fin (n + 1 + 1 + 1 + 1) → C}
    (i1 j1 : Fin (n + 1 + 1 + 1 + 1)) (i2 j2 : Fin (n + 1 + 1))
    (hij1 : i1 ≠ j1) (hij2 : i2 ≠ j2) :
    let i2'
```

lemma TensorSpecies.Tensor.IsReindexing.succAbove_succSuccAbove_comm [source]

*Removing one entry and then a pair of entries from ‘c‘ in either order gives the same colour list: removing the ‘k‘-th entry and then the (shifted) ‘i‘-th and ‘j‘-th entries matches removing the ‘i‘-th and ‘j‘-th entries first and then the (shifted) ‘k‘-th entry, via the identity permutation. This is used for the commutation of a *contraction* with an *evaluation*.*

```
lemma succAbove_succSuccAbove_comm {n : ℕ} {c : Fin (n + 1 + 1 + 1) → C}
    (k : Fin (n + 1 + 1 + 1)) (i j : Fin (n + 1 + 1)) (hij : i ≠ j) :
    let i'
```

lemma TensorSpecies.Tensor.IsReindexing.succAbove_succAbove_comm [source]

*Removing two single entries from ‘c‘ in either order gives the same colour list: removing the ‘k1‘-th entry and then the (shifted) ‘k2‘-th entry matches removing the ‘k2‘-th entry first and then the (shifted) ‘k1‘-th entry, via the identity permutation. This is used for the commutation of two *evaluations* of indices.*

```
lemma succAbove_succAbove_comm {n : ℕ} {c : Fin (n + 1 + 1) → C}
    (k1 : Fin (n + 1 + 1)) (k2 : Fin (n + 1)) :
    let k2'
```

lemma TensorSpecies.Tensor.IsReindexing.append_succ_last [source]

Splitting a list of colours ‘c : Fin (n + 1) → C‘ into its first ‘n‘ entries and its last entry recovers ‘c‘: the identity permutation matches ‘Fin.append (c ∘ (Fin.last n).succAbove) ![c (Fin.last n)]‘ with ‘c‘.

```
lemma append_succ_last {n : ℕ} (c : Fin (n + 1) → C) :
    IsReindexing (Fin.append (c ∘ (Fin.last n).succAbove) ![c (Fin.last n)]) c id
```

lemma TensorSpecies.Tensor.IsReindexing.append_of_first [source]

Splitting a list of colours ‘c : Fin (n + 1) → C‘ into its first entry and its remaining ‘n‘ entries recovers ‘c‘: the canonical reindexing ‘Fin (1 + n) ≃ Fin (n + 1)‘ matches ‘Fin.append ![c 0] (c ∘ Fin.succAbove 0)‘ with ‘c‘.

```
lemma append_of_first {n : ℕ} (c : Fin (n + 1) → C) :
    IsReindexing (Fin.append ![c 0] (c ∘ Fin.succAbove 0)) c (Fin.cast (by grind))
```

lemma TensorSpecies.Tensor.IsReindexing.fin_cast_isReindexing [source]

Casting the domain along an equality ‘n1 = n‘ of lengths is a reindexing of ‘c‘ by ‘c ∘ Fin.cast h‘.

```
lemma fin_cast_isReindexing (n n1 : ℕ) {c : Fin n → C} (h : n1 = n) :
    IsReindexing c (c ∘ Fin.cast h) (Fin.cast h)
```

2.66 Physlib.Relativity.Tensors.Tensorial

Physlib/Relativity/Tensors/Tensorial.lean on GitHub.

lemma TensorSpecies.Tensorial.prod_eq_sum_eval [source]

Double basis expansion of an element of a tensor product ‘M ⊗[k] M_2‘ of two ‘Tensorial‘ one-index spaces. Given bases ‘b‘, ‘b2‘ of ‘M‘, ‘M_2‘ coming from the single-index tensor bases, every ‘x : M ⊗[k] M_2‘ is the double sum over ‘i, j‘ of the iterated evaluation coefficient ‘toField (evalT 0 j (evalT 0 i (toTensor x)))‘ times ‘b i ⊗ₜ b2 j‘.

```
lemma prod_eq_sum_eval {c c2 : C} {₂M : Type}
    [Tensorial S ![c] M] [AddCommMonoid ₂M] [Module k ₂M]
    [Tensorial S ![c2] ₂M] {b : Module.Basis (basisIdx c) k M}
    {b2 : Module.Basis (basisIdx c2) k ₂M}
    (h : b = ((Tensor.basis (S := S) ![c]).map toTensor.symm).reindex ComponentIdx.single)
    (h2 : b2 = ((Tensor.basis (S := S) ![c2]).map toTensor.symm).reindex ComponentIdx.single)
```

```
    (x : M ⊗[k] ₂M) : x = ∑ i : (basisIdx c), ∑ j : (basisIdx c2),
    toField (evalT 0 (basisIdxCongr (by rfl) j)
      (evalT 0 (basisIdxCongr (by rfl) i) (toTensor x))) • b i ⊗ₜ[k] b2 j
```

2.67 Physlib.Relativity.Tensors.TensorSpecies.Basic

Physlib/Relativity/Tensors/TensorSpecies/Basic.lean on GitHub.

lemma TensorSpecies.basisIdxCongr_heq_arg [source]

'basisIdxCongr' only depends on its endpoints up to 'HEq' of the arguments: equal target colours and heterogeneously equal labels give equal casts.

```
lemma basisIdxCongr_heq_arg {c1 c2 d : C} (h1 : c1 = d) (h2 : c2 = d)
    {x : basisIdx c1} {y : basisIdx c2} (hxy : HEq x y) :
    basisIdxCongr h1 x = basisIdxCongr h2 y
```

2.68 Physlib.SpaceAndTime.GalileanGroup.Basic

Physlib/SpaceAndTime/GalileanGroup/Basic.lean on GitHub.

structure GalileanGroup [source]

A Galilean transformation in 'd' spatial dimensions. The fields are, in order, the spatial orthogonal part, boost velocity, spatial translation, and time translation.

```
@[ext]
structure GalileanGroup (d : ℕ := 3) where
  rotation : Matrix.orthogonalGroup (Fin d) ℝ
  velocity : EuclideanSpace ℝ (Fin d)
  spaceTranslation : EuclideanSpace ℝ (Fin d)
  timeTranslation : Time
```

lemma GalileanGroup.orthogonal_smul_smul [source]

```
lemma orthogonal_smul_smul (R : Matrix.orthogonalGroup (Fin d) ℝ) (c : ℝ)
    (v : EuclideanSpace ℝ (Fin d)) :
    R • (c • v) = c • (R • v)
```

instance GalileanGroup.«instance@f34d5bc7» [source]

The identity Galilean transformation.

```
instance : One (GalileanGroup d) where
```

lemma GalileanGroup.one_rotation [source]

```
@[simp]
lemma one_rotation : (1 : GalileanGroup d).rotation = 1
```

lemma GalileanGroup.one_velocity [source]

```
@[simp]
lemma one_velocity : (1 : GalileanGroup d).velocity = 0
```

lemma GalileanGroup.one_spaceTranslation [source]

```
@[simp]
lemma one_spaceTranslation : (1 : GalileanGroup d).spaceTranslation = 0
```

lemma GalileanGroup.one_timeTranslation [source]

```
@[simp]
lemma one_timeTranslation : (1 : GalileanGroup d).timeTranslation = 0
```

instance GalileanGroup.«instance@2d436bb6» [source]

*The product whose action is composition: ‘(g * h) • tx = g • h • tx‘.*

```
instance : Mul (GalileanGroup d) where
```

lemma GalileanGroup.mul_rotation [source]

```
@[simp]
lemma mul_rotation (g h : GalileanGroup d) :
    (g * h).rotation = g.rotation * h.rotation
```

lemma GalileanGroup.mul_velocity [source]

```
@[simp]
lemma mul_velocity (g h : GalileanGroup d) :
    (g * h).velocity = g.rotation • h.velocity + g.velocity
```

lemma GalileanGroup.mul_spaceTranslation [source]

```
@[simp]
lemma mul_spaceTranslation (g h : GalileanGroup d) :
    (g * h).spaceTranslation =
      g.spaceTranslation + g.rotation • h.spaceTranslation + h.timeTranslation.val • g.velocity
```

lemma GalileanGroup.mul_timeTranslation [source]

```
@[simp]
lemma mul_timeTranslation (g h : GalileanGroup d) :
    (g * h).timeTranslation = g.timeTranslation + h.timeTranslation
```

instance GalileanGroup.«instance@3107b840» [source]

The inverse Galilean transformation.

```
instance : Inv (GalileanGroup d) where
```

lemma GalileanGroup.inv_rotation [source]

```
@[simp]
lemma inv_rotation (g : GalileanGroup d) :
    ⁻g¹.rotation = g.⁻rotation¹
```

lemma GalileanGroup.inv_velocity [source]

The inverse boost velocity formula.

```
@[simp]
lemma inv_velocity (g : GalileanGroup d) :
    ⁻g¹.velocity = -(g.⁻rotation¹ • g.velocity)
```

lemma GalileanGroup.inv_spaceTranslation [source]

```
@[simp]
lemma inv_spaceTranslation (g : GalileanGroup d) :
    ⁻g¹.spaceTranslation =
      -(g.⁻rotation¹ • g.spaceTranslation) +
        g.timeTranslation.val • (g.⁻rotation¹ • g.velocity)
```

lemma GalileanGroup.inv_timeTranslation [source]

```
@[simp]
lemma inv_timeTranslation (g : GalileanGroup d) :
    ⁻g¹.timeTranslation = -g.timeTranslation
```

instance GalileanGroup.«instance@33c7352c» [source]

The Galilean transformations form a group under composition.

```
instance : Group (GalileanGroup d) where
```

instance GalileanGroup.«instance@5e5b6b3a» [source]

```
instance : Inhabited (GalileanGroup d) where
```

def GalileanGroup.actSpace [source]

The Galilean action on space at a fixed time.

```
noncomputable def actSpace (g : GalileanGroup d) (t : Time) (x : Space d) : Space d
```

def GalileanGroup.act [source]

The Galilean action on 'Time × Space d'.

```
noncomputable def act (g : GalileanGroup d) (tx : Time × Space d) : Time × Space d
```

instance GalileanGroup.«instance@c761f5d5» [source]

```
noncomputable instance : MulAction (GalileanGroup d) (Time × Space d) where
```

lemma GalileanGroup.smul_fst [source]

```
@[simp]
lemma smul_fst (g : GalileanGroup d) (tx : Time × Space d) :
    (g • tx).1 = tx.1 + g.timeTranslation
```

lemma GalileanGroup.smul_snd [source]

```
@[simp]
lemma smul_snd (g : GalileanGroup d) (tx : Time × Space d) :
    (g • tx).2 = g.actSpace tx.1 tx.2
```

lemma GalileanGroup.smul_mk [source]

```
@[simp]
lemma smul_mk (g : GalileanGroup d) (t : Time) (x : Space d) :
    g • ((t, x) : Time × Space d) = (t + g.timeTranslation, g.actSpace t x)
```

lemma GalileanGroup.actSpace_apply [source]

```
@[simp]
lemma actSpace_apply (g : GalileanGroup d) (t : Time) (x : Space d) (i : Fin d) :
    g.actSpace t x i =
      (g.rotation • (x ᵥ- (0 : Space d))) i + t.val * g.velocity i + g.spaceTranslation i
```

def GalileanGroup.ofEuclidean [source]

A Euclidean spatial transformation as a Galilean transformation with zero boost and no time translation.

```
def ofEuclidean (g : EuclideanGroup d) : GalileanGroup d
```

lemma GalileanGroup.ofEuclidean_rotation [source]

```
@[simp]
lemma ofEuclidean_rotation (g : EuclideanGroup d) :
    (ofEuclidean g).rotation = g.linear
```

lemma GalileanGroup.ofEuclidean_velocity [source]

```
@[simp]
lemma ofEuclidean_velocity (g : EuclideanGroup d) :
    (ofEuclidean g).velocity = 0
```

lemma GalileanGroup.ofEuclidean_spaceTranslation [source]

```
@[simp]
lemma ofEuclidean_spaceTranslation (g : EuclideanGroup d) :
    (ofEuclidean g).spaceTranslation = g.translation
```

lemma GalileanGroup.ofEuclidean_timeTranslation [source]

```
@[simp]
lemma ofEuclidean_timeTranslation (g : EuclideanGroup d) :
    (ofEuclidean g).timeTranslation = 0
```

def GalileanGroup.euclidean.incl [source]

Inclusion of the Euclidean group into the Galilean group.

```
def euclidean.incl : EuclideanGroup d →* GalileanGroup d where
```

def GalileanGroup.ofOrthogonal [source]

A pure orthogonal spatial transformation as a Galilean transformation.

```
def ofOrthogonal (R : Matrix.orthogonalGroup (Fin d) ℝ) : GalileanGroup d
```

lemma GalileanGroup.ofOrthogonal_rotation [source]

```
@[simp]
lemma ofOrthogonal_rotation (R : Matrix.orthogonalGroup (Fin d) ℝ) :
    (ofOrthogonal R).rotation = R
```

lemma GalileanGroup.ofOrthogonal_velocity [source]

```
@[simp]
lemma ofOrthogonal_velocity (R : Matrix.orthogonalGroup (Fin d) ℝ) :
    (ofOrthogonal R).velocity = 0
```

lemma GalileanGroup.ofOrthogonal_spaceTranslation [source]

```
@[simp]
lemma ofOrthogonal_spaceTranslation (R : Matrix.orthogonalGroup (Fin d) ℝ) :
    (ofOrthogonal R).spaceTranslation = 0
```

lemma GalileanGroup.ofOrthogonal_timeTranslation [source]

```
@[simp]
lemma ofOrthogonal_timeTranslation (R : Matrix.orthogonalGroup (Fin d) ℝ) :
    (ofOrthogonal R).timeTranslation = 0
```

def GalileanGroup.orthogonal.incl [source]

Inclusion of the orthogonal group into the Galilean group.

```
def orthogonal.incl : Matrix.orthogonalGroup (Fin d) ℝ →* GalileanGroup d where
```

def GalileanGroup.rotation.incl [source]

Inclusion of the existing spatial rotation subgroup into the Galilean group.

```
noncomputable def rotation.incl : EuclideanGroup.RotationGroup d →* GalileanGroup d where
```

def GalileanGroup.ofSpaceTranslation [source]

A pure spatial translation as a Galilean transformation.

```
def ofSpaceTranslation (a : EuclideanSpace ℝ (Fin d)) : GalileanGroup d
```

lemma GalileanGroup.ofSpaceTranslation_rotation [source]

```
@[simp]
lemma ofSpaceTranslation_rotation (a : EuclideanSpace ℝ (Fin d)) :
    (ofSpaceTranslation a).rotation = 1
```

lemma GalileanGroup.ofSpaceTranslation_velocity [source]

```
@[simp]
lemma ofSpaceTranslation_velocity (a : EuclideanSpace ℝ (Fin d)) :
    (ofSpaceTranslation a).velocity = 0
```

lemma GalileanGroup.ofSpaceTranslation_spaceTranslation [source]

```
@[simp]
lemma ofSpaceTranslation_spaceTranslation (a : EuclideanSpace ℝ (Fin d)) :
    (ofSpaceTranslation a).spaceTranslation = a
```

lemma GalileanGroup.ofSpaceTranslation_timeTranslation [source]

```
@[simp]
lemma ofSpaceTranslation_timeTranslation (a : EuclideanSpace ℝ (Fin d)) :
    (ofSpaceTranslation a).timeTranslation = 0
```

def GalileanGroup.spaceTranslation.incl [source]

Inclusion of spatial translations into the Galilean group.

```
def spaceTranslation.incl :
    Multiplicative (EuclideanSpace ℝ (Fin d)) →* GalileanGroup d where
```

def GalileanGroup.spaceTranslationGroup.incl [source]

Inclusion of the existing spatial translation subgroup into the Galilean group.

```
noncomputable def spaceTranslationGroup.incl :
    EuclideanGroup.TranslationGroup d →* GalileanGroup d
```

def GalileanGroup.ofTimeTranslation [source]

A pure time translation as a Galilean transformation.

```
def ofTimeTranslation (b : Time) : GalileanGroup d
```

lemma GalileanGroup.ofTimeTranslation_rotation [source]

```
@[simp]
lemma ofTimeTranslation_rotation (b : Time) :
    (ofTimeTranslation (d := d) b).rotation = 1
```

lemma GalileanGroup.ofTimeTranslation_velocity [source]

```
@[simp]
lemma ofTimeTranslation_velocity (b : Time) :
    (ofTimeTranslation (d := d) b).velocity = 0
```

lemma GalileanGroup.ofTimeTranslation_spaceTranslation [source]

```
@[simp]
lemma ofTimeTranslation_spaceTranslation (b : Time) :
    (ofTimeTranslation (d := d) b).spaceTranslation = 0
```

lemma GalileanGroup.ofTimeTranslation_timeTranslation [source]

```
@[simp]
lemma ofTimeTranslation_timeTranslation (b : Time) :
    (ofTimeTranslation (d := d) b).timeTranslation = b
```

def GalileanGroup.timeTranslation.incl [source]

Inclusion of time translations into the Galilean group.

```
def timeTranslation.incl : Multiplicative Time →* GalileanGroup d where
```

2.69 Physlib.SpaceAndTime.Space.Basic

Physlib/SpaceAndTime/Space/Basic.lean on GitHub.

instance Space.instNontrivial [source]

```
instance instNontrivial {d : ℕ} [NeZero d] : Nontrivial (Space d) where
```

def Space.homEuclideanSpaceSpace [source]

'Space d' is homoemorphic to d-dimensional euclidean space. This is used to define the 'IsManifold' instance on 'Space d'.

```
noncomputable def homEuclideanSpaceSpace (d : ℕ) : EuclideanSpace ℝ (Fin d) ≃ₜ Space d where
```

instance Space.«instance@b9d497fa» [source]

```
noncomputable instance (d : ℕ) : ChartedSpace (EuclideanSpace ℝ (Fin d)) (Space d)
```

instance Space.«instance@d376417d» [source]

```
instance (d : ℕ) : IsManifold 𝓘( d) ⊤ (Space d)
```

lemma Space.chartAt_eq [source]

```
lemma chartAt_eq (d : ℕ) (p : Space d) :
    chartAt (EuclideanSpace ℝ (Fin d)) p =
    (homEuclideanSpaceSpace d).symm.toOpenPartialHomeomorph
```

2.70 Physlib.SpaceAndTime.Space.Derivatives.Basic

Physlib/SpaceAndTime/Space/Derivatives/Basic.lean on GitHub.

lemma Space.deriv_sub [source]

Derivatives on space distribute over subtraction.

```
@[to_fun]
lemma deriv_sub [NormedAddCommGroup M] [NormedSpace ℝ M]
    (f1 f2 : Space d → M) (hf1 : Differentiable ℝ f1) (hf2 : Differentiable ℝ f2) :
    ∂[u] (f1 - f2) = ∂[u] f1 - ∂[u] f2
```

2.71 Physlib.SpaceAndTime.Space.Derivatives.Curl

Physlib/SpaceAndTime/Space/Derivatives/Curl.lean on GitHub.

lemma Space.distCurl_coord_apply [source]

```
lemma distCurl_coord_apply {i : Fin 3}
    (f : Space →d[ℝ] (EuclideanSpace ℝ (Fin 3))) (η( : □(Space, ℝ)) :
    ∇ᵈ( × f) η i = - f (SchwartzMap.evalCLM ℝ Space ℝ (basis (i+2)) (fderivCLM ℝ Space ℝ η)) (i
    +1)
    + f (SchwartzMap.evalCLM ℝ Space ℝ (basis (i+1)) (fderivCLM ℝ Space ℝ η)) (i+2)
```

2.72 Physlib.SpaceAndTime.Space.Derivatives.Grad

Physlib/SpaceAndTime/Space/Derivatives/Grad.lean on GitHub.

lemma Space.grad_eq_gradient [source]

```
lemma grad_eq_gradient {d} (f : Space d → ℝ) :
    ∇ f = basis.repr ∘ gradient f
```

lemma Space.distGrad_const [source]

```
@[simp]
lemma distGrad_const {d} (c : ℝ) :
    ∇ᵈ (Distribution.const ℝ (Space d) c) = 0
```

2.73 Physlib.SpaceAndTime.Space.Derivatives.Laplacian

Physlib/SpaceAndTime/Space/Derivatives/Laplacian.lean on GitHub.

lemma Space.laplacian_eq_sum_snd_deriv [source]

```
lemma laplacian_eq_sum_snd_deriv {d} (f : Space d → ℝ) :
    Δ f = fun x => ∑ i, ∂[i] ∂([i] f) x
```

def Space.distLaplacian [source]

The distributional 'distLaplacian' operator.

```
noncomputable def distLaplacian {d} :
    ((Space d) →d[ℝ] ℝ) →ₗ[ℝ] (Space d) →d[ℝ] ℝ
```

lemma Space.distLaplacian_const [source]

```
@[simp]
lemma distLaplacian_const {d : ℕ} (c : ℝ) :
    Δᵈ (Distribution.const ℝ (Space d) c) = 0
```

2.74 Physlib.SpaceAndTime.Space.DistOfFunction

Physlib/SpaceAndTime/Space/DistOfFunction.lean on GitHub.

lemma Space.distDeriv_distOfFunction [source]

The distributional spatial derivative of the distribution associated to a differentiable function is the distribution associated to the corresponding pointwise spatial derivative.

```
lemma distDeriv_distOfFunction {d : ℕ} μ( : Fin d) (f : Space d → F)
    (hf : IsDistBounded f) (hf_deriv : IsDistBounded ∂μ([] f))
    (hf_diff : Differentiable ℝ f) :
    ∂ᵈμ[] (distOfFunction f hf) = distOfFunction ∂μ([] f) hf_deriv
```

2.75 Physlib.SpaceAndTime.Space.EuclideanGroup.Action

Physlib/SpaceAndTime/Space/EuclideanGroup/Action.lean on GitHub.

instance EuclideanGroup.«instance@6372eb27» [source]

The action of the Euclidean group on the affine space of points ‘Space d‘: ‘$g = \langle t, Q \rangle$‘ rotates ‘p‘ about the coordinate origin by the orthogonal part ‘Q‘ and then translates by ‘t‘.

```
noncomputable instance : MulAction (EuclideanGroup d) (Space d) where
```

lemma EuclideanGroup.dist_smul [source]

*The Euclidean group acts on ‘Space d‘ **by isometries**: every rigid motion preserves distance.*

```
lemma dist_smul (g : EuclideanGroup d) (p q : Space d) :
    dist (g • p) (g • q) = dist p q
```

lemma EuclideanGroup.rotation_smul_vsub_origin [source]

A rotation acts on the displacement from the origin by its orthogonal part, for every ‘p‘: ‘$(r \bullet p) -_v \text{origin} = Q \bullet (p -_v \text{origin})$‘.

```
lemma rotation_smul_vsub_origin (r : RotationGroup d) (p : Space d) :
    (r • p) ᵥ- (0 : Space d) = (r : EuclideanGroup d).linear • (p ᵥ- (0 : Space d))
```

lemma EuclideanGroup.rotation_dist_smul [source]

*The rotation group acts on ‘Space d‘ **by isometries** (inherited from ‘dist_smul‘).*

```
lemma rotation_dist_smul (r : RotationGroup d) (p q : Space d) :
    dist (r • p) (r • q) = dist p q
```

lemma EuclideanGroup.chartEuclidean_smul [source]

Under the standard chart, the Euclidean action on ‘Space d‘ is the transport of ‘toAffineIsometryMulEquiv‘ acting on ‘EuclideanSpace‘: ‘chart (g • p) = (toAffineIsometryMulEquiv g) (chart p)‘.

```
lemma chartEuclidean_smul (g : EuclideanGroup d) (p : Space d) :
    Space.chartEuclidean d (g • p) = toAffineIsometryMulEquiv g (Space.chartEuclidean d p)
```

2.76 Physlib.SpaceAndTime.Space.EuclideanGroup.AffineGroup

Physlib/SpaceAndTime/Space/EuclideanGroup/AffineGroup.lean on GitHub.

def EuclideanGroup.orthogonalToLinearIsometryEquiv [source]

An orthogonal matrix viewed as a linear isometry equivalence of ‘EuclideanSpace ℝ (Fin n)‘; the linear ingredient of the first leg ‘toAffineIsometryHom‘.

```
noncomputable def orthogonalToLinearIsometryEquiv
    (Q : Matrix.orthogonalGroup (Fin n) ℝ) :
    EuclideanSpace ℝ (Fin n) ≃ₗᵢ[ℝ] EuclideanSpace ℝ (Fin n)
```

def EuclideanGroup.toAffineIsometryHom [source]

The first leg of the inclusion: ‘⟨t, Q⟩ ↦ (x ↦ Q x + t)‘, bundled as a monoid homomorphism from the Euclidean group into the affine isometry group of ‘EuclideanSpace ℝ (Fin n)‘.

```
noncomputable def toAffineIsometryHom :
    EuclideanGroup n →*
      AffineIsometryEquiv ℝ (EuclideanSpace ℝ (Fin n)) (EuclideanSpace ℝ (Fin n)) where
```

def AffineIsometryEquiv.toAffineEquivHom [source]

The second leg of the inclusion: ‘AffineIsometryEquiv.toAffineEquiv‘ bundled as a monoid homomorphism into the affine automorphism group.

```
noncomputable def _root_.AffineIsometryEquiv.toAffineEquivHom :
    AffineIsometryEquiv ℝ (EuclideanSpace ℝ (Fin n)) (EuclideanSpace ℝ (Fin n)) →*
      AffineEquiv ℝ (EuclideanSpace ℝ (Fin n)) (EuclideanSpace ℝ (Fin n)) where
```

def EuclideanGroup.toAffineEquiv [source]

The inclusion of the Euclidean group into Mathlib’s affine automorphism group: the composite of the two legs ‘toAffineIsometryHom‘ and ‘AffineIsometryEquiv.toAffineEquivHom‘.

```
noncomputable def toAffineEquiv :
    EuclideanGroup n →* AffineEquiv ℝ (EuclideanSpace ℝ (Fin n)) (EuclideanSpace ℝ (Fin n))
```

def EuclideanGroup.linearIsometryEquivToOrthogonal [source]

The ‘invFun‘ ingredient of ‘toAffineIsometryMulEquiv‘: a linear isometry equivalence read back as an orthogonal matrix, inverse to the linear bridge ‘orthogonalToLinearIsometryEquiv‘ (see the round-trip ‘@[simp]‘ lemmas below).

```
noncomputable def linearIsometryEquivToOrthogonal
    (L : EuclideanSpace ℝ (Fin n) ≃ₗᵢ[ℝ] EuclideanSpace ℝ (Fin n)) :
    Matrix.orthogonalGroup (Fin n) ℝ
```

def EuclideanGroup.toAffineIsometryMulEquiv [source]

'EuclideanGroup n ≃ AffineIsometryEquiv ℝ (EuclideanSpace ℝ (Fin n)) _': the Euclidean group is the full group of affine isometries of Euclidean space. This upgrades 'toAffineIsometryHom' to a group isomorphism with the same underlying map.*

```
noncomputable def toAffineIsometryMulEquiv :
    EuclideanGroup n ≃*
      AffineIsometryEquiv ℝ (EuclideanSpace ℝ (Fin n)) (EuclideanSpace ℝ (Fin n)) where
```

2.77 Physlib.SpaceAndTime.Space.EuclideanGroup.Basic

Physlib/SpaceAndTime/Space/EuclideanGroup/Basic.lean on GitHub.

structure EuclideanGroup [source]

An n-dimensional 'Euclidean group' is a group of rotations, reflections, and translations.

```
@[ext]
structure EuclideanGroup (n : ℕ) where
  translation : EuclideanSpace ℝ (Fin n)
  linear : Matrix.orthogonalGroup (Fin n) ℝ
```

instance EuclideanGroup.«instance@134bed7b» [source]

*Group structure on 'E(n) = ℝ^n ⋊ O(n)'. The orthogonal component acts on translations by the inherited matrix-vector action on 'EuclideanSpace ℝ (Fin n)', transported through 'WithLp' from the coordinate action 'Q.val *ᵥ v.ofLp'.*

```
noncomputable instance : Group (EuclideanGroup n) where
```

lemma EuclideanGroup.det_coe_inv [source]

```
private lemma det_coe_inv {n : ℕ} (Q : Matrix.orthogonalGroup (Fin n) ℝ) :
    (⁻Q¹).val.det = (Q.val.det)⁻¹
```

lemma EuclideanGroup.coe_inv [source]

```
private lemma coe_inv {n : ℕ} (Q : Matrix.orthogonalGroup (Fin n) ℝ) :
    (⁻Q¹).val = (Q.val)⁻¹
```

def EuclideanGroup.SpecialEuclideanGroup [source]

Special Euclidean Group is the subgroup with det(Q) = 1 where Q ∈ O(n)

```
def SpecialEuclideanGroup (n : ℕ) : Subgroup (EuclideanGroup n) where
```

def EuclideanGroup.SpecialEuclideanGroup.incl [source]

The inclusion of the special Euclidean group into the Euclidean group.

```
noncomputable def SpecialEuclideanGroup.incl (n : ℕ) :
    SpecialEuclideanGroup n →* EuclideanGroup n
```

def EuclideanGroup.TranslationGroup [source]

Translation is the subgroup with $Q = 1$.

```
def TranslationGroup (n : ℕ) : Subgroup (EuclideanGroup n) where
```

def EuclideanGroup.TranslationGroup.incl [source]

The inclusion of the translation subgroup into the Euclidean group.

```
noncomputable def TranslationGroup.incl (n : ℕ) :
    TranslationGroup n →* EuclideanGroup n
```

def EuclideanGroup.translationVector.incl [source]

MonoidHom including a translation vector into the Euclidean Group.

```
def translationVector.incl (n : ℕ) :
    Multiplicative (EuclideanSpace ℝ (Fin n)) →* EuclideanGroup n where
```

lemma EuclideanGroup.translationVector.incl_range [source]

An API feature: the translation vector inclusion image is the 'TranslationGroup' carrier.

```
lemma translationVector.incl_range :
    Set.range (@translationVector.incl n) = (TranslationGroup n : Set (EuclideanGroup n))
```

lemma EuclideanGroup.translation_zero [source]

The translation by the zero vector is the identity of the Euclidean group.

```
lemma translation_zero : translationVector.incl n
    (Multiplicative.ofAdd (0 : EuclideanSpace ℝ (Fin n))) = 1
```

def EuclideanGroup.OriginStabilizer [source]

The subgroup of 'EuclideanGroup n' whose elements fix the origin (translation = 0). This is the copy of 'O(n)' sitting inside 'E(n)'.

```
def OriginStabilizer (n : ℕ) : Subgroup (EuclideanGroup n) where
```

def EuclideanGroup.RotationGroup [source]

Rotation Group is the subgroup of E(n) consisting of rotations about the origin: elements with 'det = 1' (orientation-preserving) and 'translation = 0' (origin-fixing).

```
noncomputable def RotationGroup (n : ℕ) : Subgroup (EuclideanGroup n)
```

def EuclideanGroup.RotationGroup.incl [source]

The inclusion of the rotation subgroup into the Euclidean group.

```
noncomputable def RotationGroup.incl (n : ℕ) :
    RotationGroup n →* EuclideanGroup n
```

def EuclideanGroup.RotationsAbout [source]

*The subgroup of rotation about a spatial point ‘p : EuclideanSpace ℝ (Fin n)‘ consists of elements of the form T(p) * r * T(-p) with T(·) : translationVector.incl n (Multiplicative.ofAdd ·) and r : RotationGroup where r is viewed as a rotation about the origin. Note T(-p) = T(p)$^{-1}$.*

```
noncomputable def RotationsAbout : Subgroup (EuclideanGroup n) where
```

def EuclideanGroup.RotationsAbout.incl [source]

The inclusion of rotations about ‘p‘ into the Euclidean group.

```
noncomputable def RotationsAbout.incl : RotationsAbout p →* EuclideanGroup n
```

def EuclideanGroup.RotationsAbout.toOrigin [source]

Conjugate a rotation about ‘p‘ back to a rotation about the origin.

```
noncomputable def RotationsAbout.toOrigin :
    RotationsAbout p →* RotationGroup n where
```

def EuclideanGroup.RotationsAbout.fromOrigin [source]

Conjugate a rotation about the origin to a rotation about ‘p‘.

```
noncomputable def RotationsAbout.fromOrigin :
    RotationGroup n →* RotationsAbout p where
```

lemma EuclideanGroup.RotationsAbout.fromOrigin_comp_toOrigin [source]

‘RotationsAbout.toOrigin p‘ followed by ‘RotationsAbout.fromOrigin p‘ is the identity; the forward leg of the isomorphism ‘RotationsAboutEquiv‘.

```
lemma RotationsAbout.fromOrigin_comp_toOrigin :
    (RotationsAbout.fromOrigin p).comp (RotationsAbout.toOrigin p) =
      MonoidHom.id (RotationsAbout p)
```

lemma EuclideanGroup.RotationsAbout.toOrigin_comp_fromOrigin [source]

‘RotationsAbout.fromOrigin p‘ followed by ‘RotationsAbout.toOrigin p‘ is the identity; the backward leg of the isomorphism ‘RotationsAboutEquiv‘.

```
lemma RotationsAbout.toOrigin_comp_fromOrigin :
    (RotationsAbout.toOrigin p).comp (RotationsAbout.fromOrigin p) =
      MonoidHom.id (RotationGroup n)
```

def EuclideanGroup.RotationsAboutEquiv [source]

API feature: conjugation by the translation ‘T(p)‘ exhibits the rotations about ‘p‘ as isomorphic to the rotations about the origin ‘RotationGroup n‘.

```
noncomputable def RotationsAboutEquiv : RotationsAbout p ≃* RotationGroup n
```

lemma EuclideanGroup.RotationsAbout_zero [source]

API feature: the degenerate identity that ‘RotationsAbout 0 = RotationGroup n‘

```
lemma RotationsAbout_zero : RotationsAbout (0 : EuclideanSpace ℝ (Fin n)) = RotationGroup n
```

def EuclideanGroup.specialOrthogonal.incl [source]

Rotations are members of special orthogonal groups and can be viewed as members of orthogonal groups.

```
def specialOrthogonal.incl (n : ℕ) :
    Matrix.specialOrthogonalGroup (Fin n) ℝ →* Matrix.orthogonalGroup (Fin n) ℝ
```

def EuclideanGroup.ofRotation [source]

The Euclidean group element given by a rotation about the origin (zero translation).

```
def ofRotation (Q : Matrix.specialOrthogonalGroup (Fin n) ℝ) :
    EuclideanGroup n
```

def EuclideanGroup.ofRotationTranslation [source]

Specialization to a group element from a rotation and a translation.

```
def ofRotationTranslation (Q : Matrix.specialOrthogonalGroup (Fin n) ℝ)
    (t : EuclideanSpace ℝ (Fin n)) : EuclideanGroup n
```

lemma EuclideanGroup.ofRotationTranslation_translation [source]

The specialization projects back to the translation component.

```
@[simp]
lemma ofRotationTranslation_translation (Q : Matrix.specialOrthogonalGroup (Fin n) ℝ)
    (t : EuclideanSpace ℝ (Fin n)) :
    (ofRotationTranslation Q t).translation = t
```

lemma EuclideanGroup.ofRotationTranslation_linear [source]

The specialization projects back to the linear (rotation) component.

```
@[simp]
lemma ofRotationTranslation_linear (Q : Matrix.specialOrthogonalGroup (Fin n) ℝ)
    (t : EuclideanSpace ℝ (Fin n)) :
    (ofRotationTranslation Q t).linear = specialOrthogonal.incl n Q
```

lemma EuclideanGroup.ofRotationTranslation_decompose [source]

API feature: the inclusion image decomposes as group product.

```
@[simp]
lemma ofRotationTranslation_decompose (Q : Matrix.specialOrthogonalGroup (Fin n) ℝ)
    (t : EuclideanSpace ℝ (Fin n)) :
    (ofRotationTranslation Q t) =
    (translationVector.incl n (Multiplicative.ofAdd t)) * (ofRotation (Q))
```

def EuclideanGroup.specialOrthogonal.toRotation [source]

The isomorphism's forward map: a special orthogonal matrix as a rotation about the origin.

```
noncomputable def specialOrthogonal.toRotation (n : ℕ) :
    Matrix.specialOrthogonalGroup (Fin n) ℝ →* RotationGroup n where
```

def EuclideanGroup.specialOrthogonal.fromRotation [source]

The isomorphism's inverse map: the linear part of a rotation about the origin, as a special orthogonal matrix.

```
noncomputable def specialOrthogonal.fromRotation (n : ℕ) :
    RotationGroup n →* Matrix.specialOrthogonalGroup (Fin n) ℝ where
```

lemma EuclideanGroup.specialOrthogonal.fromRotation_comp_toRotation [source]

'specialOrthogonal.toRotation n' followed by 'specialOrthogonal.fromRotation n' is the identity; the forward leg of the isomorphism 'specialOrthogonalEquiv'.

```
lemma specialOrthogonal.fromRotation_comp_toRotation :
    (specialOrthogonal.fromRotation n).comp (specialOrthogonal.toRotation n) =
      MonoidHom.id (Matrix.specialOrthogonalGroup (Fin n) ℝ)
```

lemma EuclideanGroup.specialOrthogonal.toRotation_comp_fromRotation [source]

'specialOrthogonal.fromRotation n' followed by 'specialOrthogonal.toRotation n' is the identity; the backward leg of the isomorphism 'specialOrthogonalEquiv'.

```
lemma specialOrthogonal.toRotation_comp_fromRotation :
    (specialOrthogonal.toRotation n).comp (specialOrthogonal.fromRotation n) =
      MonoidHom.id (RotationGroup n)
```

def EuclideanGroup.specialOrthogonalEquiv [source]

API feature: SO(n) ≃ RotationGroup n*

```
noncomputable def specialOrthogonalEquiv :
    Matrix.specialOrthogonalGroup (Fin n) ℝ ≃* RotationGroup n
```

2.78 Physlib.SpaceAndTime.Space.Integrals.Basic

Physlib/SpaceAndTime/Space/Integrals/Basic.lean on GitHub.

lemma Space.integrable_spherical_of_integrable [source]

```
lemma integrable_spherical_of_integrable {d : ℕ} {F : Type*}
    [NormedAddCommGroup F] [NormedSpace ℝ F] {f : Space d → F}
    (hf : Integrable f volume) :
    Integrable
      (fun x : ↑(Metric.sphere (0 : Space d) 1) × Set.Ioi (0 : ℝ) =>
        f (x.2.1 • x.1.1))
      (volume α( := Space d).toSphere.prod
        (Measure.volumeIoiPow (Module.finrank ℝ (Space d) - 1)))
```

lemma Space.integral_volume_eq_spherical_iterated [source]

```
lemma integral_volume_eq_spherical_iterated {d : ℕ} [NeZero d] {F : Type*}
    [NormedAddCommGroup F] [NormedSpace ℝ F] (f : Space d → F)
    (hf : Integrable f volume) :
    ∫ x : Space d, f x =
      ∫ n : ↑(Metric.sphere (0 : Space d) 1),
        ∫ r : Set.Ioi (0 : ℝ), f (r.1 • n.1)
          ∂(Measure.volumeIoiPow (Module.finrank ℝ (Space d) - 1))
        ∂(volume α( := Space d).toSphere)
```

lemma Space.integral_volume_eq_spherical_integral [source]

```
lemma integral_volume_eq_spherical_integral {d : ℕ} [NeZero d] {F : Type*}
    [NormedAddCommGroup F] [NormedSpace ℝ F] (f : Space d → F)
    (hf : Integrable f volume) :
    ∫ x : Space d, f x =
      ∫ n : ↑(Metric.sphere (0 : Space d) 1),
        ∫ r : Set.Ioi (0 : ℝ), (r.1 ^ (d - 1)) • f (r.1 • n.1)
          ∂(.comap Subtype.val volume)
        ∂(volume α( := Space d).toSphere)
```

2.79 Physlib.SpaceAndTime.Space.Integrals.NormPow

Physlib/SpaceAndTime/Space/Integrals/NormPow.lean on GitHub.

lemma Space.radial_jacobian_zpow_mul_self [source]

```
lemma radial_jacobian_zpow_mul_self
    {d p : ℕ} [NeZero d] {q : ℤ} (hp_int : (p : ℤ) = q + (d : ℤ))
    {r : ℝ} (hr : 0 < r) :
    r ^ (d - 1) * (r ^ q * r) = r ^ p
```

lemma Space.radial_jacobian_zpow [source]

```
private lemma radial_jacobian_zpow
    {d p : ℕ} [NeZero d] {q : ℤ} (hp_int : (p : ℤ) = q + (d : ℤ))
    (hp_pos : 0 < p) {r : ℝ} (hr : 0 < r) :
    r ^ (d - 1) * r ^ q = r ^ (p - 1)
```

lemma Space.radial_norm_power_spherical_integral_eq_space_integral [source]

```
lemma radial_norm_power_spherical_integral_eq_space_integral
    {d p : ℕ} [NeZero d] {q : ℤ} (hp_int : (p : ℤ) = q + (d : ℤ))
```

```
    (hp_pos : 0 < p) η( : □(Space d, ℝ)) :
    ∫ x : Space d, η x * |‖x ^ q =
      ∫ (n : ↥(Metric.sphere (0 : Space d) 1)),
        ∫ (r : Set.Ioi (0 : ℝ)),
          r.1 ^ (p - 1) * η (r.1 • n.1)
          ∂(.comap Subtype.val volume)
        ∂(volume α( := Space d).toSphere)
```

2.80 Physlib.SpaceAndTime.Space.Module

Physlib/SpaceAndTime/Space/Module.lean on GitHub.

lemma Space.norm_smul_sphere [source]

```
lemma norm_smul_sphere {d : ℕ} (n : ↥(Metric.sphere (0 : Space d) 1))
    {r : ℝ} (hr : 0 ≤ r) :
    ‖(r • (n : Space d))‖ = r
```

2.81 Physlib.SpaceAndTime.Space.Norm.Basic

Physlib/SpaceAndTime/Space/Norm/Basic.lean on GitHub.

lemma Space.integrable_real_pow_mul_schwartz [source]

```
private lemma integrable_real_pow_mul_schwartz
    ψ( : □ℝ(, ℝ)) (k : ℕ) :
    Integrable (fun x : ℝ => x ^ k * ψ x) volume
```

lemma Space.radial_power_deriv_integral_by_parts [source]

```
private lemma radial_power_deriv_integral_by_parts
    {d : ℕ} η( : □(Space d, ℝ))
    (n : ↥(Metric.sphere (0 : Space d) 1))
    (p : ℕ) (hp : 0 < p) :
    - ∫ (r : Set.Ioi (0 : ℝ)),
        r.1 ^ p * (_root_.deriv (fun a => η (a • n.1)) r.1)
        ∂(.comap Subtype.val volume)
      =
      (p : ℝ) * ∫ (r : Set.Ioi (0 : ℝ)),
        r.1 ^ (p - 1) * η (r.1 • n.1)
        ∂(.comap Subtype.val volume)
```

lemma Space.distDiv_norm_zpow_smul_repr_self_apply_eq_radial_deriv [source]

```
private lemma distDiv_norm_zpow_smul_repr_self_apply_eq_radial_deriv
    {d p : ℕ} [NeZero d] (q : ℤ) (hq : 0 < q + (d : ℤ))
    (hp_int : (p : ℤ) = q + (d : ℤ))
    η( : □(Space d, ℝ)) :
    ∇ᵈ( □ (distOfFunction (fun x : Space d => |‖x ^ q • basis.repr x)
      (IsDistBounded.zpow_smul_repr_self q (by omega)))) η =
      - ∫ n, ∫( (r : Set.Ioi (0 : ℝ)),
        r.1 ^ p * (_root_.deriv (fun a => η (a • n.1)) r.1)
        ∂(.comap Subtype.val volume))
        ∂(volume α( := Space d).toSphere)
```

lemma Space.distDiv_norm_zpow_smul_repr_self_eq_smul [source]

```
lemma distDiv_norm_zpow_smul_repr_self_eq_smul
    {d : ℕ} [NeZero d] (q : ℤ) (hq : 0 < q + (d : ℤ)) :
    ∇ᵈ □ (distOfFunction (fun x : Space d => ‖‖x ^ q • basis.repr x)
      (IsDistBounded.zpow_smul_repr_self q (by omega))) =
      (((q + d : ℤ) : ℝ) •
        distOfFunction (fun x : Space d => ‖‖x ^ q)
          (IsDistBounded.pow q (by omega)))
```

lemma Space.distLaplacian_distOfFunction_norm_zpow [source]

```
lemma distLaplacian_distOfFunction_norm_zpow {d : ℕ} [NeZero d] (m : ℤ)
    (hdiv : 0 < m - 2 + (d : ℤ)) :
    Δᵈ (distOfFunction (fun x : Space d => ‖‖x ^ m)
      (IsDistBounded.pow m (by omega))) =
      (((m : ℝ) * (((m - 2 + d : ℤ) : ℝ))) •
        distOfFunction (fun x : Space d => ‖‖x ^ (m - 2))
          (IsDistBounded.pow (m - 2) (by omega)))
```

lemma Space.distLaplacian_fundamentalSolution_norm_zpow [source]

*The distributional Laplacian of ‘‖x‖ ^ (2 - d)‘ is ‘(2 - d) * d * volume (Metric.ball 0 1)‘ times the Dirac delta at the origin. For ‘d ≥ 3‘ this ‘‖x‖ ^ (2 - d)‘ is the (singular) fundamental solution of the Laplacian, and for ‘d = 1‘ it is ‘‖x‖‘. When ‘d = 2‘ the exponent vanishes, so the identity collapses to the trivial ‘Δ^d 1 = 0‘; the genuine two-dimensional fundamental solution is the logarithm, proved in ‘distLaplacian_fundamentalSolution_log_norm‘. The statement also holds vacuously for ‘d = 0‘, where the space is trivial.*

```
lemma distLaplacian_fundamentalSolution_norm_zpow {d : ℕ} :
    Δᵈ (distOfFunction (fun x : Space d => ‖‖x ^ (- ((d : ℤ) - 2)))
      (IsDistBounded.pow _ (by omega))) =
      ((- ((d : ℝ) - 2)) * d *
        (volume α( := Space d)).real (Metric.ball 0 1)) • diracDelta ℝ 0
```

lemma Space.distLaplacian_fundamentalSolution_log_norm [source]

*In dimension two the fundamental solution of the Laplacian is the logarithm: the distributional Laplacian of ‘Real.log ‖x‖‘ is ‘2 * volume (Metric.ball 0 1)‘ times the Dirac delta at the origin.*

```
lemma distLaplacian_fundamentalSolution_log_norm :
    Δᵈ (distOfFunction (fun x : Space 2 => Real.log ‖‖x) IsDistBounded.log_norm) =
      (2 * (volume α( := Space 2)).real (Metric.ball 0 1)) • diracDelta ℝ 0
```

2.82 Physlib.SpaceAndTime.Space.Norm.IteratedLaplacian

Physlib/SpaceAndTime/Space/Norm/IteratedLaplacian.lean on GitHub.

def Space.oddNormIteratedLaplacianCoeff [source]

The scalar factor in the odd-dimensional iterated Laplacian of the norm.

```
noncomputable def oddNormIteratedLaplacianCoeff (m : ℕ) : ℝ
```

lemma Space.oddNormIteratedLaplacianCoeff_factor_ne_zero [source]

```
private lemma oddNormIteratedLaplacianCoeff_factor_ne_zero {m k : ℕ} (hk : k < m) :
    ((((1 : ℤ) - 2 * (k : ℤ) : ℤ) : ℝ) *
      ((((1 : ℤ) - 2 * (k : ℤ) - 2 + (2 * m + 1 : ℤ) : ℤ) : ℝ))) ≠ 0
```

lemma Space.oddNormIteratedLaplacianCoeff_ne_zero [source]

The scalar factor in the odd-dimensional iterated Laplacian of the norm is nonzero.

```
lemma oddNormIteratedLaplacianCoeff_ne_zero (m : ℕ) :
    oddNormIteratedLaplacianCoeff m ≠ 0
```

lemma Space.distLaplacian_norm_zpow_odd_boundary [source]

```
private lemma distLaplacian_norm_zpow_odd_boundary (m : ℕ) :
    Δᵈ (distOfFunction (fun x : Space (2 * m + 1) =>
      |‖x ^ ((1 : ℤ) - 2 * (m : ℤ))|
      (IsDistBounded.pow _ (by omega))) =
      (((1 : ℤ) - 2 * (m : ℤ) : ℝ) * (2 * m + 1 : ℝ) *
        (volume α( := Space (2 * m + 1))).real (Metric.ball 0 1)) •
          diracDelta ℝ 0
```

lemma Space.iterated_distLaplacian_norm_zpow_odd_until_boundary [source]

```
private lemma iterated_distLaplacian_norm_zpow_odd_until_boundary
    (m k : ℕ) (hk : k ≤ m) :
    ((distLaplacian (d := 2 * m + 1))^[k])
      (distOfFunction (fun x : Space (2 * m + 1) => |‖x ^ (1 : ℤ))
        (IsDistBounded.pow 1 (by omega))) =
      ∏( j ∈ Finset.range k,
          ((((1 : ℤ) - 2 * (j : ℤ) : ℤ) : ℝ) *
            ((((1 : ℤ) - 2 * (j : ℤ) - 2 + (2 * m + 1 : ℤ) : ℤ) : ℝ)))) •
        distOfFunction (fun x : Space (2 * m + 1) => |‖x ^ ((1 : ℤ) - 2 * (k : ℤ)))
          (IsDistBounded.pow _ (by omega))
```

lemma Space.iterated_distLaplacian_norm_zpow_odd_eq_smul_diracDelta [source]

*In dimension ‘2 * m + 1‘, the ‘(m + 1)‘-fold distributional Laplacian of the distribution induced by the norm is a nonzero multiple of the Dirac delta at the origin.*

```
lemma iterated_distLaplacian_norm_zpow_odd_eq_smul_diracDelta (m : ℕ) :
    ((distLaplacian (d := 2 * m + 1))^[m + 1])
      (distOfFunction (fun x : Space (2 * m + 1) => |‖x ^ (1 : ℤ))
        (IsDistBounded.pow 1 (by omega))) =
      oddNormIteratedLaplacianCoeff m • diracDelta ℝ 0
```

2.83 Physlib.SpaceAndTime.Space.Norm.Regularized

Physlib/SpaceAndTime/Space/Norm/Regularized.lean on GitHub.

def Space.normRegularizedPow [source]

Power of regularized norm, ‘(‖x‖² + ε²)^(s/2)‘.

```
def normRegularizedPow (d : ℕ) ε( s : ℝ) : Space d → ℝ
```

lemma Space.normRegularizedPow_eq [source]

```
lemma normRegularizedPow_eq (d : ℕ) ε( s : ℝ) :
    normRegularizedPow d ε s = fun x ↦ ‖(‖x ^ 2 + ε ^ 2) ^ (s / 2)
```

lemma Space.norm_sq_add_unit_sq_pos [source]

For a nonzero regularization parameter, '$\|x\|^2 + \varepsilon^2$' is positive.

```
lemma norm_sq_add_unit_sq_pos {d : ℕ} ε( : ℝˣ) (x : Space d) : 0 < ‖‖x ^ 2 + ε ^ 2
```

lemma Space.normRegularizedPow_pos [source]

The regularized norm power is positive for nonzero regularization parameter.

```
lemma normRegularizedPow_pos (d : ℕ) ε( : ℝˣ) (s : ℝ) (x : Space d) :
    0 < normRegularizedPow d ε s x
```

lemma Space.normRegularizedPow_hasTemperateGrowth [source]

The regularized norm power has temperate growth.

```
lemma normRegularizedPow_hasTemperateGrowth (d : ℕ) ε( : ℝˣ) (s : ℝ) :
    HasTemperateGrowth (normRegularizedPow d ε s)
```

lemma Space.normRegularizedPow_measurable [source]

```
@[fun_prop]
lemma normRegularizedPow_measurable (d : ℕ) ε( s : ℝ) :
    Measurable (normRegularizedPow d ε s)
```

2.84 Physlib.SpaceAndTime.Space.Origin

Physlib/SpaceAndTime/Space/Origin.lean on GitHub.

instance Space.«instance@570dcd05» [source]

```
instance {d} : Zero (Space d) where
```

lemma Space.zero_val [source]

```
@[simp]
lemma zero_val {d : ℕ} : (0 : Space d).val = fun _ => 0
```

lemma Space.zero_apply [source]

```
@[simp]
lemma zero_apply {d : ℕ} (i : Fin d) :
    (0 : Space d) i = 0
```

def Space.vectorToSpace [source]

A Euclidean vector, based at the chosen origin, viewed as a point of ‘Space d‘.

```
noncomputable def vectorToSpace {d : ℕ} (v : EuclideanSpace ℝ (Fin d)) : Space d
```

lemma Space.vectorToSpace_apply [source]

```
@[simp]
lemma vectorToSpace_apply {d : ℕ} (v : EuclideanSpace ℝ (Fin d)) (i : Fin d) :
    vectorToSpace v i = v i
```

lemma Space.vectorToSpace_vsub_zero [source]

```
@[simp]
lemma vectorToSpace_vsub_zero {d : ℕ} (v : EuclideanSpace ℝ (Fin d)) :
    vectorToSpace v -ᵥ (0 : Space d) = v
```

def Space.chartEuclidean [source]

The standard chart ‘Space d ≃ᵃⁱ[ℝ] EuclideanSpace ℝ (Fin d)‘, ‘p ↦ p -ᵥ 0‘, identifying a point with its coordinate vector relative to the origin (the vector-space zero ‘(0 : Space d)‘).

```
noncomputable def chartEuclidean (d : ℕ) :
    Space d ≃ᵃⁱ[ℝ] EuclideanSpace ℝ (Fin d)
```

2.85 Physlib.SpaceAndTime.SpaceTime.Derivatives

Physlib/SpaceAndTime/SpaceTime/Derivatives.lean on GitHub.

def SpaceTime.manifoldDeriv [source]

The derivative of a function from ‘SpaceTime d‘ to a manifold along the ‘μ‘ coordinate, as a tangent vector at the value of the function.

```
noncomputable def manifoldDeriv {E H N : Type} [NormedAddCommGroup E] [NormedSpace ℝ E]
    [TopologicalSpace H] (I : ModelWithCorners ℝ E H) [TopologicalSpace N]
    [ChartedSpace H N] {d : ℕ} μ( : Fin 1 ⊕ Fin d) (f : SpaceTime d → N) :
    (y : SpaceTime d) → TangentSpace I (f y)
```

lemma SpaceTime.manifoldDeriv_eq [source]

```
lemma manifoldDeriv_eq {E H N : Type} [NormedAddCommGroup E] [NormedSpace ℝ E]
    [TopologicalSpace H] (I : ModelWithCorners ℝ E H) [TopologicalSpace N]
    [ChartedSpace H N] {d : ℕ} μ( : Fin 1 ⊕ Fin d) (f : SpaceTime d → N)
    (y : SpaceTime d) :
    manifoldDeriv I μ f y =
      mfderiv 𝓘(ℝ, SpaceTime d) I f y
        ((Lorentz.Vector.basis μ : SpaceTime d) : TangentSpace 𝓘(ℝ, SpaceTime d) y)
```

lemma SpaceTime.deriv_eq_mfderiv [source]

The spacetime derivative is the manifold derivative for functions into normed spaces.

```
lemma deriv_eq_mfderiv {M : Type} [NormedAddCommGroup M] [NormedSpace ℝ M]
    {d : ℕ} μ( : Fin 1 ⊕ Fin d) (f : SpaceTime d → M) (y : SpaceTime d) :
    deriv μ f y =
      mfderiv □ℝ(, SpaceTime d) □ℝ(, M) f y
        ((Lorentz.Vector.basis μ : SpaceTime d) : TangentSpace □ℝ(, SpaceTime d) y)
```

lemma SpaceTime.deriv_eq_manifoldDeriv [source]

```
lemma deriv_eq_manifoldDeriv {M : Type} [NormedAddCommGroup M] [NormedSpace ℝ M]
    {d : ℕ} μ( : Fin 1 ⊕ Fin d) (f : SpaceTime d → M) (y : SpaceTime d) :
    deriv μ f y = manifoldDeriv □ℝ(, M) μ f y
```

lemma SpaceTime.manifoldDeriv_const [source]

```
@[simp]
lemma manifoldDeriv_const {E H N : Type} [NormedAddCommGroup E] [NormedSpace ℝ E]
    [TopologicalSpace H] (I : ModelWithCorners ℝ E H) [TopologicalSpace N]
    [ChartedSpace H N] {d : ℕ} μ( : Fin 1 ⊕ Fin d) (n : N) (y : SpaceTime d) :
    manifoldDeriv I μ (fun _ : SpaceTime d => n) y = 0
```

lemma SpaceTime.contDiff_deriv [source]

If ‘f‘ is ‘C^{n+1}‘ then ‘∂_ μ f‘ is ‘C^n‘.

```
@[fun_prop]
lemma contDiff_deriv {M : Type} [NormedAddCommGroup M] [NormedSpace ℝ M] {d : ℕ}
    {n : WithTop ℕ∞} μ( : Fin 1 ⊕ Fin d) (f : SpaceTime d → M) (hf : ContDiff ℝ (n + 1) f) :
    ContDiff ℝ n ∂(_ μ f)
```

lemma SpaceTime.differentiable_deriv [source]

If ‘f‘ is ‘C^2‘ then ‘∂_ μ f‘ is differentiable.

```
@[fun_prop]
lemma differentiable_deriv {M : Type} [NormedAddCommGroup M] [NormedSpace ℝ M] {d : ℕ}
    μ( : Fin 1 ⊕ Fin d) (f : SpaceTime d → M) (hf : ContDiff ℝ 2 f) :
    Differentiable ℝ ∂(_ μ f)
```

lemma SpaceTime.deriv_commute [source]

Derivatives on spacetime commute with one another (Clairaut's theorem).

```
lemma deriv_commute {M : Type} [NormedAddCommGroup M] [NormedSpace ℝ M] {d : ℕ}
    μ( ν : Fin 1 ⊕ Fin d) (f : SpaceTime d → M) (hf : ContDiff ℝ 2 f) :
    ∂_ μ ∂(_ ν f) = ∂_ ν ∂(_ μ f)
```

2.86 Physlib.SpaceAndTime.Time.Derivatives

Physlib/SpaceAndTime/Time/Derivatives.lean on GitHub.

def Time.manifoldDeriv [source]

The time derivative of a function from 'Time' to a manifold, as a tangent vector at the value of the function.

```
noncomputable def manifoldDeriv {E H N : Type} [NormedAddCommGroup E] [NormedSpace ℝ E]
    [TopologicalSpace H] (I : ModelWithCorners ℝ E H) [TopologicalSpace N]
    [ChartedSpace H N] (f : Time → N) : (t : Time) → TangentSpace I (f t)
```

lemma Time.manifoldDeriv_eq [source]

```
lemma manifoldDeriv_eq {E H N : Type} [NormedAddCommGroup E] [NormedSpace ℝ E]
    [TopologicalSpace H] (I : ModelWithCorners ℝ E H) [TopologicalSpace N]
    [ChartedSpace H N] (f : Time → N) (t : Time) :
    manifoldDeriv I f t =
      mfderiv 𝓘(ℝ, Time) I f t ((1 : Time) : TangentSpace 𝓘(ℝ, Time) t)
```

lemma Time.deriv_eq_mfderiv [source]

The time derivative is the manifold derivative for functions into normed spaces.

```
lemma deriv_eq_mfderiv [NormedAddCommGroup M] [NormedSpace ℝ M]
    (f : Time → M) (t : Time) :
    deriv f t =
      mfderiv 𝓘(ℝ, Time) 𝓘(ℝ, M) f t
        ((1 : Time) : TangentSpace 𝓘(ℝ, Time) t)
```

lemma Time.deriv_eq_manifoldDeriv [source]

```
lemma deriv_eq_manifoldDeriv [NormedAddCommGroup M] [NormedSpace ℝ M]
    (f : Time → M) (t : Time) :
    deriv f t = manifoldDeriv 𝓘(ℝ, M) f t
```

lemma Time.manifoldDeriv_const [source]

```
@[simp]
lemma manifoldDeriv_const {E H N : Type} [NormedAddCommGroup E] [NormedSpace ℝ E]
    [TopologicalSpace H] (I : ModelWithCorners ℝ E H) [TopologicalSpace N]
    [ChartedSpace H N] (n : N) :
    manifoldDeriv I (fun _ : Time => n) t = 0
```

2.87 Physlib.SpaceAndTime.TimeAndSpace.Basic

Physlib/SpaceAndTime/TimeAndSpace/Basic.lean on GitHub.

abbrev TimeAndSpace [source]

Euclidean spacetime as the product of time and 'd'-dimensional space.

```
abbrev TimeAndSpace (d : ℕ := 3)
```

def TimeAndSpace.time [source]

The time-coordinate projection from 'TimeAndSpace d'.

```
noncomputable def time {d : ℕ} : TimeAndSpace d →L[ℝ] Time
```

def TimeAndSpace.space [source]

The spatial-coordinate projection from ‘TimeAndSpace d‘.

```
noncomputable def space {d : ℕ} : TimeAndSpace d →L[ℝ] Space d
```

lemma TimeAndSpace.time_apply [source]

```
@[simp]
lemma time_apply (tx : TimeAndSpace d) :
    time tx = tx.1
```

lemma TimeAndSpace.space_apply [source]

```
@[simp]
lemma space_apply (tx : TimeAndSpace d) :
    space tx = tx.2
```

lemma TimeAndSpace.dist_time_le [source]

The time projection is nonexpanding for the product metric.

```
lemma dist_time_le (tx ty : TimeAndSpace d) :
    dist (time tx) (time ty) ≤ dist tx ty
```

lemma TimeAndSpace.dist_space_le [source]

The spatial projection is nonexpanding for the product metric.

```
lemma dist_space_le (tx ty : TimeAndSpace d) :
    dist (space tx) (space ty) ≤ dist tx ty
```

2.88 Physlib.SpaceAndTime.TimeAndSpace.EuclideanGroup.Action

Physlib/SpaceAndTime/TimeAndSpace/EuclideanGroup/Action.lean on GitHub.

instance TimeAndSpace.instMulActionEuclideanGroup [source]

The Euclidean group acts on ‘TimeAndSpace d‘ by fixing time and acting on space.

```
noncomputable instance instMulActionEuclideanGroup :
    MulAction (EuclideanGroup d) (TimeAndSpace d) where
```

lemma TimeAndSpace.smul_mk [source]

Coordinate formula for the Euclidean-group action on ‘TimeAndSpace d‘.

```
@[simp]
lemma smul_mk (g : EuclideanGroup d) (t : Time) (x : Space d) :
    g • ((t, x) : TimeAndSpace d) = (t, g • x)
```

lemma TimeAndSpace.fst_smul [source]

The Euclidean-group action fixes the first coordinate.

```
@[simp]
lemma fst_smul (g : EuclideanGroup d) (tx : TimeAndSpace d) :
    (g • tx).1 = tx.1
```

lemma TimeAndSpace.snd_smul [source]

The Euclidean-group action applies to the second coordinate.

```
@[simp]
lemma snd_smul (g : EuclideanGroup d) (tx : TimeAndSpace d) :
    (g • tx).2 = g • tx.2
```

lemma TimeAndSpace.time_smul [source]

The Euclidean-group action fixes the time projection.

```
lemma time_smul (g : EuclideanGroup d) (tx : TimeAndSpace d) :
    time (g • tx) = time tx
```

lemma TimeAndSpace.space_smul [source]

The Euclidean-group action applies to the space projection.

```
lemma space_smul (g : EuclideanGroup d) (tx : TimeAndSpace d) :
    space (g • tx) = g • space tx
```

lemma TimeAndSpace.dist_smul [source]

The Euclidean-group action on ‘TimeAndSpace d‘ preserves product distance.

```
lemma dist_smul (g : EuclideanGroup d) (tx ty : TimeAndSpace d) :
    dist (g • tx) (g • ty) = dist tx ty
```

def TimeAndSpace.actionLinearMap [source]

The linear part of the Euclidean-group action on ‘TimeAndSpace d‘.

```
noncomputable def actionLinearMap (g : EuclideanGroup d) :
    TimeAndSpace d →L[ℝ] TimeAndSpace d
```

def TimeAndSpace.actionTranslation [source]

The translation part of the Euclidean-group action on ‘TimeAndSpace d‘.

```
noncomputable def actionTranslation (g : EuclideanGroup d) : TimeAndSpace d
```

lemma TimeAndSpace.smul_eq_actionLinearMap_add [source]

```
lemma smul_eq_actionLinearMap_add (g : EuclideanGroup d) (tx : TimeAndSpace d) :
    g • tx = actionLinearMap g tx + actionTranslation g
```

lemma TimeAndSpace.smul_hasTemperateGrowth [source]

The Euclidean-group action on ‘TimeAndSpace d‘ has temperate growth.

```
lemma smul_hasTemperateGrowth (g : EuclideanGroup d) :
    Function.HasTemperateGrowth (fun tx : TimeAndSpace d => g • tx)
```

lemma TimeAndSpace.isometry_smul [source]

The Euclidean-group action on ‘TimeAndSpace d‘ is an isometry.

```
lemma isometry_smul (g : EuclideanGroup d) :
    Isometry (fun tx : TimeAndSpace d => g • tx)
```

lemma TimeAndSpace.antilipschitz_smul [source]

The Euclidean-group action on ‘TimeAndSpace d‘ is antilipschitz.

```
lemma antilipschitz_smul (g : EuclideanGroup d) :
    AntilipschitzWith 1 (fun tx : TimeAndSpace d => g • tx)
```

2.89 Physlib.SpaceAndTime.TimeAndSpace.EuclideanGroup.SchwartzAction

Physlib/SpaceAndTime/TimeAndSpace/EuclideanGroup/SchwartzAction.lean on GitHub.

def TimeAndSpace.schwartzEuclideanAction [source]

The Euclidean-group pullback action on Schwartz maps over ‘TimeAndSpace d‘, as a continuous representation ‘ContRepresentation ℝ (EuclideanGroup d) 𝒮(TimeAndSpace d, F)‘.

‘ContRepresentation R G V‘ unfolds to the monoid homomorphism ‘G → V →L[R] V‘ into the *continuous*-linear maps: the builder ‘SchwartzMap.compCLMOfAntilipschitz‘ already yields a continuous-linear map for free, and the ‘→L‘ codomain lets the action compose under ‘∘L‘. The plain mathlib ‘Representation‘ (‘G →* V →ₗ[ℝ] V‘) is recovered via ‘ContRepresentation.toRepresentation‘.*

```
noncomputable def schwartzEuclideanAction {d : ℕ} :
    ContRepresentation ℝ (EuclideanGroup d) 𝓢(TimeAndSpace d, F) where
```

lemma TimeAndSpace.schwartzEuclideanAction_apply [source]

Pointwise formula for the monoid-homomorphism form of the Schwartz-map pullback action.

```
@[simp]
lemma schwartzEuclideanAction_apply {d : ℕ} (g : EuclideanGroup d) η( : 𝓢(TimeAndSpace d, F))
    (tx : TimeAndSpace d) :
    (schwartzEuclideanAction g η) tx = η (⁻g¹ • tx)
```

instance TimeAndSpace.instMulActionSchwartzMap [source]

The Euclidean group acts on Schwartz maps over ‘TimeAndSpace d‘ by pullback.

```
noncomputable instance instMulActionSchwartzMap {d : ℕ} :
    MulAction (EuclideanGroup d) 𝓢(TimeAndSpace d, F) where
```

lemma TimeAndSpace.smul_schwartzMap_apply [source]

Pointwise formula for the 'MulAction' instance on Schwartz maps.

```
@[simp]
lemma smul_schwartzMap_apply {d : ℕ} (g : EuclideanGroup d) η( : 𝓢(TimeAndSpace d, F))
    (tx : TimeAndSpace d) :
    (g • η) tx = η (⁻g¹ • tx)
```

lemma TimeAndSpace.schwartzEuclideanAction_mul_apply [source]

*Applying 'g' and then 'h' to a Schwartz map is the pullback action of 'h * g'.*

```
lemma schwartzEuclideanAction_mul_apply {d : ℕ} (g h : EuclideanGroup d)
    η( : 𝓢(TimeAndSpace d, F)) :
    schwartzEuclideanAction h (schwartzEuclideanAction g η) =
      schwartzEuclideanAction (h * g) η
```

lemma TimeAndSpace.schwartzEuclideanAction_injective [source]

Each Euclidean-group pullback action on Schwartz maps is injective.

```
lemma schwartzEuclideanAction_injective {d : ℕ} (g : EuclideanGroup d) :
    Function.Injective (schwartzEuclideanAction (F := F) g)
```

lemma TimeAndSpace.schwartzEuclideanAction_surjective [source]

Each Euclidean-group pullback action on Schwartz maps is surjective.

```
lemma schwartzEuclideanAction_surjective {d : ℕ} (g : EuclideanGroup d) :
    Function.Surjective (schwartzEuclideanAction (F := F) g)
```

2.90 Physlib.Thermodynamics.Temperature.Basic

Physlib/Thermodynamics/Temperature/Basic.lean on GitHub.

lemma Temperature.β_toReal [source]

```
lemma β_toReal (T : Temperature) : β( T : ℝ) = 1 / (kB * (T : ℝ))
```

lemma Temperature.ofβ_toReal [source]

```
lemma βof_toReal β( : ℝ≥0) : (βof β).toReal = 1 / (kB * β( : ℝ))
```

2.91 Physlib.Units.WithDim.Basic

Physlib/Units/WithDim/Basic.lean on GitHub.

instance WithDim.«instance@ec98a162» [source]

```
instance (d : Dimension) (M : Type) [Inhabited M] : Inhabited (WithDim d M) where
```

instance WithDim.«instance@c39a3b54» [source]

```
instance (d : Dimension) (M : Type) [Zero M] : Zero (WithDim d M) where
```

lemma WithDim.val_zero [source]

```
@[simp]
lemma val_zero {d : Dimension} {M : Type} [Zero M] :
    (0 : WithDim d M).val = 0
```

instance WithDim.«instance@ccb84cd6» [source]

```
instance (d : Dimension) (M : Type) [Add M] : Add (WithDim d M) where
```

lemma WithDim.val_add [source]

```
@[simp]
lemma val_add {d : Dimension} {M : Type} [Add M] (m1 m2 : WithDim d M) :
    (m1 + m2).val = m1.val + m2.val
```

instance WithDim.«instance@61de96eb» [source]

```
instance (d : Dimension) (M : Type) [Neg M] : Neg (WithDim d M) where
```

lemma WithDim.val_neg [source]

```
@[simp]
lemma val_neg {d : Dimension} {M : Type} [Neg M] (m : WithDim d M) :
    (-m).val = -m.val
```

instance WithDim.«instance@9b66d0ce» [source]

```
instance (d : Dimension) (M : Type) [Sub M] : Sub (WithDim d M) where
```

lemma WithDim.val_sub [source]

```
@[simp]
lemma val_sub {d : Dimension} {M : Type} [Sub M] (m1 m2 : WithDim d M) :
    (m1 - m2).val = m1.val - m2.val
```

instance WithDim.«instance@a15a8cbf» [source]

```
instance (d : Dimension) (M : Type) [AddSemigroup M] :
    AddSemigroup (WithDim d M) where
```

instance WithDim.«instance@e53aef60» [source]

```
instance (d : Dimension) (M : Type) [AddCommSemigroup M] :
    AddCommSemigroup (WithDim d M) where
```

instance WithDim.«instance@a79c96d0» [source]

```
instance (d : Dimension) (M : Type) [AddMonoid M] :
    AddMonoid (WithDim d M) where
```

instance WithDim.«instance@2b3671f3» [source]

```
instance (d : Dimension) (M : Type) [AddCommMonoid M] :
    AddCommMonoid (WithDim d M) where
```

instance WithDim.«instance@17f8c5c8» [source]

```
instance (d : Dimension) (M : Type) [AddGroup M] :
    AddGroup (WithDim d M) where
```

instance WithDim.«instance@060f3ddb» [source]

```
instance (d : Dimension) (M : Type) [AddCommGroup M] :
    AddCommGroup (WithDim d M) where
```

instance WithDim.«instance@84bf74c6» [source]

```
instance (d : Dimension) (M : Type) [LE M] : LE (WithDim d M) where
```

lemma WithDim.le_def [source]

```
@[simp]
lemma le_def {d : Dimension} {M : Type} [LE M] (m1 m2 : WithDim d M) :
    m1 ≤ m2 ↔ m1.val ≤ m2.val
```

instance WithDim.«instance@fac005f7» [source]

```
instance (d : Dimension) (M : Type) [LT M] : LT (WithDim d M) where
```

lemma WithDim.lt_def [source]

```
@[simp]
lemma lt_def {d : Dimension} {M : Type} [LT M] (m1 m2 : WithDim d M) :
    m1 < m2 ↔ m1.val < m2.val
```

instance WithDim.«instance@0a2efc58» [source]

```
instance (d : Dimension) (M : Type) [Preorder M] :
    Preorder (WithDim d M) where
```

instance WithDim.«instance@86c24b45» [source]

```
instance (d : Dimension) (M : Type) [PartialOrder M] :
    PartialOrder (WithDim d M) where
```

2.92 PhyslibAlpha.ClassicalFieldTheory.Local.Action

PhyslibAlpha/ClassicalFieldTheory/Local/Action.lean on GitHub.

def ClassicalFieldTheory.Local.actionDensity [source]

The action density determined by a local Lagrangian and a field.

```
noncomputable def actionDensity (L : Lagrangian d m k)
    (f : Space d → EuclideanSpace ℝ (Fin m)) : Space d → ℝ
```

def ClassicalFieldTheory.Local.HasFiniteAction [source]

The integrability condition for the action density of a field.

```
def HasFiniteAction (L : Lagrangian d m k) (f : Space d → EuclideanSpace ℝ (Fin m)) : Prop
```

def ClassicalFieldTheory.Local.IsAdmissibleForAction [source]

Symmetric admissibility predicate for the action functional: the pair ‘(L, f)‘ is admissible when the field is smooth and the corresponding action density is integrable.

```
def IsAdmissibleForAction (L : Lagrangian d m k)
    (f : Space d → EuclideanSpace ℝ (Fin m)) : Prop
```

def ClassicalFieldTheory.Local.action [source]

The action associated with a local Lagrangian.

```
noncomputable def action (L : Lagrangian d m k)
    (f : Space d → EuclideanSpace ℝ (Fin m)) : ℝ
```

lemma ClassicalFieldTheory.Local.actionDensity_apply [source]

```
@[simp]
lemma actionDensity_apply (L : Lagrangian d m k)
    (f : Space d → EuclideanSpace ℝ (Fin m)) (x : Space d) :
    actionDensity L f x = L (jetAt k f x)
```

lemma ClassicalFieldTheory.Local.action_eq_integral [source]

```
@[simp]
lemma action_eq_integral (L : Lagrangian d m k) (f : Space d → EuclideanSpace ℝ (Fin m)) :
    action L f = ∫ x, actionDensity L f x
```

def ClassicalFieldTheory.Local.variedField [source]

The field obtained by varying ‘f‘ in the direction ‘η‘ with parameter ‘s‘.

```
noncomputable def variedField (f : Space d → EuclideanSpace ℝ (Fin m))
    η( : AdmissibleVariation d (EuclideanSpace ℝ (Fin m)))
    (s : ℝ) :
    Space d → EuclideanSpace ℝ (Fin m)
```

lemma ClassicalFieldTheory.Local.variedField_contDiff [source]

Smoothness of the varied field for smooth ‘f‘ and admissible ‘η‘.

```
lemma variedField_contDiff (f : Space d → EuclideanSpace ℝ (Fin m))
    η( : AdmissibleVariation d (EuclideanSpace ℝ (Fin m)))
    (s : ℝ)
    (hf : ContDiff ℝ ∞ f) :
    ContDiff ℝ ∞ (variedField f η s)
```

def ClassicalFieldTheory.Local.actionVariation [source]

The action of a field under an admissible variation.

```
noncomputable def actionVariation (L : Lagrangian d m k)
    (f : Space d → EuclideanSpace ℝ (Fin m))
    η( : AdmissibleVariation d (EuclideanSpace ℝ (Fin m))) : ℝ → ℝ
```

def ClassicalFieldTheory.Local.HasFiniteActionVariation [source]

Explicit integrability condition for the family of varied action densities.

```
def HasFiniteActionVariation (L : Lagrangian d m k) (f : Space d → EuclideanSpace ℝ (Fin m))
    η( : AdmissibleVariation d (EuclideanSpace ℝ (Fin m))) : Prop
```

def ClassicalFieldTheory.Local.HasCompactlySupportedActionVariationDifference [source]

Locality package for the action density under compactly supported variations: for every variation parameter ‘s‘, the change in the action density is a continuous compactly supported function. Combined with finiteness of the base action, this implies finiteness of the varied action.

```
def HasCompactlySupportedActionVariationDifference (L : Lagrangian d m k)
    (f : Space d → EuclideanSpace ℝ (Fin m))
    η( : AdmissibleVariation d (EuclideanSpace ℝ (Fin m))) : Prop
```

lemma ClassicalFieldTheory.Local.hasFiniteActionVariation_of_hasFiniteAction_of_compactlySupportedDiff
[source]

If the base action is finite and every varied action density differs from the base density by a continuous compactly supported function, then the whole varied family has finite action.

```
lemma hasFiniteActionVariation_of_hasFiniteAction_of_compactlySupportedDifference
    (L : Lagrangian d m k) (f : Space d → EuclideanSpace ℝ (Fin m))
    η( : AdmissibleVariation d (EuclideanSpace ℝ (Fin m)))
    (hbase : HasFiniteAction L f)
    (hdiff : HasCompactlySupportedActionVariationDifference L f η) :
    HasFiniteActionVariation L f η
```

lemma ClassicalFieldTheory.Local.variedField_apply [source]

```
@[simp]
lemma variedField_apply (f : Space d → EuclideanSpace ℝ (Fin m))
    η( : AdmissibleVariation d (EuclideanSpace ℝ (Fin m))) (s : ℝ)
```

```
    (x : Space d) :
    variedField f η s x = f x + s • η x
```

lemma ClassicalFieldTheory.Local.actionVariation_apply [source]

```
@[simp]
lemma actionVariation_apply (L : Lagrangian d m k) (f : Space d → EuclideanSpace ℝ (Fin m))
    η( : AdmissibleVariation d (EuclideanSpace ℝ (Fin m))) (s : ℝ) :
    actionVariation L f η s = action L (variedField f η s)
```

lemma ClassicalFieldTheory.Local.jetAt_variedField_eq_of_notMem_tsupport [source]

```
lemma jetAt_variedField_eq_of_notMem_tsupport
    (k : ℕ) (f : Space d → EuclideanSpace ℝ (Fin m))
    η( : AdmissibleVariation d (EuclideanSpace ℝ (Fin m))) (s : ℝ)
    {x : Space d}
    (hf : ContDiff ℝ ∞ f) (hx : x ∉ tsupport η.toFun) :
    jetAt k (variedField f η s) x = jetAt k f x
```

lemma ClassicalFieldTheory.Local.actionDensity_variedField_eq_of_notMem_tsupport [source]

```
lemma actionDensity_variedField_eq_of_notMem_tsupport
    (L : Lagrangian d m k) (f : Space d → EuclideanSpace ℝ (Fin m))
    η( : AdmissibleVariation d (EuclideanSpace ℝ (Fin m)))
    (s : ℝ)
    {x : Space d} (hf : ContDiff ℝ ∞ f) (hx : x ∉ tsupport η.toFun) :
    actionDensity L (variedField f η s) x = actionDensity L f x
```

lemma ClassicalFieldTheory.Local.hasCompactlySupportedActionVariationDifference_of_continuousInCoordi
[source]

```
lemma hasCompactlySupportedActionVariationDifference_of_continuousInCoordinates
    (L : Lagrangian d m k) (hcontL : Lagrangian.ContinuousInCoordinates L)
    (f : Space d → EuclideanSpace ℝ (Fin m)) (hf : ContDiff ℝ ∞ f)
    η( : AdmissibleVariation d (EuclideanSpace ℝ (Fin m))) :
    HasCompactlySupportedActionVariationDifference L f η
```

def ClassicalFieldTheory.Local.IsCritical [source]

A field is critical for a local action if every admissible compactly supported variation has vanishing first derivative at ‘s = 0‘, under the explicit finite-action hypothesis for the varied family.

```
def IsCritical (L : Lagrangian d m k) (f : Space d → EuclideanSpace ℝ (Fin m)) : Prop
```

2.93 PhyslibAlpha.ClassicalFieldTheory.Local.EulerLagrange

PhyslibAlpha/ClassicalFieldTheory/Local/EulerLagrange.lean on GitHub.

def ClassicalFieldTheory.Local.eulerLagrangeTerm [source]

The summand ‘(-1)^|I| D_I (∂L/∂u^a_I)‘ in the local Euler-Lagrange formula.

```
noncomputable def eulerLagrangeTerm (L : Lagrangian d m k) (I : DerivativeIndex d k) (a : Fin m
    )
    (f : Space d → EuclideanSpace ℝ (Fin m)) : Space d → ℝ
```

lemma ClassicalFieldTheory.Local.eulerLagrangeTerm_apply [source]

```
@[simp]
lemma eulerLagrangeTerm_apply (L : Lagrangian d m k) (I : DerivativeIndex d k) (a : Fin m)
    (f : Space d → EuclideanSpace ℝ (Fin m)) (x : Space d) :
    eulerLagrangeTerm L I a f x =
      (-1 : ℝ) ^ I.1.order * iteratedTotalDerivative k I.1 (L.coordDeriv I a) f x
```

def ClassicalFieldTheory.Local.eulerLagrangeComponent [source]

The 'a'-th component of the local Euler-Lagrange operator.

```
noncomputable def eulerLagrangeComponent (L : Lagrangian d m k) (a : Fin m)
    (f : Space d → EuclideanSpace ℝ (Fin m)) : Space d → ℝ
```

lemma ClassicalFieldTheory.Local.eulerLagrangeComponent_apply [source]

```
@[simp]
lemma eulerLagrangeComponent_apply (L : Lagrangian d m k) (a : Fin m)
    (f : Space d → EuclideanSpace ℝ (Fin m)) (x : Space d) :
    eulerLagrangeComponent L a f x = ∑ I : DerivativeIndex d k, eulerLagrangeTerm L I a f x
```

def ClassicalFieldTheory.Local.eulerLagrangeOp [source]

The local Euler-Lagrange operator attached to a local Lagrangian.

```
noncomputable def eulerLagrangeOp (L : Lagrangian d m k) (f : Space d → EuclideanSpace ℝ (Fin m
    )) :
    Space d → EuclideanSpace ℝ (Fin m)
```

lemma ClassicalFieldTheory.Local.eulerLagrangeOp_apply [source]

```
lemma eulerLagrangeOp_apply (L : Lagrangian d m k) (f : Space d → EuclideanSpace ℝ (Fin m))
    (x : Space d) (a : Fin m) :
    eulerLagrangeOp L f x a = eulerLagrangeComponent L a f x
```

lemma ClassicalFieldTheory.Local.eulerLagrangeOp_eq [source]

```
@[simp]
lemma eulerLagrangeOp_eq (L : Lagrangian d m k) (f : Space d → EuclideanSpace ℝ (Fin m)) :
    eulerLagrangeOp L f = fun x => WithLp.toLp 2 fun a => eulerLagrangeComponent L a f x
```

2.94 PhyslibAlpha.ClassicalFieldTheory.Local.FirstOrder

PhyslibAlpha/ClassicalFieldTheory/Local/FirstOrder.lean on GitHub.

abbrev ClassicalFieldTheory.Local.FirstOrderJetCoordinates [source]

Coordinate data for first-order local jet points.

```
abbrev FirstOrderJetCoordinates (d m : ℕ)
```

abbrev ClassicalFieldTheory.Local.FirstOrderJetPoint [source]

Coordinate-level first-order jet points.

```
abbrev FirstOrderJetPoint (d m : ℕ)
```

abbrev ClassicalFieldTheory.Local.FirstOrderJetFiberData [source]

Fiber-direction data for first-order jet points.

```
abbrev FirstOrderJetFiberData (d m : ℕ)
```

abbrev ClassicalFieldTheory.Local.FirstOrderLagrangian [source]

First-order local lagrangians.

```
abbrev FirstOrderLagrangian (d m : ℕ)
```

def ClassicalFieldTheory.Local.firstDerivativeIndex [source]

The first-derivative coordinate index in the source direction ‘i‘.

```
def firstDerivativeIndex (d : ℕ) (i : Fin d) : DerivativeIndex d 1
```

lemma ClassicalFieldTheory.Local.firstDerivativeIndex_coe [source]

```
@[simp]
lemma firstDerivativeIndex_coe (d : ℕ) (i : Fin d) :
    ((firstDerivativeIndex d i : DerivativeIndex d 1) : MultiIndex d) =
      Physlib.MultiIndex.increment 0 i
```

lemma ClassicalFieldTheory.Local.firstDerivativeIndex_order [source]

```
lemma firstDerivativeIndex_order (d : ℕ) (i : Fin d) :
    ((firstDerivativeIndex d i : DerivativeIndex d 1) : MultiIndex d).order = 1
```

def ClassicalFieldTheory.Local.JetPoint.firstDerivCoord [source]

The first derivative coordinate ‘u^a_i‘ of a first-order jet point.

```
def firstDerivCoord (J : FirstOrderJetPoint d m) (i : Fin d) (a : Fin m) : ℝ
```

def ClassicalFieldTheory.Local.JetPoint.firstDerivVector [source]

The bundled target vector of first derivative coordinates in the source direction ‘i‘.

```
def firstDerivVector (J : FirstOrderJetPoint d m) (i : Fin d) : EuclideanSpace ℝ (Fin m)
```

lemma ClassicalFieldTheory.Local.JetPoint.firstDerivVector_apply [source]

```
@[simp]
lemma firstDerivVector_apply (J : FirstOrderJetPoint d m) (i : Fin d) (a : Fin m) :
    J.firstDerivVector i a = J.firstDerivCoord i a
```

lemma ClassicalFieldTheory.Local.JetPoint.firstDerivCoord_ofBaseCoordinates [source]

```
@[simp]
lemma firstDerivCoord_ofBaseCoordinates (x : Space d) (u : FirstOrderJetCoordinates d m)
    (i : Fin d) (a : Fin m) :
    (JetPoint.ofBaseCoordinates x u).firstDerivCoord i a = u (firstDerivativeIndex d i) a
```

abbrev ClassicalFieldTheory.Local.firstOrderJetAt [source]

The first-order coordinate-level jet point of a field at a point.

```
noncomputable abbrev firstOrderJetAt (f : Space d → EuclideanSpace ℝ (Fin m)) (x : Space d) :
    FirstOrderJetPoint d m
```

lemma ClassicalFieldTheory.Local.firstOrderJetAt_base [source]

```
@[simp]
lemma firstOrderJetAt_base (f : Space d → EuclideanSpace ℝ (Fin m)) (x : Space d) :
    (firstOrderJetAt f x).base = x
```

lemma ClassicalFieldTheory.Local.firstOrderJetAt_value [source]

```
lemma firstOrderJetAt_value (f : Space d → EuclideanSpace ℝ (Fin m)) (x : Space d) :
    (firstOrderJetAt f x).value = f x
```

lemma ClassicalFieldTheory.Local.firstOrderJetAt_firstDerivCoord [source]

```
@[simp]
lemma firstOrderJetAt_firstDerivCoord (f : Space d → EuclideanSpace ℝ (Fin m))
    (x : Space d) (i : Fin d) (a : Fin m) :
    (firstOrderJetAt f x).firstDerivCoord i a =
      ∂^[Physlib.MultiIndex.increment 0 i] (fun y => (f y) a) x
```

abbrev ClassicalFieldTheory.Local.firstOrderActionDensity [source]

Evaluate a first-order local lagrangian along the first jets of a field.

```
noncomputable abbrev firstOrderActionDensity (L : FirstOrderLagrangian d m)
    (f : Space d → EuclideanSpace ℝ (Fin m)) : Space d → ℝ
```

abbrev ClassicalFieldTheory.Local.firstOrderAction [source]

The action of a first-order local lagrangian.

```
noncomputable abbrev firstOrderAction (L : FirstOrderLagrangian d m)
    (f : Space d → EuclideanSpace ℝ (Fin m)) : ℝ
```

abbrev ClassicalFieldTheory.Local.firstOrderEulerLagrangeOp [source]

The Euler-Lagrange operator of a first-order local lagrangian.

```
noncomputable abbrev firstOrderEulerLagrangeOp (L : FirstOrderLagrangian d m)
    (f : Space d → EuclideanSpace ℝ (Fin m)) : Space d → EuclideanSpace ℝ (Fin m)
```

theorem ClassicalFieldTheory.Local.firstOrder_isCritical_iff_eulerLagrange_zero [source]

```
theorem firstOrder_isCritical_iff_eulerLagrange_zero
    (L : FirstOrderLagrangian d m) (f : Space d → EuclideanSpace ℝ (Fin m))
    (hLf : IsAdmissibleForAction L f)
    (hL : Lagrangian.SmoothInCoordinates L) :
    IsCritical L f ↔ firstOrderEulerLagrangeOp L f = 0
```

2.95 PhyslibAlpha.ClassicalFieldTheory.Local.FirstVariation

PhyslibAlpha/ClassicalFieldTheory/Local/FirstVariation.lean on GitHub.

theorem ClassicalFieldTheory.Local.hasSmoothEulerLagrangeRegularity_of_smoothInCoordinates [source]

```
theorem hasSmoothEulerLagrangeRegularity_of_smoothInCoordinates
    (L : Lagrangian d m k) (hL : Lagrangian.SmoothInCoordinates L) :
    HasSmoothEulerLagrangeRegularity L
```

theorem ClassicalFieldTheory.Local.isCritical_iff_eulerLagrange_zero_of_contDiff_and_smoothInCoordinat [source]

```
theorem isCritical_iff_eulerLagrange_zero_of_contDiff_and_smoothInCoordinates
    (L : Lagrangian d m k) (f : Space d → EuclideanSpace ℝ (Fin m)) (hf : ContDiff ℝ ∞ f)
    (hL : Lagrangian.SmoothInCoordinates L)
    (hbase : HasFiniteAction L f) :
    IsCritical L f ↔ eulerLagrangeOp L f = 0
```

theorem ClassicalFieldTheory.Local.isCritical_iff_eulerLagrange_zero_of_admissibleForAction_and_smooth [source]

```
theorem isCritical_iff_eulerLagrange_zero_of_admissibleForAction_and_smoothInCoordinates
    (L : Lagrangian d m k) (f : Space d → EuclideanSpace ℝ (Fin m))
    (hLf : IsAdmissibleForAction L f)
    (hL : Lagrangian.SmoothInCoordinates L) :
    IsCritical L f ↔ eulerLagrangeOp L f = 0
```

2.96 PhyslibAlpha.ClassicalFieldTheory.Local.FirstVariation.Basic

PhyslibAlpha/ClassicalFieldTheory/Local/FirstVariation/Basic.lean on GitHub.

def ClassicalFieldTheory.Local.firstVariationDensityTerm [source]

A single term in the linearized first-variation density before integration by parts.

```
noncomputable def firstVariationDensityTerm (L : Lagrangian d m k)
    (f : Space d → EuclideanSpace ℝ (Fin m))
    η( : AdmissibleVariation d (EuclideanSpace ℝ (Fin m))) (I : DerivativeIndex d k)
```

```
    (a : Fin m) :
    Space d → ℝ
```

def ClassicalFieldTheory.Local.firstVariationDensity [source]

The pointwise linearized first-variation density before integration by parts.

```
noncomputable def firstVariationDensity (L : Lagrangian d m k)
    (f : Space d → EuclideanSpace ℝ (Fin m))
    η( : AdmissibleVariation d (EuclideanSpace ℝ (Fin m))) : Space d → ℝ
```

def ClassicalFieldTheory.Local.firstVariationValue [source]

The value predicted by the first-variation formula for an admissible variation.

```
noncomputable def firstVariationValue (L : Lagrangian d m k)
    (f : Space d → EuclideanSpace ℝ (Fin m))
    η( : AdmissibleVariation d (EuclideanSpace ℝ (Fin m))) : ℝ
```

lemma ClassicalFieldTheory.Local.firstVariationDensityTerm_apply [source]

```
@[simp]
lemma firstVariationDensityTerm_apply (L : Lagrangian d m k)
    (f : Space d → EuclideanSpace ℝ (Fin m))
    η( : AdmissibleVariation d (EuclideanSpace ℝ (Fin m))) (I : DerivativeIndex d k)
    (a : Fin m) (x : Space d) :
    firstVariationDensityTerm L f η I a x =
      L.coordDeriv I a (jetAt k f x) * ∂^[I.1] (fun y => η( y) a) x
```

lemma ClassicalFieldTheory.Local.firstVariationDensity_apply [source]

```
@[simp]
lemma firstVariationDensity_apply (L : Lagrangian d m k)
    (f : Space d → EuclideanSpace ℝ (Fin m))
    η( : AdmissibleVariation d (EuclideanSpace ℝ (Fin m))) (x : Space d) :
    firstVariationDensity L f η x =
      ∑ I : DerivativeIndex d k, ∑ a : Fin m, firstVariationDensityTerm L f η I a x
```

lemma ClassicalFieldTheory.Local.firstVariationValue_eq_integral [source]

```
@[simp]
lemma firstVariationValue_eq_integral (L : Lagrangian d m k)
    (f : Space d → EuclideanSpace ℝ (Fin m))
    η( : AdmissibleVariation d (EuclideanSpace ℝ (Fin m))) :
    firstVariationValue L f η = ∫ x, □eulerLagrangeOp L f x, η □ℝx_
```

lemma ClassicalFieldTheory.Local.firstVariationValue_eq_zero_of_eulerLagrange_zero [source]

```
lemma firstVariationValue_eq_zero_of_eulerLagrange_zero
    (L : Lagrangian d m k) (f : Space d → EuclideanSpace ℝ (Fin m))
    η( : AdmissibleVariation d (EuclideanSpace ℝ (Fin m))) (hEuler : eulerLagrangeOp L f = 0) :
    firstVariationValue L f η = 0
```

2.97 PhyslibAlpha.ClassicalFieldTheory.Local.FirstVariation.Criterion

PhyslibAlpha/ClassicalFieldTheory/Local/FirstVariation/Criterion.lean on GitHub.

def ClassicalFieldTheory.Local.AllVariationsHaveFiniteAction [source]

Explicit global finiteness hypothesis for all admissible variations.

```
def AllVariationsHaveFiniteAction (L : Lagrangian d m k)
    (f : Space d → EuclideanSpace ℝ (Fin m)) : Prop
```

def ClassicalFieldTheory.Local.HasFirstVariationFormula [source]

The first-variation formula for the local action, packaged as a reusable hypothesis.

```
def HasFirstVariationFormula (L : Lagrangian d m k)
    (f : Space d → EuclideanSpace ℝ (Fin m)) : Prop
```

lemma ClassicalFieldTheory.Local.isCritical_of_eulerLagrange_zero [source]

```
lemma isCritical_of_eulerLagrange_zero (L : Lagrangian d m k)
    (f : Space d → EuclideanSpace ℝ (Fin m))
    (hfirst : HasFirstVariationFormula L f) (hEuler : eulerLagrangeOp L f = 0) :
    IsCritical L f
```

lemma ClassicalFieldTheory.Local.eulerLagrange_zero_of_isCritical [source]

```
lemma eulerLagrange_zero_of_isCritical (L : Lagrangian d m k)
    (f : Space d → EuclideanSpace ℝ (Fin m))
    (hfirst : HasFirstVariationFormula L f)
    (hfin : AllVariationsHaveFiniteAction L f)
    (hcont : Continuous (eulerLagrangeOp L f))
    (hcrit : IsCritical L f) :
    eulerLagrangeOp L f = 0
```

theorem ClassicalFieldTheory.Local.isCritical_iff_eulerLagrange_zero [source]

```
theorem isCritical_iff_eulerLagrange_zero (L : Lagrangian d m k)
    (f : Space d → EuclideanSpace ℝ (Fin m))
    (hfirst : HasFirstVariationFormula L f)
    (hfin : AllVariationsHaveFiniteAction L f)
    (hcont : Continuous (eulerLagrangeOp L f)) :
    IsCritical L f ↔ eulerLagrangeOp L f = 0
```

lemma ClassicalFieldTheory.Local.hasFirstVariationFormula_of_underIntegral_linearized_and_parts [source]

```
private lemma hasFirstVariationFormula_of_underIntegral_linearized_and_parts
    (L : Lagrangian d m k) (f : Space d → EuclideanSpace ℝ (Fin m))
    (hint : HasActionVariationDerivativeUnderIntegral L f)
    (hpoint : HasPointwiseLinearizedDensityFormula L f)
    (hibp : HasIntegratedByPartsFormula L f) :
    HasFirstVariationFormula L f
```

lemma ClassicalFieldTheory.Local.hasFirstVariationFormula_of_contDiff_underIntegral_and_parts
[source]

```
private lemma hasFirstVariationFormula_of_contDiff_underIntegral_and_parts
    (L : Lagrangian d m k) (f : Space d → EuclideanSpace ℝ (Fin m)) (hf : ContDiff ℝ ∞ f)
    (hint : HasActionVariationDerivativeUnderIntegral L f)
    (hibp : HasIntegratedByPartsFormula L f) :
    HasFirstVariationFormula L f
```

lemma ClassicalFieldTheory.Local.hasFirstVariationFormula_of_underIntegral_linearized_and_termwise
[source]

```
private lemma hasFirstVariationFormula_of_underIntegral_linearized_and_termwise
    (L : Lagrangian d m k) (f : Space d → EuclideanSpace ℝ (Fin m))
    (hint : HasActionVariationDerivativeUnderIntegral L f)
    (hpoint : HasPointwiseLinearizedDensityFormula L f)
    (hterm : HasTermwiseIntegratedByPartsFormula L f) :
    HasFirstVariationFormula L f
```

lemma ClassicalFieldTheory.Local.hasFirstVariationFormula_of_contDiff_underIntegral_and_termwise
[source]

```
private lemma hasFirstVariationFormula_of_contDiff_underIntegral_and_termwise
    (L : Lagrangian d m k) (f : Space d → EuclideanSpace ℝ (Fin m)) (hf : ContDiff ℝ ∞ f)
    (hint : HasActionVariationDerivativeUnderIntegral L f)
    (hterm : HasTermwiseIntegratedByPartsFormula L f) :
    HasFirstVariationFormula L f
```

theorem ClassicalFieldTheory.Local.isCritical_iff_eulerLagrange_zero_of_underIntegral_linearized_and_p
[source]

The local Euler-Lagrange criterion obtained from the packaged analytic ingredients of the first-variation formula. This is the current formalized form of Theorem 5.2.

```
private theorem isCritical_iff_eulerLagrange_zero_of_underIntegral_linearized_and_parts
    (L : Lagrangian d m k) (f : Space d → EuclideanSpace ℝ (Fin m))
    (hint : HasActionVariationDerivativeUnderIntegral L f)
    (hpoint : HasPointwiseLinearizedDensityFormula L f)
    (hibp : HasIntegratedByPartsFormula L f)
    (hfin : AllVariationsHaveFiniteAction L f)
    (hcont : Continuous (eulerLagrangeOp L f)) :
    IsCritical L f ↔ eulerLagrangeOp L f = 0
```

theorem ClassicalFieldTheory.Local.isCritical_iff_eulerLagrange_zero_of_underIntegral_linearized_and_t
[source]

Variant of the local Euler-Lagrange criterion where the integration-by-parts input is reduced to a termwise hypothesis.

```
private theorem isCritical_iff_eulerLagrange_zero_of_underIntegral_linearized_and_termwise
    (L : Lagrangian d m k) (f : Space d → EuclideanSpace ℝ (Fin m))
    (hint : HasActionVariationDerivativeUnderIntegral L f)
    (hpoint : HasPointwiseLinearizedDensityFormula L f)
    (hterm : HasTermwiseIntegratedByPartsFormula L f)
    (hfin : AllVariationsHaveFiniteAction L f)
```

```
    (hcont : Continuous (eulerLagrangeOp L f)) :
    IsCritical L f ↔ eulerLagrangeOp L f = 0
```

theorem ClassicalFieldTheory.Local.isCritical_iff_eulerLagrange_zero_of_contDiff_underIntegral_and_ter
[source]

```
private theorem isCritical_iff_eulerLagrange_zero_of_contDiff_underIntegral_and_termwise
    (L : Lagrangian d m k) (f : Space d → EuclideanSpace ℝ (Fin m)) (hf : ContDiff ℝ ∞ f)
    (hint : HasActionVariationDerivativeUnderIntegral L f)
    (hterm : HasTermwiseIntegratedByPartsFormula L f)
    (hfin : AllVariationsHaveFiniteAction L f)
    (hcont : Continuous (eulerLagrangeOp L f)) :
    IsCritical L f ↔ eulerLagrangeOp L f = 0
```

theorem ClassicalFieldTheory.Local.isCritical_iff_eulerLagrange_zero_of_contDiff_continuous_coordDeriv
[source]

```
private theorem
    isCritical_iff_eulerLagrange_zero_of_contDiff_continuous_coordDeriv_and_termwise
    (L : Lagrangian d m k) (f : Space d → EuclideanSpace ℝ (Fin m)) (hf : ContDiff ℝ ∞ f)
    (hcontVar : ∀ η : AdmissibleVariation d (EuclideanSpace ℝ (Fin m)),
      ∀ I : DerivativeIndex d k, ∀ a : Fin m,
      Continuous (fun p : ℝ × Space d =>
        L.coordDeriv I a (jetAt k (variedField f η p.1) p.2)))
    (hterm : HasTermwiseIntegratedByPartsFormula L f)
    (hfin : AllVariationsHaveFiniteAction L f)
    (hcont : Continuous (eulerLagrangeOp L f)) :
    IsCritical L f ↔ eulerLagrangeOp L f = 0
```

theorem ClassicalFieldTheory.Local.isCritical_iff_eulerLagrange_zero_of_contDiff_and_regular
[source]

```
private theorem isCritical_iff_eulerLagrange_zero_of_contDiff_and_regular
    (L : Lagrangian d m k) (f : Space d → EuclideanSpace ℝ (Fin m)) (hf : ContDiff ℝ ∞ f)
    (hcontVar : ∀ η : AdmissibleVariation d (EuclideanSpace ℝ (Fin m)),
      Lagrangian.ContinuousCoordDerivAlongFamily L (fun s : ℝ => variedField f η s))
    (hcoeff : Lagrangian.ContDiffCoordDerivAlongField L f)
    (hfin : AllVariationsHaveFiniteAction L f)
    (hcont : Continuous (eulerLagrangeOp L f)) :
    IsCritical L f ↔ eulerLagrangeOp L f = 0
```

theorem ClassicalFieldTheory.Local.isCritical_iff_eulerLagrange_zero_of_contDiff_and_regularityAt
[source]

```
private theorem isCritical_iff_eulerLagrange_zero_of_contDiff_and_regularityAt
    (L : Lagrangian d m k) (f : Space d → EuclideanSpace ℝ (Fin m)) (hf : ContDiff ℝ ∞ f)
    (hreg : HasEulerLagrangeRegularityAt L f)
    (hfin : AllVariationsHaveFiniteAction L f)
    (hcont : Continuous (eulerLagrangeOp L f)) :
    IsCritical L f ↔ eulerLagrangeOp L f = 0
```

theorem ClassicalFieldTheory.Local.isCritical_iff_eulerLagrange_zero_of_contDiff_and_smoothRegularity
[source]

```
private theorem isCritical_iff_eulerLagrange_zero_of_contDiff_and_smoothRegularity
    (L : Lagrangian d m k) (f : Space d → EuclideanSpace ℝ (Fin m)) (hf : ContDiff ℝ ∞ f)
    (hreg : HasSmoothEulerLagrangeRegularity L)
    (hfin : AllVariationsHaveFiniteAction L f)
    (hcont : Continuous (eulerLagrangeOp L f)) :
    IsCritical L f ↔ eulerLagrangeOp L f = 0
```

theorem ClassicalFieldTheory.Local.isCritical_iff_eulerLagrange_zero_of_contDiff_and_coordinateRegular
[source]

```
private theorem isCritical_iff_eulerLagrange_zero_of_contDiff_and_coordinateRegularity
    (L : Lagrangian d m k) (f : Space d → EuclideanSpace ℝ (Fin m)) (hf : ContDiff ℝ ∞ f)
    (hcoord : Lagrangian.ContDiffCoordDerivInCoordinates L)
    (hfin : AllVariationsHaveFiniteAction L f)
    (hcont : Continuous (eulerLagrangeOp L f)) :
    IsCritical L f ↔ eulerLagrangeOp L f = 0
```

theorem ClassicalFieldTheory.Local.isCritical_iff_eulerLagrange_zero_of_contDiff_and_regularityAt'
[source]

```
private theorem isCritical_iff_eulerLagrange_zero_of_contDiff_and_regularityAt'
    (L : Lagrangian d m k) (f : Space d → EuclideanSpace ℝ (Fin m)) (hf : ContDiff ℝ ∞ f)
    (hreg : HasEulerLagrangeRegularityAt L f)
    (hfin : AllVariationsHaveFiniteAction L f) :
    IsCritical L f ↔ eulerLagrangeOp L f = 0
```

theorem ClassicalFieldTheory.Local.isCritical_iff_eulerLagrange_zero_of_contDiff_and_smoothRegularity'
[source]

```
private theorem isCritical_iff_eulerLagrange_zero_of_contDiff_and_smoothRegularity'
    (L : Lagrangian d m k) (f : Space d → EuclideanSpace ℝ (Fin m)) (hf : ContDiff ℝ ∞ f)
    (hreg : HasSmoothEulerLagrangeRegularity L)
    (hfin : AllVariationsHaveFiniteAction L f) :
    IsCritical L f ↔ eulerLagrangeOp L f = 0
```

theorem ClassicalFieldTheory.Local.isCritical_iff_eulerLagrange_zero_of_contDiff_and_coordinateRegular
[source]

```
private theorem isCritical_iff_eulerLagrange_zero_of_contDiff_and_coordinateRegularity'
    (L : Lagrangian d m k) (f : Space d → EuclideanSpace ℝ (Fin m)) (hf : ContDiff ℝ ∞ f)
    (hcoord : Lagrangian.ContDiffCoordDerivInCoordinates L)
    (hfin : AllVariationsHaveFiniteAction L f) :
    IsCritical L f ↔ eulerLagrangeOp L f = 0
```

theorem ClassicalFieldTheory.Local.«theorem@4a640fdf» [source]

```
private theorem
    isCritical_iff_eulerLagrange_zero_of_contDiff_and_coordinateRegularity_of_hasFiniteAction
    (L : Lagrangian d m k) (f : Space d → EuclideanSpace ℝ (Fin m)) (hf : ContDiff ℝ ∞ f)
    (hcoord : Lagrangian.ContDiffCoordDerivInCoordinates L)
    (hbase : HasFiniteAction L f)
    (hlocal :
      ∀ η : AdmissibleVariation d (EuclideanSpace ℝ (Fin m)),
```

```
      HasCompactlySupportedActionVariationDifference L f η) :
    IsCritical L f ↔ eulerLagrangeOp L f = 0
```

theorem ClassicalFieldTheory.Local.isCritical_iff_eulerLagrange_zero_of_hasFiniteAction_and_continuous
[source]

```
theorem isCritical_iff_eulerLagrange_zero_of_hasFiniteAction_and_continuousInCoordinates
    (L : Lagrangian d m k) (f : Space d → EuclideanSpace ℝ (Fin m)) (hf : ContDiff ℝ ∞ f)
    (hcoord : Lagrangian.ContDiffCoordDerivInCoordinates L)
    (hcontL : Lagrangian.ContinuousInCoordinates L)
    (hbase : HasFiniteAction L f) :
    IsCritical L f ↔ eulerLagrangeOp L f = 0
```

2.98 PhyslibAlpha.ClassicalFieldTheory.Local.FirstVariation.Density

PhyslibAlpha/ClassicalFieldTheory/Local/FirstVariation/Density.lean on GitHub.

def ClassicalFieldTheory.Local.HasActionVariationDerivativeUnderIntegral [source]

Differentiation of the varied action under the integral sign, packaged as a hypothesis.

```
def HasActionVariationDerivativeUnderIntegral (L : Lagrangian d m k)
    (f : Space d → EuclideanSpace ℝ (Fin m)) :
    Prop
```

def ClassicalFieldTheory.Local.HasPointwiseLinearizedDensityFormula [source]

Pointwise linearization of the action density before integration by parts.

```
def HasPointwiseLinearizedDensityFormula (L : Lagrangian d m k)
    (f : Space d → EuclideanSpace ℝ (Fin m)) : Prop
```

lemma ClassicalFieldTheory.Local.actionVariation_hasDerivAt_of_dominated_loc_of_deriv_le
[source]

A direct bridge from Mathlib's dominated differentiation-under-the-integral theorem to the local action variation. This isolates the measure-theoretic input needed to prove 'HasActionVariationDerivativeUnderIntegral'.

```
lemma actionVariation_hasDerivAt_of_dominated_loc_of_deriv_le
    (L : Lagrangian d m k) (f : Space d → EuclideanSpace ℝ (Fin m))
    η( : AdmissibleVariation d (EuclideanSpace ℝ (Fin m)))
    {F' : ℝ → Space d → ℝ} {bound : Space d → ℝ} ε{ : ℝ} (εh : 0 < ε)
    (hmeas :
      ∀ᶠ s in nhds (0 : ℝ),
        AEStronglyMeasurable (actionDensity L (variedField f η s)) volume)
    (hfinite0 : Integrable (actionDensity L (variedField f η 0)))
    (hF'_meas : AEStronglyMeasurable (F' 0) volume)
    (hbound :
      ∀ᵐ x ∂volume, ∀ s ∈ Metric.ball (0 : ℝ) ε, ‖F' s ‖x ≤ bound x)
    (hbound_int : Integrable bound volume)
    (hderiv :
      ∀ᵐ x ∂volume, ∀ s ∈ Metric.ball (0 : ℝ) ε,
        HasDerivAt (fun r : ℝ => actionDensity L (variedField f η r) x) (F' s x) s) :
    HasDerivAt (actionVariation L f η) ∫( x, F' 0 x) 0
```

def ClassicalFieldTheory.Local.HasDominatedVariationDerivativeNear [source]

A dominated bound for the pointwise first variation along the varied-field family near `s = 0`. This packages the analytic domination needed to differentiate the action under the integral sign.

```
def HasDominatedVariationDerivativeNear (L : Lagrangian d m k)
    (f : Space d → EuclideanSpace ℝ (Fin m))
    (η( : AdmissibleVariation d (EuclideanSpace ℝ (Fin m))) : Prop
```

lemma ClassicalFieldTheory.Local.continuous_coordDeriv_at_zero [source]

```
private lemma continuous_coordDeriv_at_zero
    (L : Lagrangian d m k) (f : Space d → EuclideanSpace ℝ (Fin m))
    (η( : AdmissibleVariation d (EuclideanSpace ℝ (Fin m)))
    (hcont : ∀ I : DerivativeIndex d k, ∀ a : Fin m,
      Continuous (fun p : ℝ × Space d =>
        L.coordDeriv I a (jetAt k (variedField f η p.1) p.2)))
    (I : DerivativeIndex d k) (a : Fin m) :
    Continuous (fun x : Space d => L.coordDeriv I a (jetAt k f x))
```

lemma ClassicalFieldTheory.Local.firstVariationDensityTerm_norm_le_of_coeff_bound [source]

```
private lemma firstVariationDensityTerm_norm_le_of_coeff_bound
    (L : Lagrangian d m k) (f : Space d → EuclideanSpace ℝ (Fin m))
    (η( : AdmissibleVariation d (EuclideanSpace ℝ (Fin m)))
    (ψ( : DerivativeIndex d k → Fin m → Space d → ℝ)
    (ψh : ∀ I : DerivativeIndex d k, ∀ a : Fin m, ∀ x : Space d,
      ψ I a x = ∂^[I.1] (fun y => η( y) a) x)
    (C : DerivativeIndex d k → Fin m → ℝ)
    (K : DerivativeIndex d k → Fin m → Set (Space d))
    (hKzero : ∀ I : DerivativeIndex d k, ∀ a : Fin m, ∀ x, x ∉ K I a → ψ I a x = 0)
    (hC : ∀ I : DerivativeIndex d k, ∀ a : Fin m,
      ∀ p ∈ Metric.closedBall (0 : ℝ) 1 ˢ× K I a,
        ‖L.coordDeriv I a (jetAt k (variedField f η p.1) p.2)‖ ≤ C I a)
    {x : Space d} {s : ℝ} (hs : s ∈ Metric.ball (0 : ℝ) 1)
    (I : DerivativeIndex d k) (a : Fin m) :
    ‖firstVariationDensityTerm L (variedField f η s) η I a ‖x ≤ C I a * ‖ψ I a ‖x
```

lemma ClassicalFieldTheory.Local.norm_firstVariationDensity_le_bound [source]

```
private lemma norm_firstVariationDensity_le_bound
    (L : Lagrangian d m k) (f : Space d → EuclideanSpace ℝ (Fin m))
    (η( : AdmissibleVariation d (EuclideanSpace ℝ (Fin m)))
    (ψ( : DerivativeIndex d k → Fin m → Space d → ℝ)
    (ψh : ∀ I : DerivativeIndex d k, ∀ a : Fin m, ∀ x : Space d,
      ψ I a x = ∂^[I.1] (fun y => η( y) a) x)
    (C : DerivativeIndex d k → Fin m → ℝ)
    (K : DerivativeIndex d k → Fin m → Set (Space d))
    (hKzero : ∀ I : DerivativeIndex d k, ∀ a : Fin m, ∀ x, x ∉ K I a → ψ I a x = 0)
    (hC : ∀ I : DerivativeIndex d k, ∀ a : Fin m,
      ∀ p ∈ Metric.closedBall (0 : ℝ) 1 ˢ× K I a,
        ‖L.coordDeriv I a (jetAt k (variedField f η p.1) p.2)‖ ≤ C I a)
    {x : Space d} {s : ℝ} (hs : s ∈ Metric.ball (0 : ℝ) 1) :
    ‖firstVariationDensity L (variedField f η s) η ‖x
      ≤ ∑ I : DerivativeIndex d k, ∑ a : Fin m, C I a * ‖ψ I a ‖x
```

lemma ClassicalFieldTheory.Local.hasDominatedVariationDerivativeNear_of_continuous_coordDeriv [source]

A concrete dominated bound for the varied first-variation density, obtained from continuity of the jet-coordinate coefficient family in '(s, x)'. The compact support of the iterated derivatives of the admissible variation supplies the integrable majorant.

```
lemma hasDominatedVariationDerivativeNear_of_continuous_coordDeriv
    (L : Lagrangian d m k) (f : Space d → EuclideanSpace ℝ (Fin m))
    η( : AdmissibleVariation d (EuclideanSpace ℝ (Fin m)))
    (hcont : ∀ I : DerivativeIndex d k, ∀ a : Fin m,
      Continuous (fun p : ℝ × Space d =>
        L.coordDeriv I a (jetAt k (variedField f η p.1) p.2))) :
    HasDominatedVariationDerivativeNear L f η
```

lemma ClassicalFieldTheory.Local.hasDerivAt_actionDensity_variedField_zero_of_contDiff [source]

```
lemma hasDerivAt_actionDensity_variedField_zero_of_contDiff
    (L : Lagrangian d m k) (f : Space d → EuclideanSpace ℝ (Fin m)) (hf : ContDiff ℝ ∞ f) :
    ∀ η : AdmissibleVariation d (EuclideanSpace ℝ (Fin m)), ∀ x : Space d,
      HasDerivAt (fun s : ℝ => actionDensity L (variedField f η s) x)
        (firstVariationDensity L f η x) 0
```

lemma ClassicalFieldTheory.Local.hasDerivAt_actionDensity_variedField_of_contDiff [source]

```
lemma hasDerivAt_actionDensity_variedField_of_contDiff
    (L : Lagrangian d m k) (f : Space d → EuclideanSpace ℝ (Fin m)) (hf : ContDiff ℝ ∞ f)
    η( : AdmissibleVariation d (EuclideanSpace ℝ (Fin m))) (x : Space d) (s : ℝ) :
    HasDerivAt (fun r : ℝ => actionDensity L (variedField f η r) x)
      (firstVariationDensity L (variedField f η s) η x) s
```

lemma ClassicalFieldTheory.Local.hasPointwiseLinearizedDensityFormula_of_contDiff [source]

```
lemma hasPointwiseLinearizedDensityFormula_of_contDiff
    (L : Lagrangian d m k) (f : Space d → EuclideanSpace ℝ (Fin m)) (hf : ContDiff ℝ ∞ f) :
    HasPointwiseLinearizedDensityFormula L f
```

lemma ClassicalFieldTheory.Local.hasActionVariationDerivativeUnderIntegral_of_contDiff_of_dominatedVar [source]

```
lemma hasActionVariationDerivativeUnderIntegral_of_contDiff_of_dominatedVariation
    (L : Lagrangian d m k) (f : Space d → EuclideanSpace ℝ (Fin m)) (hf : ContDiff ℝ ∞ f)
    (hdom : ∀ η : AdmissibleVariation d (EuclideanSpace ℝ (Fin m)),
      HasFiniteActionVariation L f η →
      HasDominatedVariationDerivativeNear L f η) :
    HasActionVariationDerivativeUnderIntegral L f
```

lemma ClassicalFieldTheory.Local.hasActionVariationDerivativeUnderIntegral_of_contDiff_of_continuous_c [source]

```
lemma hasActionVariationDerivativeUnderIntegral_of_contDiff_of_continuous_coordDeriv
    (L : Lagrangian d m k) (f : Space d → EuclideanSpace ℝ (Fin m)) (hf : ContDiff ℝ ∞ f)
    (hcontVar : ∀ η : AdmissibleVariation d (EuclideanSpace ℝ (Fin m)),
      ∀ I : DerivativeIndex d k, ∀ a : Fin m,
```

```
    Continuous (fun p : ℝ × Space d =>
      L.coordDeriv I a (jetAt k (variedField f η p.1) p.2))) :
  HasActionVariationDerivativeUnderIntegral L f
```

lemma ClassicalFieldTheory.Local.hasActionVariationDerivativeUnderIntegral_of_contDiff_of_regular [source]

```
lemma hasActionVariationDerivativeUnderIntegral_of_contDiff_of_regular
    (L : Lagrangian d m k) (f : Space d → EuclideanSpace ℝ (Fin m)) (hf : ContDiff ℝ ∞ f)
    (hcontVar : ∀ η : AdmissibleVariation d (EuclideanSpace ℝ (Fin m)),
      Lagrangian.ContinuousCoordDerivAlongFamily L (fun s : ℝ => variedField f η s)) :
    HasActionVariationDerivativeUnderIntegral L f
```

2.99 PhyslibAlpha.ClassicalFieldTheory.Local.FirstVariation.IntegrationByParts

PhyslibAlpha/ClassicalFieldTheory/Local/FirstVariation/IntegrationByParts.lean on GitHub.

def ClassicalFieldTheory.Local.HasIntegratedByPartsFormula [source]

The integration-by-parts step sending the linearized density to the Euler-Lagrange pairing.

```
def HasIntegratedByPartsFormula (L : Lagrangian d m k)
    (f : Space d → EuclideanSpace ℝ (Fin m)) : Prop
```

def ClassicalFieldTheory.Local.HasTermwiseIntegratedByPartsFormula [source]

Termwise integration-by-parts data for the linearized first-variation density.

```
def HasTermwiseIntegratedByPartsFormula (L : Lagrangian d m k)
    (f : Space d → EuclideanSpace ℝ (Fin m)) : Prop
```

lemma ClassicalFieldTheory.Local.integral_mul_space_deriv_eq_neg_deriv_mul [source]

```
private lemma integral_mul_space_deriv_eq_neg_deriv_mul (i : Fin d) {g h : Space d → ℝ}
    (hg : ContDiff ℝ ∞ g) (hh : IsTestFunction h) :
    ∫ x, g x * ∂[i] h x = ∫ x, ∂(-[i] g x) * h x
```

lemma ClassicalFieldTheory.Local.integral_mul_iteratedDerivList_eq_sign [source]

```
private lemma integral_mul_iteratedDerivList_eq_sign (L : List (Fin d)) {g h : Space d → ℝ}
    (hg : ContDiff ℝ ∞ g) (hh : IsTestFunction h) :
    ∫ x, g x * L.foldr (fun i f => ∂[i] f) h x =
      ∫ x, (((-1 : ℝ) ^ L.length) * L.foldr (fun i f => ∂[i] f) g x) * h x
```

lemma ClassicalFieldTheory.Local.integral_mul_iteratedDeriv_eq_sign [source]

Repeated integration by parts for iterated coordinate derivatives against a test function.

```
lemma integral_mul_iteratedDeriv_eq_sign {g h : Space d → ℝ} (I : MultiIndex d)
    (hg : ContDiff ℝ ∞ g) (hh : IsTestFunction h) :
    ∫ x, g x * ∂^[I] h x =
      ∫ x, (((-1 : ℝ) ^ I.order) * ∂^[I] g x) * h x
```

lemma ClassicalFieldTheory.Local.hasTermwiseIntegratedByPartsFormula_of_contDiff_coordDeriv [source]

Concrete termwise integration by parts for the first-variation density, assuming the jet coordinate coefficient functions are smooth along the field.

```
lemma hasTermwiseIntegratedByPartsFormula_of_contDiff_coordDeriv
    (L : Lagrangian d m k) (f : Space d → EuclideanSpace ℝ (Fin m))
    (hcoeff : ∀ I : DerivativeIndex d k, ∀ a : Fin m,
      ContDiff ℝ ∞ (fun x => L.coordDeriv I a (jetAt k f x))) :
    HasTermwiseIntegratedByPartsFormula L f
```

lemma ClassicalFieldTheory.Local.hasTermwiseIntegratedByPartsFormula_of_regular [source]

```
lemma hasTermwiseIntegratedByPartsFormula_of_regular
    (L : Lagrangian d m k) (f : Space d → EuclideanSpace ℝ (Fin m))
    (hcoeff : Lagrangian.ContDiffCoordDerivAlongField L f) :
    HasTermwiseIntegratedByPartsFormula L f
```

lemma ClassicalFieldTheory.Local.integral_firstVariationDensity_eq_termwise_sum [source]

```
private lemma integral_firstVariationDensity_eq_termwise_sum
    (L : Lagrangian d m k) (f : Space d → EuclideanSpace ℝ (Fin m))
    η( : AdmissibleVariation d (EuclideanSpace ℝ (Fin m)))
    (hterm : HasTermwiseIntegratedByPartsFormula L f) :
    ∫ x, firstVariationDensity L f η x =
      ∑ I : DerivativeIndex d k, ∑ a : Fin m,
        ∫ x, firstVariationDensityTerm L f η I a x
```

lemma ClassicalFieldTheory.Local.integral_eulerLagrange_termwise_sum_eq_pairing [source]

```
private lemma integral_eulerLagrange_termwise_sum_eq_pairing
    (L : Lagrangian d m k) (f : Space d → EuclideanSpace ℝ (Fin m))
    η( : AdmissibleVariation d (EuclideanSpace ℝ (Fin m)))
    (hterm : HasTermwiseIntegratedByPartsFormula L f) :
    ∑ I : DerivativeIndex d k, ∑ a : Fin m, ∫ x, eulerLagrangeTerm L I a f x * η( x) a
      = firstVariationValue L f η
```

lemma ClassicalFieldTheory.Local.hasIntegratedByPartsFormula_of_termwise [source]

```
lemma hasIntegratedByPartsFormula_of_termwise (L : Lagrangian d m k)
    (f : Space d → EuclideanSpace ℝ (Fin m))
    (hterm : HasTermwiseIntegratedByPartsFormula L f) :
    HasIntegratedByPartsFormula L f
```

2.100 PhyslibAlpha.ClassicalFieldTheory.Local.FirstVariation.Regularity

PhyslibAlpha/ClassicalFieldTheory/Local/FirstVariation/Regularity.lean on GitHub.

def ClassicalFieldTheory.Local.HasEulerLagrangeRegularityAt [source]

The combined regularity package currently needed to derive the local Euler-Lagrange criterion for a field ‘f‘: smooth coefficient functions along ‘f‘, and continuity of the

corresponding coefficient families along every admissible varied-field family based at 'f'.

```
def HasEulerLagrangeRegularityAt (L : Lagrangian d m k)
    (f : Space d → EuclideanSpace ℝ (Fin m)) : Prop
```

def ClassicalFieldTheory.Local.HasSmoothEulerLagrangeRegularity [source]

A local Lagrangian has smooth Euler-Lagrange regularity if every smooth field satisfies the regularity package currently needed by the local first-variation proof.

```
def HasSmoothEulerLagrangeRegularity (L : Lagrangian d m k) : Prop
```

lemma ClassicalFieldTheory.Local.continuous_eulerLagrangeTerm_of_regular [source]

```
lemma continuous_eulerLagrangeTerm_of_regular
    (L : Lagrangian d m k) (f : Space d → EuclideanSpace ℝ (Fin m))
    (hcoeff : Lagrangian.ContDiffCoordDerivAlongField L f)
    (I : DerivativeIndex d k) (a : Fin m) :
    Continuous (eulerLagrangeTerm L I a f)
```

lemma ClassicalFieldTheory.Local.continuous_eulerLagrangeComponent_of_regular [source]

```
lemma continuous_eulerLagrangeComponent_of_regular
    (L : Lagrangian d m k) (f : Space d → EuclideanSpace ℝ (Fin m))
    (hcoeff : Lagrangian.ContDiffCoordDerivAlongField L f)
    (a : Fin m) :
    Continuous (eulerLagrangeComponent L a f)
```

lemma ClassicalFieldTheory.Local.continuous_eulerLagrangeOp_of_regular [source]

```
lemma continuous_eulerLagrangeOp_of_regular
    (L : Lagrangian d m k) (f : Space d → EuclideanSpace ℝ (Fin m))
    (hcoeff : Lagrangian.ContDiffCoordDerivAlongField L f) :
    Continuous (eulerLagrangeOp L f)
```

theorem ClassicalFieldTheory.Local.hasSmoothEulerLagrangeRegularity_of_contDiffCoordDerivInCoordinates [source]

```
theorem hasSmoothEulerLagrangeRegularity_of_contDiffCoordDerivInCoordinates
    (L : Lagrangian d m k) (hcoord : Lagrangian.ContDiffCoordDerivInCoordinates L) :
    HasSmoothEulerLagrangeRegularity L
```

2.101 PhyslibAlpha.ClassicalFieldTheory.Local.FirstVariation.Support

PhyslibAlpha/ClassicalFieldTheory/Local/FirstVariation/Support.lean on GitHub.

lemma ClassicalFieldTheory.Local.variedField_zero [source]

```
@[simp]
lemma variedField_zero (f : Space d → EuclideanSpace ℝ (Fin m))
    η( : AdmissibleVariation d (EuclideanSpace ℝ (Fin m))) :
    variedField f η 0 = f
```

lemma ClassicalFieldTheory.Local.variedField_variedField [source]

```
@[simp]
lemma variedField_variedField (f : Space d → EuclideanSpace ℝ (Fin m))
    η( : AdmissibleVariation d (EuclideanSpace ℝ (Fin m)))
    (s t : ℝ) :
    variedField (variedField f η s) η t = variedField f η (s + t)
```

lemma ClassicalFieldTheory.Local.contDiff_space_deriv [source]

```
lemma contDiff_space_deriv {g : Space d → ℝ} (hg : ContDiff ℝ ∞ g)
    (i : Fin d) : ContDiff ℝ ∞ ∂([i] g)
```

lemma ClassicalFieldTheory.Local.isTestFunction_space_deriv [source]

```
lemma isTestFunction_space_deriv {g : Space d → ℝ} (hg : IsTestFunction g) (i : Fin d) :
    IsTestFunction ∂([i] g)
```

lemma ClassicalFieldTheory.Local.iteratedDerivList_contDiff [source]

```
lemma iteratedDerivList_contDiff (L : List (Fin d)) {g : Space d → ℝ}
    (hg : ContDiff ℝ ∞ g) :
    ContDiff ℝ ∞ (L.foldr (fun i h => ∂[i] h) g)
```

lemma ClassicalFieldTheory.Local.iteratedDerivList_isTestFunction [source]

```
lemma iteratedDerivList_isTestFunction (L : List (Fin d)) {g : Space d → ℝ}
    (hg : IsTestFunction g) :
    IsTestFunction (L.foldr (fun i h => ∂[i] h) g)
```

lemma ClassicalFieldTheory.Local.iteratedDerivList_commute_deriv [source]

```
lemma iteratedDerivList_commute_deriv (L : List (Fin d)) (i : Fin d)
    {g : Space d → ℝ} (hg : ContDiff ℝ ∞ g) :
    L.foldr (fun j h => ∂[j] h) ∂([i] g) = ∂[i] (L.foldr (fun j h => ∂[j] h) g)
```

lemma ClassicalFieldTheory.Local.iteratedDeriv_coord_isTestFunction [source]

```
lemma iteratedDeriv_coord_isTestFunction η( : AdmissibleVariation d (EuclideanSpace ℝ (Fin m)))
    (I : DerivativeIndex d k) (a : Fin m) :
    IsTestFunction (fun x => ∂^[I.1] (fun y => η( y) a) x)
```

lemma ClassicalFieldTheory.Local.firstVariationDensityTerm_integrable_of_continuous_coordDeriv [source]

```
lemma firstVariationDensityTerm_integrable_of_continuous_coordDeriv
    (L : Lagrangian d m k) (f : Space d → EuclideanSpace ℝ (Fin m))
    η( : AdmissibleVariation d (EuclideanSpace ℝ (Fin m)))
    (I : DerivativeIndex d k) (a : Fin m)
    (hcont : Continuous (fun x : Space d =>
      L.coordDeriv I a (jetAt k f x))) :
    Integrable (firstVariationDensityTerm L f η I a)
```

lemma ClassicalFieldTheory.Local.continuous_jetBaseCoordinates_variedField [source]

```
lemma continuous_jetBaseCoordinates_variedField
    (k : ℕ) (f : Space d → EuclideanSpace ℝ (Fin m))
    η( : AdmissibleVariation d (EuclideanSpace ℝ (Fin m)))
    (hf : ContDiff ℝ ∞ f) :
    Continuous
      (fun p : ℝ × Space d => (p.2, jetCoordinatesAt k (variedField f η p.1) p.2))
```

2.102 PhyslibAlpha.ClassicalFieldTheory.Local.JetPoint

PhyslibAlpha/ClassicalFieldTheory/Local/JetPoint.lean on GitHub.

abbrev ClassicalFieldTheory.Local.JetCoordinates [source]

Coordinate data ‘u^a_I‘ for ‘k‘-th order jet points.

For each derivative index ‘I‘, these are the scalar components of the corresponding ‘EuclideanSpace ℝ (Fin m)‘ fiber coordinate. We keep this componentwise form because the local Euler-Lagrange formulas are indexed by both ‘I‘ and ‘a‘.

```
abbrev JetCoordinates (d m k : ℕ)
```

structure ClassicalFieldTheory.Local.JetPoint [source]

The coordinate-level points of the locally trivialized ‘k‘-jet bundle for fields ‘Space d → EuclideanSpace ℝ (Fin m)‘.

```
structure JetPoint (d m k : ℕ) where
  base : Space d
  fiber : JetCoordinates d m k
```

def ClassicalFieldTheory.Local.JetPoint.coord [source]

The coordinate ‘u^a_I‘ of a jet point.

```
def coord (J : JetPoint d m k) (I : DerivativeIndex d k) (a : Fin m) : ℝ
```

def ClassicalFieldTheory.Local.JetPoint.value [source]

The zero-th order value encoded by a jet point.

```
def value (J : JetPoint d m k) : EuclideanSpace ℝ (Fin m)
```

lemma ClassicalFieldTheory.Local.JetPoint.value_apply [source]

```
@[simp]
lemma value_apply (J : JetPoint d m k) (a : Fin m) :
    J.value a = J.coord 0 a
```

lemma ClassicalFieldTheory.Local.JetPoint.ext [source]

```
@[ext]
lemma ext (J K : JetPoint d m k) (hbase : J.base = K.base)
    (hcoord : ∀ I a, J.coord I a = K.coord I a) :
```

```
    J = K
```

def ClassicalFieldTheory.Local.JetPoint.ofBaseCoordinates [source]

Build a jet point from a base point and its jet coordinates.

```
def ofBaseCoordinates (x : Space d) (u : JetCoordinates d m k) : JetPoint d m k where
```

def ClassicalFieldTheory.Local.JetPoint.toBaseCoordinates [source]

The base point and jet coordinates of a jet point.

```
def toBaseCoordinates (J : JetPoint d m k) : Space d × JetCoordinates d m k
```

lemma ClassicalFieldTheory.Local.JetPoint.ofBaseCoordinates_base [source]

```
@[simp]
lemma ofBaseCoordinates_base (x : Space d) (u : JetCoordinates d m k) :
    (ofBaseCoordinates x u).base = x
```

lemma ClassicalFieldTheory.Local.JetPoint.ofBaseCoordinates_fiber [source]

```
@[simp]
lemma ofBaseCoordinates_fiber (x : Space d) (u : JetCoordinates d m k) :
    (ofBaseCoordinates x u).fiber = u
```

lemma ClassicalFieldTheory.Local.JetPoint.ofBaseCoordinates_coord [source]

```
@[simp]
lemma ofBaseCoordinates_coord (x : Space d) (u : JetCoordinates d m k)
    (I : DerivativeIndex d k) (a : Fin m) :
    (ofBaseCoordinates x u).coord I a = u I a
```

lemma ClassicalFieldTheory.Local.JetPoint.ofBaseCoordinates_base_fiber [source]

```
@[simp]
lemma ofBaseCoordinates_base_fiber (J : JetPoint d m k) :
    ofBaseCoordinates J.base J.fiber = J
```

lemma ClassicalFieldTheory.Local.JetPoint.toBaseCoordinates_ofBaseCoordinates [source]

```
@[simp]
lemma toBaseCoordinates_ofBaseCoordinates (x : Space d) (u : JetCoordinates d m k) :
    (ofBaseCoordinates x u).toBaseCoordinates = (x, u)
```

def ClassicalFieldTheory.Local.jetCoordinatesAt [source]

The jet-coordinate function determined by a field ‘g‘.

```
noncomputable def jetCoordinatesAt (k : ℕ) (g : Space d → EuclideanSpace ℝ (Fin m))
    (x : Space d) :
    JetCoordinates d m k
```

def ClassicalFieldTheory.Local.jetAt [source]

The coordinate-level ‘k‘-jet point of a field ‘f‘ at the point ‘x‘.

```
noncomputable def jetAt (k : ℕ) (f : Space d → EuclideanSpace ℝ (Fin m)) (x : Space d) :
    JetPoint d m k where
```

lemma ClassicalFieldTheory.Local.jetAt_base [source]

```
@[simp]
lemma jetAt_base (k : ℕ) (f : Space d → EuclideanSpace ℝ (Fin m)) (x : Space d) :
    (jetAt k f x).base = x
```

lemma ClassicalFieldTheory.Local.jetAt_value [source]

```
@[simp]
lemma jetAt_value (k : ℕ) (f : Space d → EuclideanSpace ℝ (Fin m)) (x : Space d) :
    (jetAt k f x).value = f x
```

lemma ClassicalFieldTheory.Local.jetAt_coord [source]

```
@[simp]
lemma jetAt_coord (k : ℕ) (f : Space d → EuclideanSpace ℝ (Fin m)) (x : Space d)
    (I : DerivativeIndex d k) (a : Fin m) :
    (jetAt k f x).coord I a = ∂^[I.1] (fun y => (f y) a) x
```

lemma ClassicalFieldTheory.Local.jetAt_coord_zero [source]

```
lemma jetAt_coord_zero (k : ℕ) (f : Space d → EuclideanSpace ℝ (Fin m)) (x : Space d)
    (a : Fin m) :
    (jetAt k f x).coord 0 a = (f x) a
```

lemma ClassicalFieldTheory.Local.jetCoordinatesAt_eq [source]

```
@[simp]
lemma jetCoordinatesAt_eq (k : ℕ) (g : Space d → EuclideanSpace ℝ (Fin m)) (x : Space d)
    (I : DerivativeIndex d k) (a : Fin m) :
    jetCoordinatesAt k g x I a = ∂^[I.1] (fun y => (g y) a) x
```

lemma ClassicalFieldTheory.Local.jetCoordinatesAt_zero [source]

```
lemma jetCoordinatesAt_zero (k : ℕ) (g : Space d → EuclideanSpace ℝ (Fin m))
    (x : Space d) (a : Fin m) :
    jetCoordinatesAt k g x 0 a = (g x) a
```

lemma ClassicalFieldTheory.Local.jetAt_eq_ofBaseCoordinates [source]

```
lemma jetAt_eq_ofBaseCoordinates (k : ℕ) (f : Space d → EuclideanSpace ℝ (Fin m))
    (x : Space d) :
    jetAt k f x = JetPoint.ofBaseCoordinates x (jetCoordinatesAt k f x)
```

2.103 PhyslibAlpha.ClassicalFieldTheory.Local.JetPointFiber

PhyslibAlpha/ClassicalFieldTheory/Local/JetPointFiber.lean on GitHub.

structure ClassicalFieldTheory.Local.JetFiberData [source]

Fiber-coordinate data for jet points of order ‘k‘. This records only the coordinates ‘u^a_I‘, not the base point in ‘Space d‘.

```
structure JetFiberData (d m k : ℕ) where
  coord : JetCoordinates d m k
```

instance ClassicalFieldTheory.Local.JetFiberData.«instance@24e9455d» [source]

```
instance : CoeFun (JetFiberData d m k) (fun _ => DerivativeIndex d k → Fin m → ℝ) where
```

lemma ClassicalFieldTheory.Local.JetFiberData.ext [source]

```
@[ext]
lemma ext (V W : JetFiberData d m k) (hcoord : ∀ I a, V.coord I a = W.coord I a) : V = W
```

def ClassicalFieldTheory.Local.JetFiberData.value [source]

The zero-th order component of a jet-fiber direction, corresponding to the field value.

```
def value (V : JetFiberData d m k) : EuclideanSpace ℝ (Fin m)
```

lemma ClassicalFieldTheory.Local.JetFiberData.value_apply [source]

```
@[simp]
lemma value_apply (V : JetFiberData d m k) (a : Fin m) :
    V.value a = V.coord 0 a
```

instance ClassicalFieldTheory.Local.JetFiberData.«instance@14f5f97d» [source]

```
instance : Zero (JetFiberData d m k) where
```

instance ClassicalFieldTheory.Local.JetFiberData.«instance@47407776» [source]

```
instance : Add (JetFiberData d m k) where
```

instance ClassicalFieldTheory.Local.JetFiberData.«instance@cddd31c6» [source]

```
instance : SMul ℝ (JetFiberData d m k) where
```

lemma ClassicalFieldTheory.Local.JetFiberData.zero_coord [source]

```
@[simp]
lemma zero_coord (I : DerivativeIndex d k) (a : Fin m) :
    (0 : JetFiberData d m k).coord I a = 0
```

lemma ClassicalFieldTheory.Local.JetFiberData.add_coord [source]

```
@[simp]
lemma add_coord (V W : JetFiberData d m k) (I : DerivativeIndex d k) (a : Fin m) :
    (V + W).coord I a = V.coord I a + W.coord I a
```

lemma ClassicalFieldTheory.Local.JetFiberData.smul_coord [source]

```
@[simp]
lemma smul_coord (c : ℝ) (V : JetFiberData d m k) (I : DerivativeIndex d k) (a : Fin m) :
    (c • V).coord I a = c * V.coord I a
```

def ClassicalFieldTheory.Local.JetPoint.addFiber [source]

Translate a jet point by a fiber-direction increment.

```
def addFiber (J : JetPoint d m k) (V : JetFiberData d m k) : JetPoint d m k where
```

def ClassicalFieldTheory.Local.JetPoint.lineMap [source]

The affine line in jet space through ‘J‘ in the fiber direction ‘V‘.

```
def lineMap (J : JetPoint d m k) (V : JetFiberData d m k) (s : ℝ) : JetPoint d m k
```

lemma ClassicalFieldTheory.Local.JetPoint.addFiber_base [source]

```
@[simp]
lemma addFiber_base (J : JetPoint d m k) (V : JetFiberData d m k) :
    (J.addFiber V).base = J.base
```

lemma ClassicalFieldTheory.Local.JetPoint.addFiber_value [source]

```
@[simp]
lemma addFiber_value (J : JetPoint d m k) (V : JetFiberData d m k) :
    (J.addFiber V).value = J.value + V.value
```

lemma ClassicalFieldTheory.Local.JetPoint.addFiber_coord [source]

```
@[simp]
lemma addFiber_coord (J : JetPoint d m k) (V : JetFiberData d m k)
    (I : DerivativeIndex d k) (a : Fin m) :
    (J.addFiber V).coord I a = J.coord I a + V.coord I a
```

lemma ClassicalFieldTheory.Local.JetPoint.lineMap_base [source]

```
@[simp]
lemma lineMap_base (J : JetPoint d m k) (V : JetFiberData d m k) (s : ℝ) :
    (J.lineMap V s).base = J.base
```

lemma ClassicalFieldTheory.Local.JetPoint.lineMap_coord [source]

```
@[simp]
lemma lineMap_coord (J : JetPoint d m k) (V : JetFiberData d m k) (s : ℝ)
```

```
    (I : DerivativeIndex d k) (a : Fin m) :
    (J.lineMap V s).coord I a = J.coord I a + s * V.coord I a
```

def ClassicalFieldTheory.Local.jetDirectionAt [source]

The jet-fiber direction determined by a field ‘g‘ at a point ‘x‘.

```
noncomputable def jetDirectionAt (k : ℕ) (g : Space d → EuclideanSpace ℝ (Fin m))
    (x : Space d) :
    JetFiberData d m k where
```

lemma ClassicalFieldTheory.Local.jetDirectionAt_coord [source]

```
@[simp]
lemma jetDirectionAt_coord (k : ℕ) (g : Space d → EuclideanSpace ℝ (Fin m)) (x : Space d)
    (I : DerivativeIndex d k) (a : Fin m) :
    (jetDirectionAt k g x).coord I a = ∂^[I.1] (fun y => (g y) a) x
```

lemma ClassicalFieldTheory.Local.jetDirectionAt_coord_zero [source]

```
lemma jetDirectionAt_coord_zero (k : ℕ) (g : Space d → EuclideanSpace ℝ (Fin m))
    (x : Space d) (a : Fin m) :
    (jetDirectionAt k g x).coord 0 a = (g x) a
```

lemma ClassicalFieldTheory.Local.jetCoordinatesAt_add_smul [source]

```
lemma jetCoordinatesAt_add_smul (k : ℕ) (f g : Space d → EuclideanSpace ℝ (Fin m))
    (x : Space d) (s : ℝ)
    (hf : ContDiff ℝ ∞ f) (hg : ContDiff ℝ ∞ g) :
    jetCoordinatesAt k (fun y => f y + s • g y) x =
      jetCoordinatesAt k f x + s • jetCoordinatesAt k g x
```

lemma ClassicalFieldTheory.Local.jetAt_add_smul [source]

```
lemma jetAt_add_smul (k : ℕ) (f g : Space d → EuclideanSpace ℝ (Fin m))
    (x : Space d) (s : ℝ)
    (hf : ContDiff ℝ ∞ f) (hg : ContDiff ℝ ∞ g) :
    jetAt k (fun y => f y + s • g y) x = (jetAt k f x).lineMap (jetDirectionAt k g x) s
```

2.104 PhyslibAlpha.ClassicalFieldTheory.Local.JetPointRegularity

PhyslibAlpha/ClassicalFieldTheory/Local/JetPointRegularity.lean on GitHub.

lemma ClassicalFieldTheory.Local.jetCoordinatesAt_contDiff [source]

Smoothness of the jet-coordinate map of a smooth field.

```
lemma jetCoordinatesAt_contDiff (k : ℕ) (g : Space d → EuclideanSpace ℝ (Fin m))
    (hg : ContDiff ℝ ∞ g) :
    ContDiff ℝ ∞ (jetCoordinatesAt k g)
```

lemma ClassicalFieldTheory.Local.jetBaseCoordinates_contDiff [source]

Smoothness of the base-plus-coordinate jet map of a smooth field.

```
lemma jetBaseCoordinates_contDiff (k : ℕ) (g : Space d → EuclideanSpace ℝ (Fin m))
    (hg : ContDiff ℝ ∞ g) :
    ContDiff ℝ ∞ (fun x : Space d => (x, jetCoordinatesAt k g x))
```

lemma ClassicalFieldTheory.Local.notMem_tsupport_coord [source]

```
private lemma notMem_tsupport_coord {g : Space d → EuclideanSpace ℝ (Fin m)} {x : Space d}
    (hx : x ∉ tsupport g) (a : Fin m) :
    x ∉ tsupport (fun y => (g y) a)
```

lemma ClassicalFieldTheory.Local.jetCoordinatesAt_eq_zero_of_notMem_tsupport [source]

Outside the topological support of a field, all of its jet coordinates vanish.

```
lemma jetCoordinatesAt_eq_zero_of_notMem_tsupport (k : ℕ)
    (g : Space d → EuclideanSpace ℝ (Fin m)) {x : Space d}
    (hx : x ∉ tsupport g) :
    jetCoordinatesAt k g x = 0
```

lemma ClassicalFieldTheory.Local.jetDirectionAt_eq_zero_of_notMem_tsupport [source]

Outside the topological support of a field, the corresponding jet-fiber direction vanishes.

```
lemma jetDirectionAt_eq_zero_of_notMem_tsupport (k : ℕ)
    (g : Space d → EuclideanSpace ℝ (Fin m)) {x : Space d}
    (hx : x ∉ tsupport g) :
    jetDirectionAt k g x = 0
```

2.105 PhyslibAlpha.ClassicalFieldTheory.Local.Lagrangian

PhyslibAlpha/ClassicalFieldTheory/Local/Lagrangian.lean on GitHub.

structure ClassicalFieldTheory.Local.Lagrangian [source]

A local 'k'-th order Lagrangian for fields 'Space d → EuclideanSpace ℝ (Fin m)'.

```
structure Lagrangian (d m k : ℕ) where
  toFun : JetPoint d m k → ℝ
  coordDeriv : DerivativeIndex d k → Fin m → JetPoint d m k → ℝ
  hasDerivAlongFiber :
    ∀ J : JetPoint d m k, ∀ V : JetFiberData d m k,
      HasDerivAt (fun s : ℝ => toFun (J.lineMap V s))
        ∑( I : DerivativeIndex d k, ∑ a : Fin m, coordDeriv I a J * V.coord I a) 0
```

instance ClassicalFieldTheory.Local.Lagrangian.«instance@43bf9c65» [source]

```
instance : CoeFun (Lagrangian d m k) (fun _ => JetPoint d m k → ℝ) where
```

def ClassicalFieldTheory.Local.Lagrangian.fiberDerivative [source]

The derivative of ‘L‘ along the fiber-jet direction ‘V‘ based at ‘J‘.

```
noncomputable def fiberDerivative (L : Lagrangian d m k) (J : JetPoint d m k)
    (V : JetFiberData d m k) : ℝ
```

lemma ClassicalFieldTheory.Local.Lagrangian.hasDerivAt_lineMap [source]

```
lemma hasDerivAt_lineMap (L : Lagrangian d m k) (J : JetPoint d m k) (V : JetFiberData d m k) :
    HasDerivAt (fun s : ℝ => L (J.lineMap V s)) (L.fiberDerivative J V) 0
```

def ClassicalFieldTheory.Local.Lagrangian.ContinuousInCoordinates [source]

Continuity of a local lagrangian in the explicit local jet coordinates ‘(x, u^a_I)‘.

```
def ContinuousInCoordinates (L : Lagrangian d m k) : Prop
```

def ClassicalFieldTheory.Local.Lagrangian.ContDiffCoordDerivInCoordinates [source]

Intrinsic smoothness of the jet-coordinate derivatives of a local Lagrangian in the explicit local jet coordinates ‘(x, u^a_I)‘.

```
def ContDiffCoordDerivInCoordinates (L : Lagrangian d m k) : Prop
```

def ClassicalFieldTheory.Local.Lagrangian.SmoothInCoordinates [source]

Public regularity package for a smooth local lagrangian in explicit local jet coordinates: continuity of the lagrangian itself together with smoothness of all jet-coordinate derivatives.

```
def SmoothInCoordinates (L : Lagrangian d m k) : Prop
```

def ClassicalFieldTheory.Local.Lagrangian.ContDiffCoordDerivAlongField [source]

Smoothness of all coefficient functions ‘x ↦ ∂L/∂u_I^a (j^k f(x))‘ along a field ‘f‘.

```
def ContDiffCoordDerivAlongField (L : Lagrangian d m k)
    (f : Space d → EuclideanSpace ℝ (Fin m)) : Prop
```

def ClassicalFieldTheory.Local.Lagrangian.ContinuousCoordDerivAlongFamily [source]

Continuity of the coefficient family ‘(s, x) ↦ ∂L/∂u_I^a (j^k(F s)(x))‘ along a one-parameter family of fields ‘F : ℝ → Space d → EuclideanSpace ℝ (Fin m)‘.

```
def ContinuousCoordDerivAlongFamily (L : Lagrangian d m k)
    (F : ℝ → Space d → EuclideanSpace ℝ (Fin m)) : Prop
```

lemma ClassicalFieldTheory.Local.Lagrangian.continuousAlongField_of_inCoordinates [source]

```
lemma continuousAlongField_of_inCoordinates (L : Lagrangian d m k)
    (hcont : ContinuousInCoordinates L) (f : Space d → EuclideanSpace ℝ (Fin m))
    (hf : ContDiff ℝ ∞ f) :
    Continuous (fun x : Space d => L (jetAt k f x))
```

lemma ClassicalFieldTheory.Local.Lagrangian.contDiffCoordDerivAlongField_of_inCoordinates [source]

```
lemma contDiffCoordDerivAlongField_of_inCoordinates (L : Lagrangian d m k)
    (hcoord : ContDiffCoordDerivInCoordinates L) (f : Space d → EuclideanSpace ℝ (Fin m))
    (hf : ContDiff ℝ ∞ f) :
    ContDiffCoordDerivAlongField L f
```

lemma ClassicalFieldTheory.Local.Lagrangian.continuousCoordDerivAlongFamily_of_inCoordinates [source]

```
lemma continuousCoordDerivAlongFamily_of_inCoordinates (L : Lagrangian d m k)
    (hcoord : ContDiffCoordDerivInCoordinates L)
    (F : ℝ → Space d → EuclideanSpace ℝ (Fin m))
    (hjet : Continuous (fun p : ℝ × Space d => (p.2, jetCoordinatesAt k (F p.1) p.2))) :
    ContinuousCoordDerivAlongFamily L F
```

def ClassicalFieldTheory.Local.Lagrangian.alongField [source]

Evaluate a local Lagrangian along the 'k'-jets of a field.

```
noncomputable def alongField (L : Lagrangian d m k)
    (f : Space d → EuclideanSpace ℝ (Fin m)) : Space d → ℝ
```

lemma ClassicalFieldTheory.Local.Lagrangian.alongField_apply [source]

```
@[simp]
lemma alongField_apply (L : Lagrangian d m k)
    (f : Space d → EuclideanSpace ℝ (Fin m)) (x : Space d) :
    L.alongField f x = L (jetAt k f x)
```

2.106 PhyslibAlpha.ClassicalFieldTheory.Local.TotalDerivative

PhyslibAlpha/ClassicalFieldTheory/Local/TotalDerivative.lean on GitHub.

def ClassicalFieldTheory.Local.evalOnJet [source]

Evaluate a local jet-dependent function along the 'k'-jet of a field.

```
noncomputable def evalOnJet (k : ℕ) (F : JetPoint d m k → ℝ)
    (f : Space d → EuclideanSpace ℝ (Fin m)) :
    Space d → ℝ
```

def ClassicalFieldTheory.Local.totalDerivative [source]

The total derivative of a jet-dependent function in the coordinate direction 'i'.

```
noncomputable def totalDerivative (k : ℕ) (i : Fin d) (F : JetPoint d m k → ℝ)
    (f : Space d → EuclideanSpace ℝ (Fin m)) : Space d → ℝ
```

def ClassicalFieldTheory.Local.iteratedTotalDerivative [source]

The iterated total derivative indexed by a multi-index.

```
noncomputable def iteratedTotalDerivative (k : ℕ) (I : MultiIndex d) (F : JetPoint d m k → ℝ)
    (f : Space d → EuclideanSpace ℝ (Fin m)) : Space d → ℝ
```

lemma ClassicalFieldTheory.Local.evalOnJet_apply [source]

```
@[simp]
lemma evalOnJet_apply (k : ℕ) (F : JetPoint d m k → ℝ)
    (f : Space d → EuclideanSpace ℝ (Fin m)) (x : Space d) :
    evalOnJet k F f x = F (jetAt k f x)
```

lemma ClassicalFieldTheory.Local.totalDerivative_eq [source]

```
@[simp]
lemma totalDerivative_eq (k : ℕ) (i : Fin d) (F : JetPoint d m k → ℝ)
    (f : Space d → EuclideanSpace ℝ (Fin m)) :
    totalDerivative k i F f = ∂[i] (fun x => F (jetAt k f x))
```

lemma ClassicalFieldTheory.Local.iteratedTotalDerivative_eq [source]

```
@[simp]
lemma iteratedTotalDerivative_eq (k : ℕ) (I : MultiIndex d) (F : JetPoint d m k → ℝ)
    (f : Space d → EuclideanSpace ℝ (Fin m)) :
    iteratedTotalDerivative k I F f = ∂^[I] (fun x => F (jetAt k f x))
```

lemma ClassicalFieldTheory.Local.iteratedTotalDerivative_zero [source]

```
lemma iteratedTotalDerivative_zero (k : ℕ) (F : JetPoint d m k → ℝ)
    (f : Space d → EuclideanSpace ℝ (Fin m)) :
    iteratedTotalDerivative k (0 : MultiIndex d) F f = evalOnJet k F f
```

lemma ClassicalFieldTheory.Local.iteratedTotalDerivative_single [source]

```
lemma iteratedTotalDerivative_single (k : ℕ) (i : Fin d) (F : JetPoint d m k → ℝ)
    (f : Space d → EuclideanSpace ℝ (Fin m)) :
    iteratedTotalDerivative k (Physlib.MultiIndex.increment 0 i) F f =
      totalDerivative k i F f
```

2.107 PhyslibAlpha.ClassicalFieldTheory.Local.TotalDivergence

PhyslibAlpha/ClassicalFieldTheory/Local/TotalDivergence.lean on GitHub.

def ClassicalFieldTheory.Local.HasTotalDivergenceDensity [source]

A lagrangian density has total-divergence form if, along every field, it is the sum of the total derivatives of a jet-dependent current. A current of order ‘k‘ gives a density of order ‘k + 1‘.

```
def HasTotalDivergenceDensity (L : Lagrangian d m (k + 1))
    (current : Fin d → JetPoint d m k → ℝ) : Prop
```

def ClassicalFieldTheory.Local.IsEulerLagrangeTrivial [source]

A local lagrangian is Euler-Lagrange-trivial if its Euler-Lagrange operator vanishes along every field.

```
def IsEulerLagrangeTrivial (L : Lagrangian d m k) : Prop
```

structure ClassicalFieldTheory.Local.TotalDivergence [source]

A packaged local total divergence.

The current has order ‘k‘, while the associated lagrangian has order ‘k + 1‘, matching the coordinate expression ‘$L = \sum i$, D_i B_i‘. The Euler-Lagrange-triviality field records the variationally trivial direction needed by later equivalence results.

```
structure TotalDivergence (d m k : ℕ) where
  lagrangian : Lagrangian d m (k + 1)
  current : Fin d → JetPoint d m k → ℝ
  hasTotalDivergenceDensity : HasTotalDivergenceDensity lagrangian current
  isEulerLagrangeTrivial : IsEulerLagrangeTrivial lagrangian
```

instance ClassicalFieldTheory.Local.TotalDivergence.«instance@5c9b98ce» [source]

```
instance : CoeFun (TotalDivergence d m k) (fun _ => Lagrangian d m (k + 1)) where
```

lemma ClassicalFieldTheory.Local.TotalDivergence.actionDensity_eq [source]

```
@[simp]
lemma actionDensity_eq (T : TotalDivergence d m k)
    (f : Space d → EuclideanSpace ℝ (Fin m)) :
    actionDensity T.lagrangian f =
      fun x => ∑ i : Fin d, totalDerivative k i (T.current i) f x
```

lemma ClassicalFieldTheory.Local.TotalDivergence.eulerLagrangeOp_eq_zero [source]

```
lemma eulerLagrangeOp_eq_zero (T : TotalDivergence d m k)
    (f : Space d → EuclideanSpace ℝ (Fin m)) :
    eulerLagrangeOp T.lagrangian f = 0
```

lemma ClassicalFieldTheory.Local.TotalDivergence.isCritical [source]

```
lemma isCritical (T : TotalDivergence d m k)
    (f : Space d → EuclideanSpace ℝ (Fin m))
    (hTf : IsAdmissibleForAction T.lagrangian f)
    (hT : Lagrangian.SmoothInCoordinates T.lagrangian) :
    IsCritical T.lagrangian f
```

2.108 PhyslibAlpha.ClassicalFieldTheory.Local.TotalDivergenceEquivalence

PhyslibAlpha/ClassicalFieldTheory/Local/TotalDivergenceEquivalence.lean on GitHub.

def ClassicalFieldTheory.Local.HasTotalDivergenceDifference [source]

A lagrangian 'target' differs from 'source' by the density of a packaged total divergence.

The total divergence has current order 'k', hence its associated lagrangian and both compared lagrangians have order 'k + 1'.

```
def HasTotalDivergenceDifference (source target : Lagrangian d m (k + 1))
    (T : TotalDivergence d m k) : Prop
```

def ClassicalFieldTheory.Local.IsEulerLagrangeEquivalent [source]

Two local lagrangians are Euler-Lagrange equivalent when they determine the same local Euler-Lagrange operator along every field.

```
def IsEulerLagrangeEquivalent (source target : Lagrangian d m k) : Prop
```

lemma ClassicalFieldTheory.Local.IsEulerLagrangeEquivalent.refl [source]

```
lemma refl (L : Lagrangian d m k) : IsEulerLagrangeEquivalent L L
```

lemma ClassicalFieldTheory.Local.IsEulerLagrangeEquivalent.symm [source]

```
lemma symm {L1 L2 : Lagrangian d m k} (h : IsEulerLagrangeEquivalent L1 L2) :
    IsEulerLagrangeEquivalent L2 L1
```

lemma ClassicalFieldTheory.Local.IsEulerLagrangeEquivalent.trans [source]

```
lemma trans {L1 L2 L3 : Lagrangian d m k}
    (h12 : IsEulerLagrangeEquivalent L1 L2) (h23 : IsEulerLagrangeEquivalent L2 L3) :
    IsEulerLagrangeEquivalent L1 L3
```

lemma ClassicalFieldTheory.Local.IsEulerLagrangeEquivalent.eulerLagrangeOp_eq [source]

```
lemma eulerLagrangeOp_eq {source target : Lagrangian d m k}
    (h : IsEulerLagrangeEquivalent source target)
    (f : Space d → EuclideanSpace ℝ (Fin m)) :
    eulerLagrangeOp target f = eulerLagrangeOp source f
```

lemma ClassicalFieldTheory.Local.IsEulerLagrangeEquivalent.eulerLagrangeOp_eq_zero_iff [source]

```
lemma eulerLagrangeOp_eq_zero_iff {source target : Lagrangian d m k}
    (h : IsEulerLagrangeEquivalent source target)
    (f : Space d → EuclideanSpace ℝ (Fin m)) :
    eulerLagrangeOp target f = 0 ↔ eulerLagrangeOp source f = 0
```

lemma ClassicalFieldTheory.Local.IsEulerLagrangeEquivalent.isCritical_iff [source]

```
lemma isCritical_iff {source target : Lagrangian d m k}
    (h : IsEulerLagrangeEquivalent source target)
    (f : Space d → EuclideanSpace ℝ (Fin m))
    (hsourcef : IsAdmissibleForAction source f)
    (htargetf : IsAdmissibleForAction target f)
    (hsource : Lagrangian.SmoothInCoordinates source)
```

```
    (htarget : Lagrangian.SmoothInCoordinates target) :
    IsCritical target f ↔ IsCritical source f
```

lemma ClassicalFieldTheory.Local.IsEulerLagrangeEquivalent.isCritical_target_of_source [source]

```
lemma isCritical_target_of_source {source target : Lagrangian d m k}
    (h : IsEulerLagrangeEquivalent source target)
    (f : Space d → EuclideanSpace ℝ (Fin m))
    (hsourcef : IsAdmissibleForAction source f)
    (htargetf : IsAdmissibleForAction target f)
    (hsource : Lagrangian.SmoothInCoordinates source)
    (htarget : Lagrangian.SmoothInCoordinates target)
    (hcrit : IsCritical source f) :
    IsCritical target f
```

lemma ClassicalFieldTheory.Local.IsEulerLagrangeEquivalent.isCritical_source_of_target [source]

```
lemma isCritical_source_of_target {source target : Lagrangian d m k}
    (h : IsEulerLagrangeEquivalent source target)
    (f : Space d → EuclideanSpace ℝ (Fin m))
    (hsourcef : IsAdmissibleForAction source f)
    (htargetf : IsAdmissibleForAction target f)
    (hsource : Lagrangian.SmoothInCoordinates source)
    (htarget : Lagrangian.SmoothInCoordinates target)
    (hcrit : IsCritical target f) :
    IsCritical source f
```

structure ClassicalFieldTheory.Local.TotalDivergenceEquivalence [source]

A packaged equivalence between two local lagrangians whose densities differ by a total divergence.

The field ‘sameEulerLagrangeOp‘ records the variational consequence in the current Alpha API. It will become a theorem once the stack has enough symbolic calculus for coordinate derivatives of total divergences.

```
structure TotalDivergenceEquivalence (d m k : ℕ) where
  source : Lagrangian d m (k + 1)
  target : Lagrangian d m (k + 1)
  divergence : TotalDivergence d m k
  hasTotalDivergenceDifference :
    HasTotalDivergenceDifference source target divergence
  sameEulerLagrangeOp : IsEulerLagrangeEquivalent source target
```

lemma ClassicalFieldTheory.Local.TotalDivergenceEquivalence.actionDensity_eq [source]

```
@[simp]
lemma actionDensity_eq (E : TotalDivergenceEquivalence d m k)
    (f : Space d → EuclideanSpace ℝ (Fin m)) :
    actionDensity E.target f =
      fun x => actionDensity E.source f x + actionDensity E.divergence.lagrangian f x
```

lemma ClassicalFieldTheory.Local.TotalDivergenceEquivalence.eulerLagrangeOp_eq [source]

```
lemma eulerLagrangeOp_eq (E : TotalDivergenceEquivalence d m k)
    (f : Space d → EuclideanSpace ℝ (Fin m)) :
    eulerLagrangeOp E.target f = eulerLagrangeOp E.source f
```

lemma ClassicalFieldTheory.Local.TotalDivergenceEquivalence.eulerLagrangeOp_eq_zero_iff [source]

```
lemma eulerLagrangeOp_eq_zero_iff (E : TotalDivergenceEquivalence d m k)
    (f : Space d → EuclideanSpace ℝ (Fin m)) :
    eulerLagrangeOp E.target f = 0 ↔ eulerLagrangeOp E.source f = 0
```

lemma ClassicalFieldTheory.Local.TotalDivergenceEquivalence.isCritical_iff [source]

```
lemma isCritical_iff (E : TotalDivergenceEquivalence d m k)
    (f : Space d → EuclideanSpace ℝ (Fin m))
    (hsourcef : IsAdmissibleForAction E.source f)
    (htargetf : IsAdmissibleForAction E.target f)
    (hsource : Lagrangian.SmoothInCoordinates E.source)
    (htarget : Lagrangian.SmoothInCoordinates E.target) :
    IsCritical E.target f ↔ IsCritical E.source f
```

lemma ClassicalFieldTheory.Local.TotalDivergenceEquivalence.isCritical_target_of_source [source]

```
lemma isCritical_target_of_source (E : TotalDivergenceEquivalence d m k)
    (f : Space d → EuclideanSpace ℝ (Fin m))
    (hsourcef : IsAdmissibleForAction E.source f)
    (htargetf : IsAdmissibleForAction E.target f)
    (hsource : Lagrangian.SmoothInCoordinates E.source)
    (htarget : Lagrangian.SmoothInCoordinates E.target)
    (hcrit : IsCritical E.source f) :
    IsCritical E.target f
```

lemma ClassicalFieldTheory.Local.TotalDivergenceEquivalence.isCritical_source_of_target [source]

```
lemma isCritical_source_of_target (E : TotalDivergenceEquivalence d m k)
    (f : Space d → EuclideanSpace ℝ (Fin m))
    (hsourcef : IsAdmissibleForAction E.source f)
    (htargetf : IsAdmissibleForAction E.target f)
    (hsource : Lagrangian.SmoothInCoordinates E.source)
    (htarget : Lagrangian.SmoothInCoordinates E.target)
    (hcrit : IsCritical E.target f) :
    IsCritical E.source f
```

2.109 PhyslibAlpha.ClassicalFieldTheory.Local.Variation

PhyslibAlpha/ClassicalFieldTheory/Local/Variation.lean on GitHub.

lemma ClassicalFieldTheory.Local.AdmissibleVariation.coord_euclidean [source]

A Euclidean component of an admissible variation is again a test function.

```
@[fun_prop]
lemma coord_euclidean η( : AdmissibleVariation d (EuclideanSpace ℝ (Fin m))) (a : Fin m) :
    IsTestFunction (fun x => η(.toFun x) a)
```

2.110 PhyslibAlpha.ClassicalMechanics.CoupledSpringPotential

PhyslibAlpha/ClassicalMechanics/CoupledSpringPotential.lean on GitHub.

def couplingPotential [source]

The potential energy of a pair of coupled springs.

```
noncomputable def couplingPotential (x : EuclideanSpace ℝ (Fin 2)) : ℝ
```

lemma couplingPotential_analyticAt [source]

```
lemma couplingPotential_analyticAt (z : EuclideanSpace ℝ (Fin 2)) :
    AnalyticAt ℝ couplingPotential z
```

lemma couplingPotential_hasFDerivAt [source]

```
lemma couplingPotential_hasFDerivAt (x : EuclideanSpace ℝ (Fin 2)) :
    HasFDerivAt couplingPotential
      ((2 * x 0 + x 1) • (EuclideanSpace.proj □( := ℝ) 0)
        + (x 0 + 2 * x 1) • (EuclideanSpace.proj □( := ℝ) 1)) x
```

lemma couplingPotential_gradient_zero [source]

```
lemma couplingPotential_gradient_zero :
    gradient couplingPotential ₂(![0, 0] : EuclideanSpace ℝ (Fin 2)) = 0
```

lemma couplingPotential_iteratedFDeriv_two [source]

```
lemma couplingPotential_iteratedFDeriv_two (z : EuclideanSpace ℝ (Fin 2))
    (a b : EuclideanSpace ℝ (Fin 2)) :
    iteratedFDeriv ℝ 2 couplingPotential z ![a, b]
      = 2 * a 0 * b 0 + a 0 * b 1 + a 1 * b 0 + 2 * a 1 * b 1
```

lemma couplingPotential_posDef [source]

```
lemma couplingPotential_posDef :
    (iteratedFDerivQuadraticMap couplingPotential
    ₂(![0, 0] : EuclideanSpace ℝ (Fin 2))).PosDef
```

lemma coupled_spring_potential [source]

```
lemma coupled_spring_potential :
    let U
```

2.111 PhyslibAlpha.Mathematics.PartialDerivativeTest

PhyslibAlpha/Mathematics/PartialDerivativeTest.lean on GitHub.

lemma Function.update$_0$ [source]

Update a vector of length 2 in coordinate 0.

```
@[simp]
lemma Function.₀update α{ : Type*} {a b c : α} : Function.update ![a,b] 0 c = ![c,b]
```

lemma Function.update$_1$ [source]

Update a vector of length 2 in coordinate 1.

```
@[simp]
lemma Function.₁update α{ : Type*} {a b c : α} : Function.update ![a,b] 1 c = ![a,c]
```

def hessianBilinearCompanion [source]

The Hessian companion as a bilinear map.

```
noncomputable def hessianBilinearCompanion {V : Type*} [NormedAddCommGroup V]
    [NormedSpace ℝ V] (f : V → ℝ) (₀x : V) : V →₁ℝ[] V →₁ℝ[] ℝ
```

def iteratedFDerivQuadraticMap [source]

The second iterated Frechét derivative as a quadratic map.

```
noncomputable def iteratedFDerivQuadraticMap {V : Type*} [NormedAddCommGroup V]
    [NormedSpace ℝ V] (f : V → ℝ) (₀x : V) : QuadraticMap ℝ V ℝ
```

def continuousBilinearMapOfContinuousMultilinearMap [source]

A continuous multilinear map is bilinear.

```
noncomputable def continuousBilinearMapOfContinuousMultilinearMap
    □{ : Type*} [NontriviallyNormedField □] [CompleteSpace □]
    {V : Type*}
    [NormedAddCommGroup V] [NormedSpace □ V] [FiniteDimensional □ V]
    (g : ContinuousMultilinearMap □ (fun _ : Fin 2 => V) □) : V →L□[] V →L□[] □
```

def QuadraticMap.toMultilinearMap [source]

.

```
def QuadraticMap.toMultilinearMap {V : Type*} [AddCommGroup V] [Module ℝ V]
    (Q : QuadraticMap ℝ V ℝ) : MultilinearMap ℝ (fun _ : Fin 2 => V) ℝ
```

def QuadraticMap.toMultilinearMapHalfPolarBilin [source]

.

```
noncomputable def QuadraticMap.toMultilinearMapHalfPolarBilin
    {V : Type*} [AddCommGroup V] [Module ℝ V]
    (Q : QuadraticMap ℝ V ℝ) :
    MultilinearMap ℝ (fun _ : Fin 2 => V) ℝ
```

theorem QuadraticMap.toMultilinearMap_continuous [source]

The multilinear map associated to a quadratic map is continuous.

```
theorem QuadraticMap.toMultilinearMap_continuous {V : Type*}
    [NormedAddCommGroup V] [NormedSpace ℝ V]
    [FiniteDimensional ℝ V] (Q : QuadraticMap ℝ V ℝ) : Continuous Q.toMultilinearMap
```

theorem QuadraticMap.toMultilinearMapHalfPolarBilin_continuous [source]

.

```
theorem QuadraticMap.toMultilinearMapHalfPolarBilin_continuous {V : Type*}
    [NormedAddCommGroup V] [NormedSpace ℝ V]
    [FiniteDimensional ℝ V] (Q : QuadraticMap ℝ V ℝ) :
    Continuous Q.toMultilinearMapHalfPolarBilin
```

def QuadraticMap.toContinuousMultilinearMap [source]

Construct a continuous multilinear map from a quadratic map on a finite dimensional space.

```
def QuadraticMap.toContinuousMultilinearMap {V : Type*} [NormedAddCommGroup V] [NormedSpace ℝ V
    ]
    [FiniteDimensional ℝ V] (Q : QuadraticMap ℝ V ℝ) :
    ContinuousMultilinearMap ℝ (fun _ : Fin 2 ↦ V) ℝ
```

def QuadraticMap.toContinuousMultilinearMapHalfPolarBilin [source]

.

```
noncomputable def QuadraticMap.toContinuousMultilinearMapHalfPolarBilin {V : Type*}
    [NormedAddCommGroup V] [NormedSpace ℝ V] [FiniteDimensional ℝ V] (Q : QuadraticMap ℝ V ℝ) :
    ContinuousMultilinearMap ℝ (fun _ : Fin 2 ↦ V) ℝ
```

theorem QuadraticMap.toContinuousMultilinearMap_apply [source]

The constructed continuous multilinear map agrees with the polar bilinear form.

```
theorem QuadraticMap.toContinuousMultilinearMap_apply {V : Type*} [NormedAddCommGroup V]
    [NormedSpace ℝ V] [FiniteDimensional ℝ V] (Q : QuadraticMap ℝ V ℝ) (x y : V) :
    Q.toContinuousMultilinearMap ![x, y] = Q.polarBilin x y
```

theorem QuadraticMap.toContinuousMultilinearMap_applyHalf [source]

.

```
theorem QuadraticMap.toContinuousMultilinearMap_applyHalf {V : Type*} [NormedAddCommGroup V]
    [NormedSpace ℝ V] [FiniteDimensional ℝ V] (Q : QuadraticMap ℝ V ℝ) (x y : V) :
    Q.toContinuousMultilinearMapHalfPolarBilin ![x, y] = (1/2) * Q.polarBilin x y
```

lemma coercive_of_posdefHalf [source]

.

```
lemma coercive_of_posdefHalf {V : Type*} [NormedAddCommGroup V] [NormedSpace ℝ V]
    [FiniteDimensional ℝ V] {F : QuadraticMap ℝ V ℝ}
    (hf' : F.PosDef) :
    IsCoercive (continuousBilinearMapOfContinuousMultilinearMap
        F.toContinuousMultilinearMapHalfPolarBilin)
```

lemma polarBilin_iteratedFDerivQuadraticMap [source]

The polar bilinear form of the second-derivative quadratic map is the symmetrized Hessian.

```
lemma polarBilin_iteratedFDerivQuadraticMap {V : Type*} [NormedAddCommGroup V] [NormedSpace ℝ V
    ]
    (f : V → ℝ) (₀x : V) (x y : V) :
    (iteratedFDerivQuadraticMap f ₀x).polarBilin x y
      = iteratedFDeriv ℝ 2 f ₀x ![x, y] + iteratedFDeriv ℝ 2 f ₀x ![y, x]
```

lemma iteratedFDeriv_two_swap [source]

Symmetry of the second iterated Fréchet derivative (Schwarz's theorem).

```
lemma iteratedFDeriv_two_swap {V : Type*} [NormedAddCommGroup V] [NormedSpace ℝ V]
    {f : V → ℝ} {₀x : V} (hf : ContDiffAt ℝ ⊤ f ₀x) (x y : V) :
    iteratedFDeriv ℝ 2 f ₀x ![y, x] = iteratedFDeriv ℝ 2 f ₀x ![x, y]
```

lemma coercive_of_posdef_of_contdiff [source]

Positive definiteness implies coercivity. The proof uses the general fact 'coercive_of_posdefHalf' but requires a differentiability assumption on 'f'.

```
lemma coercive_of_posdef_of_contdiff {V : Type*}
    [NormedAddCommGroup V] [NormedSpace ℝ V]
    [FiniteDimensional ℝ V] {f : V → ℝ} {₀x : V}
    (hf : ContDiffAt ℝ ⊤ f ₀x)
    (hf' : (iteratedFDerivQuadraticMap f ₀x).PosDef) :
    IsCoercive (continuousBilinearMapOfContinuousMultilinearMap
        (iteratedFDeriv ℝ 2 f ₀x))
```

lemma coercive_of_posdef [source]

Positive definiteness implies coercivity.

```
lemma coercive_of_posdef {V : Type*} [NormedAddCommGroup V] [NormedSpace ℝ V]
    [FiniteDimensional ℝ V] {f : V → ℝ} {₀x : V}
    (hf' : (iteratedFDerivQuadraticMap f ₀x).PosDef) :
    IsCoercive (continuousBilinearMapOfContinuousMultilinearMap
        (iteratedFDeriv ℝ 2 f ₀x))
```

theorem le_of_littleO [source]

.

```
theorem le_of_littleO {V : Type*}
    [NormedAddCommGroup V] [InnerProductSpace ℝ V]
    {f : V → ℝ} {₀x x : V} {C : ℝ}
```

```
    (hx : C * ‖(x - ₀‖x ^ 2) ≤ iteratedFDeriv ℝ 2 f ₀x fun _ ↦ x - ₀x)
    (₀hx : fderiv ℝ f ₀x x = fderiv ℝ f ₀x ₀x)
    (₁h : ‖f x - ∑ i ∈ range 3, 1 / ↑i ! * iteratedFDeriv ℝ i f ₀x fun _ ↦ x - ₀‖x
      ≤ C / 2 * ‖x - ₀‖x ^ 2) :
  f ₀x ≤ f x
```

theorem isLocalMin_of_posDef_of_littleo [source]

Second partial derivative test, "little oh" form.

```
theorem isLocalMin_of_posDef_of_littleo {V : Type*} [NormedAddCommGroup V]
    [InnerProductSpace ℝ V] [FiniteDimensional ℝ V] {f : V → ℝ} {₀x : V}
    (h : (fun x => f x - ∑ i ∈ range 3, 1 / (i)! * iteratedFDeriv ℝ i f ₀x fun _ => x - ₀x)
      =o[nhds ₀x] fun x => ‖x - ₀‖x ^ 2) (₀h : gradient f ₀x = 0) (hcd : ContDiffAt ℝ ⊤ f ₀x)
    (hf : (iteratedFDerivQuadraticMap f ₀x).PosDef) : IsLocalMin f ₀x
```

lemma littleO_of_powerseries [source]

Having a power series implies quadratic approximation.

```
lemma littleO_of_powerseries {V : Type*} [NormedAddCommGroup V] [NormedSpace ℝ V]
    {f : V → ℝ} {₀x : V} {p : FormalMultilinearSeries ℝ V ℝ}
    {r : NNReal} (h : HasFPowerSeriesOnBall f p ₀x r) :
    (fun x => f x - p.partialSum 3 (x - ₀x)) =o[nhds ₀x] fun x => ‖x - ₀‖x ^ 2
```

lemma isLocalMin.of_subsingleton [source]

```
@[nontriviality]
lemma isLocalMin.of_subsingleton {V : Type*} [TopologicalSpace V]
    [Subsingleton V] {f : V → ℝ} {₀x : V} : IsLocalMin f ₀x
```

theorem second_derivative_test [source]

The second partial derivative test.

```
theorem second_derivative_test {V : Type*} [NormedAddCommGroup V] [InnerProductSpace ℝ V]
    [FiniteDimensional ℝ V] {f : V → ℝ} {₀x : V} {p : FormalMultilinearSeries ℝ V ℝ}
    (₀h : gradient f ₀x = 0) {r : NNReal}
    (₁h : HasFPowerSeriesOnBall f p ₀x r)
    (hf : (iteratedFDerivQuadraticMap f ₀x).PosDef) : IsLocalMin f ₀x
```

theorem second_derivative_test_analyticAt [source]

A more convenient form of the second partial derivative test: it suffices that 'f' is analytic at 'x_0' (so that an explicit power series need not be produced by hand).

```
theorem second_derivative_test_analyticAt {V : Type*} [NormedAddCommGroup V]
    [InnerProductSpace ℝ V] [FiniteDimensional ℝ V] {f : V → ℝ} {₀x : V}
    (₀h : gradient f ₀x = 0) (han : AnalyticAt ℝ f ₀x)
    (hf : (iteratedFDerivQuadraticMap f ₀x).PosDef) : IsLocalMin f ₀x
```

2.112 PhyslibAlpha.QuantumMechanics.QuantumHarmonicOscillator

PhyslibAlpha/QuantumMechanics/QuantumHarmonicOscillator.lean on GitHub.

def a [source]

Annihilation operator.

```
def a (x : ℕ → ℂ) : ℕ → ℂ
```

def aLin [source]

```
def aLin : ℕ( → ℂ) →ₗ[ℂ] ℕ( → ℂ)
```

def a_dag [source]

Creation operator.

```
def a_dag (x : ℕ → ℂ) : ℕ → ℂ
```

def a_dagLin [source]

```
def a_dagLin : ℕ( → ℂ) →ₗ[ℂ] ℕ( → ℂ)
```

def ε [source]

```
def ε (n : ℕ) (c : ℂ) : ℕ → ℂ
```

lemma a_dag_eq [source]

Verify that a_dag really is the transpose of a.

```
lemma a_dag_eq (i j : ℕ) : a_dag ε( i 1) j = star (a ε( j 1) i)
```

lemma verify_a [source]

Verify that a $|n + 1\rangle = \sqrt{(n + 1)}\ |n\rangle$

```
lemma verify_a (n : ℕ) :
    a ε( (n + 1) 1) =
       ε n √((n + 1))
```

lemma verify_a_dag [source]

Verify that $a^\dagger\ |n\rangle = \sqrt{n+1}\ |n+1\rangle$.

```
lemma verify_a_dag (n : ℕ) :
    a_dag ε( n 1) = ε (n + 1) √((n + 1))
```

lemma verify_a_dag_a [source]

```
lemma verify_a_dag_a (n : ℕ) (x : ℕ → ℂ) :
    a_dag (a x) n = n * x n
```

lemma verify_a_a_dag [source]

```
lemma verify_a_a_dag (n : ℕ) (x : ℕ → ℂ) :
    a (a_dag x) n = (n + 1) * x n
```

lemma commutation_relation [source]

```
lemma commutation_relation :
    a ∘ a_dag - a_dag ∘ a = id
```

def coherentState [source]

```
def coherentState α( : ℂ) : ℕ → ℂ
```

def probabilityOf [source]

```
def probabilityOf (n : ℕ) α( : ℂ) : NNReal
```

lemma probabilityOf_eq_poisson_C [source]

Coherent state has a Poisson distribution.

```
lemma probabilityOf_eq_poisson_C (n : ℕ) α( : ℂ) :
    let Λ
```

lemma coherentState_only_eigenvector [source]

The only eigenvectors of ‘a‘ are the coherent states.

```
lemma coherentState_only_eigenvector α( : ℂ) (v : ℕ → ℂ) :
    a v = α • v ↔
    v = (Complex.exp ↑‖α‖( ^ 2 / 2) * v 0) • coherentState α
```

lemma eigenvector_coherentState [source]

```
lemma eigenvector_coherentState α( : ℂ) :
    a (coherentState α) = α • coherentState α
```

lemma distinct_eigenvectors_a [source]

None of the ‘coherentState‘ eigenvectors are proportional.

```
lemma distinct_eigenvectors_a α( β c : ℂ)
    (hc : coherentState α = c • coherentState β) : α = β
```

lemma a_dagalmost_eigenvector [source]

Formal eigenvectors for ‘a_dag‘ (not in ‘$\ell^2(\mathbb{C})$‘) (fails at n=0) .

```
lemma a_dagalmost_eigenvector α{ : ℂ} (αh : α ≠ 0) {n : ℕ} (hn : n ≠ 0) :
    a_dag (fun n => √(n.factorial) / α^n) n =
      α • (fun n => √(n.factorial) / α^n) n
```

lemma a_dag_nullspace [source]

```
lemma a_dag_nullspace
    {v : ℕ → ℂ} (hv : a_dag v = 0) : v = 0
```

lemma no_a_dag_eigenvector [source]

```
lemma no_a_dag_eigenvector α( : ℂ) (v : ℕ → ℂ) :
    a_dag v = α • v ↔ v = 0
```

lemma commutationRelation [source]

```
lemma commutationRelation : ⁅aLin, ⁆a_dagLin = 1
```

def coherentState_ℓ2 [source]

The coherent state belongs to '$\ell^2(\mathbb{C})$'.

```
def □coherentState_2 α( : ℂ) : lp (fun _ : ℕ => ℂ) 2
```

2.113 PhyslibAlpha.QuantumMechanics.StinespringDilation

PhyslibAlpha/QuantumMechanics/StinespringDilation.lean on GitHub.

def krausApply [source]

Completely positive map given by a (not necessarily minimal) Kraus family.

```
def krausApply {R : Type*} [Mul R] [Star R] [AddCommMonoid R]
    {q r : Type*} [Fintype q] [Fintype r]
    (K : r → Matrix q q R) ρ( : Matrix q q R) : Matrix q q R
```

lemma krausApply.posSemidef [source]

Kraus operator preserves PSD property.

```
lemma krausApply.posSemidef {R : Type*} [Ring R] [PartialOrder R] [StarRing R]
    [AddLeftMono R]
    {q r : Type*} [Fintype q] [Fintype r]
    (K : r → Matrix q q R)
    ρ{ : Matrix q q R} (ρh : ρ.PosSemidef) :
    (krausApply K ρ).PosSemidef
```

def QuantumChannel [source]

Quantum channel.

```
def QuantumChannel {R : Type*} [Mul R] [One R] [Star R] [AddCommMonoid R]
    {q r : Type*} [Fintype q] [Fintype r] [DecidableEq q]
    (K : r → Matrix q q R)
```

def QuantumOperation [source]

Quantum operation.

```
def QuantumOperation {R : Type*} [RCLike R]
    {q r : Type*} [Fintype q] [Fintype r] [DecidableEq q]
    (K : r → Matrix q q R)
```

def densityMatrix [source]

Density matrix.

```
def densityMatrix {R : Type*} [Ring R] [PartialOrder R] [StarRing R] (d : Type*) [Fintype d]
```

def densityMatrix.convex_comb [source]

Density matrices are closed under real convex combinations.

```
def densityMatrix.convex_comb {R : Type*} [RCLike R]
    {d : ℕ} ρ₀( ρ₁ : densityMatrix (Fin d) (R := R)) {t : R}
    (₀hp : 0 ≤ t) (₁hp : 0 ≤ 1 - t) : densityMatrix (Fin d) (R := R)
```

def $\mathbf{tr}_2$ [source]

Also known as 'partialTraceRight'.

```
def ₂tr {R : Type*} [Ring R] {m n m' : Type*} [Fintype n]
    ρ( : Matrix (m × n) (m' × n) R) : Matrix m m' R
```

def stinespringOp [source]

'stinespringOp' is often written as 'V'.

```
def stinespringOp {R : Type*} [Ring R]
    {m r : Type*} [Fintype r] [DecidableEq r]
  (K : r → Matrix m m R) : Matrix (m × r) m R
```

def stinespringDilation [source]

The Stinespring dilation.

```
def stinespringDilation {R : Type*} [Ring R] [StarRing R]
    {m r : Type*} [Fintype r] [DecidableEq r] [Fintype m]
    (K : r → Matrix m m R)
    ρ( : Matrix m m R)
```

def stinespringForm [source]

The partial trace of the Stinespring dilation.

```
def stinespringForm {R : Type*} [Ring R] [StarRing R]
    {m r : Type*} [Fintype r] [DecidableEq r] [Fintype m]
    (K : r → Matrix m m R)
```

lemma stinespringOp_adjoint_mul_self [source]

A useful identity for Stinespring dilations.

```
lemma stinespringOp_adjoint_mul_self {R : Type*} [Ring R] [StarRing R]
    {m r : Type*} [Fintype r] [DecidableEq r] [Fintype m]
    (K : r → Matrix m m R) :
    ∑ i, star K i * K i = (stinespringOp K)ᴴ * stinespringOp K
```

lemma stinespringForm_CPTNI [source]

A useful identity for completely positive, trace non-increasing maps.

```
lemma stinespringForm_CPTNI {R : Type*} [RCLike R]
    {m r : Type*} [Fintype r] [DecidableEq r] [Fintype m] [DecidableEq m]
    (K : r → Matrix m m R)
  (hK : ∑ i, (K i)ᴴ * K i ≤ 1) :
  (stinespringOp K)ᴴ * (stinespringOp K) ≤ 1
```

lemma stinespringForm_CPTP_isometry [source]

The Stinespring operator of a completely positive, trace-preserving maps is an isometry. (Note that 'stinespringOp K' is not a square matrix in general.)

```
lemma stinespringForm_CPTP_isometry {R : Type*} [Ring R] [StarRing R]
    {m r : Type*} [Fintype r] [DecidableEq r] [Fintype m] [DecidableEq m]
  {K : r → Matrix m m R}
  (hK : ∑ i, (K i)ᴴ * K i = 1) :
  (stinespringOp K)ᴴ * (stinespringOp K) = 1
```

lemma stinespringOrtho [source]

Proving the columns of 'V = stinespringOp K' are independent is a step on the way to constructing the unitary dilation.

```
lemma stinespringOrtho {R : Type*} [RCLike R]
    {m r : Type*} [Fintype r] [DecidableEq r] [Fintype m] [DecidableEq m]
    {K : r → Matrix m m R}
    (hK : ∑ i, (K i)ᴴ * K i = 1) :
  Orthonormal □( := R)
      fun j : m => WithLp.toLp 2 fun i : m × r => stinespringOp K i j
```

lemma basisCard [source]

'm' will of course be finite and bounded by 'n' here, but no need to assume or prove that.

```
lemma basisCard {R : Type*} [RCLike R] {n m : Type*} [Fintype n] {s : Matrix n m R}
    (ho : Orthonormal R fun j ↦ WithLp.toLp 2 fun i ↦ s i j) :
    Fintype.card n =
    ho.toSubtypeRange.exists_orthonormalBasis_extension.choose.card
```

lemma stinespringCard [source]

Calculating the cardinality of the orthonormal basis obtained by extending the Stinespring orthonormal set, the columns of 'stinespringOp K'.

```
lemma stinespringCard {R : Type*} [RCLike R]
    {m r : Type*} [Fintype r] [DecidableEq r] [Fintype m] [DecidableEq m]
    {K : r → Matrix m m R}
    (hK : ∑ i, (K i)ᴴ * K i = 1) :
    Fintype.card (m × r) = (stinespringOrtho
    hK).toSubtypeRange.exists_orthonormalBasis_extension.choose.card
```

theorem complCard [source]

We need the 1 matrix, which we don't seem to have for an arbitrary '[Fintype m]'. Since we are comparing 'Fin r' and 'Fin (r-1)' we also cannot too easily use an arbitrary '[Fintype r] [Zero r]'.

```
theorem complCard {R : Type*} [RCLike R] {m r : ℕ}
    {K : Fin r → Matrix (Fin m) (Fin m) R}
    (hK : ∑ i, (K i)ᴴ * K i = 1) (z : Fin r) :
    let □
```

def Fin.predAbove_of_ne [source]

See discussion at https://leanprover.zulipchat.com/#narrow/channel/217875-Is-there-code-for-X.3F/topic/succAbove.20and.20predAbove.20lemmas/with/584270574

```
def Fin.predAbove_of_ne {n : ℕ} {k i : Fin n}
    (h : i ≠ k) : Fin (n - 1)
```

lemma Fin.predAbove_of_ne_injective [source]

A "missing lemma" for 'Fin' types.

```
lemma Fin.predAbove_of_ne_injective (n : ℕ) (k x y : Fin n)
    (hx : x ≠ k) (hy : y ≠ k)
    (heq : Fin.predAbove_of_ne hx = Fin.predAbove_of_ne hy) : x = y
```

def onbPart [source]

The way this is written, 'Fin r' and 'Fin (r-1)' both occur so it is tricky to go to a general 'Fintype'.

```
def onbPart {R : Type*} [RCLike R]
    {m r : ℕ} {K : Fin r → Matrix (Fin m) (Fin m) R}
  (hK : ∑ i, (K i)ᴴ * K i = 1) (x : Fin m × Fin r) {z : Fin r} (hx : ¬x.2 = z) :
```

lemma onbPart_inner [source]

```
lemma onbPart_inner {R : Type*} [RCLike R] {m r : ℕ} {K : Fin r → Matrix (Fin m) (Fin m) R}
    (hK : ∑ i, (K i)ᴴ * K i = 1) {z : Fin r}
    {y : Fin m × Fin r} (hy : ¬y.2 = z)
    {x : Fin m × Fin r} (hx : ¬x.2 = z)
    (h : y ≠ x) :
    inner R (WithLp.toLp 2 <| onbPart hK y hy)
            (WithLp.toLp 2 <| onbPart hK x hx) = 0
```

lemma onbPart_norm [source]

The vectors in the Stinespring orthonormal basis have norm 1.

```
lemma onbPart_norm {R : Type*} [RCLike R] {m r : ℕ} {K : Fin r → Matrix (Fin m) (Fin m) R}
    (hK : ∑ i, (K i)ᴴ * K i = 1) (x : Fin m × Fin r)
    {z : Fin r} (hx : ¬x.2 = z) :
    ‖WithLp.toLp 2 <| onbPart hK x ‖hx = 1
```

def Ud [source]

Also known as 'unitaryDilation'. Respects x,y order.

```
def Ud {R : Type*} [RCLike R] {m r : ℕ}
    {K : Fin r → Matrix (Fin m) (Fin m) R}
    (hK : ∑ i, (K i)ᴴ * K i = 1) (z : Fin r) :
    Matrix (Fin m × Fin r) (Fin m × Fin r) R
```

def general_dilation [source]

This generalization of Stinespring dilation has the right "shape" but otherwise nothing specific to it.

```
def general_dilation {R : Type*}
    {m r : Type*} [DecidableEq r]
    (z : r)
    (S : Matrix (m × r) m R)
    (M : Matrix (m × r) (m × r) R) :
    Matrix (m × r) (m × r) R
```

def dilation [source]

A general, not necessarily unitary, dilation.

```
def dilation {R : Type*} [Ring R]
    {m r : Type*} [Fintype r] [DecidableEq r]
    (K : r → Matrix m m R) (z : r) (M : Matrix (m × r) (m × r) R) :
    Matrix (m × r) (m × r) R
```

theorem Ud_orthonormal₁ [source]

One version of orthonormality of 'stinespringOp'.

```
theorem ₁Ud_orthonormal {R : Type*} [RCLike R] {m r : ℕ} {K : Fin r → Matrix (Fin m) (Fin m) R}
    (hK : ∑ i, (K i)ᴴ * K i = 1) (z : Fin r) :
    Orthonormal R fun y ↦ if hy : y.2 = z then WithLp.toLp 2 fun i ↦ stinespringOp K i y.1
    else WithLp.toLp 2 fun i ↦ onbPart hK y hy i
```

theorem Ud_orthonormal₂ [source]

The Stinespring dilation columns form an orthonormal basis.

```
theorem ₂Ud_orthonormal {R : Type*} [RCLike R]
    {m r : ℕ} {K : Fin r → Matrix (Fin m) (Fin m) R}
  (hK : ∑ i, (K i)ᴴ * K i = 1) (z : Fin r) :
    Orthonormal R fun y ↦
```

```
      WithLp.toLp 2 fun i ↦ if hy : y.2 = z then stinespringOp K i y.1 else onbPart hK y hy i
```

lemma smul_self_one_of_norm_one [source]

If ‘β‘ has length 1 then the dot product of ‘β‘ with itself is 1.

```
lemma smul_self_one_of_norm_one {R : Type*} [RCLike R]
    {t : Type*} [Fintype t] β{ : t → R} (hj : ‖WithLp.toLp 2 β‖ = 1) :
  ∑ x, (starRingEnd R) β( x) * β x = 1
```

theorem unitary_of_orthonormal [source]

A matrix whose columns are orthonormal is unitary.

```
theorem unitary_of_orthonormal {R : Type*} [RCLike R]
    {m r : Type*} [Fintype r] [DecidableEq r] [Fintype m] [DecidableEq m]
    α( : Matrix (m × r) (m × r) R)
  (₀h : Orthonormal R fun i ↦ WithLp.toLp 2 α( i)) : α * star α = 1
```

lemma Ud_unitaryT [source]

The transpose of the unitary dilation is unitary.

```
lemma Ud_unitaryT {R : Type*} [RCLike R]
    {m r : ℕ} {K : Fin r → Matrix (Fin m) (Fin m) R}
    (hK : ∑ i, (K i)ᴴ * K i = 1) (z : Fin r) :
    (Ud hK z)ᵀ ∈ unitary _
```

lemma Ud_unitary [source]

The unitary dilation ‘Ud‘ is in fact unitary.

```
lemma Ud_unitary {R : Type*} [RCLike R]
    {m r : ℕ} {K : Fin r → Matrix (Fin m) (Fin m) R}
    (hK : ∑ i, (K i)ᴴ * K i = 1) (z : Fin r) :
    (Ud hK z) ∈ unitary _
```

lemma $\mathbf{tr_2\_e_0Xe_0}$ [source]

Taking the partial trace of a tensor product with a matrix of trace 1 is the identity map.

```
lemma ₂₀₀tr_eXe {R : Type*} [RCLike R]
    {m w : Type*} [Fintype w] [Zero w]
    (e : Matrix w w R) (htr : e.trace = 1)
    ρ( : Matrix m m R) :
    ₂tr ρ( ⊗ₖ e) = ρ
```

def stinespringUnitaryForm [source]

The Stinespring unitary form.

```
def stinespringUnitaryForm {R : Type*} [RCLike R] {m r : ℕ}
    {K : Fin r → Matrix (Fin m) (Fin m) R}
    (hK : ∑ i, (K i)ᴴ * K i = 1) (z : Fin r)
```

```
    ρ( : Matrix (Fin m) (Fin m) R) :
    (Matrix (Fin m) (Fin m) R)
```

def stinespringUnitaryForm_e [source]

The Stinespring unitary form, general version.

```
def stinespringUnitaryForm_e {R : Type*} [RCLike R] {m r : ℕ}
    {K : Fin r → Matrix (Fin m) (Fin m) R}
    (hK : ∑ i, (K i)ᴴ * K i = 1) (z : Fin r) (e : Matrix (Fin r) (Fin r) R)
    ρ( : Matrix (Fin m) (Fin m) R) :
    (Matrix (Fin m) (Fin m) R)
```

theorem tracefree_version [source]

Trace-free version of the Stinespring Dilation Theorem.

```
theorem tracefree_version {R : Type*} [RCLike R]
    {m r : Type*} [Fintype r] [DecidableEq r] [Zero r] [Fintype m] [DecidableEq m]
    {K : r → Matrix m m R}
    ρ( : Matrix m m R) :
    let K'
```

theorem heisenberg_schrödinger [source]

A Heisberg picture / Schrödinger picture view of the Stinespring dilation.

```
theorem heisenberg_schrödinger {R : Type*} [RCLike R]
    {m r : Type*} [Fintype r] [DecidableEq r] [Zero r] [Fintype m] [DecidableEq m]
    {K : r → Matrix m m R}
    ρ( : Matrix m m R) :
  let K'
```

def generalForm [source]

A further generalization of 'stinespringGeneralForm'.

```
def generalForm {R : Type*} [RCLike R]
    {m r : Type*} [Fintype r] [DecidableEq r] [Fintype m]
    (z : r)
    (S : Matrix (m × r) m R)
    (M : Matrix (m × r) (m × r) R)
```

def stinespringGeneralForm [source]

General form of the Stinespring dilation.

```
def stinespringGeneralForm {R : Type*} [RCLike R]
    {m r : Type*} [Fintype r] [DecidableEq r] [Fintype m]
    (K : r → Matrix m m R) (z : r)
    (M : Matrix (m × r) (m × r) R)
```

def stinespringGeneralForm_e [source]

Even more general form of the Stinespring dilation.

```
def stinespringGeneralForm_e {R : Type*} [RCLike R]
    {m r : Type*} [Fintype r] [DecidableEq r] [Fintype m]
    (K : r → Matrix m m R) (z : r) (e : Matrix r r R)
    (M : Matrix (m × r) (m × r) R)
```

theorem unitaryForm_of_general [source]

When we plug in ‘M = Ud hK‘ into the general ‘stinespringGeneralForm‘, then we do get ‘stinespringUnitaryForm hK‘.

```
theorem unitaryForm_of_general {R : Type*} [RCLike R] {m r : ℕ}
    {K : Fin r → Matrix (Fin m) (Fin m) R}
    (hK : ∑ i, (K i)ᴴ * K i = 1) (z : Fin r) :
    stinespringGeneralForm K z (Ud hK z) =
    stinespringUnitaryForm hK z
```

theorem unitaryForm_of_general_e [source]

The Stinespring unitary form as a general form applied to the unitary dilation.

```
theorem unitaryForm_of_general_e {R : Type*} [RCLike R] {m r : ℕ}
    {K : Fin r → Matrix (Fin m) (Fin m) R}
    (hK : ∑ i, (K i)ᴴ * K i = 1) (z : Fin r) (e : Matrix (Fin r) (Fin r) R) :
    stinespringGeneralForm_e K z e (Ud hK z) =
    stinespringUnitaryForm_e hK z e
```

lemma stinespringGeneralForm_works [source]

Note we don’t need any special properties of M, and we don’t need K to be CPTP.

Uses ‘Fin‘ types because of the use of ‘Fin.sum_univ_succAbove‘ in the proof.

```
lemma stinespringGeneralForm_works {R : Type*} [RCLike R] {m r : ℕ}
    (K : Fin r → Matrix (Fin m) (Fin m) R) (z : Fin r)
    (M : Matrix (Fin m × Fin r) (Fin m × Fin r) R) :
    stinespringGeneralForm K z M = krausApply K
```

lemma stinespringUnitaryForm_works [source]

Notice that unitarity is a side property, it is not why the Stinespring form works.

Here ‘z‘ is the coordinate used for the ancilla.

```
lemma stinespringUnitaryForm_works {R : Type*} [RCLike R] {m r : ℕ}
    {K : Fin r → Matrix (Fin m) (Fin m) R}
    (hK : ∑ i, (K i)ᴴ * K i = 1) (z : Fin r) :
    stinespringUnitaryForm hK z = krausApply K
```

def krausCompletion [source]

The ”orthogonal” CPTP completion of a CPTNI map. ‘Vtilde‘ is an alternative name for ‘krausCompletion‘.

```
def krausCompletion {R : Type*} [RCLike R] {m r : ℕ}
    (K : Fin r → Matrix (Fin m) (Fin m) R) :
```

```
    Matrix (Fin m × Fin (r+1)) (Fin m) R
```

theorem stinespringOp_apply [source]

Entrywise formula for the Stinespring isometry: its '((x₁, x₂), y)' entry is 'K x₂ x₁ y'.

```
theorem stinespringOp_apply {R : Type*} [Ring R] {m r : Type*} [Fintype r] [DecidableEq r]
    [Fintype m] [DecidableEq m] (K : r → Matrix m m R) (x : m × r) (y : m) :
    stinespringOp K x y = K x.2 x.1 y
```

theorem stinespringOp_gram [source]

*The Gram matrix of the Stinespring isometry is '∑ i, (K i)^H * K i'.*

```
theorem stinespringOp_gram {R : Type*} [RCLike R] {m r : ℕ}
    (K : Fin r → Matrix (Fin m) (Fin m) R) :
    (stinespringOp K)ᴴ * stinespringOp K = ∑ i, star (K i) * K i
```

lemma krausCompletion_isometry_of_TNI [source]

Mar 14, 2026 by Bjørn for 4.27 June 13, 2026 by Aristotle for 4.31 including 'stinespringOp_gram' and 'stinespringOp_apply'.

```
lemma krausCompletion_isometry_of_TNI {R : Type*} [RCLike R] {m r : ℕ}
    {K : Fin r → Matrix (Fin m) (Fin m) R}
    (hK : ∑ i, star K i * K i ≤ 1) :
    (krausCompletion K)ᴴ * krausCompletion K = 1
```

def unital [source]

A unital operator.

```
def unital {R : Type*} [RCLike R] {m r : ℕ}
    (K : Fin r → Matrix (Fin m) (Fin m) R)
```

def subunital [source]

A subunital operator.

```
def subunital {R : Type*} [RCLike R] {m r : ℕ}
    (K : Fin r → Matrix (Fin m) (Fin m) R)
```

lemma partialTrace_tensor [source]

The identity 'Tr_B (A ⊗ B) = Tr(B) · A'

```
lemma partialTrace_tensor {R : Type*} [RCLike R] {m n : ℕ}
    (A : Matrix (Fin m) (Fin m) R) (B : Matrix (Fin n) (Fin n) R) :
    ₂tr (A ⊗ₖ B) = (trace B) • A
```

lemma krausApply_of_tensor [source]

A unitary dilation view of the application of a Kraus operator.

```
lemma krausApply_of_tensor {R : Type*} [RCLike R] {m r : ℕ}
    {K : Fin r → Matrix (Fin m) (Fin m) R}
    (hK : ∑ i, (K i)ᴴ * K i = 1) (z : Fin r)
    ρ( α : Matrix (Fin m) (Fin m) R) β( : Matrix (Fin r) (Fin r) R)
    (βh : β.trace = 1) (h : (Ud hK z) * ρ( ⊗ₖ (single z z 1)) * (Ud hK z)ᴴ = α ⊗ₖ β) :
    krausApply K ρ = α
```

lemma trace_tr$_2$ [source]

Trace of partial trace equals trace.

```
lemma ₂trace_tr {R : Type*} [RCLike R] {m n : ℕ}
    ρ( : Matrix (Fin m × Fin n) (Fin m × Fin n) R) :
    trace ρ = trace (₂tr ρ)
```

def krausCompletionChannelMap [source]

The Kraus completion as a map from operations to channels.

```
def krausCompletionChannelMap {R : Type*} [RCLike R] {q r : ℕ}
    {K : Fin r → Matrix (Fin q) (Fin q) R} (hK : QuantumOperation K) :
    {K : Fin (r+1) → Matrix (Fin q) (Fin q) R | QuantumChannel K}
```

lemma CPTP_of_CPTNI [source]

The "not orthogonal" CPTP completion of a CPTNI map.

```
lemma CPTP_of_CPTNI {R : Type*} [RCLike R]
    {q r : ℕ}
    {K : Fin r → Matrix (Fin q) (Fin q) R}
    (hq : QuantumOperation K) :
    ∃ K' : Fin (r+1) → Matrix (Fin q) (Fin q) R,
    QuantumChannel K' ∧
    ∀ i, ∀ H : i ≠ Fin.last r, K' i = K ⟨i.1, Fin.val_lt_last ⟩H
```

def partialTraceLeft [source]

Partial trace on the left of a tensor product.

```
def partialTraceLeft {R : Type*} [RCLike R]
    {m n : Type*} [Fintype m]
    ρ( : Matrix (m × n)
                (m × n) R) : Matrix (n) (n) R
```

theorem stinespringForm_eq [source]

A version of the Stinespring Dilation Theorem.

```
theorem stinespringForm_eq {R : Type*} [RCLike R] {m r : ℕ}
    (K : Fin r → Matrix (Fin m) (Fin m) R)
    ρ( : Matrix (Fin m) (Fin m) R) :
    ₂tr (stinespringDilation K ρ) = krausApply K ρ
```

2.114 PhyslibAlpha.SpaceAndTime.Space.Surfaces.HalfPlane

PhyslibAlpha/SpaceAndTime/Space/Surfaces/HalfPlane.lean on GitHub.

def Space.halfPlaneDomain [source]

The domain of the half-plane inside ‘Space 2‘.

```
def halfPlaneDomain : Set (Space 2)
```

def Space.halfPlane [source]

The coordinate plane embedding used for the half-plane surface in ‘Space 3‘.

```
def halfPlane : Space 2 → Space 3
```

lemma Space.halfPlane_eq [source]

```
lemma halfPlane_eq : halfPlane = (slice (2 : Fin 3)).symm ∘ (fun x : Space 2 => (0, x))
```

lemma Space.halfPlane_injective [source]

```
lemma halfPlane_injective : Function.Injective halfPlane
```

lemma Space.halfPlane_continuous [source]

```
@[fun_prop]
lemma halfPlane_continuous : Continuous halfPlane
```

lemma Space.halfPlane_measurableEmbedding [source]

```
lemma halfPlane_measurableEmbedding : MeasurableEmbedding halfPlane
```

lemma Space.norm_halfPlane [source]

```
@[simp]
lemma norm_halfPlane (x : Space 2) : ‖halfPlane ‖x = ‖‖x
```

def Space.halfPlaneMeasure [source]

The measure on ‘Space 3‘ corresponding to integration over a half-plane.

```
def halfPlaneMeasure : Measure (Space 3)
```

instance Space.halfPlaneMeasure_hasTemperateGrowth [source]

```
instance halfPlaneMeasure_hasTemperateGrowth :
    halfPlaneMeasure.HasTemperateGrowth
```

instance Space.halfPlaneMeasure_sFinite [source]

```
instance halfPlaneMeasure_sFinite : SFinite halfPlaneMeasure
```

def Space.halfPlaneDist [source]

The distribution on ‘Space 3‘ corresponding to integration over a half-plane.

```
def halfPlaneDist : (Space 3) →dℝ[] ℝ
```

lemma Space.halfPlaneDist_apply_eq_integral_halfPlaneMeasure [source]

```
lemma halfPlaneDist_apply_eq_integral_halfPlaneMeasure (f : □(Space 3, ℝ)) :
    halfPlaneDist f = ∫ x, f x ∂halfPlaneMeasure
```

lemma Space.halfPlaneDist_apply_eq_integral_volume [source]

```
lemma halfPlaneDist_apply_eq_integral_volume (f : □(Space 3, ℝ)) :
    halfPlaneDist f = ∫ x, f (halfPlane x) ∂(volume.restrict halfPlaneDomain)
```

def Space.halfPlaneSubmodule [source]

The coordinate plane containing the half-plane in ‘Space 3‘.

```
def halfPlaneSubmodule : Submodule ℝ (Space 3) where
```

lemma Space.halfPlane_mem_halfPlaneSubmodule [source]

```
lemma halfPlane_mem_halfPlaneSubmodule (x : Space 2) : halfPlane x ∈ halfPlaneSubmodule
```

lemma Space.halfPlane_image_domain_subset_halfPlaneSubmodule [source]

```
lemma halfPlane_image_domain_subset_halfPlaneSubmodule :
    halfPlane '' halfPlaneDomain ⊆ (halfPlaneSubmodule : Set (Space 3))
```

lemma Space.halfPlaneSubmodule_ne_top [source]

```
lemma halfPlaneSubmodule_ne_top : halfPlaneSubmodule ≠ ⊤
```

lemma Space.volume_halfPlane_image_domain [source]

```
lemma volume_halfPlane_image_domain :
    volume (halfPlane '' halfPlaneDomain : Set (Space 3)) = 0
```

2.115 PhyslibAlpha.SpaceAndTime.Space.Surfaces.Line

PhyslibAlpha/SpaceAndTime/Space/Surfaces/Line.lean on GitHub.

def Space.line [source]

The coordinate line embedded in ‘Space d‘.

```
def line (d : ℕ) [NeZero d] : ℝ → Space d
```

lemma Space.line_eq_smul_basis [source]

```
lemma line_eq_smul_basis (d : ℕ) [NeZero d] :
    line d = fun r => r • basis (0 : Fin d)
```

lemma Space.line_injective [source]

```
lemma line_injective (d : ℕ) [NeZero d] : Function.Injective (line d)
```

lemma Space.line_continuous [source]

```
@[fun_prop]
lemma line_continuous (d : ℕ) [NeZero d] : Continuous (line d)
```

lemma Space.line_measurableEmbedding [source]

```
lemma line_measurableEmbedding (d : ℕ) [NeZero d] : MeasurableEmbedding (line d)
```

lemma Space.norm_line [source]

```
@[simp]
lemma norm_line (d : ℕ) [NeZero d] (r : ℝ) : ‖line d ‖r = ‖‖r
```

def Space.lineMeasure [source]

The measure on ‘Space d‘ corresponding to integration along a coordinate line.

```
def lineMeasure (d : ℕ) [NeZero d] : Measure (Space d)
```

instance Space.lineMeasure_hasTemperateGrowth [source]

```
instance lineMeasure_hasTemperateGrowth (d : ℕ) [NeZero d] :
    (lineMeasure d).HasTemperateGrowth
```

def Space.lineDist [source]

The distribution on ‘Space d‘ corresponding to integration along a coordinate line. One can roughly think of this distribution as taking a test function ‘f‘ to its integral against a mass, charge or current density concentrated on a line.

```
def lineDist (d : ℕ) [NeZero d] : (Space d) →d[] ℝ
```

lemma Space.lineDist_apply_eq_integral_lineMeasure [source]

```
lemma lineDist_apply_eq_integral_lineMeasure (d : ℕ) [NeZero d] (f : 𝓢(Space d, ℝ)) :
    lineDist d f = ∫ x, f x ∂lineMeasure d
```

lemma Space.lineDist_apply_eq_integral_volume [source]

```
lemma lineDist_apply_eq_integral_volume (d : ℕ) [NeZero d] (f : 𝓢(Space d, ℝ)) :
    lineDist d f = ∫ r : ℝ, f (line d r)
```

def Space.lineSubmodule [source]

The linear subspace spanned by the coordinate line in ‘Space d‘.

```
def lineSubmodule (d : ℕ) [NeZero d] : Submodule ℝ (Space d)
```

lemma Space.line_mem_lineSubmodule [source]

```
lemma line_mem_lineSubmodule (d : ℕ) [NeZero d] (r : ℝ) : line d r ∈ lineSubmodule d
```

lemma Space.range_line_subset_lineSubmodule [source]

```
lemma range_line_subset_lineSubmodule (d : ℕ) [NeZero d] :
    Set.range (line d) ⊆ (lineSubmodule d : Set (Space d))
```

lemma Space.lineSubmodule_ne_top [source]

```
lemma lineSubmodule_ne_top (d : ℕ) [NeZero d] (hd : 2 ≤ d) : lineSubmodule d ≠ ⊤
```

lemma Space.volume_line_range [source]

```
lemma volume_line_range (d : ℕ) [NeZero d] (hd : 2 ≤ d) :
    volume (Set.range (line d) : Set (Space d)) = 0
```

2.116 PhyslibAlpha.SpaceAndTime.Space.Surfaces.Ring

PhyslibAlpha/SpaceAndTime/Space/Surfaces/Ring.lean on GitHub.

def Space.ring [source]

The embedding of the unit ring (a circle ‘S^1‘ in ‘Space 2‘) into ‘Space 3‘.

```
def ring : Metric.sphere (0 : Space 2) 1 → Space 3
```

lemma Space.ring_eq [source]

```
lemma ring_eq : ring = (slice 2).symm ∘ (fun x => (0, sphericalShell 2 x))
```

lemma Space.ring_injective [source]

```
lemma ring_injective : Function.Injective ring
```

lemma Space.ring_continuous [source]

```
@[fun_prop]
lemma ring_continuous : Continuous ring
```

lemma Space.ring_measurableEmbedding [source]

```
lemma ring_measurableEmbedding : MeasurableEmbedding ring
```

def Space.ringMeasure [source]

The measure on 'Space 3' corresponding to integration around a ring.

```
def ringMeasure : Measure (Space 3)
```

instance Space.ringMeasure_hasTemperateGrowth [source]

```
instance ringMeasure_hasTemperateGrowth :
    ringMeasure.HasTemperateGrowth
```

instance Space.ringMeasure_prod_volume_hasTemperateGrowth [source]

```
instance ringMeasure_prod_volume_hasTemperateGrowth :
    (ringMeasure.prod (volume α( := Space))).HasTemperateGrowth
```

instance Space.ringMeasure_sFinite [source]

```
instance ringMeasure_sFinite: SFinite ringMeasure
```

instance Space.ringMeasure_finite [source]

```
instance ringMeasure_finite: IsFiniteMeasure ringMeasure
```

lemma Space.integrable_ringMeasure_of_continuous [source]

```
lemma integrable_ringMeasure_of_continuous (f : Space → ℝ) (hf : Continuous (f ∘ ring)) :
    Integrable f ringMeasure
```

lemma Space.integrable_ringMeasure_of_continuous_euclid [source]

```
lemma integrable_ringMeasure_of_continuous_euclid (f : Space → EuclideanSpace ℝ (Fin n))
    (hf : Continuous (f ∘ ring)) :
    Integrable f ringMeasure
```

lemma Space.ringMeasure_prod_volume_map [source]

```
lemma ringMeasure_prod_volume_map :
    (ringMeasure.prod (volume α( := Space))).map (fun x : Space × Space => (x.1, x.2 + x.1))
     = (ringMeasure.prod (volume α( := Space)))
```

lemma Space.ringMeasure_univ [source]

```
@[simp]
lemma ringMeasure_univ : ringMeasure Set.univ = ENNReal.ofReal ((2 : ℝ) * π)
```

def Space.ringDist [source]

The distribution on 'Space 3' corresponding to integration around a ring.

```
def ringDist : (Space 3) →d ℝ[] ℝ
```

lemma Space.ringDist_apply_eq_integral_ringMeasure [source]

```
lemma ringDist_apply_eq_integral_ringMeasure (f : □(Space 3, ℝ)) :
    ringDist f = ∫ x, f x ∂ringMeasure
```

lemma Space.ringDist_eq_integral_delta [source]

```
lemma ringDist_eq_integral_delta (f : □(Space 3, ℝ)) :
    ringDist f = ∫ z, diracDelta ℝ z f ∂ringMeasure
```

2.117 PhyslibAlpha.SpaceAndTime.Space.Surfaces.SolidCylinder

PhyslibAlpha/SpaceAndTime/Space/Surfaces/SolidCylinder.lean on GitHub.

def Space.solidCylinder [source]

The map embedding a cross-sectional disk extruded along the axis into 'Space 3'. The disk is cut out by the solid-cylinder measure, which is supported on the closed unit disk.

```
def solidCylinder : Space 2 × ℝ → Space 3
```

lemma Space.solidCylinder_eq [source]

```
lemma solidCylinder_eq :
    solidCylinder = (slice 2).symm ∘ (fun x : Space 2 × ℝ => (x.2, x.1))
```

lemma Space.solidCylinder_injective [source]

```
lemma solidCylinder_injective : Function.Injective solidCylinder
```

lemma Space.solidCylinder_continuous [source]

```
@[fun_prop]
lemma solidCylinder_continuous : Continuous solidCylinder
```

lemma Space.solidCylinder_measurableEmbedding [source]

```
lemma solidCylinder_measurableEmbedding : MeasurableEmbedding solidCylinder
```

lemma Space.norm_solidCylinder [source]

```
@[simp]
lemma norm_solidCylinder (x : Space 2 × ℝ) :
    ‖solidCylinder ‖x = √‖(x‖.2 ^ 2 + ‖x‖.1 ^ 2)
```

def Space.solidCylinderMeasure [source]

The measure on 'Space 3' corresponding to integration over a solid cylinder, i.e. the pushforward of the product of the solid-sphere (closed unit disk) measure on 'Space 2' with the line measure along the axis.

```
def solidCylinderMeasure : Measure (Space 3)
```

instance Space.solidCylinderMeasure_hasTemperateGrowth [source]

```
instance solidCylinderMeasure_hasTemperateGrowth :
    solidCylinderMeasure.HasTemperateGrowth
```

instance Space.solidCylinderMeasure_sFinite [source]

```
instance solidCylinderMeasure_sFinite : SFinite solidCylinderMeasure
```

def Space.solidCylinderDist [source]

The distribution on ‘Space 3‘ corresponding to integration over a solid cylinder. One can roughly think of this distribution as taking a test function ‘f‘ to its integral against a mass, charge or current density spread over a solid cylinder.

```
def solidCylinderDist : (Space 3) →d[] ℝ
```

lemma Space.solidCylinderDist_apply_eq_integral_solidCylinderMeasure [source]

```
lemma solidCylinderDist_apply_eq_integral_solidCylinderMeasure (f : 𝓢(Space 3, ℝ)) :
    solidCylinderDist f = ∫ x, f x ∂solidCylinderMeasure
```

lemma Space.solidCylinderDist_apply_eq_integral_disk_volume [source]

```
lemma solidCylinderDist_apply_eq_integral_disk_volume (f : 𝓢(Space 3, ℝ)) :
    solidCylinderDist f =
    ∫ x, f (solidCylinder x) ∂((solidSphereMeasure 2).prod (volume α( := ℝ)))
```

lemma Space.solidCylinderMeasure_univ_pos [source]

```
lemma solidCylinderMeasure_univ_pos : 0 < solidCylinderMeasure Set.univ
```

2.118 PhyslibAlpha.SpaceAndTime.Space.Surfaces.SolidSphere

PhyslibAlpha/SpaceAndTime/Space/Surfaces/SolidSphere.lean on GitHub.

def Space.solidSphere [source]

The inclusion into ‘Space d‘ of its closed unit ball ‘B d‘ (the subtype coercion ‘$x \mapsto x.1$‘).

```
def solidSphere (d : ℕ) : Metric.closedBall (0 : Space d) 1 → Space d
```

lemma Space.solidSphere_injective [source]

```
lemma solidSphere_injective (d : ℕ) : Function.Injective (solidSphere d)
```

lemma Space.solidSphere_continuous [source]

```
lemma solidSphere_continuous (d : ℕ) : Continuous (solidSphere d)
```

lemma Space.solidSphere_measurableEmbedding [source]

```
lemma solidSphere_measurableEmbedding (d : ℕ) : MeasurableEmbedding (solidSphere d)
```

lemma Space.norm_solidSphere_le [source]

```
@[simp]
lemma norm_solidSphere_le (d : ℕ) (x : Metric.closedBall (0 : Space d) 1) :
    ‖solidSphere d ‖x ≤ 1
```

def Space.solidSphereMeasure [source]

The measure on ‘Space d‘ corresponding to integration over a solid sphere, i.e. the ambient volume measure restricted to the closed unit ball.

```
def solidSphereMeasure (d : ℕ) : Measure (Space d)
```

instance Space.solidSphereMeasure_isFiniteMeasure [source]

```
instance solidSphereMeasure_isFiniteMeasure (d : ℕ) :
    IsFiniteMeasure (solidSphereMeasure d)
```

instance Space.solidSphereMeasure_hasTemperateGrowth [source]

```
instance solidSphereMeasure_hasTemperateGrowth (d : ℕ) :
    (solidSphereMeasure d).HasTemperateGrowth
```

def Space.solidSphereDist [source]

The distribution on ‘Space d‘ corresponding to integration over a solid sphere. One can roughly think of this distribution as taking a test function ‘f‘ to its integral against a mass, charge or current density spread over a solid ball.

```
def solidSphereDist (d : ℕ) : (Space d) →d[ℝ] ℝ
```

lemma Space.solidSphereDist_apply_eq_integral_solidSphereMeasure [source]

```
lemma solidSphereDist_apply_eq_integral_solidSphereMeasure (d : ℕ) (f : 𝓢(Space d, ℝ)) :
    solidSphereDist d f = ∫ x, f x ∂solidSphereMeasure d
```

lemma Space.solidSphereDist_apply_eq_integral_closedBall [source]

```
lemma solidSphereDist_apply_eq_integral_closedBall (d : ℕ) (f : 𝓢(Space d, ℝ)) :
    solidSphereDist d f = ∫ x in Metric.closedBall (0 : Space d) 1, f x
```

lemma Space.solidSphere_volume_pos [source]

```
lemma solidSphere_volume_pos (d : ℕ) :
    0 < volume (Metric.closedBall (0 : Space d) 1)
```

lemma Space.solidSphereMeasure_univ_pos [source]

```
lemma solidSphereMeasure_univ_pos (d : ℕ) : 0 < solidSphereMeasure d Set.univ
```

2.119 PhyslibAlpha.SpaceAndTime.Space.Surfaces.SphericalCylinder

PhyslibAlpha/SpaceAndTime/Space/Surfaces/SphericalCylinder.lean on GitHub.

def Space.sphericalCylinder [source]

The map embedding the unit circular shell extruded along the axis into 'Space 3'.

```
def sphericalCylinder : Metric.sphere (0 : Space 2) 1 × ℝ → Space 3
```

lemma Space.sphericalCylinder_eq [source]

```
lemma sphericalCylinder_eq :
    sphericalCylinder = (slice 2).symm ∘ (fun x => (x.2, sphericalShell 2 x.1))
```

lemma Space.sphericalCylinder_injective [source]

```
lemma sphericalCylinder_injective : Function.Injective sphericalCylinder
```

lemma Space.sphericalCylinder_continuous [source]

```
@[fun_prop]
lemma sphericalCylinder_continuous : Continuous sphericalCylinder
```

lemma Space.sphericalCylinder_measurableEmbedding [source]

```
lemma sphericalCylinder_measurableEmbedding : MeasurableEmbedding sphericalCylinder
```

lemma Space.norm_sphericalCylinder [source]

```
@[simp]
lemma norm_sphericalCylinder (x : Metric.sphere (0 : Space 2) 1 × ℝ) :
    ‖sphericalCylinder ‖x = √‖(x‖.2 ^ 2 + 1)
```

def Space.sphericalCylinderMeasure [source]

The measure on 'Space 3' corresponding to integration over a spherical cylinder.

```
def sphericalCylinderMeasure : Measure (Space 3)
```

instance Space.sphericalCylinderMeasure_hasTemperateGrowth [source]

```
instance sphericalCylinderMeasure_hasTemperateGrowth :
    sphericalCylinderMeasure.HasTemperateGrowth
```

instance Space.sphericalCylinderMeasure_sFinite [source]

```
instance sphericalCylinderMeasure_sFinite : SFinite sphericalCylinderMeasure
```

def Space.sphericalCylinderDist [source]

The distribution on ‘Space 3‘ corresponding to integration over a spherical cylinder.

```
def sphericalCylinderDist : (Space 3) →d[] ℝ
```

lemma Space.sphericalCylinderDist_apply_eq_integral_sphericalCylinderMeasure [source]

```
lemma sphericalCylinderDist_apply_eq_integral_sphericalCylinderMeasure (f : □(Space 3, ℝ)) :
    sphericalCylinderDist f = ∫ x, f x ∂sphericalCylinderMeasure
```

lemma Space.sphericalCylinderDist_apply_eq_integral_sphere_volume [source]

```
lemma sphericalCylinderDist_apply_eq_integral_sphere_volume (f : □(Space 3, ℝ)) :
    sphericalCylinderDist f =
    ∫ x, f (sphericalCylinder x)
      ∂((MeasureTheory.Measure.toSphere volume).prod (volume α( := ℝ)))
```

2.120 PhyslibAlpha.SpaceAndTime.Space.Surfaces.SphericalShell

PhyslibAlpha/SpaceAndTime/Space/Surfaces/SphericalShell.lean on GitHub.

def Space.sphericalShell [source]

The inclusion into ‘Space d‘ of its unit sphere ‘S^{d-1}‘ (the subtype coercion ‘x ↦ x.1‘).

```
def sphericalShell (d : ℕ) : Metric.sphere (0 : Space d) 1 → Space d
```

lemma Space.sphericalShell_injective [source]

```
lemma sphericalShell_injective (d : ℕ) : Function.Injective (sphericalShell d)
```

lemma Space.sphericalShell_continuous [source]

```
lemma sphericalShell_continuous (d : ℕ) : Continuous (sphericalShell d)
```

lemma Space.sphericalShell_measurableEmbedding [source]

```
lemma sphericalShell_measurableEmbedding (d : ℕ) : MeasurableEmbedding (sphericalShell d)
```

lemma Space.norm_sphericalShell [source]

```
@[simp]
lemma norm_sphericalShell (d : ℕ) (x : Metric.sphere (0 : Space d) 1) :
    ‖sphericalShell d ‖x = 1
```

def Space.sphericalShellMeasure [source]

The measure on ‘Space d‘ corresponding to integration around a spherical shell.

```
def sphericalShellMeasure (d : ℕ) : Measure (Space d)
```

instance Space.sphericalShellMeasure_hasTemperateGrowth [source]

```
instance sphericalShellMeasure_hasTemperateGrowth (d : ℕ) :
    (sphericalShellMeasure d).HasTemperateGrowth
```

def Space.sphericalShellDist [source]

The distribution associated with a spherical shell. One can roughly think of this distribution as the distribution which takes test functions ‘f (r)‘ to ‘∫ d^3r f(r) ρ(r)‘ where ‘ρ(r)‘ is the mass, charge or current etc. distribution.

```
def sphericalShellDist (d : ℕ) : (Space d) →d[ℝ] ℝ
```

lemma Space.sphericalShellDist_apply_eq_integral_sphericalShellMeasure [source]

```
lemma sphericalShellDist_apply_eq_integral_sphericalShellMeasure (d : ℕ) (f : 𝓢(Space d, ℝ)) :
    sphericalShellDist d f = ∫ x, f x ∂sphericalShellMeasure d
```

lemma Space.sphericalShellDist_apply_eq_integral_sphere_volume [source]

```
lemma sphericalShellDist_apply_eq_integral_sphere_volume (d : ℕ) (f : 𝓢(Space d, ℝ)) :
    sphericalShellDist d f =
    ∫ x, f (sphericalShell d x) ∂(MeasureTheory.Measure.toSphere volume)
```

2.121 QuantumInfo.States.Mixed.MState

QuantumInfo/States/Mixed/MState.lean on GitHub.

theorem MState.traceNorm_eq_one [source]

```
@[simp]
theorem traceNorm_eq_one ρ( : MState d) : ρ.m.traceNorm = 1
```

2.122 scripts.MetaPrograms.module_doc_lint

scripts/MetaPrograms/module_doc_lint.lean on GitHub.

def linterExemptions [source]

The array of modules exempt from all linters, read from ‘scripts/LinterExemption.txt‘. This is used to lint ‘QuantumInfo‘ file-by-file.

```
def linterExemptions : IO (Array FilePath)
```

def importedFilePaths [source]

The file paths of the modules imported into the module ‘mods‘ (e.g. ‘Physlib‘).

```
def importedFilePaths (mods : Name) : IO (Array FilePath)
```

2.123 scripts.MetaPrograms.runPhyslibLinters

scripts/MetaPrograms/runPhyslibLinters.lean on GitHub.

def linterExemptions [source]

The file paths of modules exempt from all linters, read from ‘scripts/LinterExemption.txt‘. This is used to lint ‘QuantumInfo‘ file-by-file.

```
def linterExemptions : IO (Array String)
```

def moduleToFilePathString [source]

The file path (as a string e.g. ‘QuantumInfo/Foo/Bar.lean‘) of the module ‘mod‘.

```
def moduleToFilePathString (mod : Name) : String
```

2.124 scripts.MetaPrograms.style_lint

scripts/MetaPrograms/style_lint.lean on GitHub.

def importedFilePaths [source]

The file paths of the modules imported into the module ‘mods‘ (e.g. ‘Physlib‘), found by reading the module’s ‘.olean‘ file.

```
def importedFilePaths (mods : Name) : IO (Array FilePath)
```

def linterExemptions [source]

The file paths of modules which should be skipped by the linters, read from ‘scripts/LinterExemption.txt‘. This is used to lint ‘QuantumInfo‘ file-by-file.

```
def linterExemptions : IO (Array String)
```

2.125 scripts.PhyslibAlpha.runPhyslibAlphaLinters

scripts/PhyslibAlpha/runPhyslibAlphaLinters.lean on GitHub.

def runLinterOnModule [source]

Run the Batteries linter on a given module and update the linter if ‘update‘ is ‘true‘.

```
unsafe def runLinterOnModule  (module : Name): IO Unit
```

def main [source]

```
unsafe def main (_ : List String) : IO Unit
```